SIEMENS

SITRAIN
Digital Industry Academy

SIMATIC WinCC Unified

TIA-UWCCBMWOEM

Register

Course version V18.00.033

Published by: Siemens AG, Digital Industries, Customer Services, P.O. Box 31 80, 91050 Erlangen, Germany

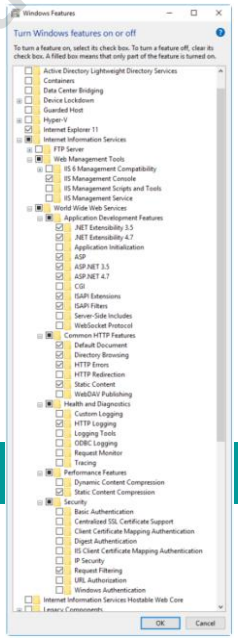
For the US published by: Siemens Industry Inc., 100 Technology Drive, Alpharetta, GA 30005, United States

SITRAIN courses on the internet: www.siemens.com/sitrain

1 System overview

1.1 Overview of Windows functions that are a precondition to use

Overview of Windows functions that are a precondition to use



WinCC Unified PC Runtime

Simulation of WinCC Unified PC systems

(on a WinCC Unified engineering station)

These windows functions must be activated before the installation of WinCC Unified !

https://support.industry.siemens.com/cs/de/en/view/109807122
https://support.industry.siemens.com/cs/de/en/view/109773589

1.2 Preconditions

1.2.1 WinCC Unified V18

Operating Systems (64-bit)	Configuration	Engineering	PC Runtime
Windows 11	Home Version 21H2	✓	✗
	Pro Version 21H2	✓	✓
	Enterprise 21H2	✗	✓
Windows 10 Pro / Enterprise	Pro / Enterprise Version 21H1	✓	✓
	Pro / Enterprise Version 21H2	✓	✓
Windows 10 IoT Enterprise (PC RT: Test for IPC)	Enterprise 2016 LTSC	✓	✓
	Enterprise 2019 LTSC	✓	✓
	Enterprise 2021 LTSC	✓	✓
Windows Server **(for Unified PC RT with more than 5 Clients)**	Windows Server 2016 Standard (Full Installation)	✓	✓
	Windows Server 2019 Standard (Full Installation)	✓	✓
	Windows Server 2022 Standard (Full Installation)	✓	✓

Operating System (Client)	Recommended Browser
Microsoft Windows	▪ Google Chrome (Preferred) recommended 107 (at least version 77 or higher) ▪ Microsoft Edge Chromium recommended 106 (at least version 88 or higher) ▪ Mozilla Firefox recommended 106 (at least version 78 or higher) ▪ Mozilla Firefox ESR recommended 102 (at least version 78.6 or higher)
Android	▪ Google Chrome (preferred) ▪ Firefox ▪ Edge
iOS, Mac	▪ Safari (preferred) ▪ Google Chrome ▪ Firefox ▪ Microsoft Edge

Reporting (Excel)
Local installation of MS Excel (32- or 64-Bit) as new as possible in accordance to operation system
Online access to Excel (Online Office 365)

Virtualization
VMware vSphere Hypervisor (ESXi) 6.7 (or higher)
VMware Workstation 12.5.5 or VMware Workstation 15.5.0 (or higher)
VMware Player 12.5.5 or VMware Player 15.5.0 (or higher)
Microsoft Hyper-V Server 2019

Anti Virus Software
Symantec Endpoint Protection 14.3
McAfee Endpoint Security (ENS) 10.7
Trend Micro "Office Scan" 14.0
Windows Defender (as part of Windows operating systems)
Qihoo 360 "Safe Guard 12.1" + "Virus Scanner"

1.3 Engineering-Licenses for WinCC Unified (TIA Portal)

WinCC Unified (TIA Portal) V18		WinCC (TIA Portal) V18	
Engineering System	Unified PC Runtime	WinCC RT Advanced	WinCC RT Professional
WinCC Unified PC (max.)	✓ (max.)	✓	✓ (max.)
WinCC Unified PC (100k)	✓ (100k)	✓	✓ (max.)
WinCC Unified PC (10k)	✓ (10k)	✓	-

The license of WinCC Unified (TIA Portal) is also valid for engineering using WinCC (TIA Portal)

WinCC (TIA Portal) V18 License present …				WinCC Unified (TIA Portal) V18 Can be projected as well…
Engineering System applicable for..	Comfort Panels	WinCC RT Adv.	WinCC RT Prof.	Unified PC Runtime
WinCC Prof. (max)	✓	✓	✓ (max.)	✓ **(100k)**
WinCC Prof. (4k)	✓	✓	✓ (4k)	✓ (10k)
WinCC Prof. (512)	✓	✓	✓ (512)	✓ (10k)
WinCC Advanced	✓	✓	-	✓ (10k)

Existing licenses of WinCC (TIA Portal) can also be used with WinCC Unified (TIA Portal).

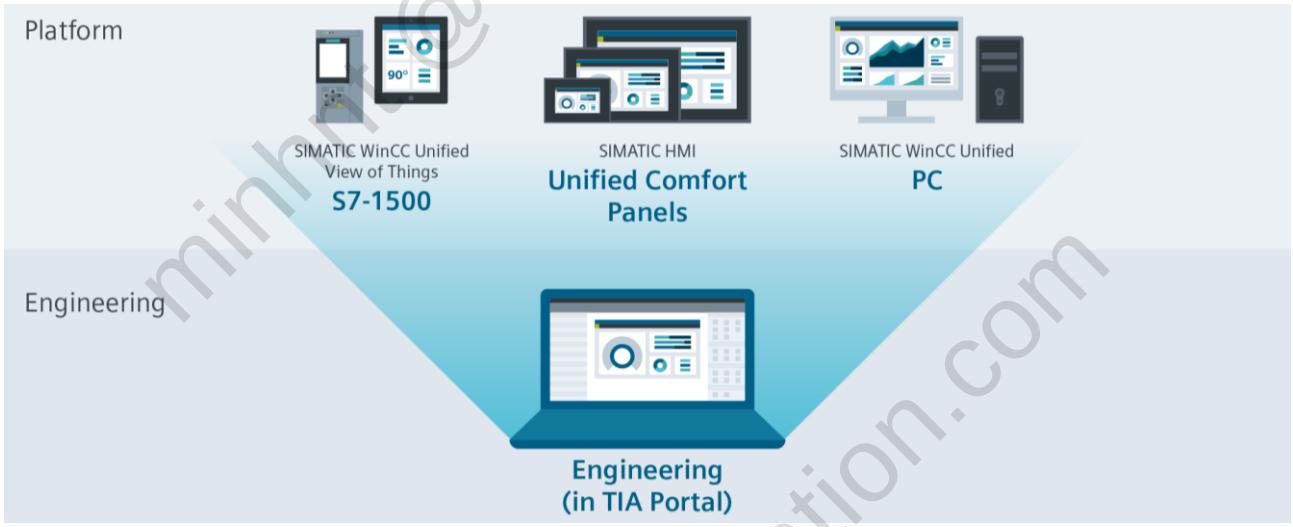
1.3.1 Compatibility Engineering and Runtime

Parallel installation	WinCC Advanced V18 ES	WinCC V7.x RC	WinCC V7.x RT	WinCC RT Professional V x ES	WinCC RT Professional V x RT
WinCC Unified PC (x) ES	**Yes**	**No**	**No**	**No**	**No**
WinCC Unified PC (x) RT	**Yes**	**No**	**No**	**No**	**No**

1.4 SIMATIC WinCC Unified System

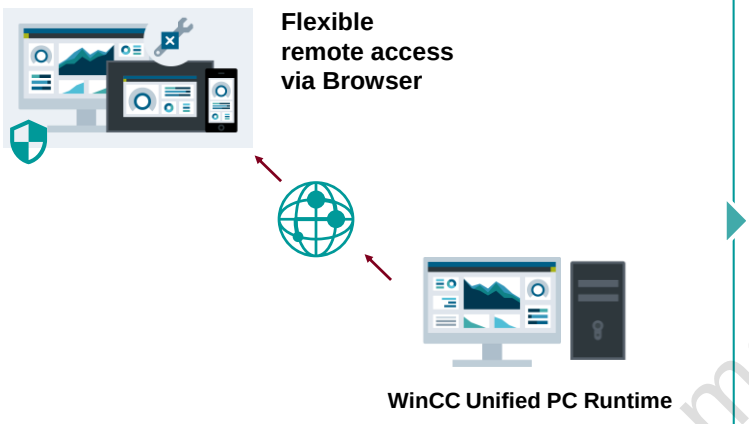
1.4.1 The DNA for the future of visualization

1.4.2 TIA Portal V18



1.5 Legend

1.6 Flexible web-based, remote Monitoring and Operation



Clients for flexible remote access

- Simultaneous remote access offers parallel operation of many users
- Use **state-of-the-art web** to access the visualization
- **"Zero Installation" Clients** via state-of-the-art web browsers – **no need to install separate apps or plug-ins**
- **Configurable access control** via user management
- **Secure communication using web standards (**https, SSL, certificates,…)

Using various Client devices

- Get Client device dimension as java script function

1.7 Remote access for “Operation” or “Monitoring”

WinCC Unified
offers Clients for remote Monitoring or Operation

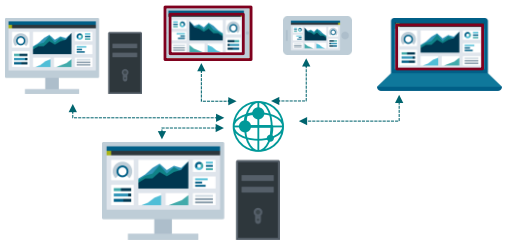
- 1 Client for remote Operation inclusive
  1 Client for remote Monitoring inclusive

The total number of Monitoring or Operation Clients is expandable upon demand.

Please note
There are separate licenses for Client Operate and Client Monitor (independent from each other)

1.8 WinCC Unified PC systems

1.8.1 Remote Control



Every WinCC Unified PC Runtime system includes:

- The local visualization Client (monitor and operate via browser)
- **ONE “Operate” Client**
- **ONE “Monitor” Client**

The maximum number of clients can be extended (optional).

Client "Operate“ enables remote operation according to assigned user role

- 1 Client for remote Operation inclusive
  (steps: 1,3,10, max)

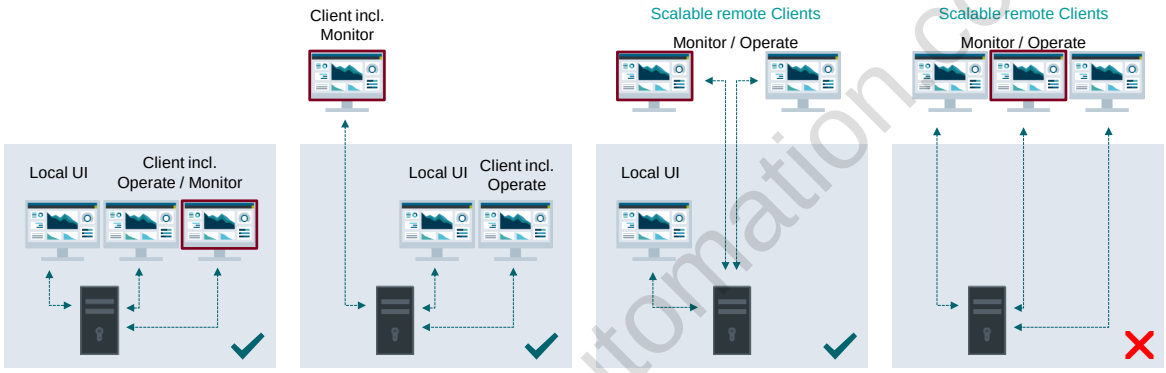
Client "Monitor“[1] enables remote operation in view-only mode

- 1 Client for remote Monitoring inclusive
  (steps: 1,3,10, max)

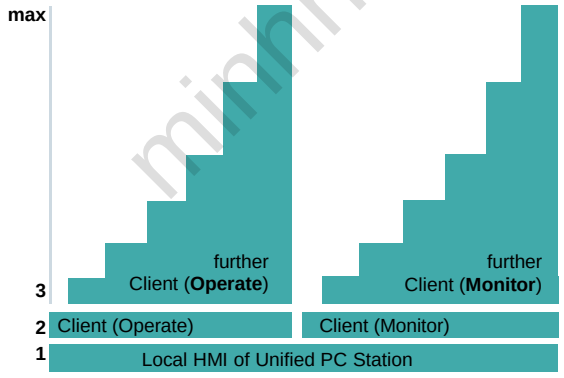
Asynchronous access of numerous users (simultaneous access is licensed)

Windows 10 Server is mandatory
for more than 5 Clients **!**

1.8.2 Remote Monitoring and Operation



1.8.3 License concept – Clients (Operate / Monitor)



Client licenses (Operate / Monitor) are pooled at the RT station – and can be extended on demand at any time.

Flexible remote access for any number of users **(concurrent access)**

- **Grow on demand** increase number of Clients **(countable) -** grow by 1, 3, 10, max

Each Unified PC RT includes:

- One Client (local HMI)
- One Client (Operate)
- One Client (Monitor)

Client (Operate) and Client (Monitor) are independent of each other

- No up- or downgrade of licenses

1.9 General Highlights

Scalable for industries
Advanced functionality like long-term databased logging, reporting

Zero installation Clients
Flexible remote access for Operation.

Available as of TIA Portal V17: Monitor-only clients in order to provide management or information screens.

Automated engineering
Create, validate and reuse standardized WinCC Unified components with TIA Portal Openness.

Open for extensions
Integrate your own controls or web-based tools directly into the visualisation of WinCC Unified.

Modern User Interface
Take full advantage of WinCC Unified technologies to provide the best possible visualisation during operation.

Security & Access Control
Secured communication and configurable access control.

Distributed Systems
Setup of distributed configurations by sharing screens and Alarms between WinCC Unified devices.

Integration Platform
Connect OT/IT Systems into one user interface and exchange data via interfaces e.g., OPC UA.

1.10 Additional Highlights and News

Scalable for industries
Advanced functionality like long-term databased logging, reporting

Zero installation Clients
Flexible remote access for Operation.

Available as of TIA Portal V17: Monitor-only clients in order to provide management or information screens.

Expand functionality
Individual application via a Server sided data interface for online and historical tags and alarms with.net or C++ programming.

Please note the general highlights and V17 news for for Panel- and PC-based systems at page 9/10 (Link).

1.11 WinCC Unified View of Things V18

1.11.1 Summery

View of Things follows configuration approach – no need for programing skills

Comfortable editor with drag & drop functionality for visual configuration of your screens

View of Things is based on WinCC Unified Technology – Screens can also be used for Panel & PC visualization

View of Things only requires an Engineering license for WinCC Unifed (TIA Portal)

1.12 Get started smoothly with the WinCC Unified System

1.12.1 Supporting Materials

Tutorial Center SIOS

Video series for an easy start with the WinCC Unified System. Learn all the things you need to get started smoothly.

Template Suite SIOS

Ready to use screen templates for PC Systems and the HMI Unified Comfort Panels in a free TIA Portal library incl. Wizard.

Switching Guideline SIOS

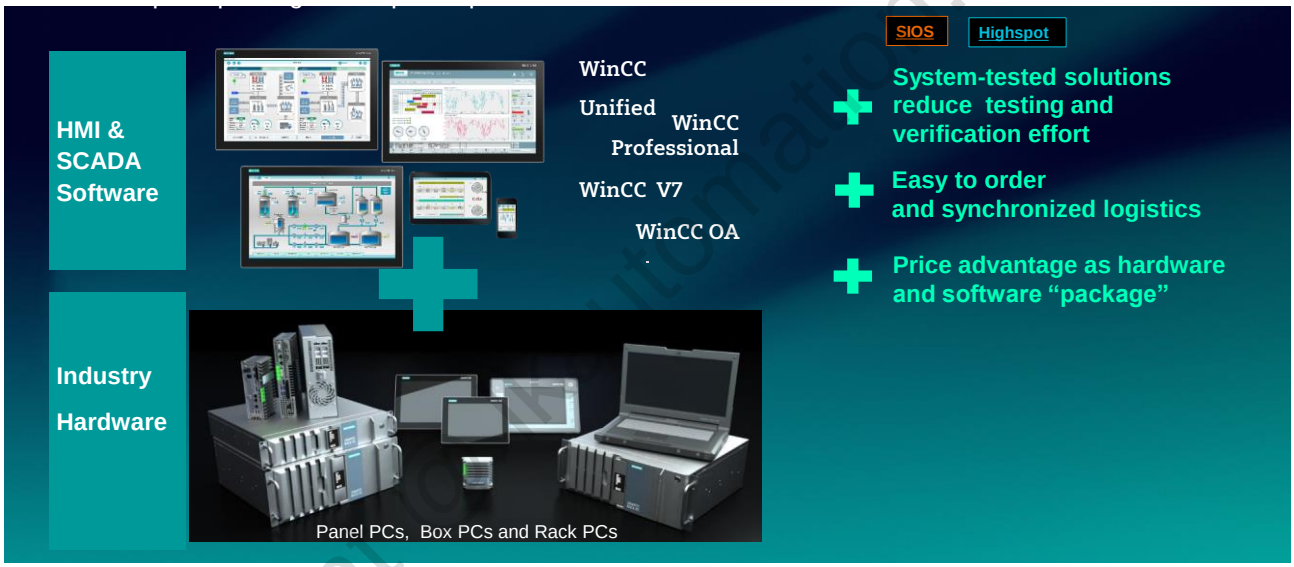
Everything you need know when you want to switch from an existing visualization to the WinCC Unified System

Demo project SIOS

Toolbox SIOS

Find all the supporting materials for the WinCC Unified System at a glance SIOS

1.13 Hardware and software from a single source The complete package at a special price



2 SIMATIC WinCC Unified tools

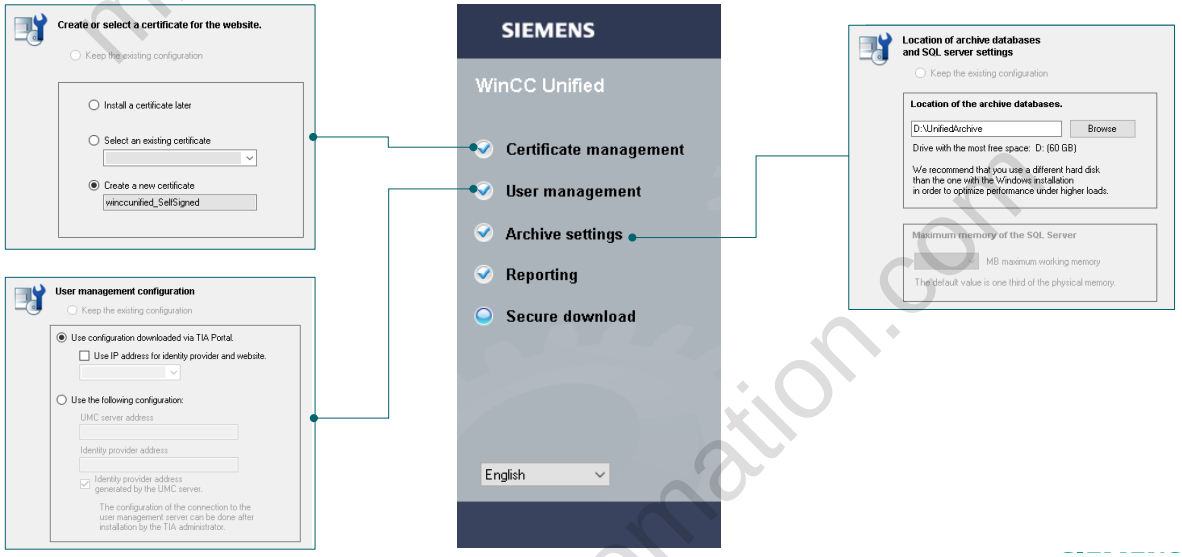
2.1 Unified Configuration

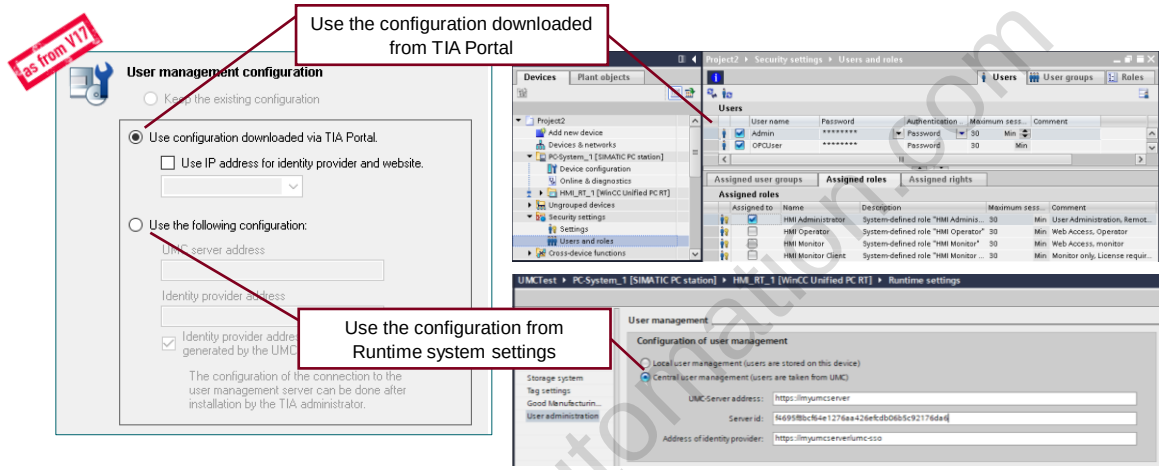
Adjust settings for:

- Web UI certificate
- User Administration
- Storage location of log databases
- Password-protected download
- Storage location of Reports

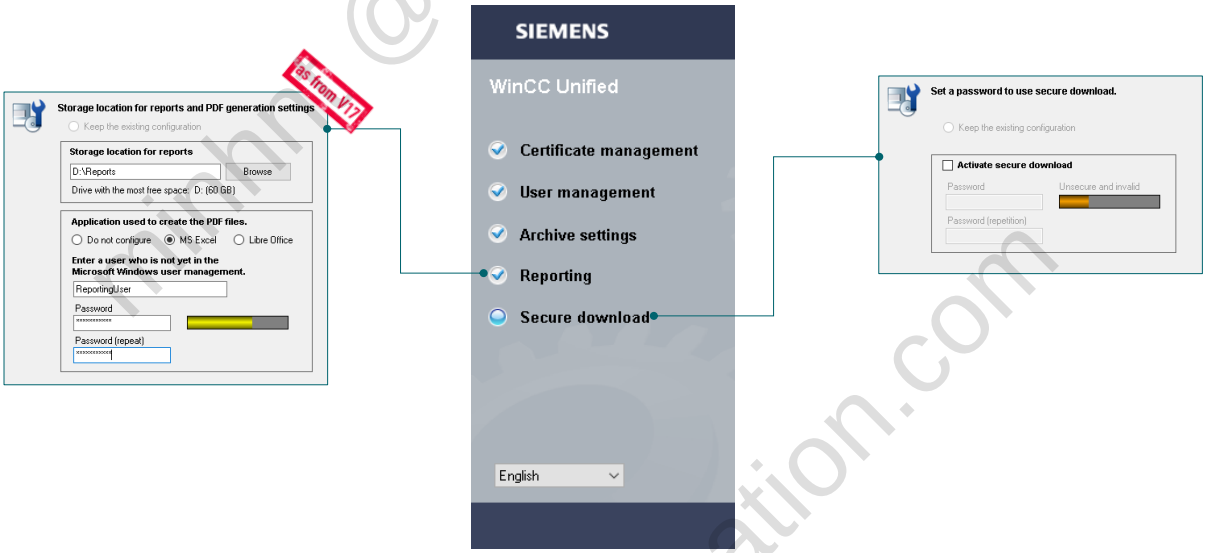
Unified Configuration dialog is displayed during installation but you can also open it after installation.

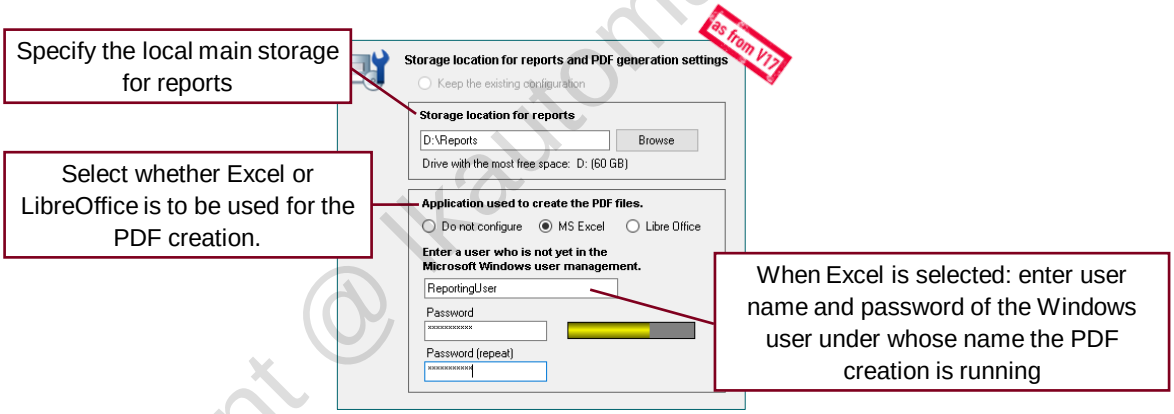
Available during installation of WinCC Unified ES or PC Runtime





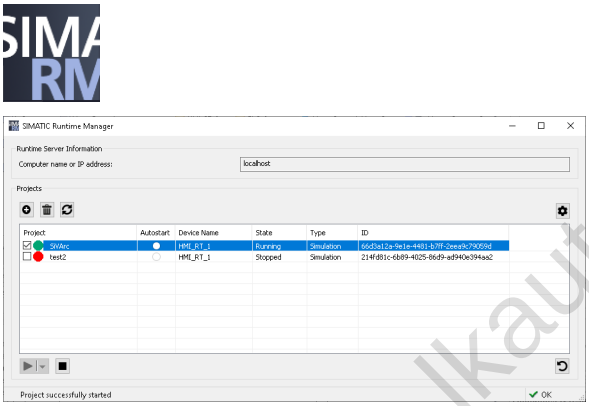
Configure the user management configuration that Runtime is to use





Configure the settings for generating reports.

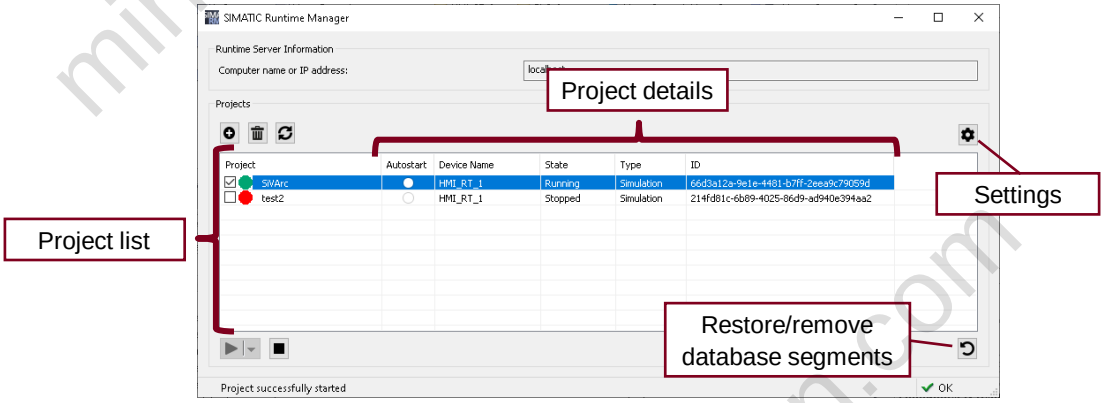
2.2 Runtime Manager

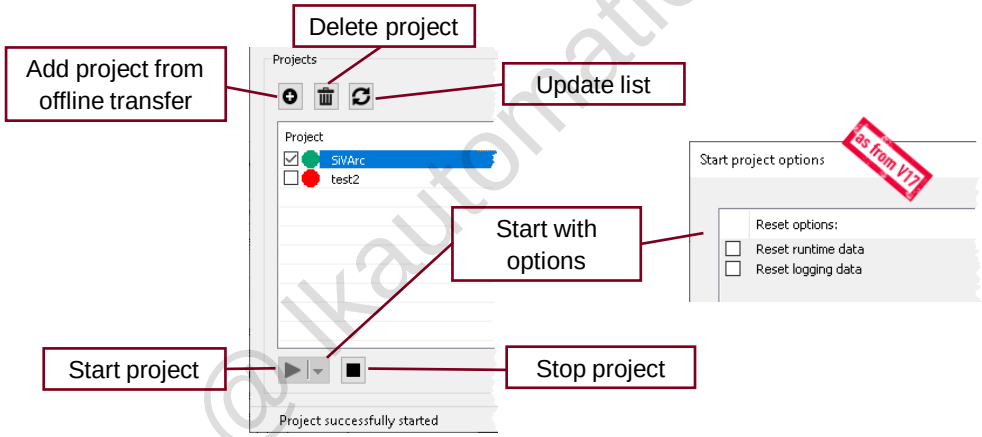


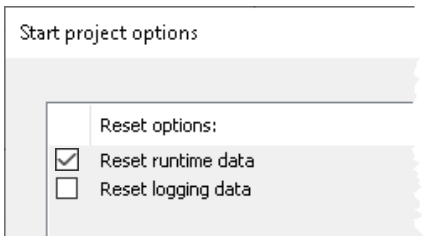
Manage WinCC Unified projects

- Run, stop or switch projects
- Show project state
- Activate / Deactivate Debugging
- OPC UA Export
- Manage 3rd party certificates
- Connect logging backup segments
- User Management
- Autostart project

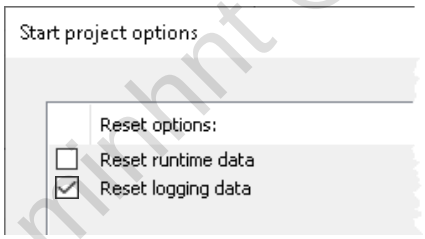
Several WinCC Unified projects can be downloaded to one PC station via TIA Portal. Just one project at a time can be running.





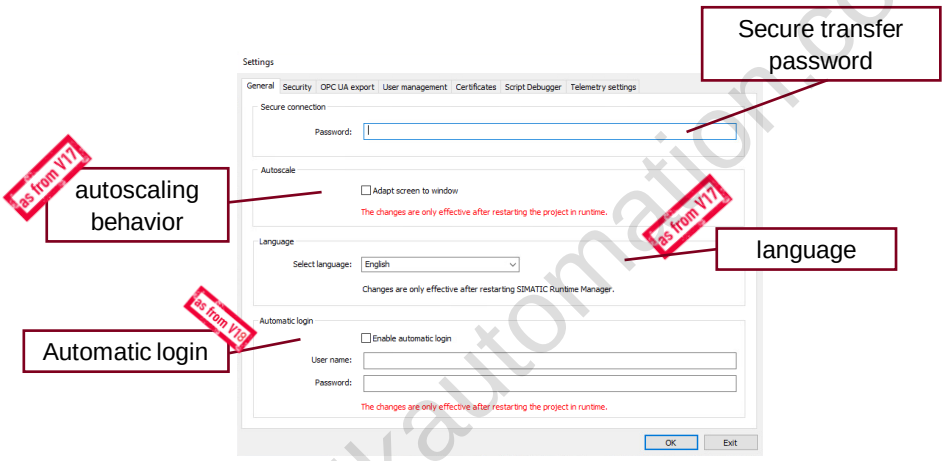


- Last values of internal, persistent tags
- Last alarm states
- Persistent attributes of the alarm annunciator
- Persistent attributes for the last logging cycle of the logging tags

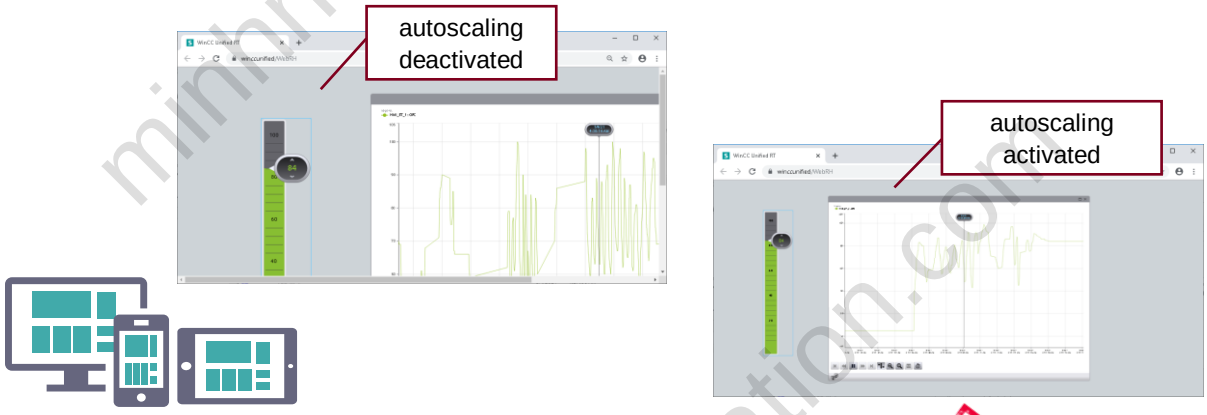


- Logging tags
- Log alarms
- Logged context values

2.2.1 Runtime Manager Settings

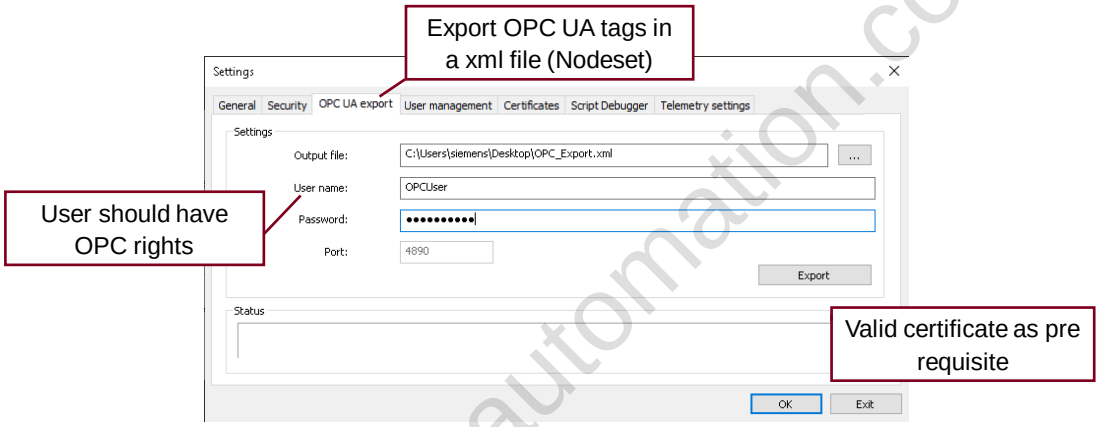


Runtime Manager settings > General tab.

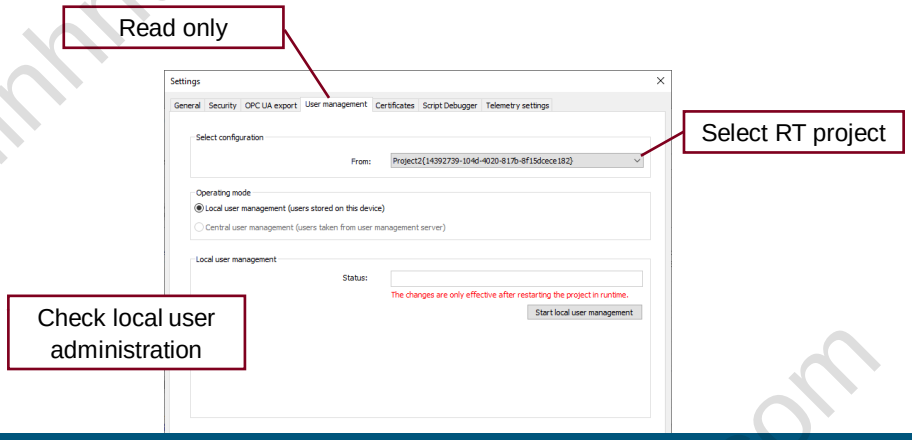


Automatically adapt the size of HMI screens to the window size of the browser in which a Runtime project is open.

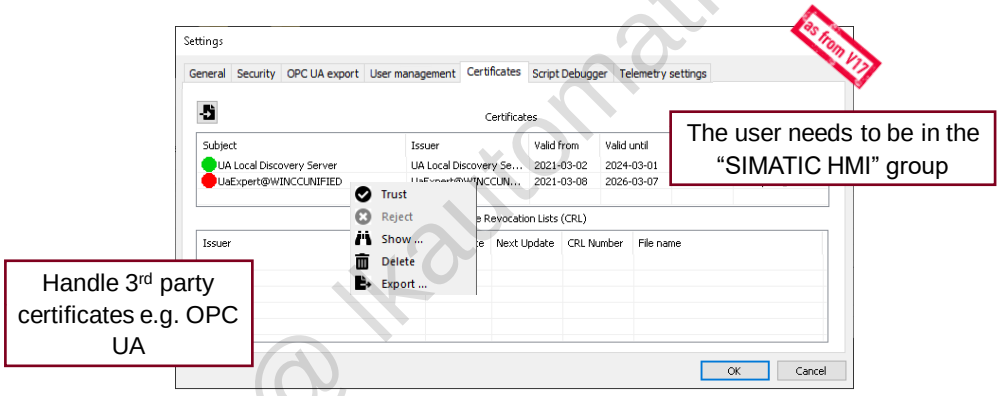
as from V17



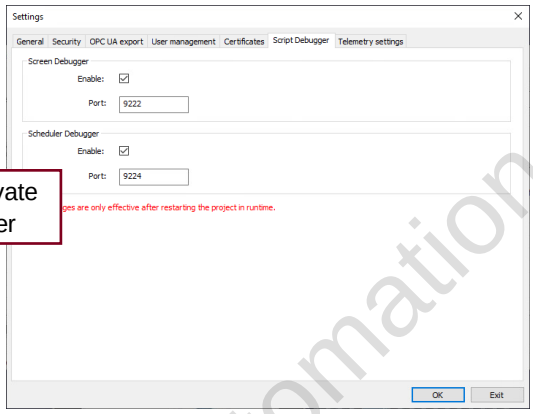
Runtime Manager settings > OPC UA Export tab.



Runtime Manager settings > User management tab.
The configuration is downloaded from TIA Portal project.



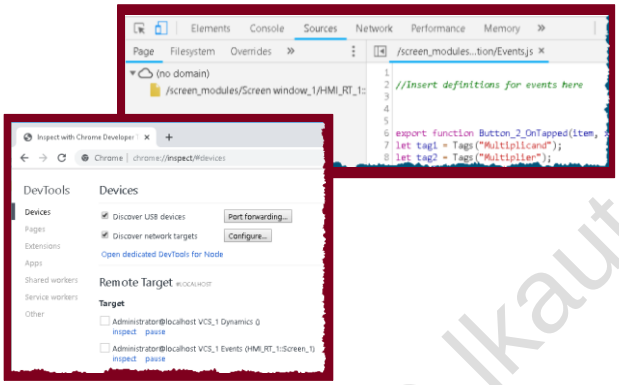
Runtime Manager settings > Certificates tab.



Activate/Deactivate script debugger

Runtime Manager settings > Script debugger tab.

2.3 Script debugger



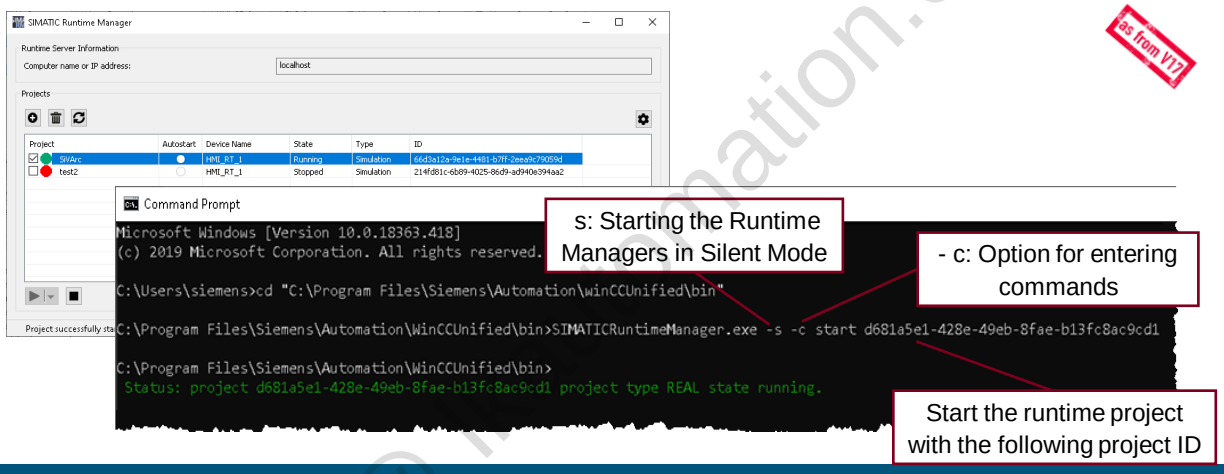
To start the debbuger:

- Chrome version >= V80: call the URL Chrome://inspect go to "Target Discovery" settings and add the address "127.0.0.1:9222“
- Chrome version < V80: call the URL http://localhost:port number (e.g. http://localhost:9222)

You can find more information in SIOS: 109779192.

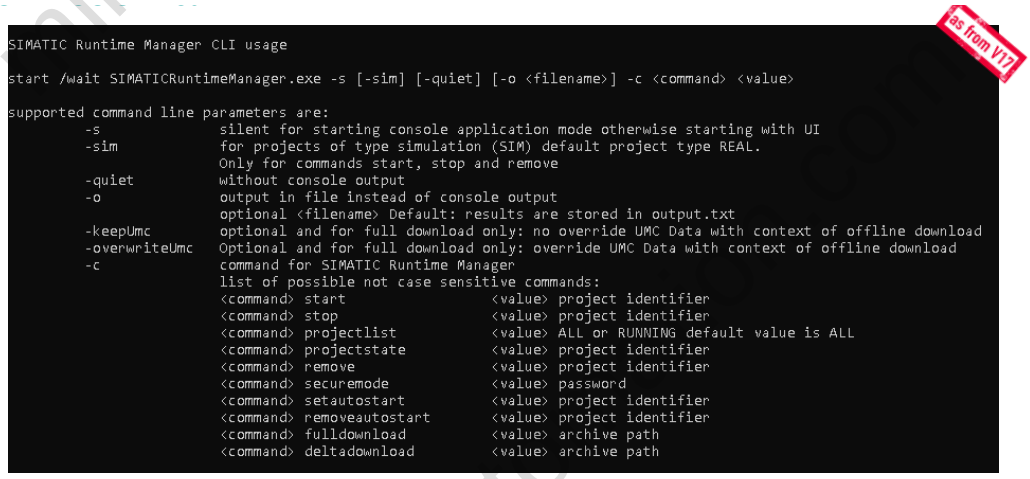
2.4 Options using the command line

2.4.1 Operation via command line



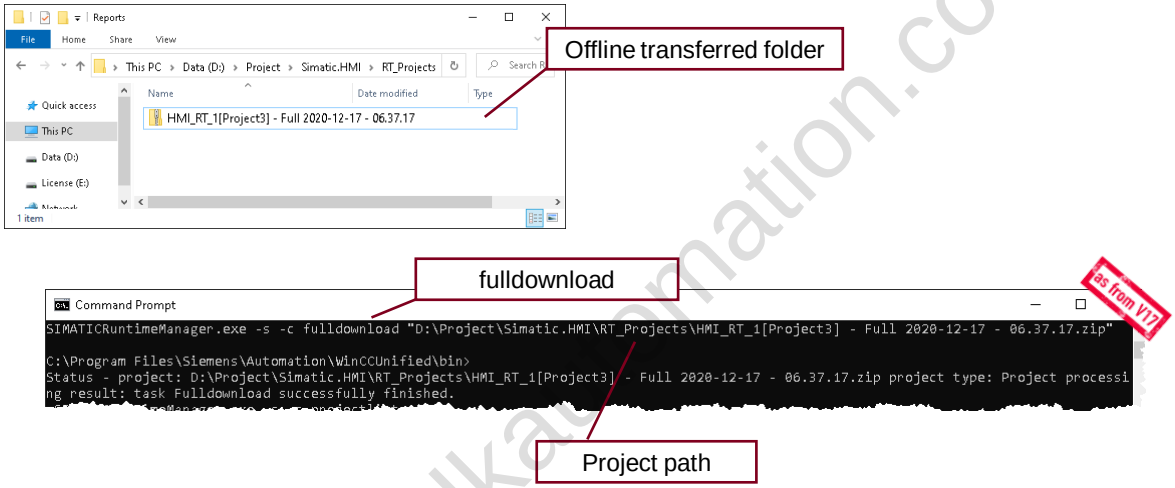
For starting/stopping projects: Projects have been loaded into Runtime.

2.4.2 Starting a project via the command line

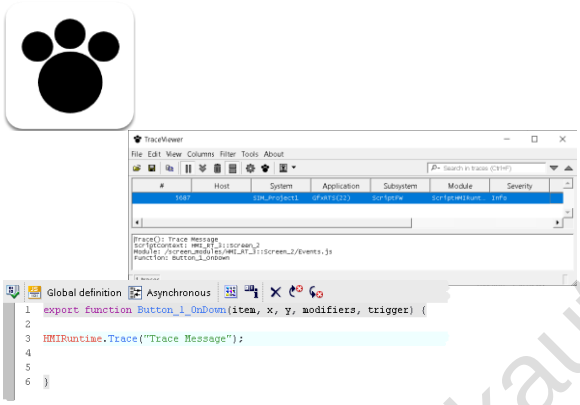


You can find more information in WinCC Unified Help in "Operation via command line" chapter.

2.4.3 Transfer a project via the command line

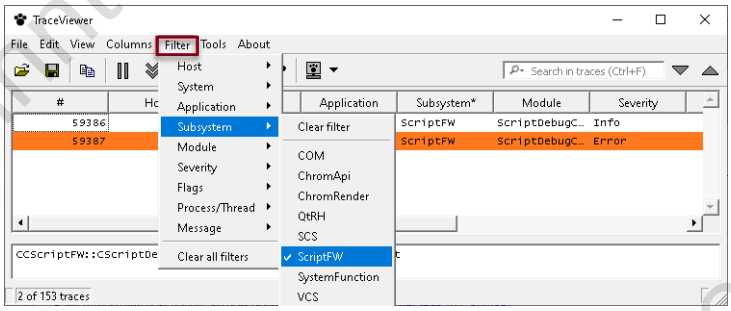


2.5 RTIL Trace viewer

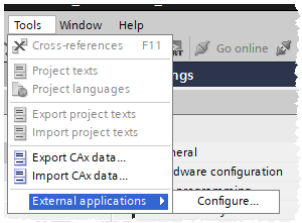


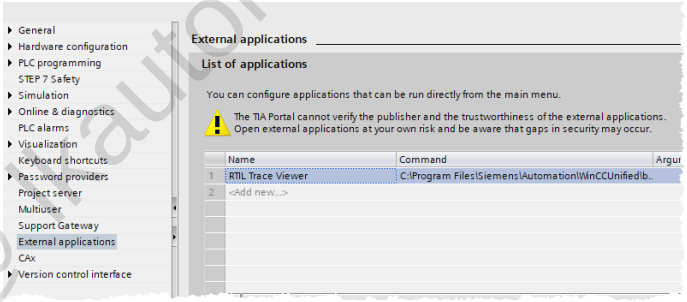
- Trace Viewer displays all runtime alarms which are listed in the configurable TraceCatalog.
- Traces are displayed in tabular form and can be filtered.
- Alarms can be exported in text or CSV files.

Location of RTIL Trace Viewer:
C:\Program Files\Siemens\Automation\WinCCUnified\bin\RTILtraceViewer



For better reading of the Trace Messages you can activate the filter:
Filter > Subsystem > ScriptFW

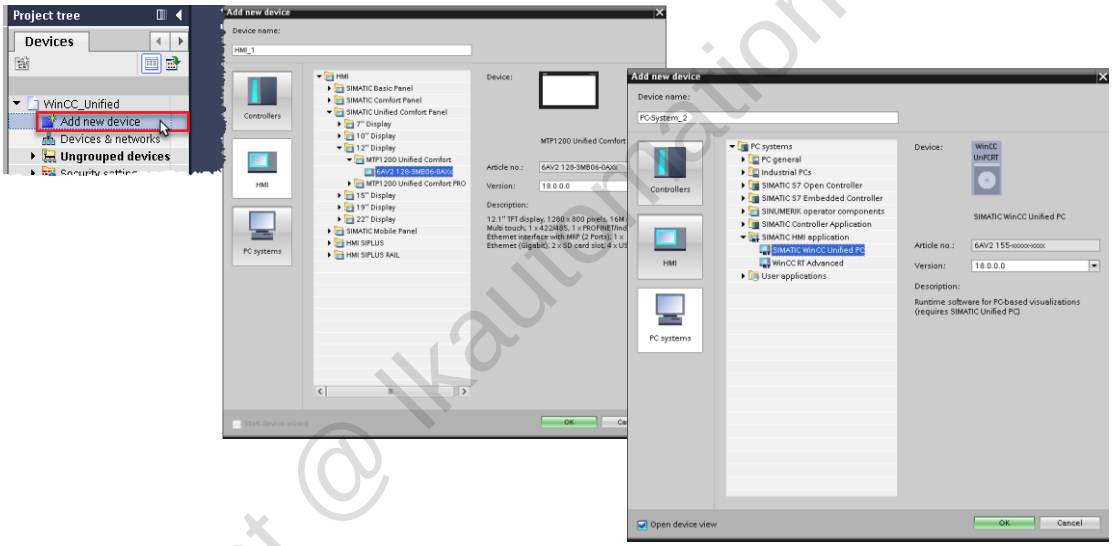




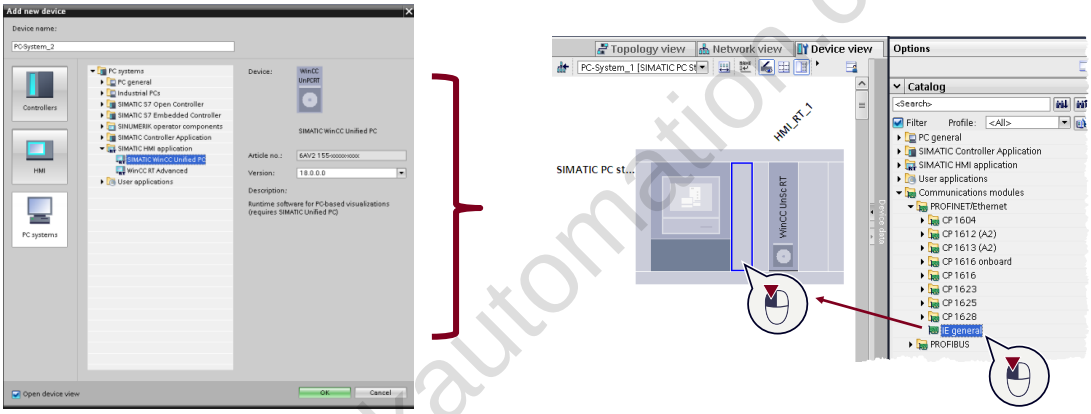
Integrate RTIL Trace Viewer as an external application.

3 Device handling

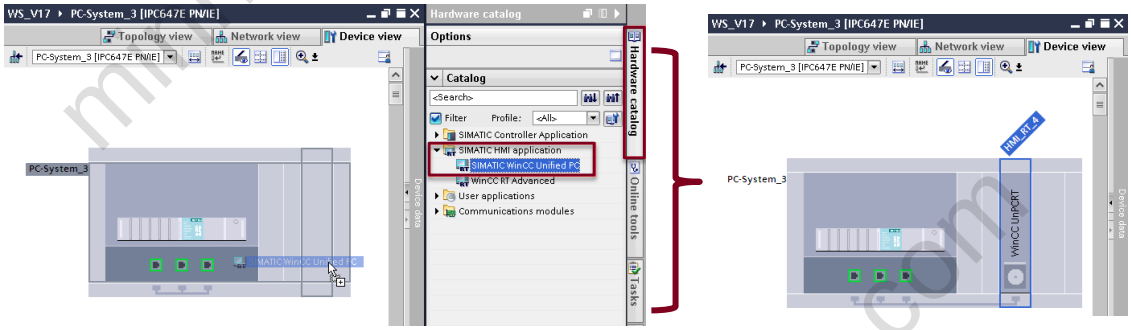
3.1 Add device



3.2 PC Station



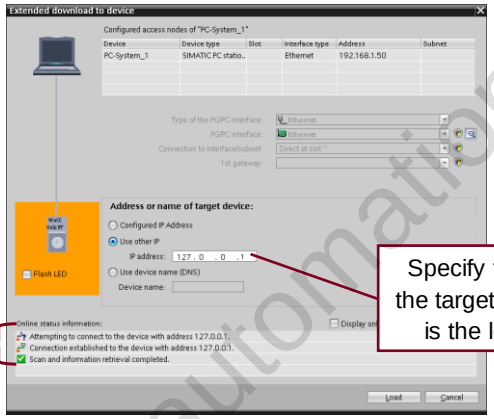
The PC station needs a separate communication module.



RT Application can be added to an E-series IPC.

as from V17

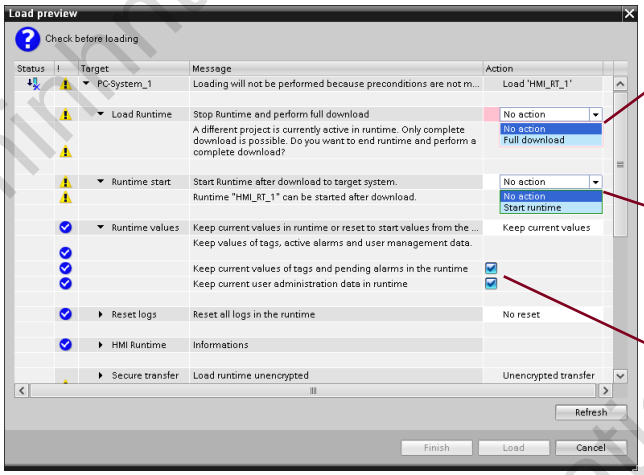
3.3 Downloading project data



Online status information

Specify the IP address of the target device, 127.0.0.1 is the local IP-address

The first time a download is performed, the "extended download to device" window is opened to set the connection parameters. This data is stored for subsequent downloads.



In this box you can select a delta or a full download if applicable

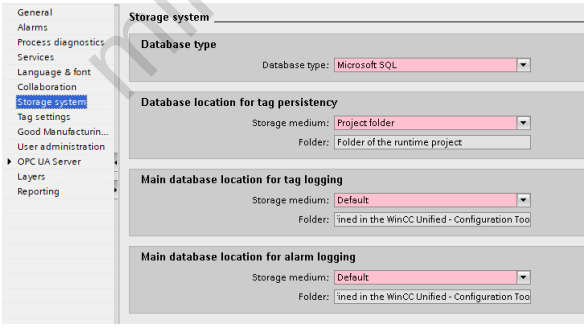
Select if you would like to start RT when download is finished or not

Select to keep or overwrite RT values

3.4 Change device

- You can use existing configurations for new devices and optimize these configurations with little manual effort.
- All data configured by you is retained in the configuration data. This means you do not need to copy individual objects of one device and paste them to another.

WinCC Unified supports change device function between PC-station and HMI Unified Comfort Panels.

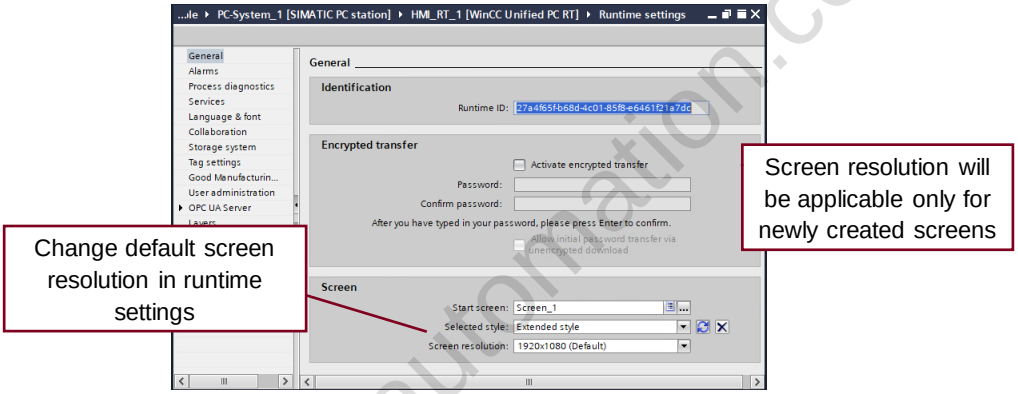


After an HMI device has been replaced, you should check:

- Appearance of configured screens
- Storage system settings
- Rename the device if necessary

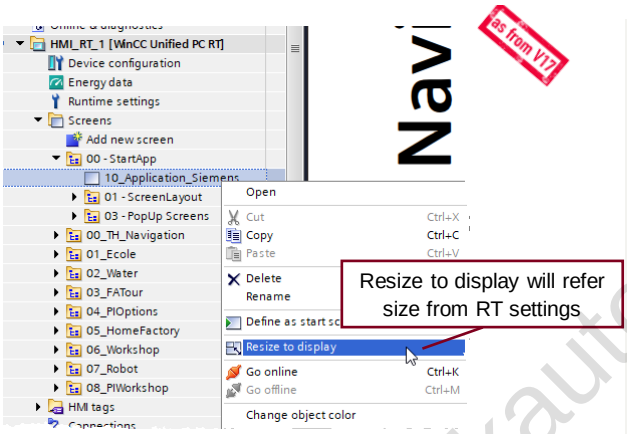
In case of unsupported functions after a device change, the compiler will report an error message.

3.5 Screen Resolution



Configure screen resolution from runtime settings

3.6 Resize screen



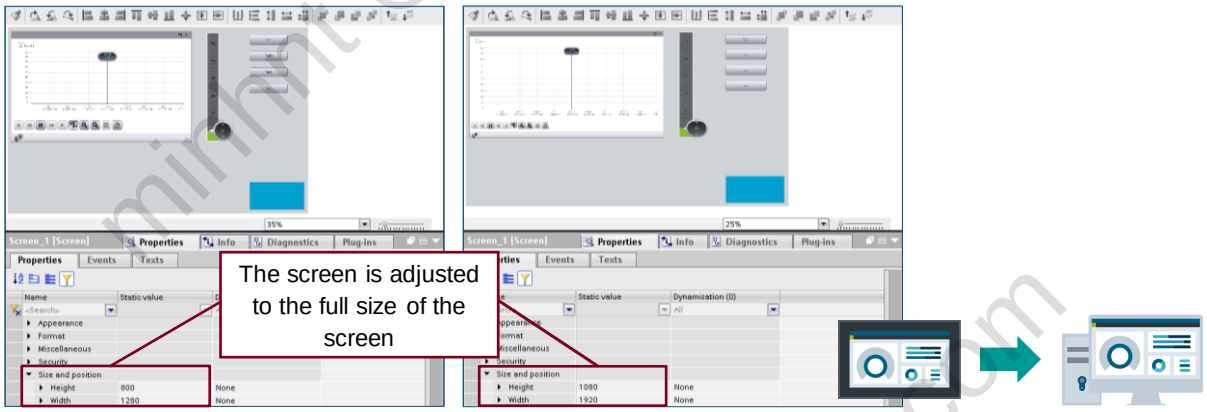
Trigger resize screen for:

- All screens
- A group of screens
- One screen

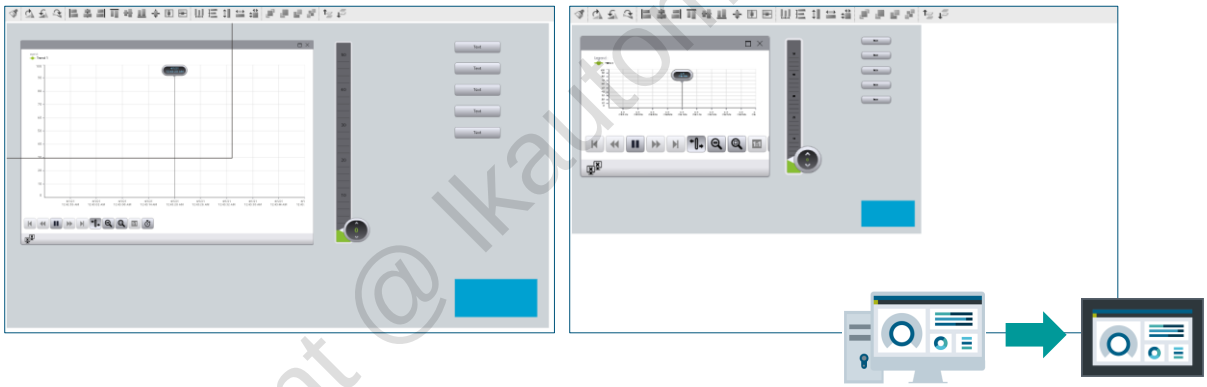
Fonts are adjusted as per ratio calculation.

After replacing the HMI device with substantially larger or smaller variants, you should **check the appearance** of the configured screens. Changed display sizes can result, for example, in the fonts being too small or too large.

The screen size, the objects and fonts used are scaled in one step in all selected screens.

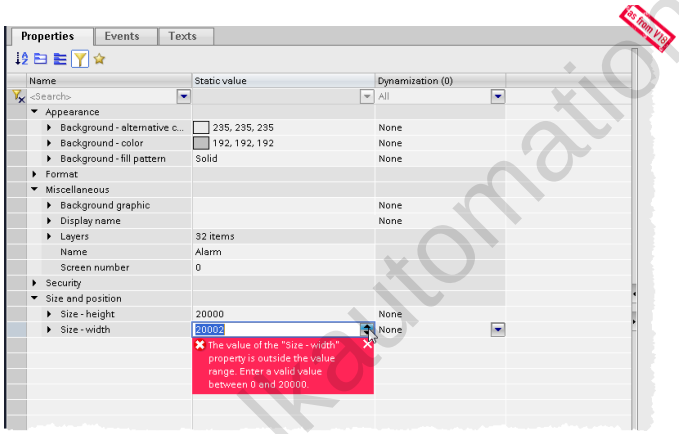


Screens with customized size should be adapted manually.



Objects out of range are not adjusted.

3.7 Screen Size improvement



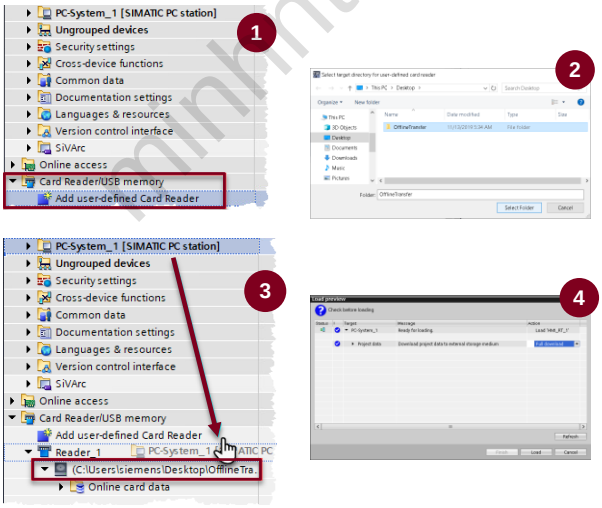
Now it is possible to use screens with a resolution of 20000x20000

3.8 Load project to external storage medium

Offline transfer

- You can use offline transfer if you cannot establish a direct connection from the configuration PC to the HMI device.
- The compiled runtime project is loaded onto an external storage medium.
- You load either the complete runtime project or only changes of a runtime project.

Only one runtime project at a time with the same project identification can be saved in the target directory.

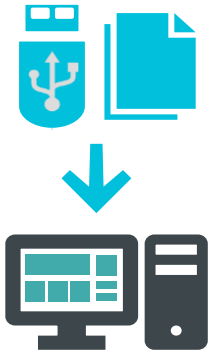


Procedure:

Jump to the "Devices" tab in the project tree.

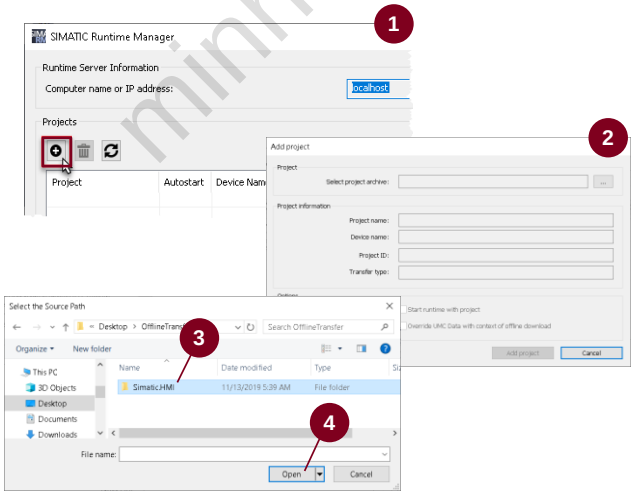
1. Double-click "Add user-defined card reader" in the "Card reader/USB storage" folder.
2. Select a target directory to save the project.
3. Drag and drop the folder of the HMI device to the added folder.
4. In the selection menu, specify how your project is to be loaded, Click "Load" to confirm.

3.9 Load project from external storage medium



Offline transfer:

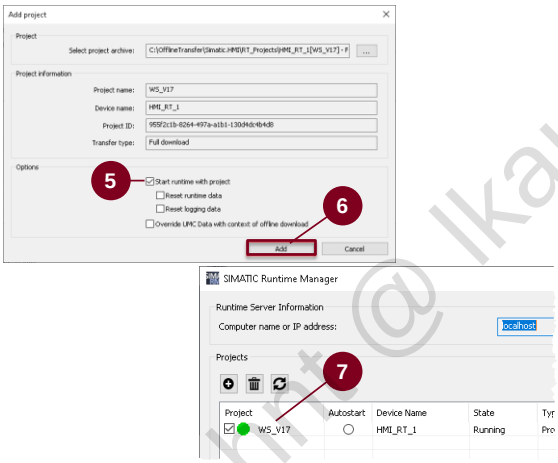
- SIMATIC Runtime Manager extracts the repository to a temporary folder on the target system.
- The transfer to runtime takes place from this folder which is then deleted again.



PC Procedure:

Start the "SIMATIC Runtime Manager" tool.

1. Click "Add project from offline transfer".
2. The "Add project" dialog opens. Click "..." under "Select project archive".
3. Select the compressed ZIP folder of the runtime project on the storage medium.
4. Click "Open".

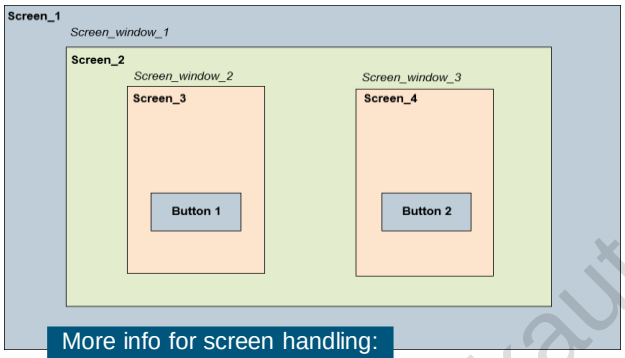


PC Procedure:

Under "Project information" you can see details of the selected project.

5. To start the project directly, select the option "Start runtime with project" under "Options".
6. Confirm with "Add project".
7. The project is running.

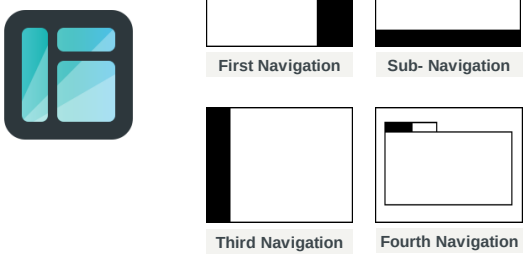
3.10 Screen navigation



More info for screen handling:
SIOS 109758536

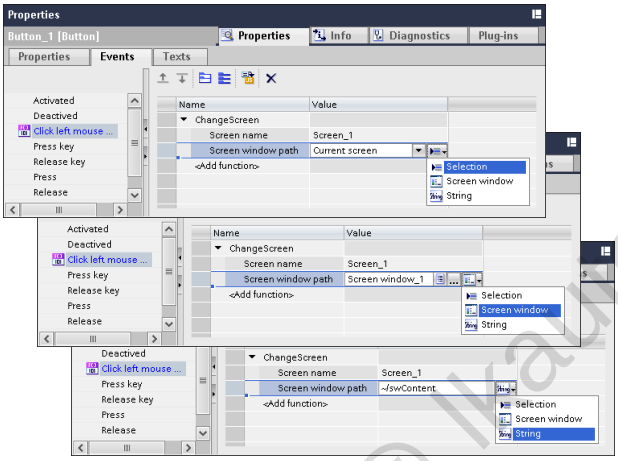
- To create the screen layout screen windows are used in WinCC Unified.
- This applies also to the Unified Comfort Panels
- Depending on the application, it may happen that several screen windows are nested.
- The ChangeScreen system function can be used to change the properties of the screen window dynamically.
- Parameter: Screen name [Object]
- Parameter: Screen window path [Object, String].

To create the screen layout screen windows are used in WinCC Unified.



HMI Template suite Wizard offers you templates for screen navigation.

3.10.1 ChangeScreen System Function



- Use as Screen window path parameter **Base screen Selection** to change the screen
- Use as Screen window path parameter **Screen window item** to change the screen in the same hierarchy
- Use as Screen window path parameter **String** to specify **absolute/relative item path** if item is not in the same hierarchy

3.10.2 Absolute / relative addressing

relative

Prefix	Description
".."	References the higher-level screen window (parent) in the context of the current screen window.
"."	References the own screen window (self).
""	A screen item of the current screen window is referenced without prefix.

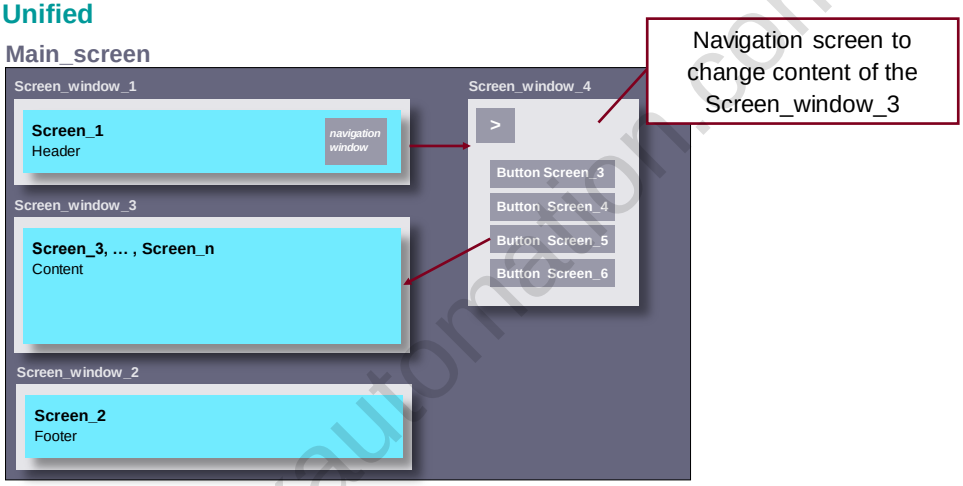
absolute

Prefix	Description
"/"	References a screen window on the highest level, whose name must follow.
"~"	References the screen window on the highest level in the own screen hierarchy.

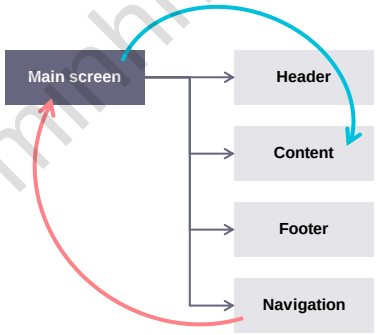
- Relative and absolute items paths are distinguished by the prefix of the item path.
- The **relative item path** is specified starting from the screen where the script is called.
- The **absolute item path** is specified starting from the "RootScreenWindow".

3.10.3 Example 1: Adjacent screen windows

SIMATIC WinCC Unified



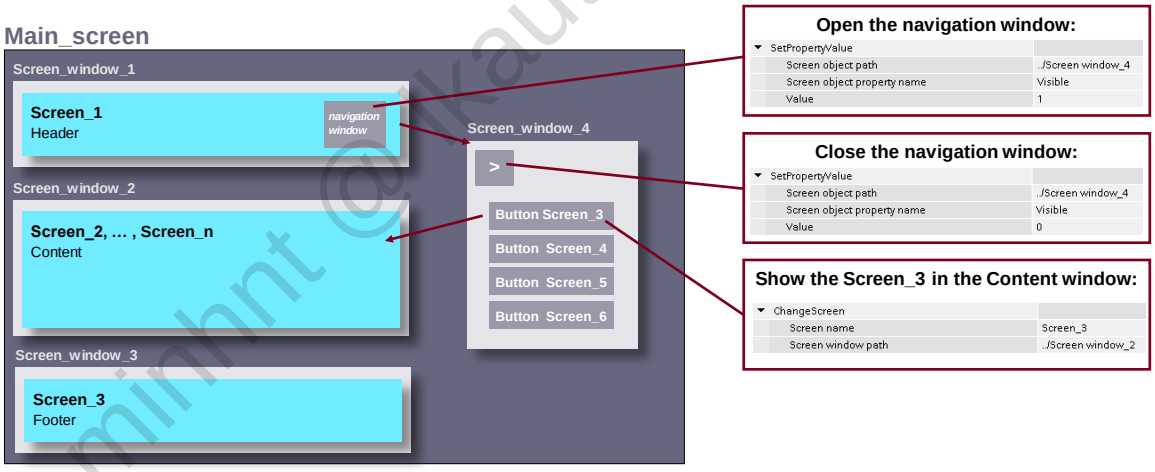
The navigation window is a good example to use ChangeScreen system function.



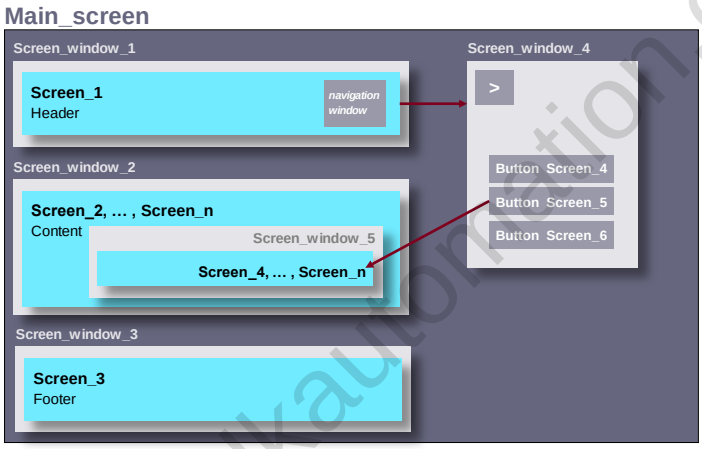
- The event to open a navigation window is located on Navigation (screen_window_4).
- The screen windows header, content, footer and navigation are adjacent screen windows, that's why the relative item path looks this way:

(„../Screen_window_2“)

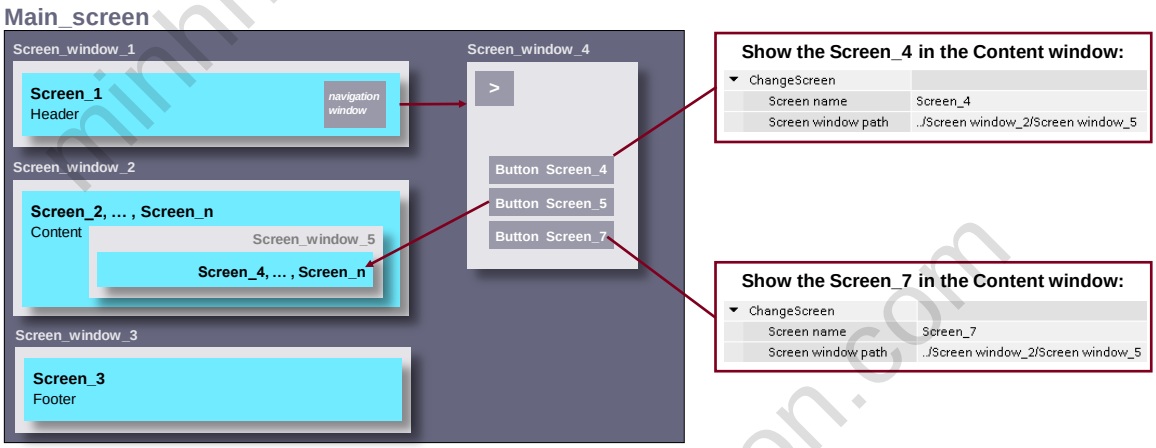
[Please note the colors!!!]



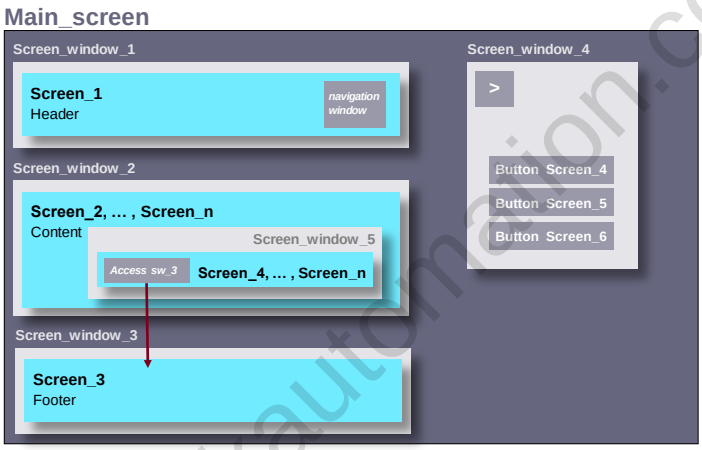
3.10.4 Example 2: Higher-level screen windows



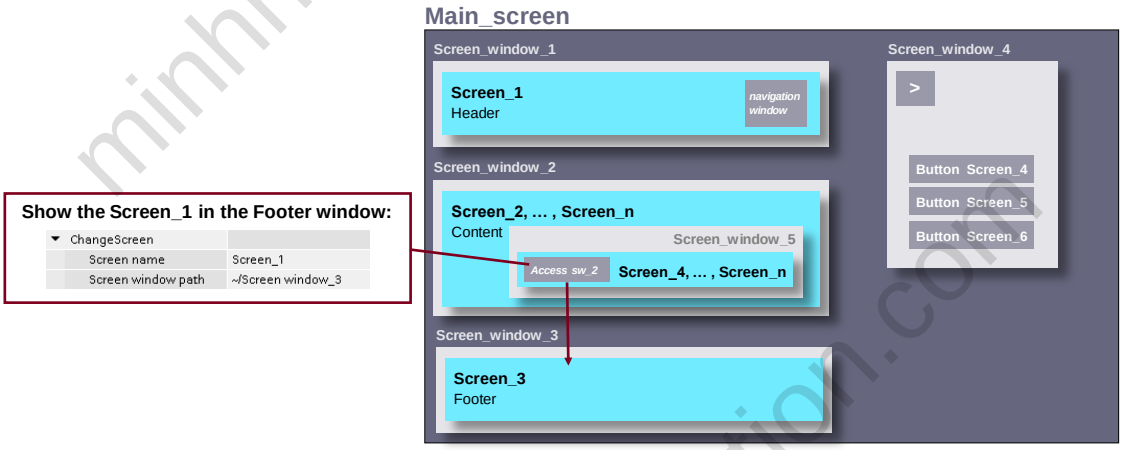
The content of a screen window needs to be displayed via button on the higher-level screen window.



3.10.5 Example 3: Lower-level screen windows

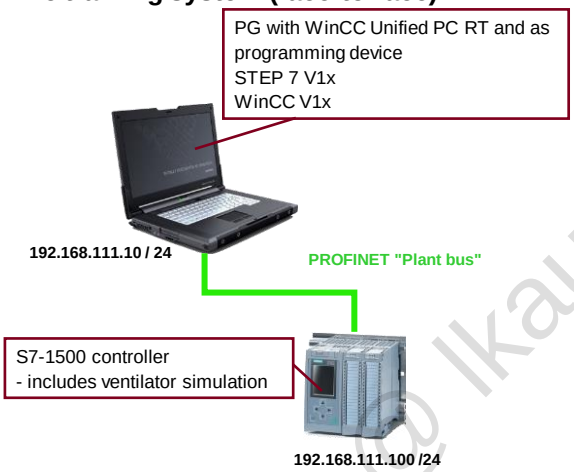


In this case the screen needs to be displayed via button, that is located on the lower-level screen window.



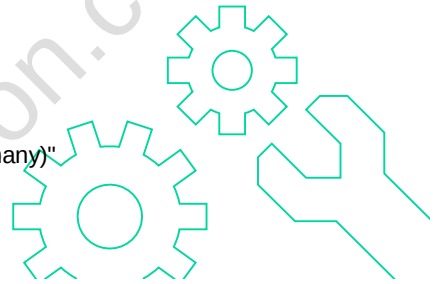
3.11 Exercise 1 (face-to-face): Add an HMI device

The training system (face-to-face)



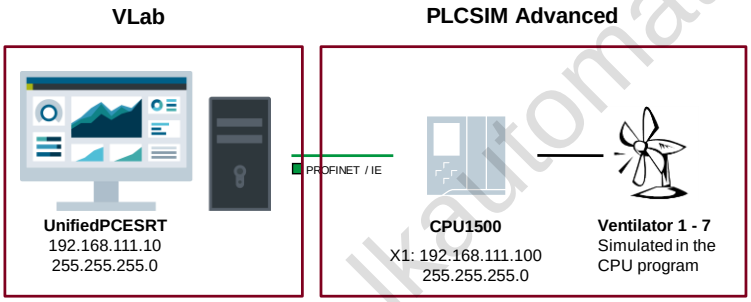
3.11.1 Procedure for exercise 1:

1. **Open project**
   - Open TIA Portal project "Project" with PLC program
     - Face-to-face: *D:\Courses\TIA-UWCCBMWOEM*
2. **Hardware download** not required. (face-to-face: PLC is already loaded; drive is simulated)
   - No activities
3. **Insert HMI device**
   - Insert HMI device (WinCC Unified PC RT)
   - Change device name to "HMI_RT"
4. **Add Ethernet interface**
   - Hardware catalog -> PC systems -> Communication modules > PROFINET/Ethernet -> IE General
5. **Make language settings**
   - Check project languages: "English (United States)" and "German (Germany)"
   - Check language setting: Editing language "English (United States)"
6. **Save the project**



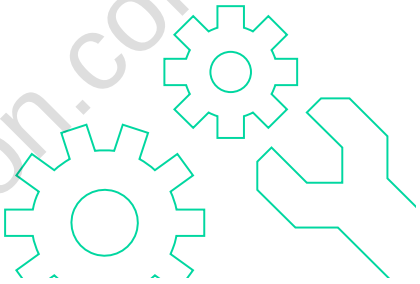
3.12 Exercise 1 (online): Add an HMI device

The training system (online)

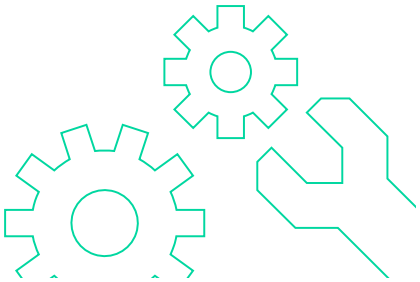


3.12.1 Procedure for exercise 1 (online): Add an HMI device

1. **Open project**
   - Copy TIA Portal project "Project" from C partition and store it here:
     - Online: *D:\Courses\TIA-UWCCBMWOEM*
   - Open TIA Portal project with PLC program
2. **Download hardware**.
   - Start PLCSIM Advanced
   - Create instance (IP address: 192.168.111.100)
   - Download
3. **Insert HMI device**
   - Insert HMI device (WinCC Unified PC RT)
   - Change device name to "HMI_RT"

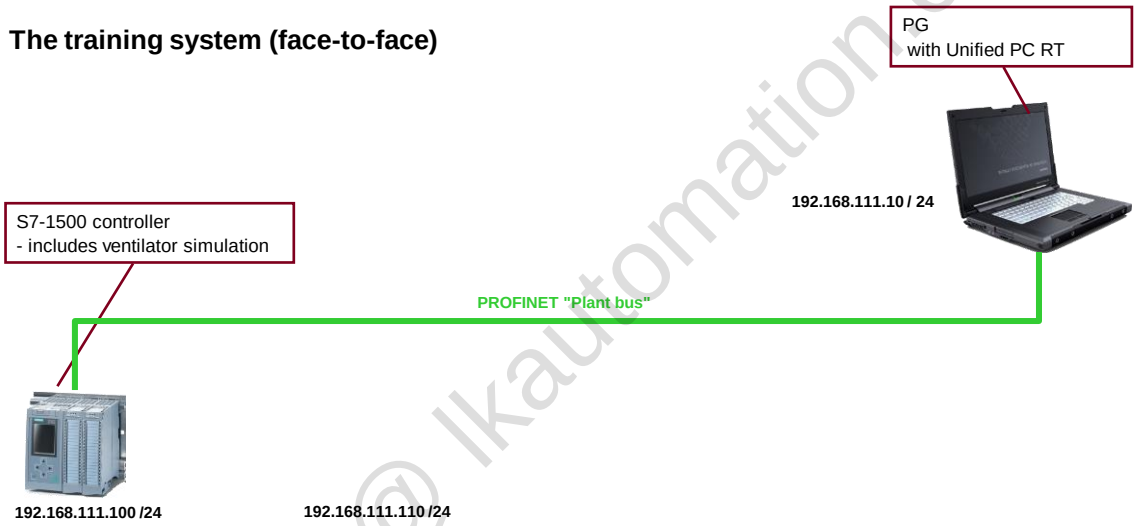


4. **Add Ethernet interface**
   - Hardware catalog -> PC systems -> Communication modules > PROFINET/Ethernet -> IE General
5. **Make language settings**
   - Check project languages: "English (United States)" and "German (Germany)"
   - Check language setting: Editing language "English (United States)"
6. **Save the project**



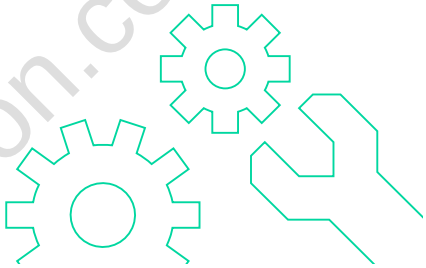
3.13 Exercise 2: Configure a connection

The training system (face-to-face)



3.13.1 Procedure for exercise 2:

1. **Set IP addresses for the PC**
   (Device configuration: Properties of HMI device)
2. **Network PC IE interface with "PROFINET" bus**
3. **Configure the HMI connection (S7-1500 with PC IE interface)**
   - Create HMI connection and rename it to HMI_PCRuntime
4. **Create an initial screen named "Start".**
   - Insert a text box with the text "Start screen".
   - Enter the screen as the start screen in the Runtime settings of the PC.
5. **Disable encrypted transfer in Runtime settings**
6. **Save the project**

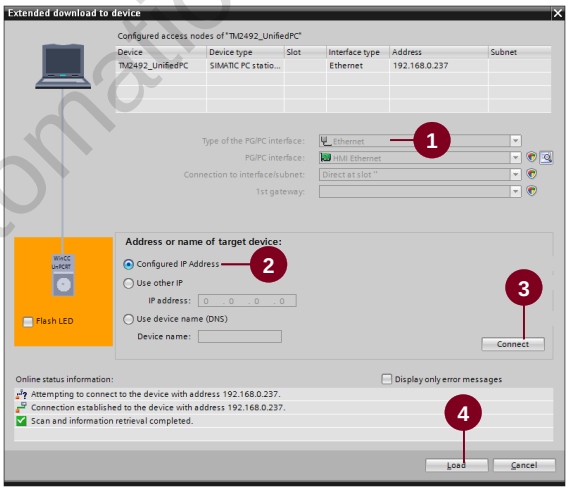


3.14 Exercise 3: Download a configuration to the HMI device

3.14.1 Procedure for exercise 3:

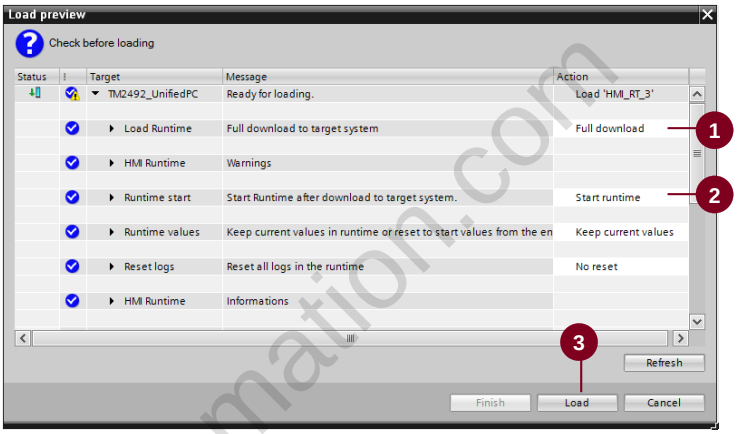
1. **Download configuration**
   - Select PC in the project tree
   - Click "Download to device"

2. **At the first download, the "Extended download to device" dialog is displayed**
   - (1) Ensure that "Ethernet" is selected
   - (2) Select "Configured connection"
   - (3) Click the "Connect" button
   - (4) Click the "Load" button



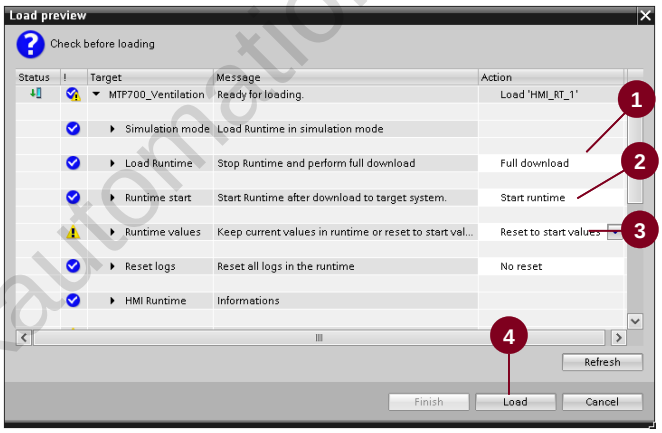
3. **"Load preview" dialog is displayed**
   - (1) Change target "Load Runtime" to "Full download"
   - (2) Change target "Runtime start" to "Start Runtime"
   - (3) Click "Load"

4. **Read the information in the Inspector window "Info > General"**

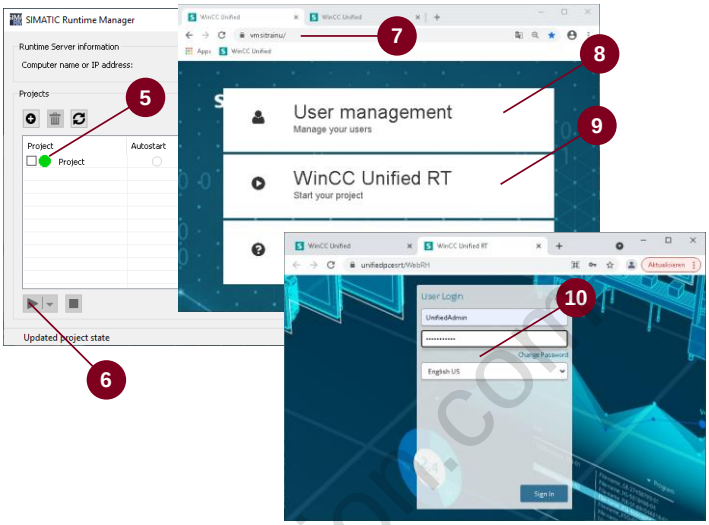


3.14.2 Procedure when downloading for the first time or simulating a configuration

1. **Configure PG/PC interface**
2. **Create user**
   - User: UnifiedAdmin
   - Password: Siemens123!
3. **Compile complete configuration**
4. **Click "Start simulation"**
5. **"Load preview" dialog is displayed**
   - (1) Change target "Load Runtime" to "Full download"
   - (2) Change target "Runtime start" to "Start runtime"
   - (3) Change "Runtime values" to "Reset to start values"
   - (4) Click "Load"



6. **Check SIMATIC Runtime Manager**
   - (5) Ensure that Runtime has started (green). If not, press the "Start" button (6).
7. **Call simulation/runtime project via the web browser:**
   - (7) Enter the following in the address field of the browser: https://localhost
   - (8) Once the connection has been established, the following screen is shown.
   - (9) Click the button "WinCC Unified RT"
   - (10) Enter the following in the login dialog:
   - User: UnifiedAdmin
   - Password: Siemens123!
8. **Check if the configured start screen can be seen**



3.15 Exercise 4: Create initial screens

3.15.1 Procedure for exercise 4:

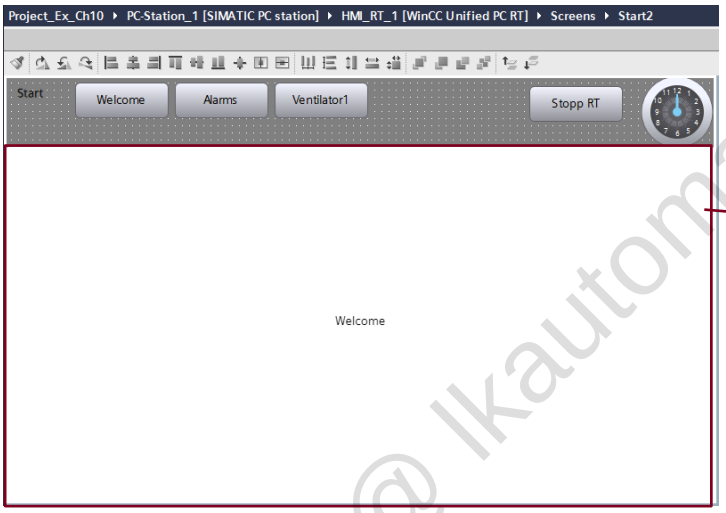
1. **Add to the "Start" screen.**
   - Insert clock
   - Insert graphic view
   - Insert button for "Stop Runtime"

2. **Create two new screens.**
   - Screens are to be named "Alarms" and "Ventilator1"
   - Output screen names in the screen at the top left with a text box

3. **Add to the screen navigation.**
   - Add two new buttons in "Start" for screen switch to the "Alarms" and "Ventilator1" screens.
   - In both the "Alarms" and "Ventilator1" screens, add a button to return to the "Start" screen.

4. **Download and test the configuration.**

3.16 Exercise 5: Work with screen windows



At the end of the exercise, the "Start2" screen should look like this.

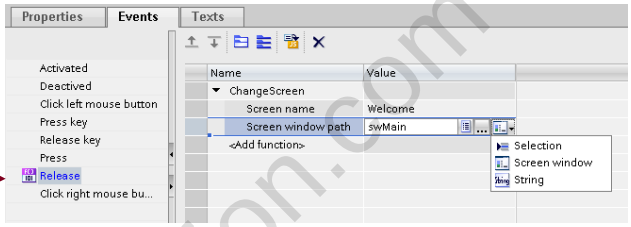
3.16.1 Procedure for exercise 5:

1. **Create a new screen "Welcome".**
   - Insert graphic view from "Start"

2. **Create an additional "Start2" screen as a copy of the "Start" screen.**

3. **Insert a screen window into screen "Start2".**
   - Delete graphic view
   - Screen for screen window: Welcome
   - Size, Position: 1920x1000, Left 0, Top 80

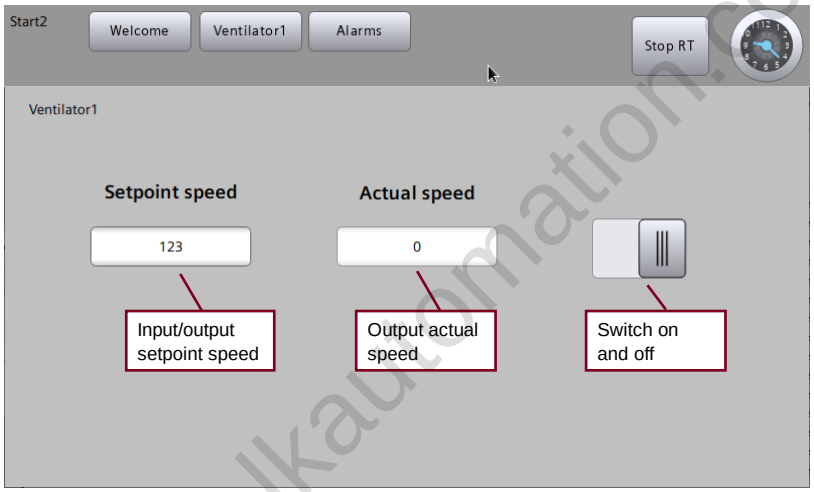
4. **Configure the screen change.**



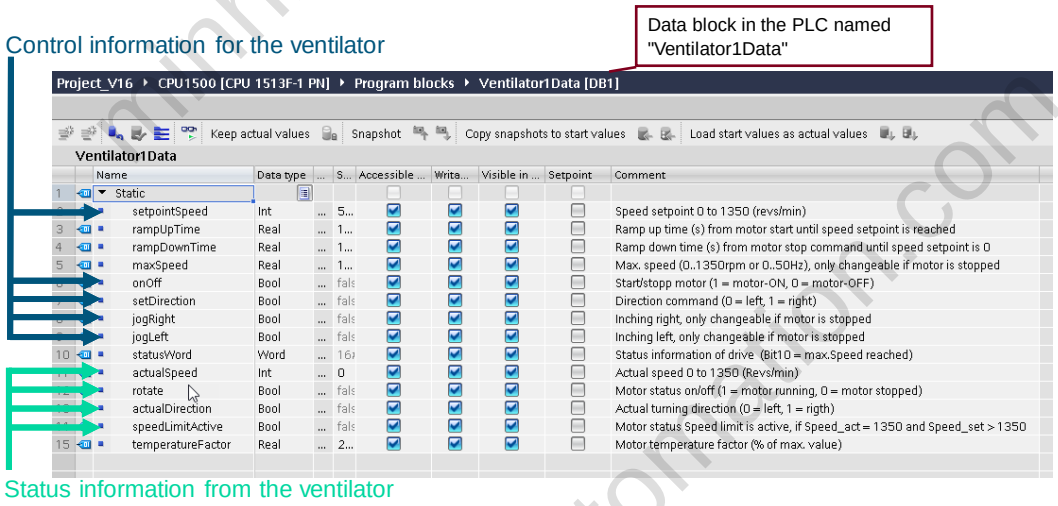
5. **Enter "Start2" as the new start screen for the HMI_RT.**

6. **Download and test the configuration.**

3.17 Exercise 6: Ventilator screen



3.17.1 Data interface



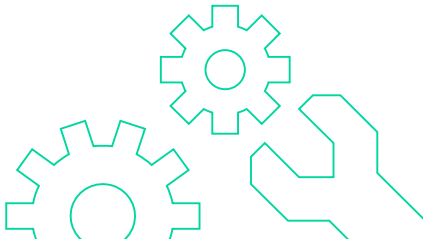
3.17.2 Procedure for exercise 6:

1. **Add two text boxes. Change the label to "Setpoint speed" and "Actual speed", respectively.**

2. **Add two IO fields. Connect these fields to the tags for "Setpoint speed" and "Actual speed" respectively.**

3. **Add a switch for switching the ventilator on and off.**

4. **Download and test the configuration on your PC. Correct any problems that may occur.**



3.18 Detailed exercise description

3.18.1 Exercise 1 (face-to-face) Add an HMI device

1. Open project

   Instruction:

- Start the TIA Portal.
  Go to the Project view.
  Menu Project > Open… -> "Browse" button -> D:\Courses\TIA-UWCCBMWOEM\Project\Project.ap18

2. Hardware download not required.
   PLC is already loaded; drive parameters have already been assigned.
   -> No activities

3. Insert HMI device
   -> Insert HMI device (WinCC Unified PC RT)
   -> Change device name to "PC system_1"

Instructions:
Project tree > Project > Add new device -> PC systems button > SIMATIC HMI application > SIMATIC WinCC Unified PC
Version 18.0.0.0

Change device name to "PC system_1"

4. Insert Ethernet interface module: "Hardware catalog -> PC systems -> Communication modules > PROFINET/Ethernet -> IE General"

5. Make language settings
   -> Check project languages: "English (United States)" and "German (Germany)"
   -> Check language setting: Editing language "English (United States)"

Instructions:
Project tree > Project > Languages and resources > Project languages

Set the editing language at the top left to "English (United States)".
Select both languages ("English (United States)" and "German (Germany)").

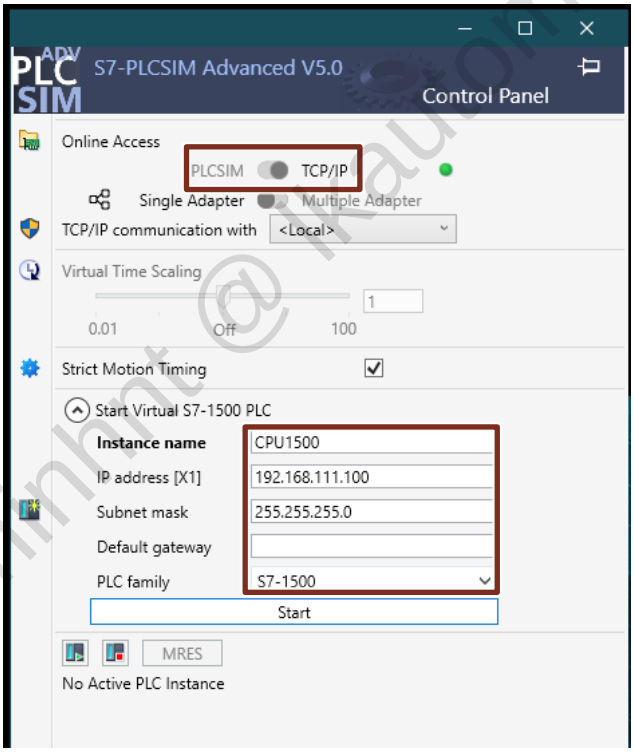
6. Save the project

3.18.2 Exercise 1 (online) Add an HMI device

1. Copy the project from the C partition and store it in the following folder:
   D:\Courses\TIA-UWCCBMWOEM\Project

2. Start PLCSIM Advanced.
   -> Windows Start menu > Siemens Automation > S7-PLCSIM Advanced V5.x

3. Set online access point to TCP/IP and configure the CPU instance:



4. Start instance.

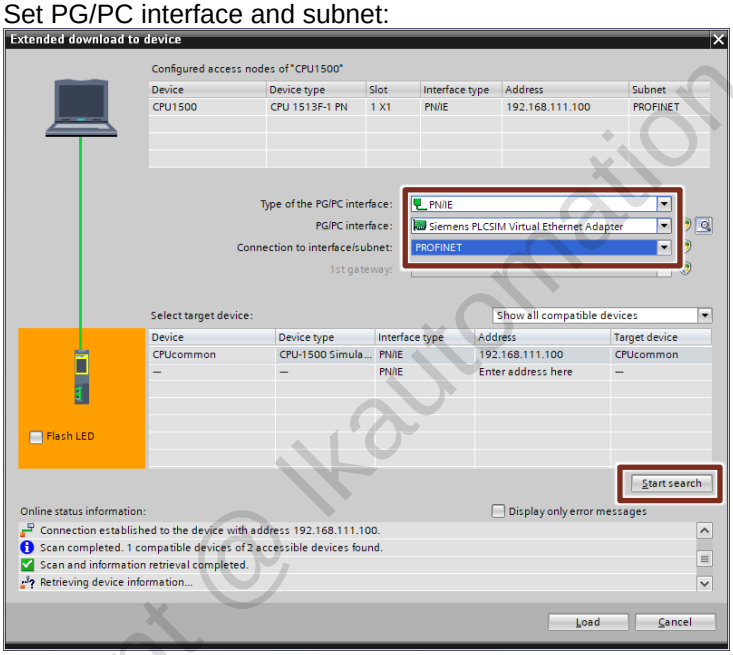
5. Open TIA project

   Instruction:

- Start the TIA Portal.
  Go to the Project view.
  Menu Project > Open… -> "Browse" button -> D:\Courses\TIA-UWCCBMWOEM\Project\Project.ap18

6. Select CPU in the project tree and download.
   Menu bar > Online > Download to device

7. Set PG/PC interface and subnet:



8. Download the configuration.

9. Insert HMI device
   -> Insert HMI device (WinCC Unified PC RT)
   -> Change device name to "PC system_1"

Instructions:
Project tree > Project > Add new device -> PC systems button > SIMATIC HMI application > SIMATIC WinCC Unified PC
Version 18.0.0.0

Change device name to "PC system_1"

10. Insert Ethernet interface module: "Hardware catalog -> PC systems -> Communication modules > PROFINET/Ethernet -> IE General"

11. Make language settings
    -> Check project languages: "English (United States)" and "German (Germany)"
    -> Check language setting: Editing language "English (United States)"

Instructions:
Project tree > Project > Languages and resources > Project languages

Set the editing language at the top left to "English (United States)".
Select both languages ("English (United States)" and "German (Germany)").

12. Save the project

3.18.3 Exercise 2 (Configure a connection)

1. Set IP address for PC
   (Device configuration: Properties of HMI device)

   Instructions:
   Project tree > Project > HMI_RT_1 > Ventilation > Open device configuration with a double-click

   First select the Ethernet interface and then open "Properties > General" in the Inspector window: PROFINET interface [X1] > Ethernet addresses: enter "192.168.111.10" with subnet mask "255.255.255.0".

2. Network IE interface with "PROFINET" network

   Instructions:
   Open "Project tree > Project > Devices & networks" with a double-click and switch to the "Network view" tab.
   Select "Network" in the toolbar of the editor.
   Using drag-and-drop, connect the Ethernet interface of the PC to the existing "PROFINET" Ethernet bus.

3. Configure the HMI connection (S7-1500 with PC Ethernet interface)
   (Create HMI connection and rename it to HMI_PCRuntime)

   Instructions:
   Stay in the "Network view" tab of the "Devices & networks" editor.
   Select "Connections" and connection type "HMI connection" in the toolbar of the editor.
   Using drag-and-drop, connect the Ethernet interface of the PC to the S7-1500.
   Result: A dashed line to the S7-1500 is displayed.
   Rename the connection to "HMI_PCRuntime".

4. Create an initial screen named "Start". Insert a text box with the text "Start screen". Enter the screen as the start screen in the Runtime settings of the PC.

   Instructions:
   Double-click "Project tree > Project > HMI_RT_1 > Screens > Add new screen".
   A new screen named "Screen_1" is created and opened.
   In the shortcut menu (open by right-clicking "Screen_1"), click "Rename" and name the screen "Start".

   Double-click "Project tree > Project > HMI_RT_1 > Runtime settings".
   In the "General" section, you can select the start screen for runtime under "Screen". Select the new "Start" screen there.

5. Disable encrypted transfer in Runtime settings

6. Save the project

3.18.4 Exercise 3 (Download configuration to PC station and open it)

1. Configure PG/PC interface
   - Open Windows Control Panel
   - Select "PG/PC Interface"
   - Select "Siemens PLCSIM Virtual Ethernet Adapter.TCPIP.1" as communication interface

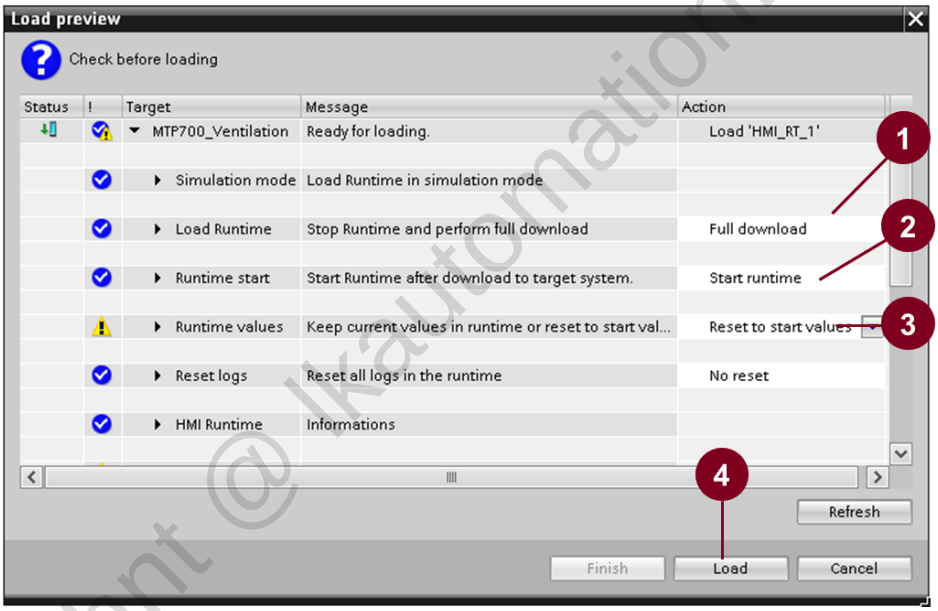
2. Create user
   - Open "Security settings > Users and roles" in the project tree
   - Add new local user:
     - User name: "UnifiedAdmin"
     - Password: "Siemens123!"
   - Under "Assigned roles", select the "HMI Administrator" check box

3. Compile complete configuration
   - Right-click the Unified Comfort Panel in the project tree
   - Select "Compile > Software (rebuild all)" in the shortcut menu
   - Wait for compilation to complete
   - The compilation result must not contain any errors

4. Download configuration
   - Select PC in the project tree
   - Click "Download to device"

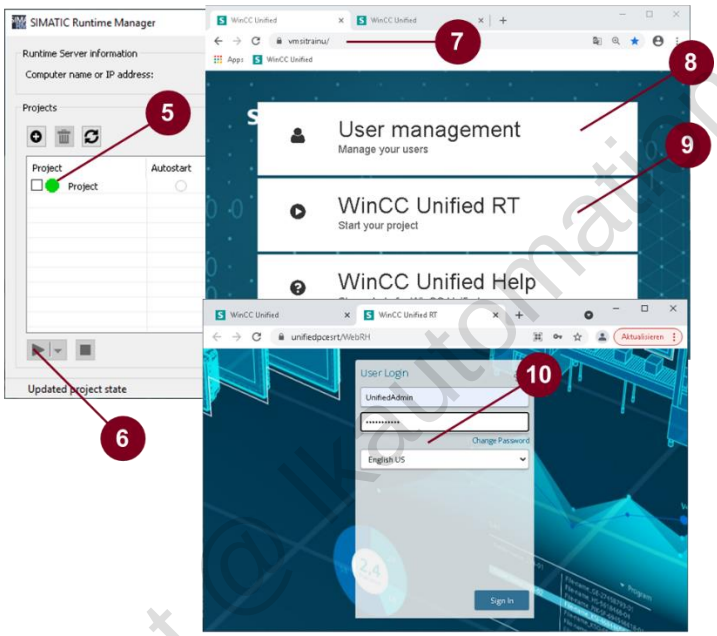
5. At the first download, the "Extended download to device" dialog is displayed
   - (1) Ensure that "Ethernet" is selected
   - (2) Select "Configured IP address"
   - (3) Click "Connect" button
   - (4) Click "Load" button

6. The "Load preview" dialog is displayed
   - (1) Change target "Load Runtime" to "Full download".
   - (2) Change target "Runtime start" to "Start runtime".
   - (3) Change target "Runtime values" to "Reset to start values".
   - (4) Click "Load".



7. Check SIMATIC Runtime Manager
   - Open the Windows search with the keyboard shortcut "Windows + Q"
   - Enter "simatic runtime manager" and confirm with "Enter"; → SIMATIC Runtime Manager is started
   - Check whether the downloaded project is displayed; if not, click the "Update list of available projects" button to update the display
   - Check if the status is "Running" (5); if not, select the project and click the "Start runtime with selected project" button (6)
   - Check whether the type is "Project"; if not, download the project once again from the TIA Portal.

8. Call the runtime project via the web browser:
   - (7) Enter the following in the address field of the browser: https://localhost
   - (8) Once the connection has been established the following screen is displayed.
   - (9) Click the button "WinCC Unified RT".
   - (10) Enter the following in the login dialog:
   - User: UnifiedAdmin
   - Password: Siemens123!



9. Check if the configured start screen can be seen

3.18.5 Exercise 4 (Create screens)

1. Add to the "Start" screen
→ Insert clock
→ Insert graphic view
→ Insert button for "Stop Runtime"

Instructions:
Project tree > Project > HMI_RT_1 > Screens:
Open the "Start" screen by double-clicking.

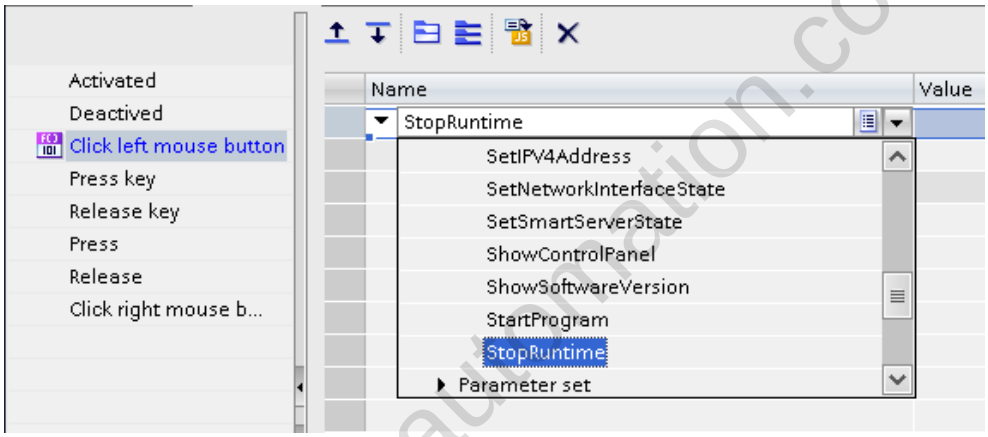
Insert clock
Taskbar > Toolbox > Open "Elements" group
Move the "Clock" object into the screen using drag-and-drop.
Select the clock in the screen and then go to "Properties > Properties" in the Inspector window.
Adapt the following properties:
"Size and position > Width": 80
"Size and position > Height": 80
"General > Clock face font > Font": Click the "…" button, then set the font size to 8
"Appearance > Label color clock face": Set color to "white"
Move the clock to the upper right corner using the mouse

Insert graphic view
Taskbar > Toolbox > "Basic objects" group
Move the "Graphic view" object into the screen using drag-and-drop.
Select the object in the screen and then go to "Properties > Properties" in the Inspector window.
Adapt the following properties:
"General > Graphic": insert new graphic:
Path "D:\Courses\TIA-UWCCBMWOEM_Graphics\SITRAIN.png" (face-to-face training)
Path "C:\Courses\TIA-UWCCBMWOEM_Graphics\SITRAIN.png" (online training)

Insert button for "Stop Runtime"
Taskbar > Toolbox > Open "Elements" group
Move the "Button" object into the screen using drag-and-drop.
Select the object in the screen and then go to "Properties > Properties" in the Inspector window.
Adapt the following properties:
"General > Text": "Stop RT"

Go to "Properties > Events" in the Inspector window.

Select the "StopRuntime" function for the "Click left mouse button" event.



2. Create two new screens
→ Screens are to be named "Alarms" and "Ventilator1"
→ Display the screen name in the upper left corner of the respective screen with a text box.

Instructions:
Project tree > Project > HMI_RT_1 > Screens:
Create two new screens by double-clicking on "Add new screen".

Select the screen names and rename them to "Alarms" and "Ventilator1" with F2.

3. Add screen navigation
→ Add two new buttons in the "Start" screen for changing to the "Alarms" and "Ventilator1" screens
→ Add a button in each of the "Alarms" and "Ventilator1" screens for switching back to the "Start" screen

Instructions:
Open the "Start" screen. Then use drag-and-drop to move the "Alarms" and "Ventilator1" screens from the project tree to the screen. Two new buttons with screen change function are then created.

Open the "Alarms" screen. Then use drag-and-drop to move the "Start" screen from the project tree to the screen.
Open the "Ventilator1" screen. Then use drag-and-drop to move the "Start" screen from the project tree to the screen.

This enables you to change back from these screens to the Start screen later in runtime.

4. Download and test the configuration

3.18.6 Exercise 5 (Screen window)

1. Create a new "Welcome" screen.

Instructions:
Create another screen and rename it to "Welcome".
Copy the graphic view from the "Start" screen and insert it in the "Welcome" screen.

2. Create an additional "Start2" screen as a copy of the "Start" screen.

Instructions:
Copy the "Start" screen.
Insert it under Screens.
Rename the copied screen to "Start2".

3. Insert a screen window into the "Start2" screen

Instructions:
Delete the graphic view in the "Start2" screen.

Then insert a screen window object:
Taskbar > Toolbox > Open "Controls" group
Use drag-and-drop to move the "Screen window" object into the screen.

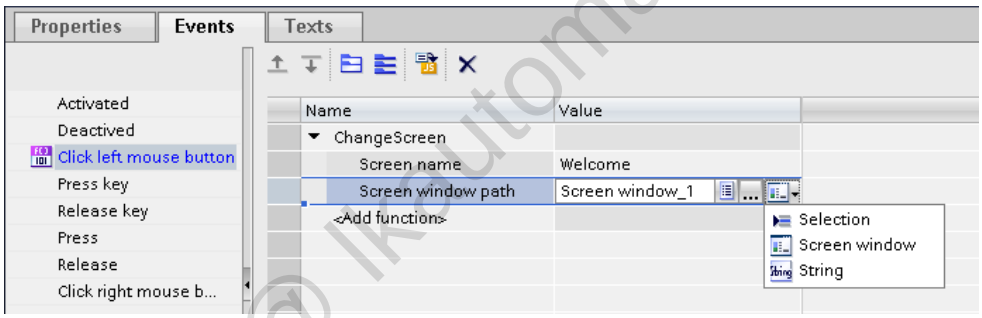
Select the object in the screen and then go to "Properties > Properties" in the Inspector window.
Adapt the following properties:
"General > Screen": "Welcome"
"Appearance > Window settings": None
"Size and position": "Width": 1920, "Height": 1000; "Left": 0; "Top": 80

4. Configure a screen change in the screen window

Instructions:
Add another button labeled "Welcome".

Go to "Properties > Events" in the Inspector window.

Call the following function when the "Click left mouse button" event occurs:



When the "Welcome" button is clicked in runtime, the "Welcome" screen is to be opened in the screen window.

Adapt the events for the "Ventilator1" and "Alarms" buttons accordingly.

5. Enter "Start2" as the new start screen for the PC.

Instructions:
Project tree > Project > PCRuntime_1 > Runtime settings:
"General > Start screen": Select "Start2"

6. Download and test the configuration.
What do you notice in runtime?

7. Change the background color of the "Start2" screen to a darker gray tone. This will improve the display of the screen division in runtime.

8. Adapt the size of all the screens displayed in the screen window so that scroll bars are no longer displayed.

9. Delete the "Start" button from the "Alarms" and "Ventilator1" screens.

3.18.7 Exercise 6 (Ventilator screen)

1. Open the "Ventilator1" screen. Add two text boxes. Change the label to "Setpoint speed" and "Actual speed".

2. Add two IO fields. Connect these to the tags for "Setpoint speed" and "Actual speed".

To do this, open the corresponding data block in the Details view:
Project tree > Project > CPU1500 > Program blocks: Select the "Ventilator1Data" data block and expand the Details view at the bottom left. Then use drag-and-drop to move the "setpointSpeed" and "actualSpeed" tags to the screen.

3. Add a switch for switching the ventilator on and off.

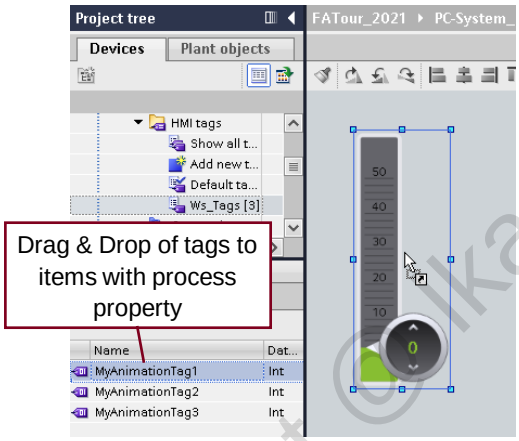
Taskbar > Toolbox > Open "Elements" group
Move the "Switch" object into the screen using drag-and-drop.

Select the object in the screen and then go to "Properties > Properties" in the Inspector window. Adapt the following properties:
"General > Switch state": Set "Dynamization" column to "Tag". Select the "onOff" tag from the "Ventilator1Data" data block as "Tag".

4. Download and test the configuration on your PC

4 Screen engineering

4.1 Drag & Drop



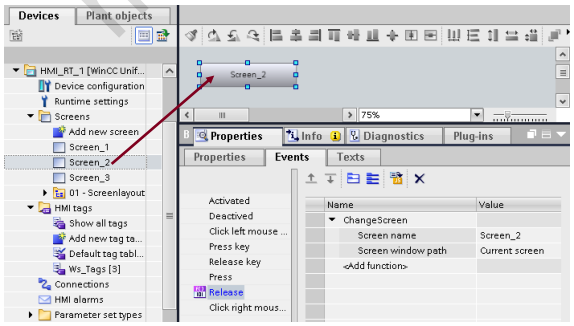
Drag & Drop Screenevents
- to existing Buttons

Drag & Drop tags to object
- To Gauge, Slider, Bar, Radio button, …

Multi Drag & Drop of tags
- Create multiple IO fields and trends

Drag & Drop graphics to objects
- To Buttons & Graphic view

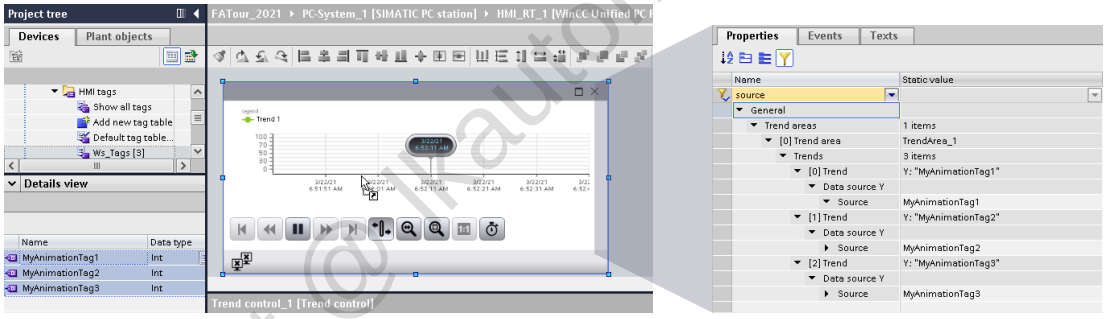
4.1.1 Examples



Drag & Drop of:
- Screen creates a new button with Screen Change event

- Tags to screens creates IO fields connected to the tag (as much io fields as tags)
- Tags to items with process property

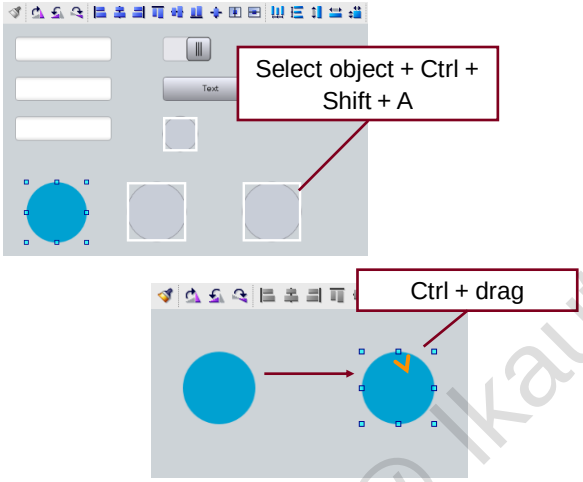
- Text list: creates a text field with resource list
- Graphic list: creates a graphic view with resource list
- Graphic: creates graphic view (Tip: Graphics can be also dragged from windows explorer)



Drag & Drop tags to a trend control link the tags with the data source property. Logging tags are not supported.

as from V17

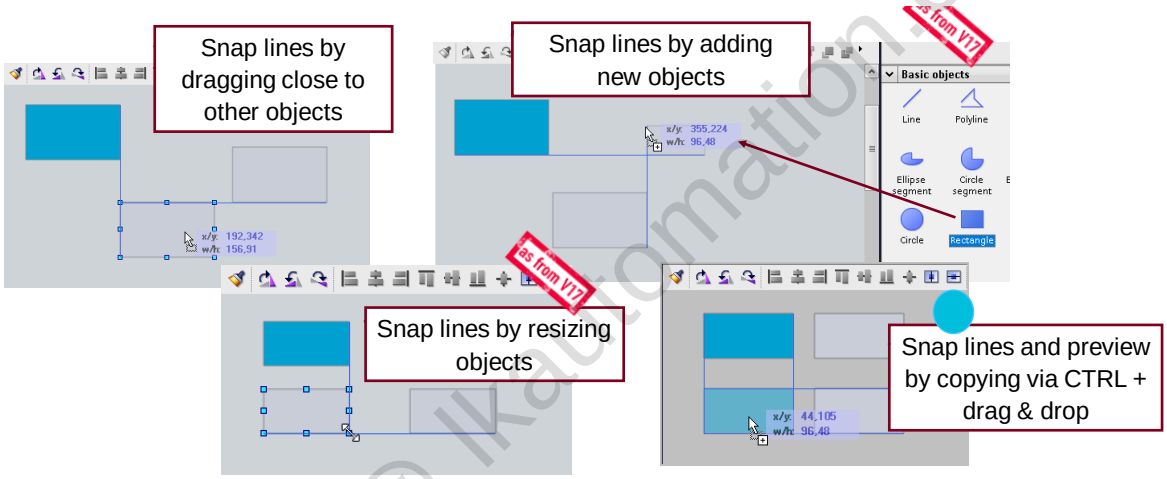
4.2 Keyboard shortcuts



Keyboard shortcuts

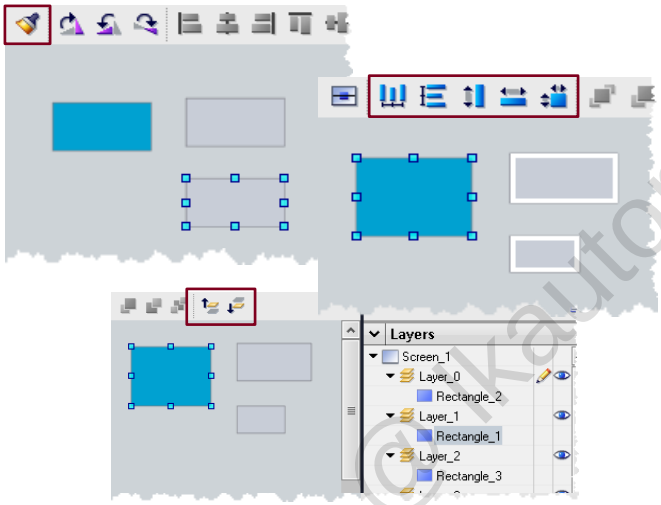
- **Ctrl + A:** select all objects
- **Ctrl + Shift + A:** select all objects from same type
- **Ctrl + drag + release:** creates a copy of the object
- **Ctrl + C, Ctrl + V:** creates a copy of the object repeated execution of Ctrl + V pastes the object depending on offset position of last pasted element

4.3 Snap lines



Snap lines in screen editor help adjusting objects to each other

4.4 Toolbar enhancements



Transfer Format *as from V17*

Copy appearance properties in between the same or different basic objects, elements and controls. For objects and properties supported, check TIA Portal help under the chapter "Configuring screens – Basics – Transfer format".

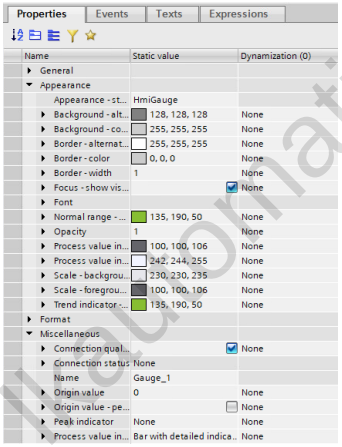
Object handling *as from V17*

- Set selected objects to the same height and/or width
- Distribute objects vertically/horizontally

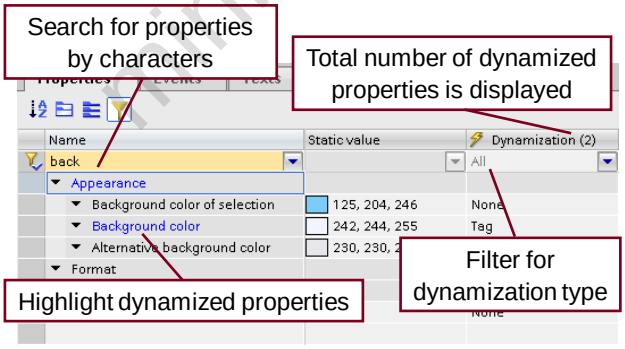
Layer handling

Move objects one layer up or down

4.5 Properties



It is possible to view and to edit the properties of an object using the property list in the Inspector window.

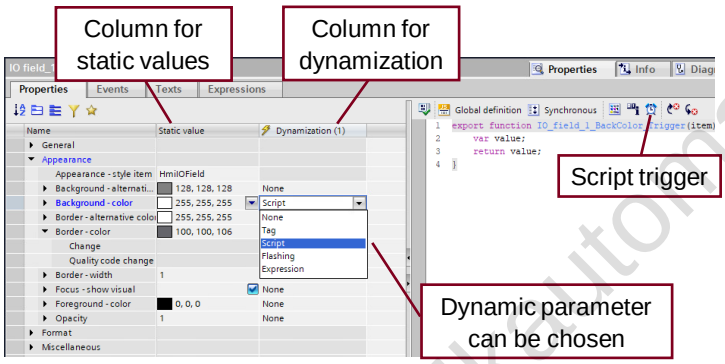


The property list can be sorted as follows:

- Display of properties in alphabetical order
- Display of the properties grouped in categories
- Search Filter:
  - Search for all properties by characters
  - Overview of used dynamizations
  - Filter of different dynamizations

Tip: most important properties of an object can always be found in the "General" category (list needs to be sorted by category)

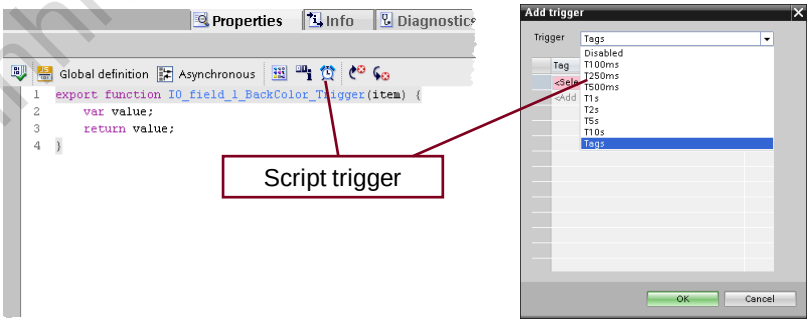
4.6 Dynamization



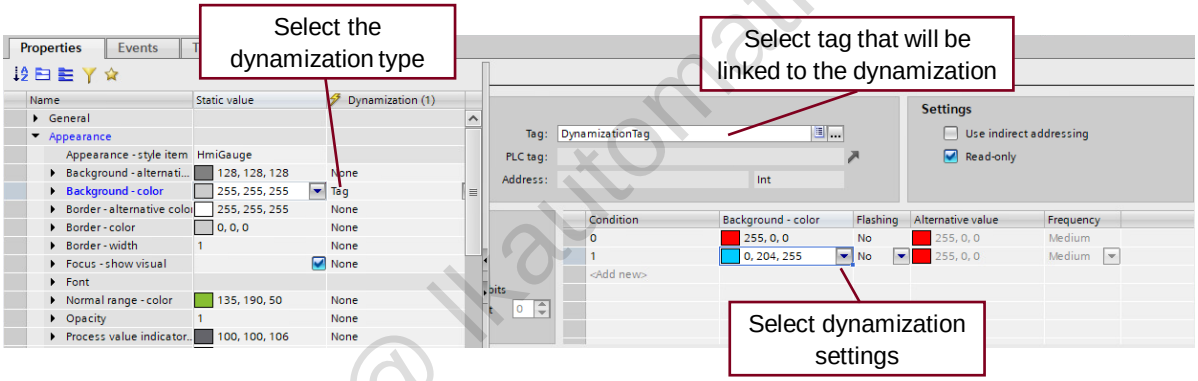
Dynamization types:

- Tag – Defines the property value depending on the tag value
- Script – Defines the property value depending on the return value
- Resource list – Defines the property value depending on an entry from a text/graphic list
- Flashing – Defines that the property flashes in configurable colors

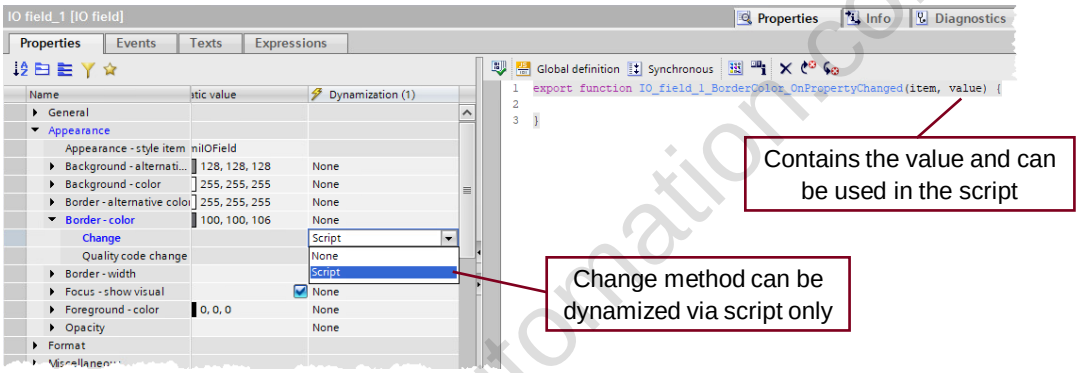
Almost all object properties can be dynamized.
Tags, Scripts and in some cases resource list, can be used for dynamization.



You can select cyclic or tag trigger.

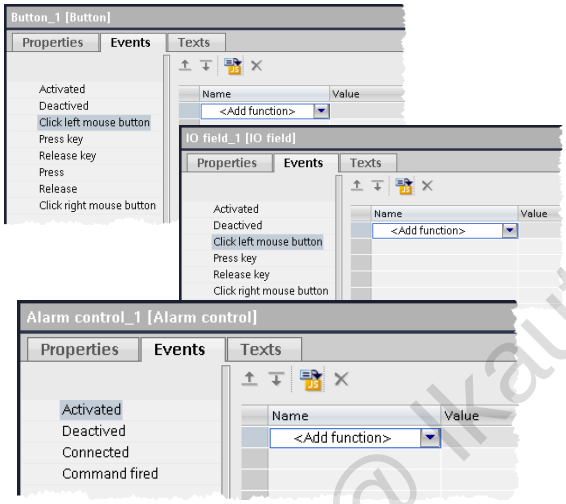


The object property is dynamized with a tag. The tag value specifies the property value in runtime, they can be copied & pasted for further screen objects.



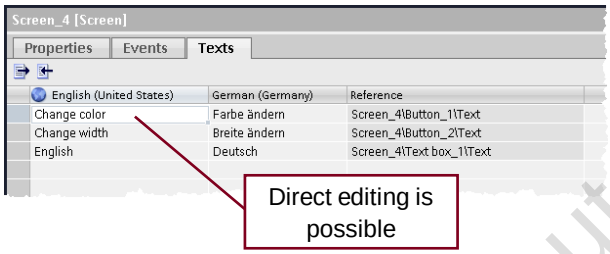
Most of the object properties include change methods that will be executed by change of the property value.

4.7 Events



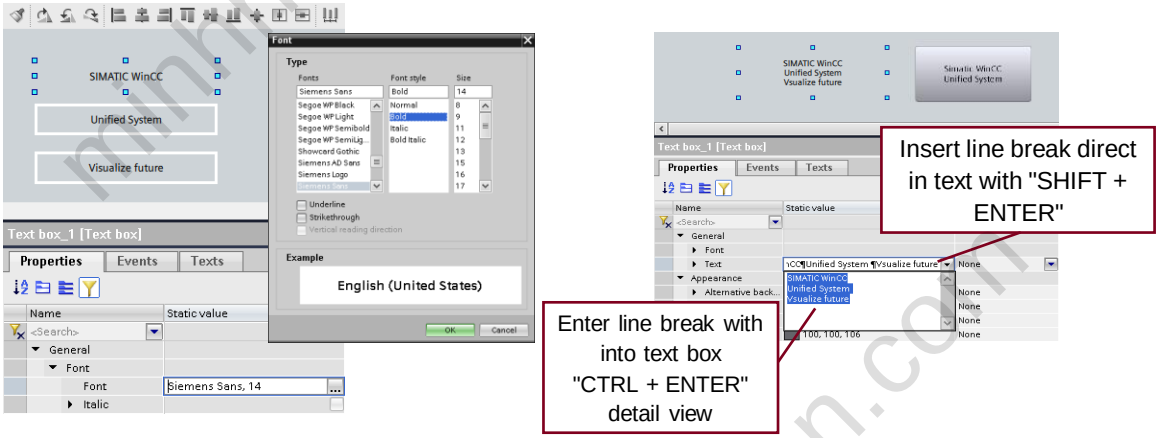
- The available events depend on the object which is selected
- System functions or scripts can be triggered at events
- Multiple system functions can be configured in a function list
- Conversion of a function list to a script is possible

4.8 Texts



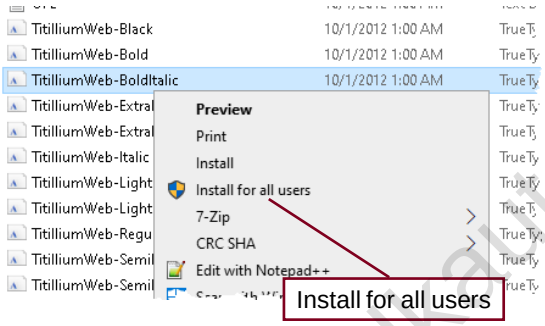
- The "Texts" tab displays all language references in a screen or depending on the selected object!
- All texts in all configured languages will be displayed
- Direct editing, copy&paste actions are possible

Tip: use the "Texts" tab to edit texts of objects instead of using the property list can save time as you do not need to search for the property



Change the font of multiple "text" objects or insert line breaks.

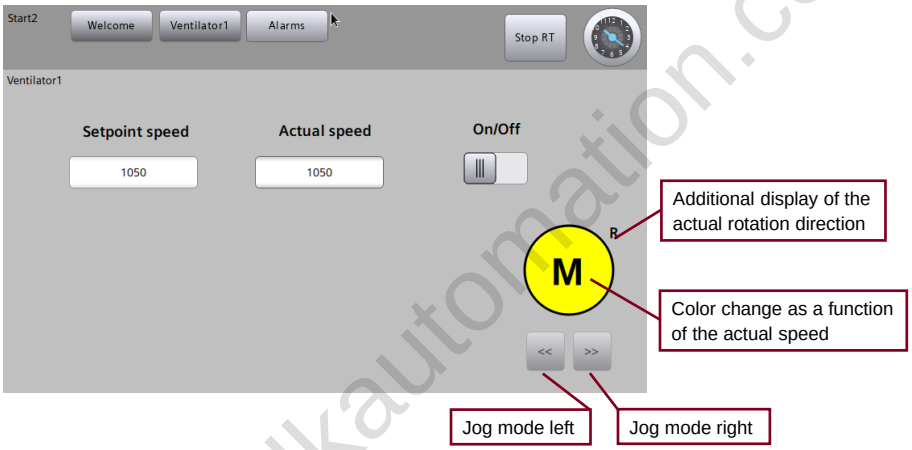
4.9 Custom fonts



- Only fonts that based on **TrueType-Fonts (TTF)** data can be used in TIA Portal.
- PostScript fonts and open type fonts that contain PostScript data are not supported.

- It is important to install the font for all Users
- After the installation of a font a restart of TIA Portal is necessary

4.10 Exercise 1: Dynamize the ventilator screen



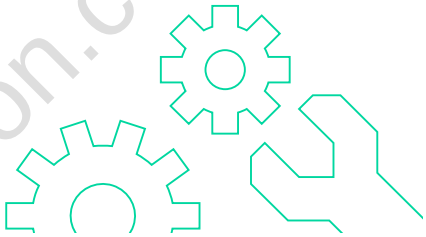
4.10.1 Procedure for exercise 1:

1. **Add two buttons and label them with "<<" and ">>", respectively. Configure jog mode for counterclockwise and clockwise rotation.**
   - Set "JogLeft" tag to 1 with "Press" event and set it back to 0 with "Release" event

2. **Add a motor symbol consisting of a circle and a text box.**

3. **Dynamize the background color of the circle as a function of the actual speed.**
   - 0 = gray; 1-1000 = white; 1001-1200 = yellow and 1201-1400 = red

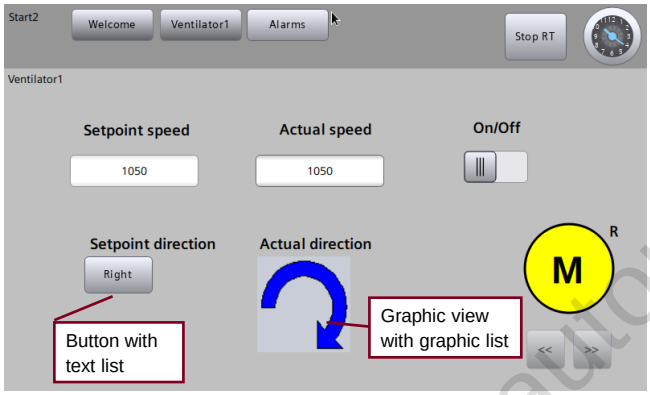
4. **Add two text boxes "L" and "R".**
   Display these boxes as a function of the actual rotation direction.



5. **Optional exercise: The buttons for jog mode are to be inoperable when the motor is switched on.**

6. **Download and test the configuration.**

4.11 Exercise 2: Add to the ventilator screen



- Change of rotation direction is to be possible
- Current rotation direction is displayed with a graphic

4.11.1 Procedure for exercise 2:

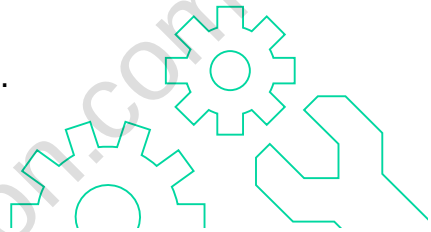
1. **Create a new text list named "LeftRight".**
   - "0" = Left, "1" = Right

2. **Create a new graphic list named "LeftRight".**

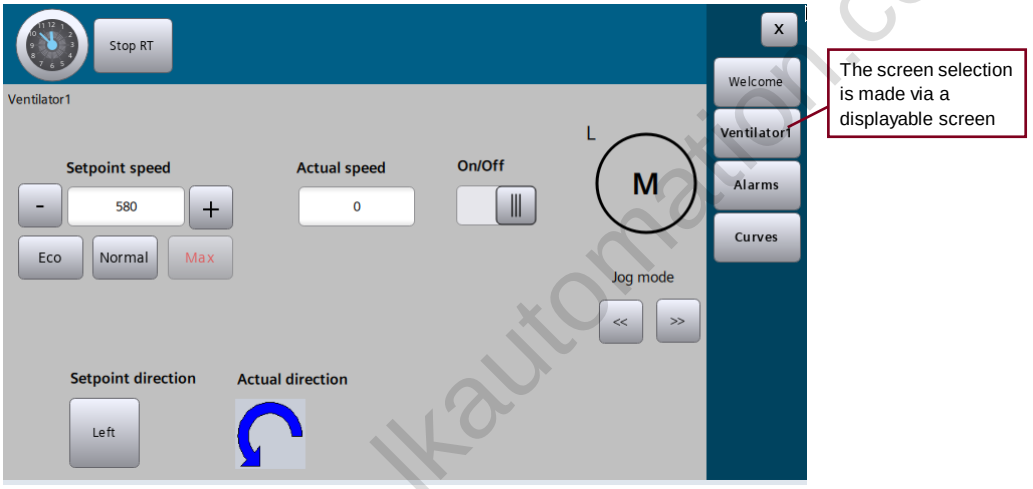
3. **Add a button for changing the setpoint rotation direction of the ventilator.**
   - Event: InvertBitInTag
   - Link text with "LeftRight" text list

4. **Add a graphic view and use it to display the actual rotation direction.**
   - Link graphic with "LeftRight" graphic list

5. **Download and test the configuration.**

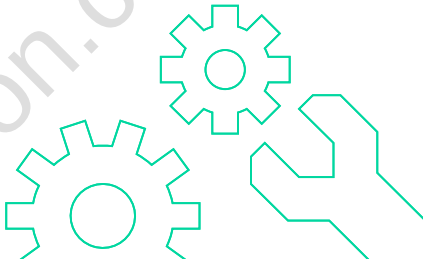


4.12 Exercise 3: Screen navigation via hamburger menu



4.12.1 Procedure for exercise 3:

1. **"Start2" screen: Rename object name from "Screen window_1" to "swMain" (screen window Main).**
2. **Copy the "Start2" screen and paste it again. Rename the copied screen to "Start3". Define "Start3" as the new start screen.**
3. **Adapt the "Start3" screen (see detailed description).**
4. **Insert screen window "swNavigation".**
5. **Show menu.**
6. **Create a new "Menu" screen.**
7. **Download and test the configuration.**

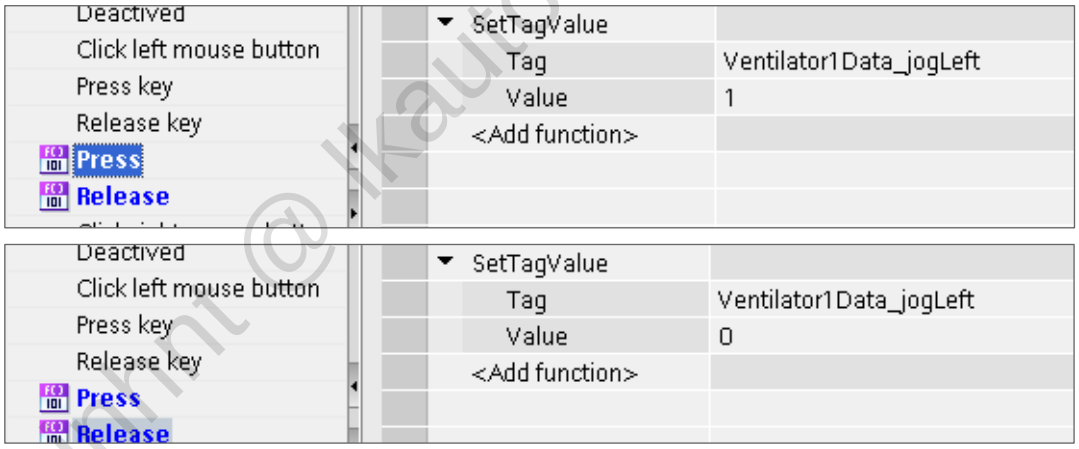


4.13 Detailed exercise description

4.13.1 Exercise 1 (Dynamize objects)

1. Add two buttons and label them with "<<" and ">>". Configure jog mode for counterclockwise and clockwise rotation.

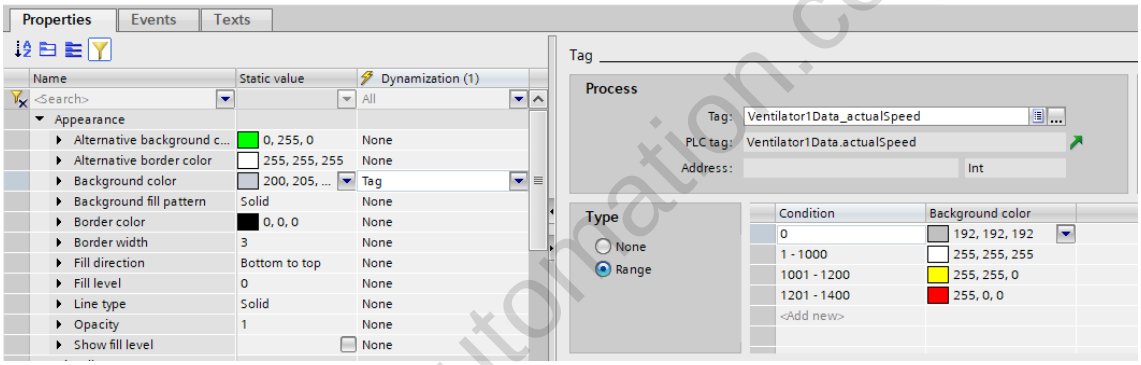
With "Press", call the "SetTagValue" function with Value = "1"; with "Release", call the "SetTagValue" function with value = "0".



Proceed accordingly for the second button with the tag "DB_Ventilator1_JogRight".

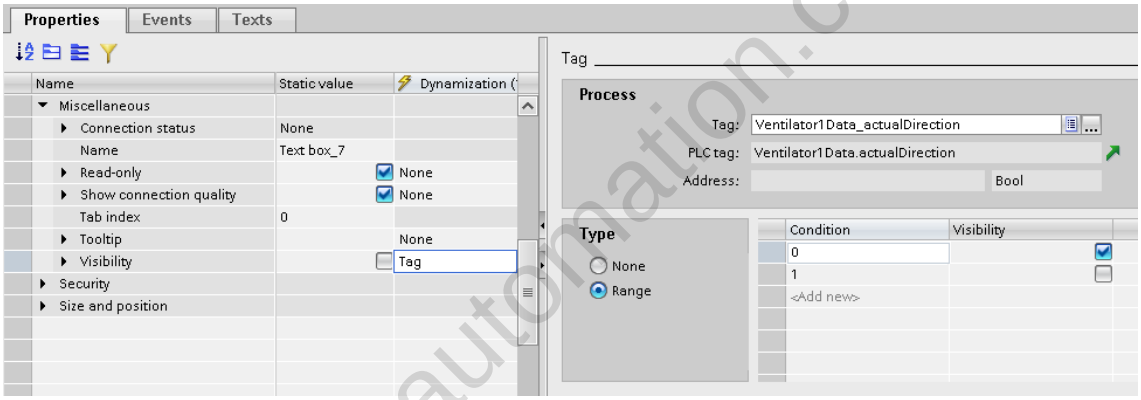
2. Add a motor symbol consisting of a circle and a text box.

3. Dynamize the background color of the circle as a function of the actual speed.



4. Add two text boxes "L" and "R". Display these as a function of the actual rotation direction.

For "L", the dynamization looks like this:



For "R", the values are interchanged.

5. Optional exercise:
The buttons for jog mode are to be inoperable as long as the motor is switched on.

6. Download and test the configuration on your PC.

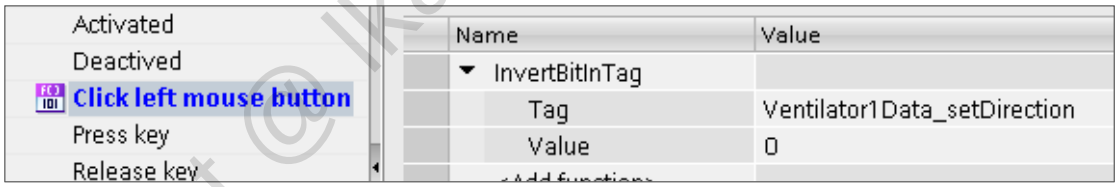
7. Reduce the update times for the tags in the screen to 500 ms, for example.

4.13.2 Exercise 2 (Text and graphic lists) (optional)

1. Create a new text list named "LeftRight".
For value "0", configure the text "Left"; for value "1", configure the text "Right".

2. Create a new graphic list named "LeftRight". In the Selection column, select "Bit (0, 1)".
For value "0", configure a graphic with a counterclockwise arrow, for value "1" configure a graphic with a clockwise arrow. The corresponding graphic files can be found in the folder "D:\Courses\TIA-UWCCBMWOEM_Graphics": "ArrowLeft.gif" and "ArrowRight.gif"

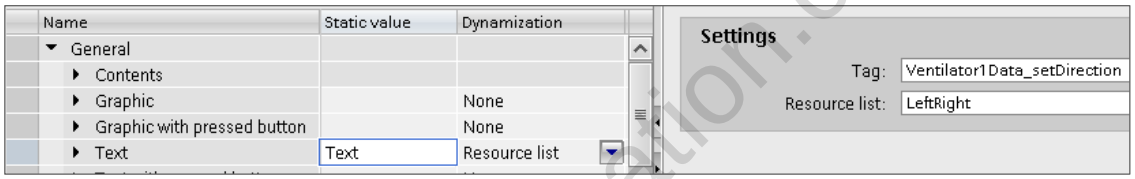
3. Add a **button** in the "Ventilator1" screen. Add a text box with the text "Setpoint direction" above the button. The purpose of the button is to allow the setpoint rotation direction of the ventilator to be changed. The "InvertBitInTag" system function can be used to invert a value with a button.



The text on the button is to be dependent on the setpoint rotation direction. Therefore, link the "Text" property with the "LeftRight" text list.

Select the button in the screen and then go to "Properties > Properties" in the Inspector window.
Adapt the following properties:
"General > Text": Set the "Dynamization" column to "Resource list"

Select tag and resource list as follows:



4. Add a text box with text "Actual direction".

5. Add a graphic view and use it to display the actual rotation direction.

Taskbar > Toolbox > Open "Basic objects" group
Insert the "Graphic view" object in the screen.

Select the object in the screen and then go to "Properties > Properties" in the Inspector window.
Adapt the following properties:
"General > Graphic": Set the "Dynamization" column to "Resource list" and select the "LeftRight" graphic list. Select "Ventilator1Data_actualDirection" as tag.

6. Download and test the configuration on your PC.

4.13.3 Exercise 3 (Hamburger menu)

1. "Start2" screen: Rename object name from "Screen window_1" to "swMain" (screen window Main).

Open the object properties:
"Miscellaneous > Name": Change to "swMain"

Check whether the screen change still works on the HMI_RT.
If so, why does it still work even though the screen window name has been changed?

2. Copy the "Start2" screen and paste it again. Rename the copied screen to "Start3". Define "Start3" as the new start screen.

3. Adapt the "Start3" screen.

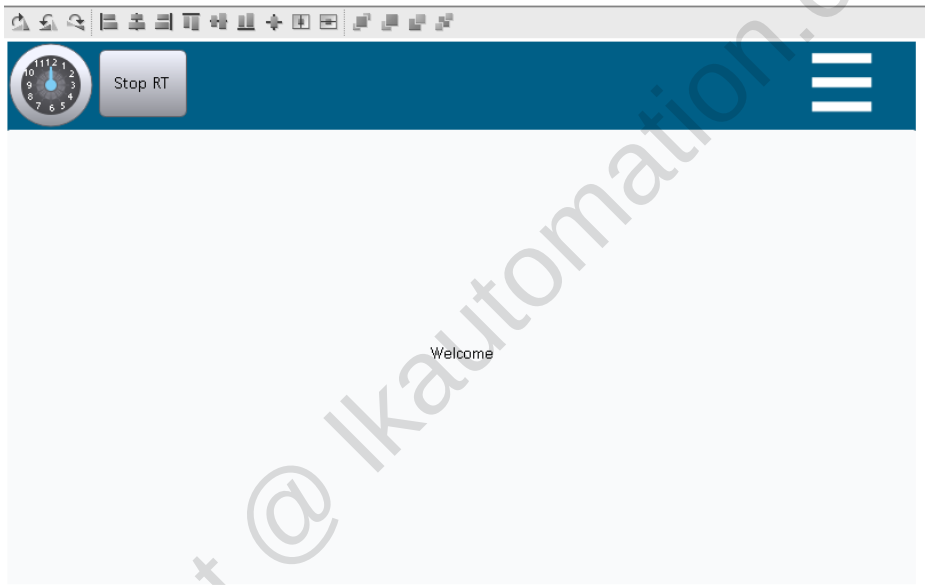
Delete the three buttons for screen selection. Move the clock and the "Stop RT" button to the left.

Properties of screen:
"Appearance > Background color": Blue (red component: 0, green component: 95, blue component: 135)

Add a graphic view for the hamburger icon.

Properties of graphic view with 3 bars ("hamburger"):
"General > Graphic": "iconMainNavigation" (graphic already exists in the project)
"Appearance > Background color": Blue (red component: 0, green component: 95, blue component: 135)

Result:



4. Insert screen window "swNavigation".

Insert another screen window and name the object "swNavigation" ("Miscellaneous > Name").

Change the window settings to "None" (Appearance > Window properties).

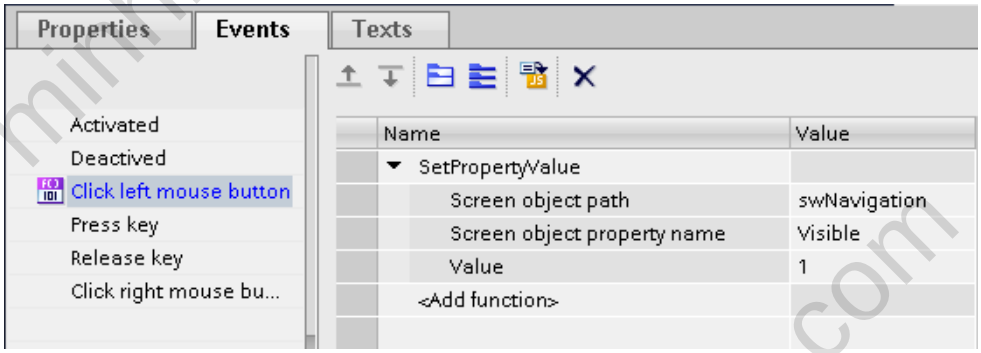
Additional properties:
"Size and position": Width 240, Height 1080, Left 1680, Top 0

The screen window is to be invisible when the screen opens:
Disable "Properties > Miscellaneous > Visibility".

Taskbar > Layout > Layers: Move the "swNavigation" object from "Layer 0" to "Layer 1".
Then hide Layer 1 by clicking on the "eye" symbol.

5. Display hamburger menu.

Select the graphic view "Hamburger" and call the "SetPropertyValue" function with the "Click left mouse button" event:

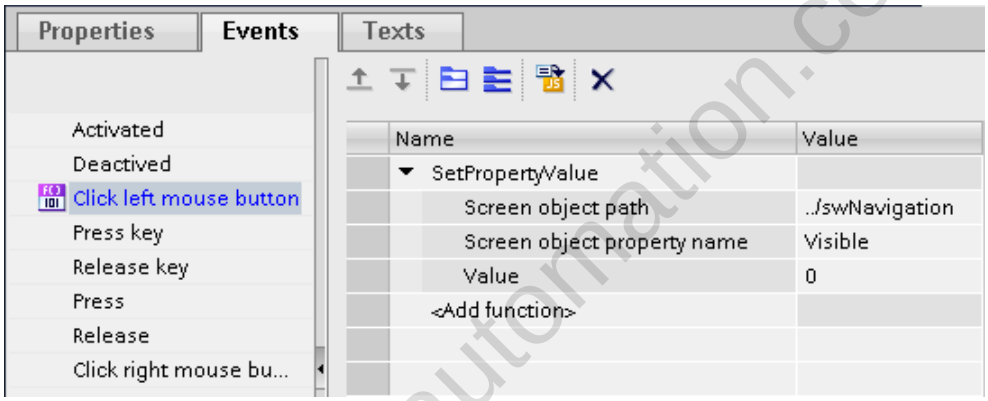


6. Create a new "Menu" screen.

The size is to correspond to the size of the screen window:
"Size and position": Width 240, Height 1080

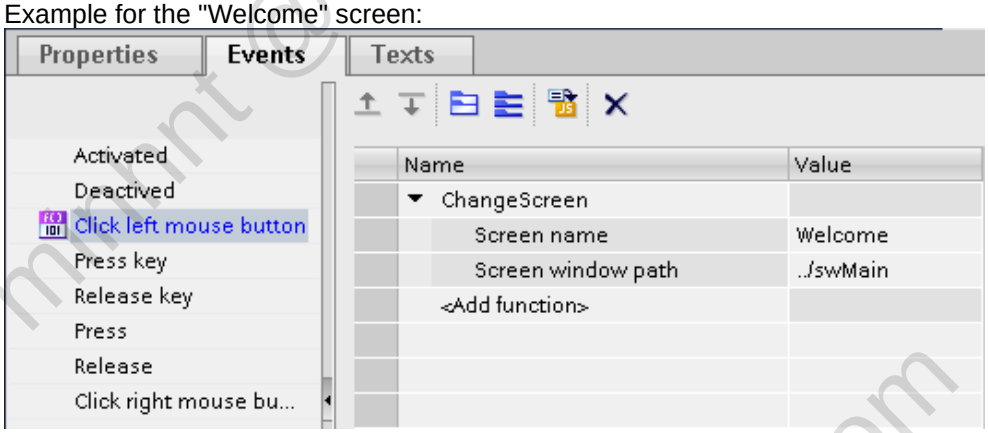
Add 3 buttons labeled: "Welcome", "Ventilator1" and "Alarms" and a small button with an "X".

The "SetPropertyValue" function is called with "X":



Add three buttons labeled: "Welcome", "Ventilator1" and "Alarms". Configure the "ChangeScreen" function in each case for the screen change in the "swMain" screen window.

Example for the "Welcome" screen:



Properties of screen:
"Appearance > Background color": Blue (red component: 0, green component: 67, blue component: 95)

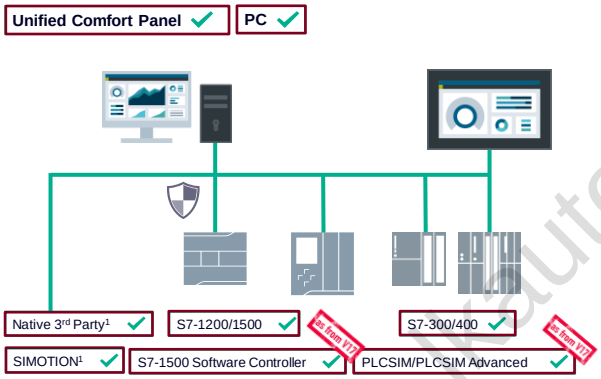
Change to the "Start3" screen and enter the "Menu" screen there for the "swNavigation" screen window:
"Properties > General > Screen": "Menu"

The screen window is to be invisible by default.

7. Download and test the configuration.

5 Communication

5.1 Overview - Connection to automation systems

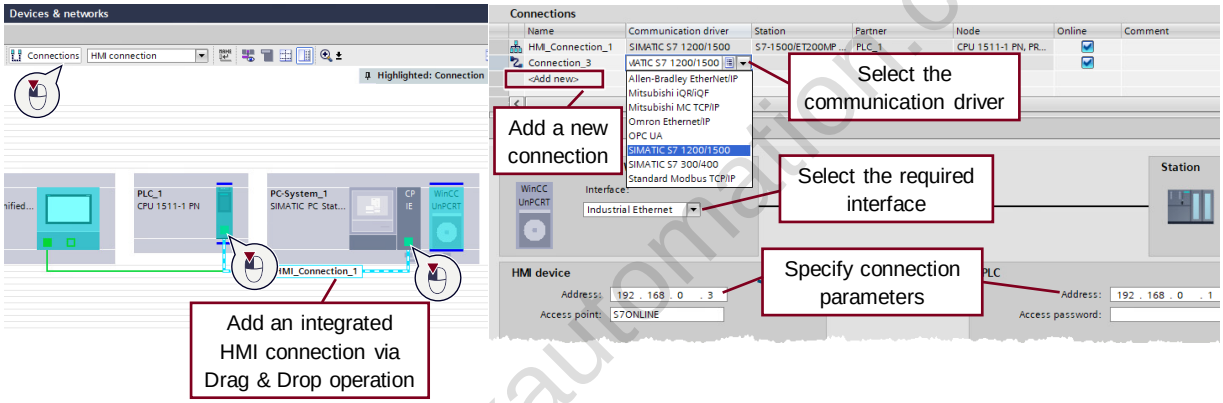


[1] Allen Bradley EtherNet/IP, Mitsubishi iQR/iQF/MC TCP/IP, Omron Ethernet/IP, Communication driver Standard Modbus TCP/IP, OPC UA are integrated

→ Other Native 3rd Party[1] communication via Channel Support Package (CSP) (Entry-ID: 109779920)

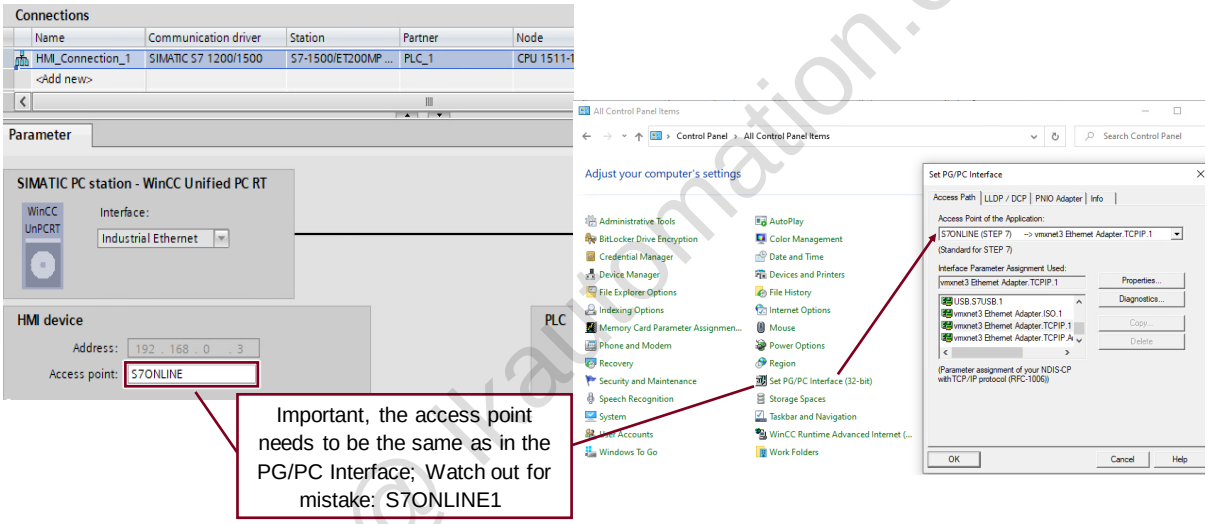
- **Perfect integration** of SIMATIC PLCs and Software PLCs (TIA Portal)
- **High number of connections** for PC systems, up to 128 PLCs (>10 with SIMATIC NET Softnet-IE license)
- **Increased number of connections** for Panel systems, up to 16 PLCs
- **Full integration in simulation** with PLCSIM and PLCSIM Advanced

5.2 “Devices & networks” or Editor “Connections”

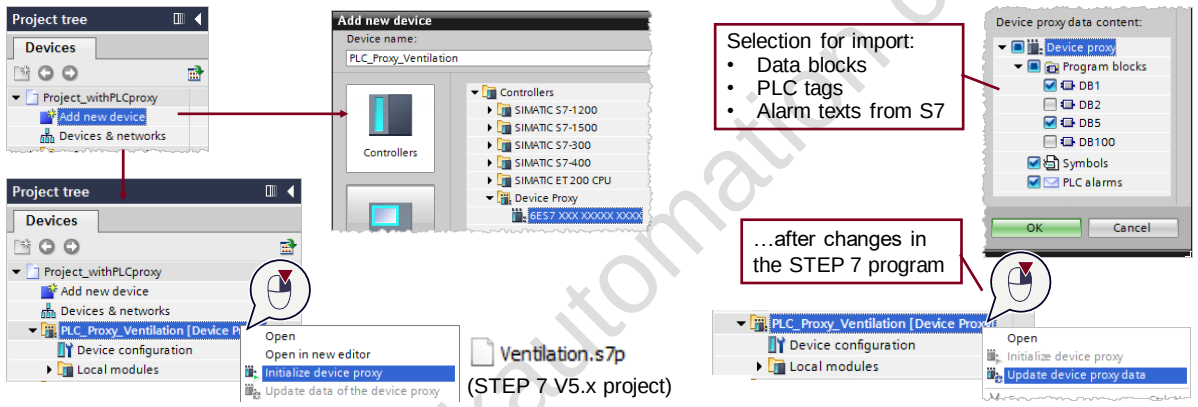


Editor “Connections” to manage and create manually integrated and non-integrated connections.

5.3 Settings and PG/PC interface in Control Panel



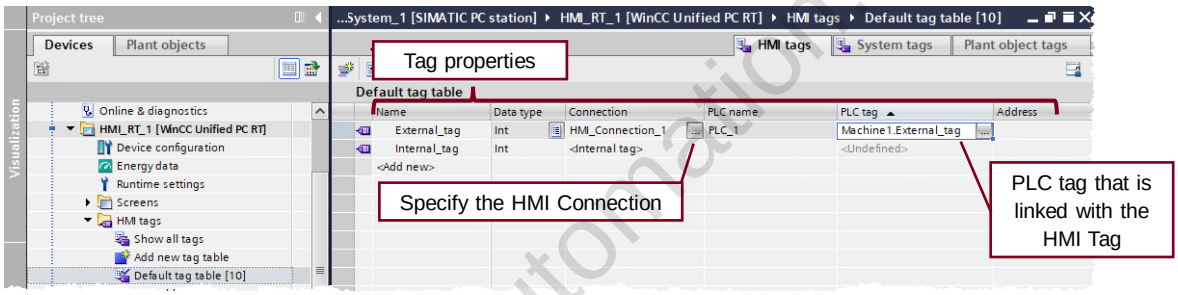
5.4 Using Device Proxy



With the "Device Proxy" you can include PLCs from STEP 7 V5.x projects and use the advantages of TIA Portal.

5.5 Tag handling

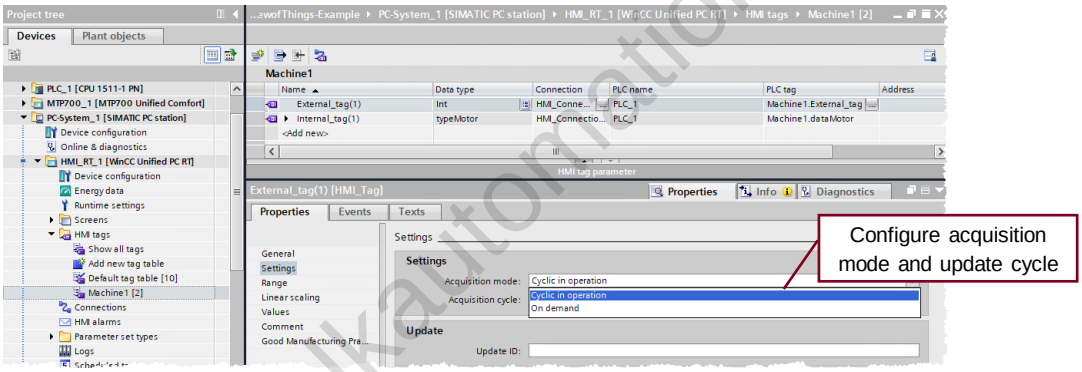
5.5.1 Editor “HMI tags”



External HMI tags will be created via SiVArc. Internal HMI tags are created in an HMI tag table.

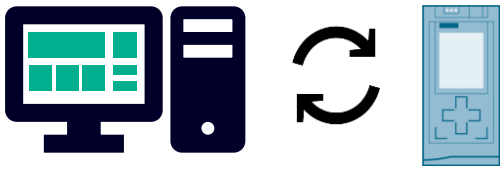
5.5.2 External Tags

5.5.2.1 Acquisition mode/cycle



Acquisition mode and update cycle is directly configured at the properties of a tag.
External tags can be configured, internal tags always “Cycle in operation”.

5.5.2.2 Addressing



Options for addressing a PLC tag:

- **Symbolic addressing** – a validity check of the tag connection is performed in runtime. If a symbol is changed in the PLC, the change is registered and an error message is issued.
- **Absolute addressing** – The linking of tags is not checked in runtime. You select the valid data type of the tags. If the tag address changes in the PLC, compile and load the HMI device again so that the change is registered in runtime.

Symbolic addressing is the default method.

5.5.2.3 Multiplexing & indirect addressing overview

Multiplexing symbolic access

Usage description

the Array index can be multiplexed

PowerTag Licensing

Less PowerTags
instead of all array elements, only one element is used in HMI

Remarks

- Additional Index tag necessary
- Alarming will trigger alarms depending on index tag
- Logging will log tag values depending on index tag

Multiplexing absolute access

Usage description

each single address part can be multiplexed

PowerTag Licensing

Less PowerTags, depending on used addressing

Remarks

- Works only with absolute addressing
- Additional index tags necessary for addressing
- Alarming will trigger alarms depending on index tag
- Logging will log tag values depending on index tag

Indirect Addressing

Usage description

Each HMI tag can be accessed in screens by using String tags

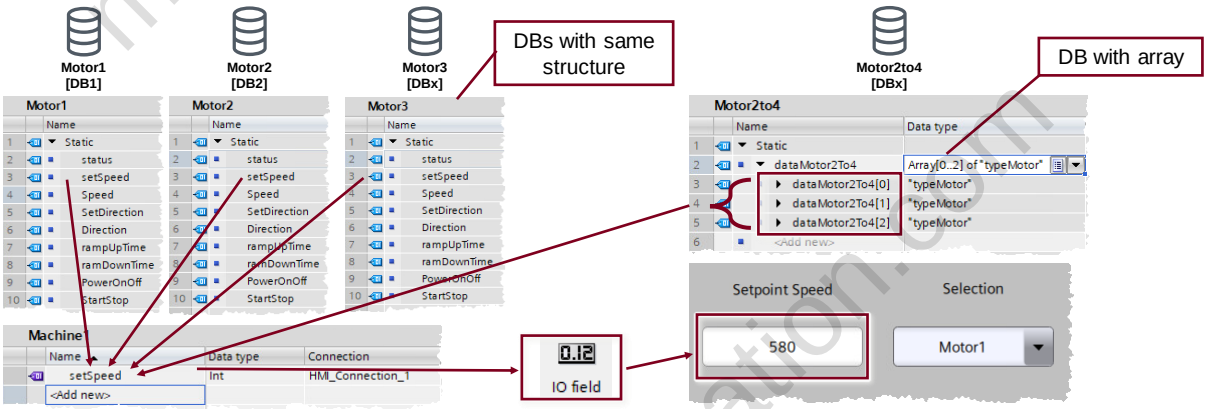
PowerTag Licensing

All necessary Tags need to be HMI Tags, flexibility comes at the usage of the tags in screens

Remarks

- Additional String tags necessary for addressing
- Can not be used with system functions

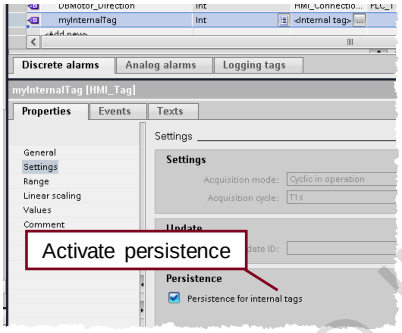
5.5.2.4 Multiplexing



One HMI tag in the screen visualize different data from the PLC
→ Reduce communication load and make data interconnection possible

5.5.3 Internal Tags

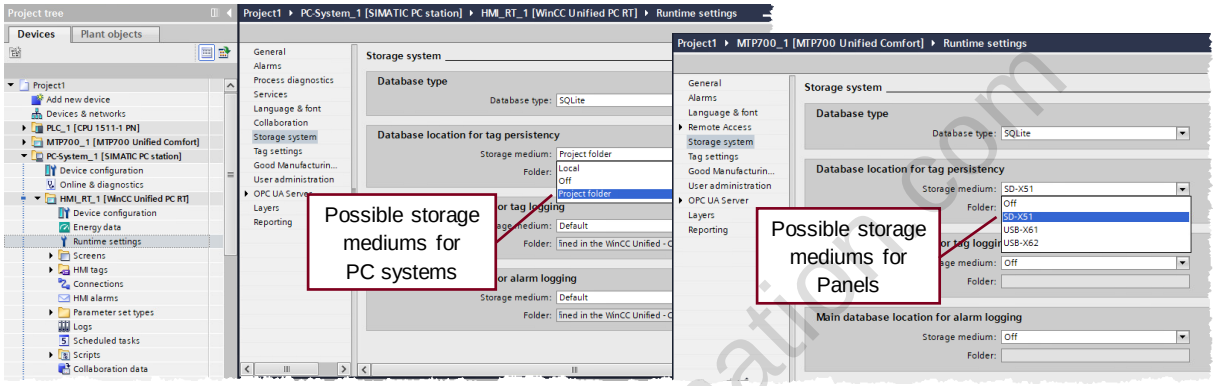
5.5.3.1 Persistence



- **Activate** the **persistence at the properties** of the internal tag
- The value will **be saved when stopping the runtime** and **used as start value** if runtime is started again
- Tag persistence **needs to be activated** in runtime settings

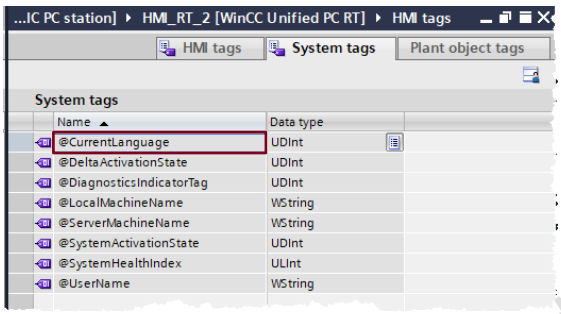
To save persistency values, the Runtime on UCP needs to be stopped via system function “StopRuntime”

5.5.3.2 Runtime settings for tag persistency



You configure the data location for tag persistency in the Runtime settings.
It depends on the device which locations are possible.

5.5.4 System tags

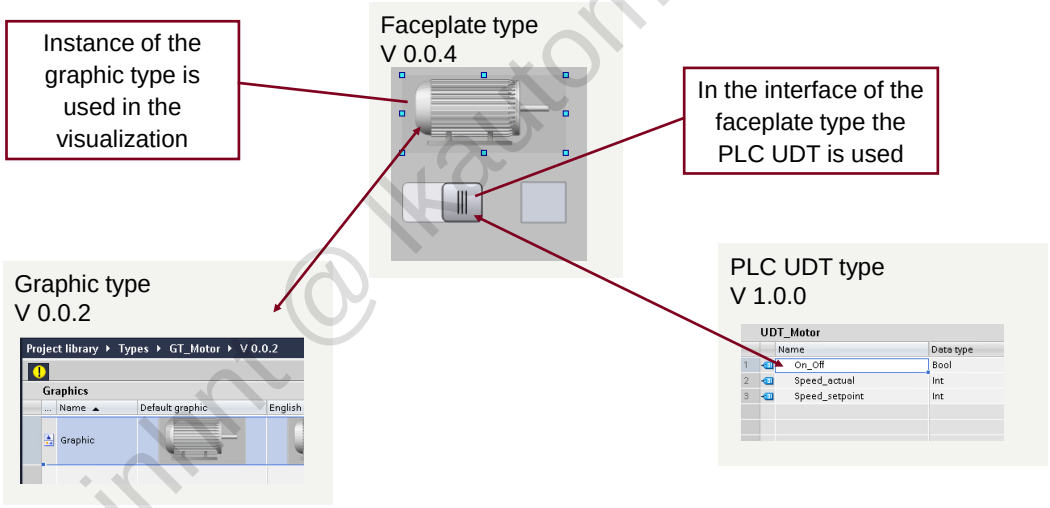


- **Internal tags** from the WinCC Unified System
- **Can be used in runtime** to display system information
- **Available System tags**:
  - CurrentLanguage
  - DeltaActivationState
  - LocalMachineName
  - ServerMachineName
  - SystemActivationState
  - SystemHealthIndex
  - UserName

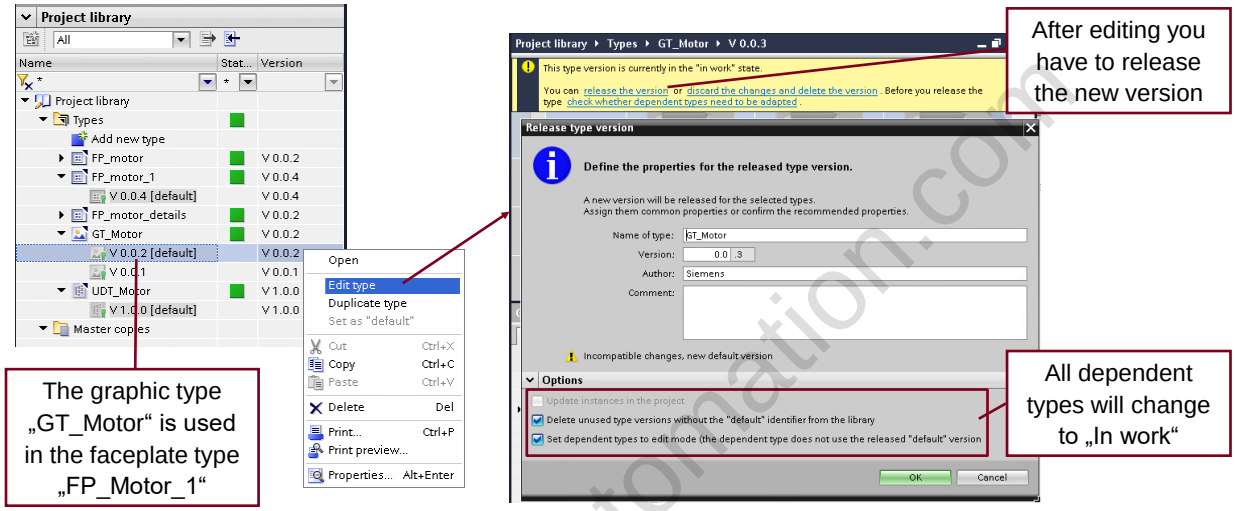
6 Libraries

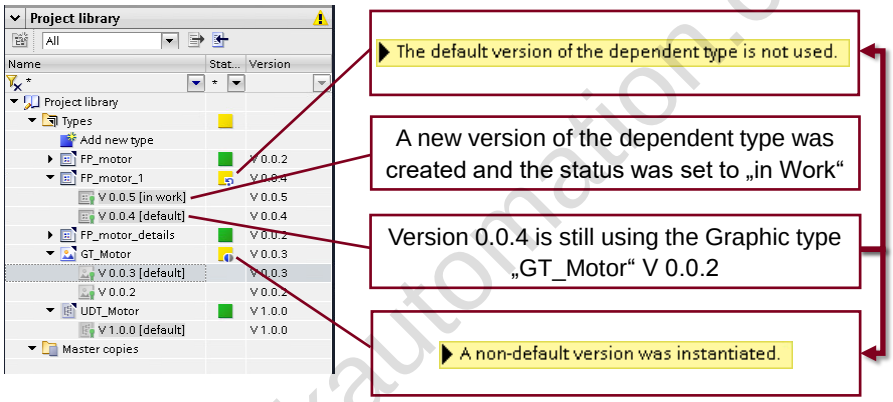
6.1 Dependencies between types

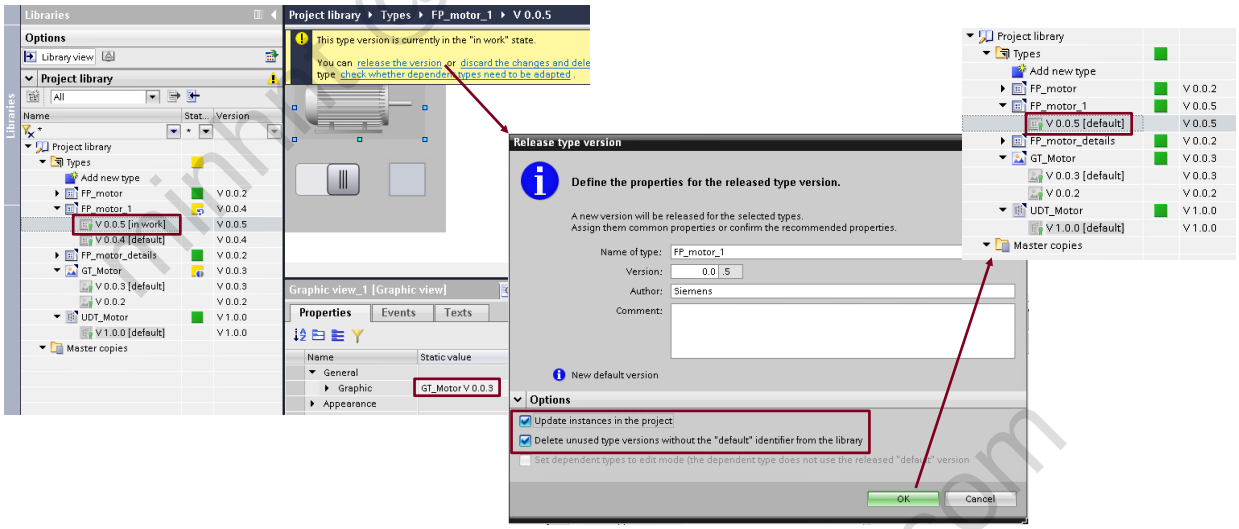
6.1.1 Overview



6.1.2 Engineering steps

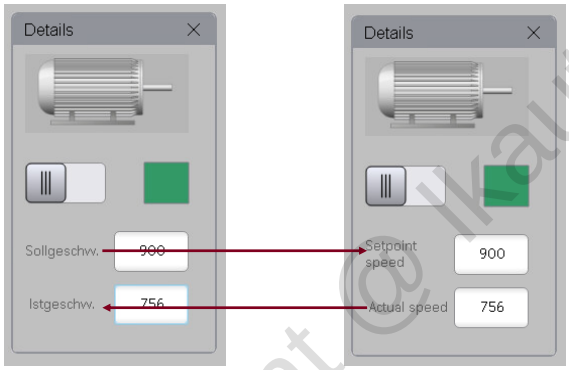






6.2 Multi Language Support

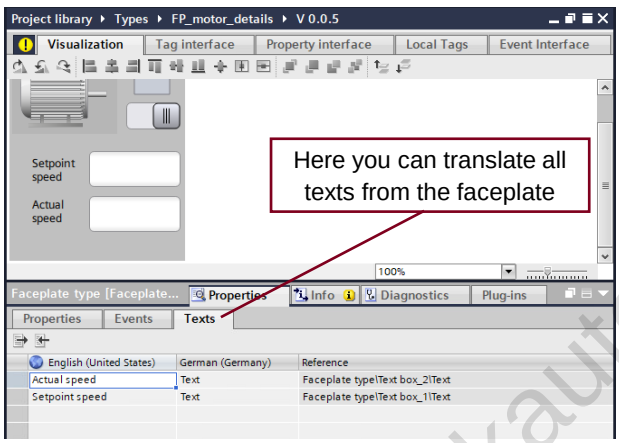
6.2.1 Overview

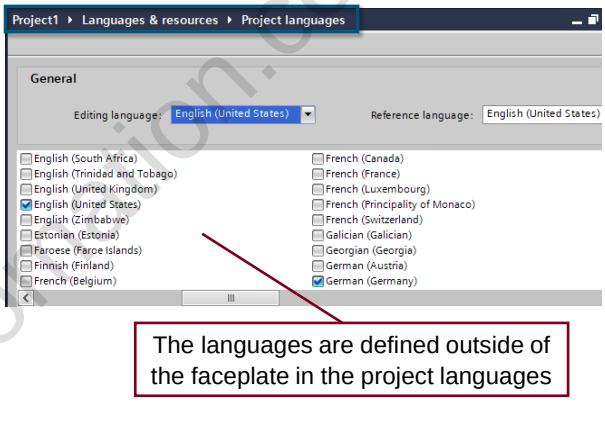


- **Runtime language**
  The definition of the Runtime languages is done at the Project languages

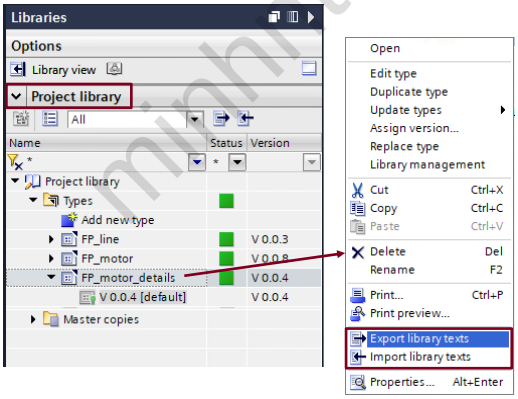
- **Translation**
  The translation can be done inside the faceplate or by export/import.

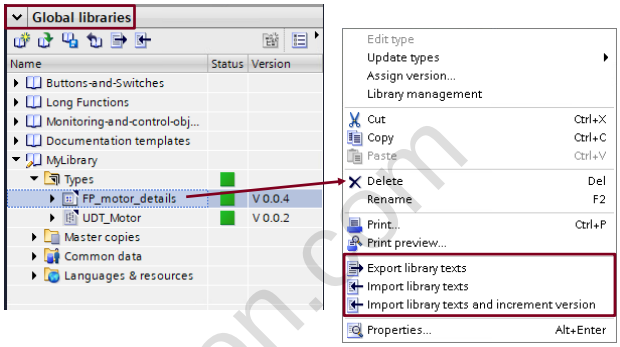
6.2.2 Engineering steps





After the release of the Faceplate type the version will be increased.

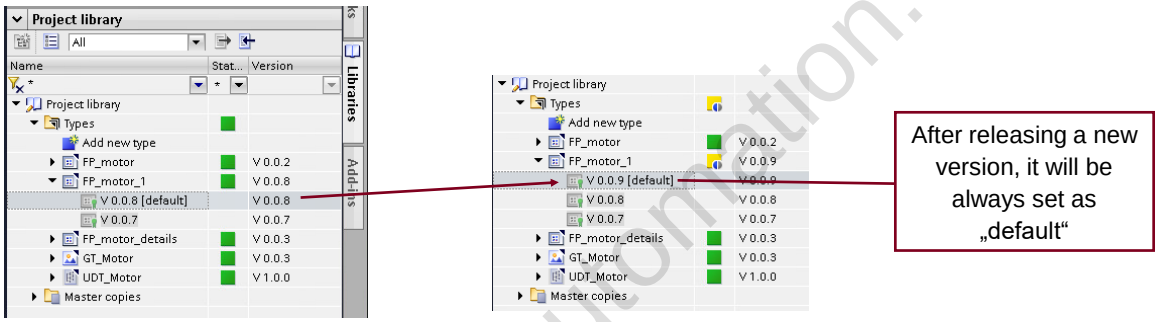




"Export library texts" will create an Excel file. Here you also can translate the texts.
"Import library texts" will <u>not</u> increase the version of the Faceplate type.

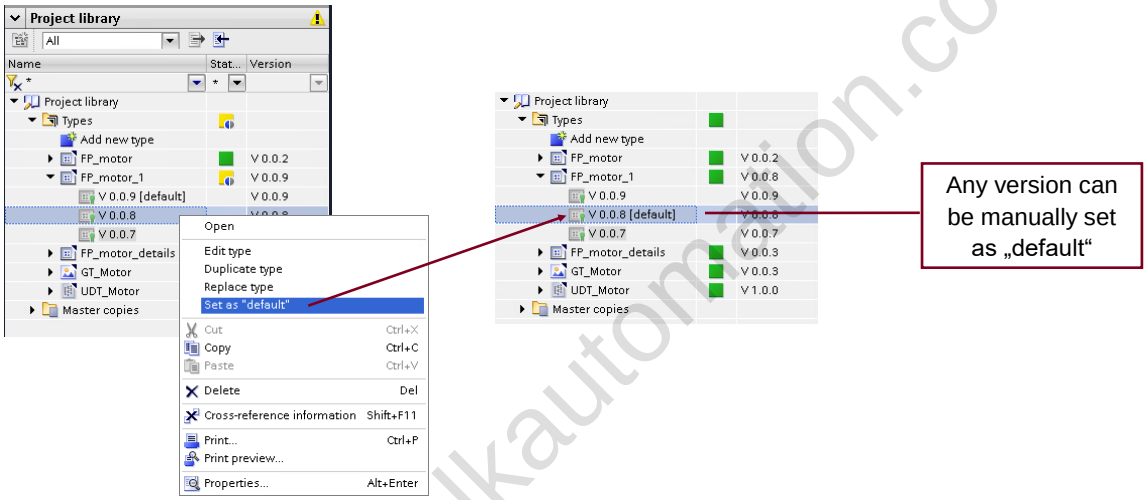
6.3 Default type

6.3.1 Overview

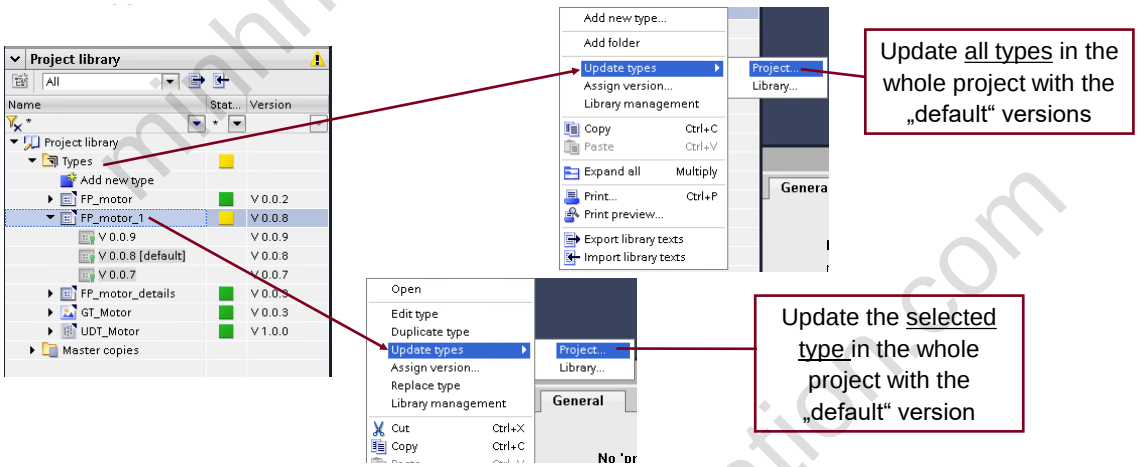


Since TIA Portal V17 type have a “default” version.

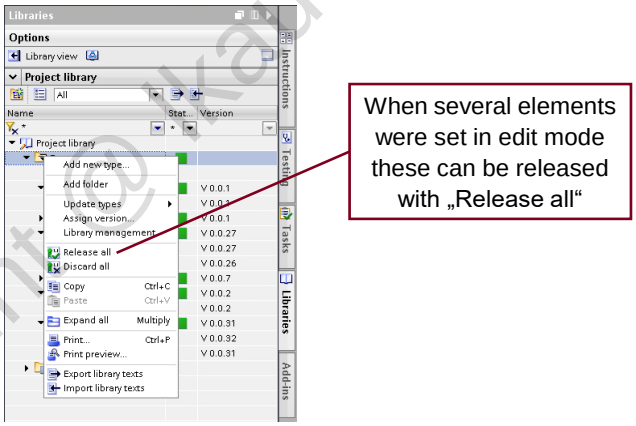
6.3.2 Engineering steps



6.3.2.1 Use case

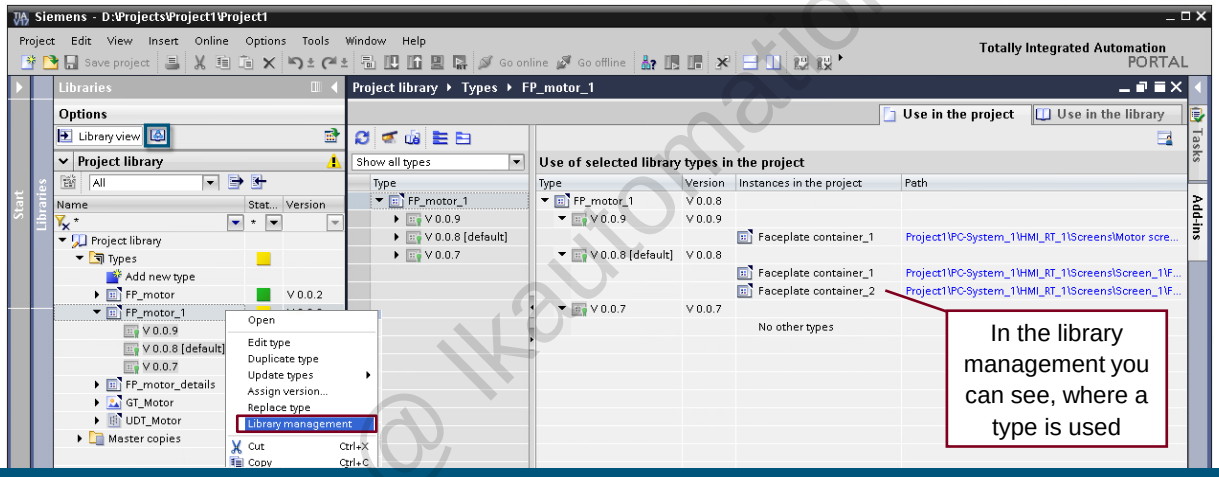


With the “default” version you can select, which type should be used in the project.
In V16 always the highest version was used for updating.



6.4 Tips and Tricks

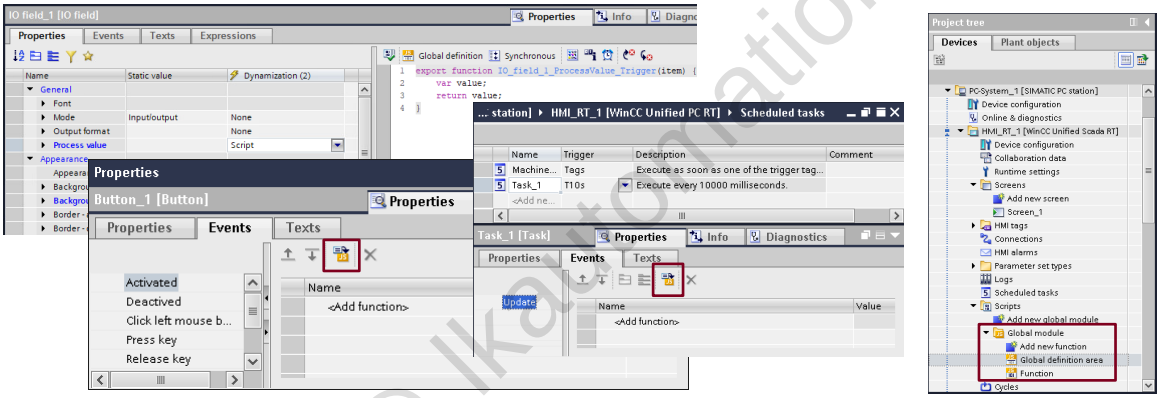
6.4.1 Libraries



Application example “Guideline on Library Handling in TIA Portal”
https://support.industry.siemens.com/cs/ww/en/view/109747503

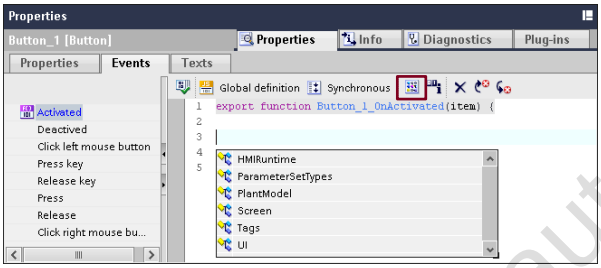
7 Scripting basics

7.1 Script editors



There are different script editors in WinCC Unified. You can find them in the events, dynamizations of screen items, scheduled tasks and also in the global module area.

7.2 Autocomplete

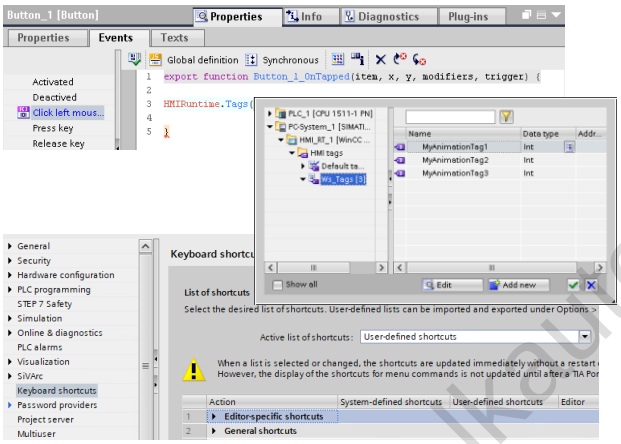


Can be opened by…

- Click on the button
- Key combination "Ctrl & Space" or "Ctrl & I"
- Adding "." after an object model element

Autocomplete helps to address WinCC elements.

7.3 Object picker

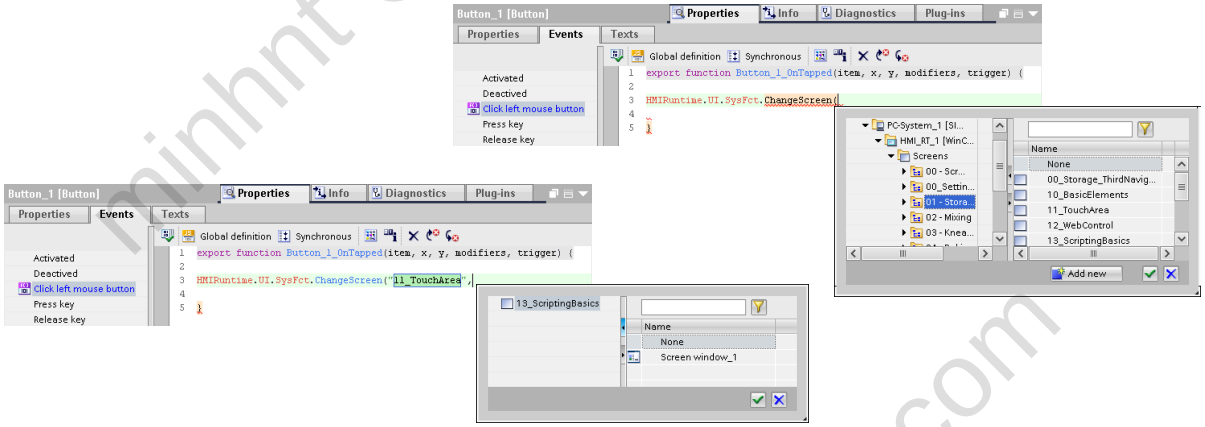


Can be opened by…

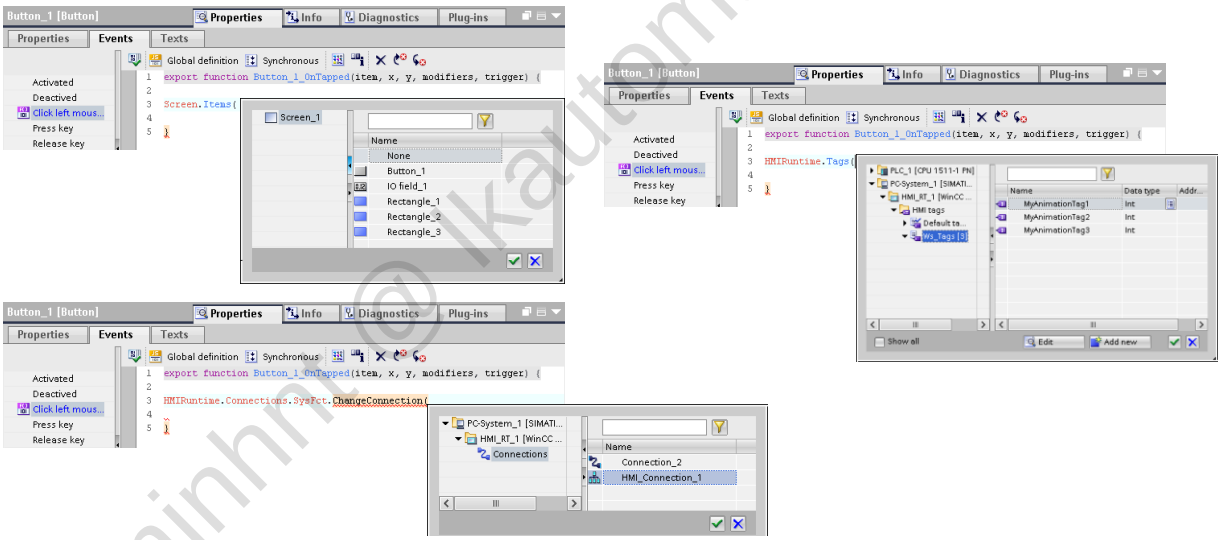
- Key combination "Ctrl & j" as from V17

You can change the short cut in TIA Portal general settings

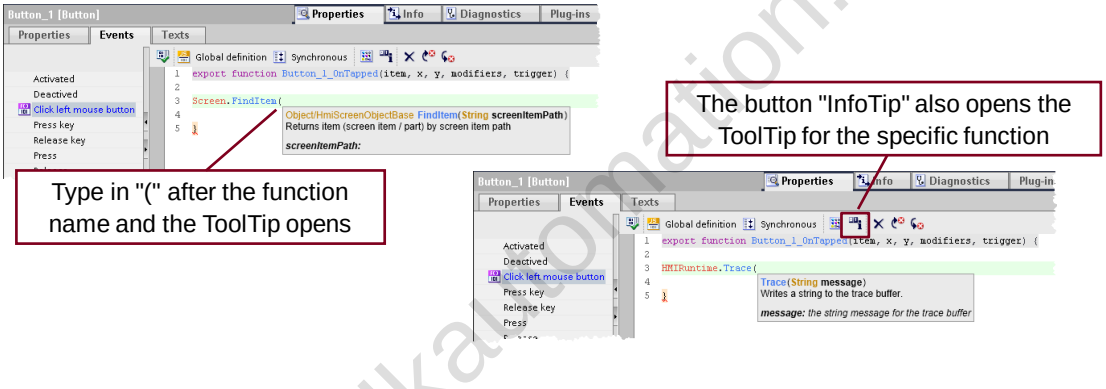
Object picker for HMI tags, HMI connections, screens & screen objects.



The object picker opens different picker depending on the context.

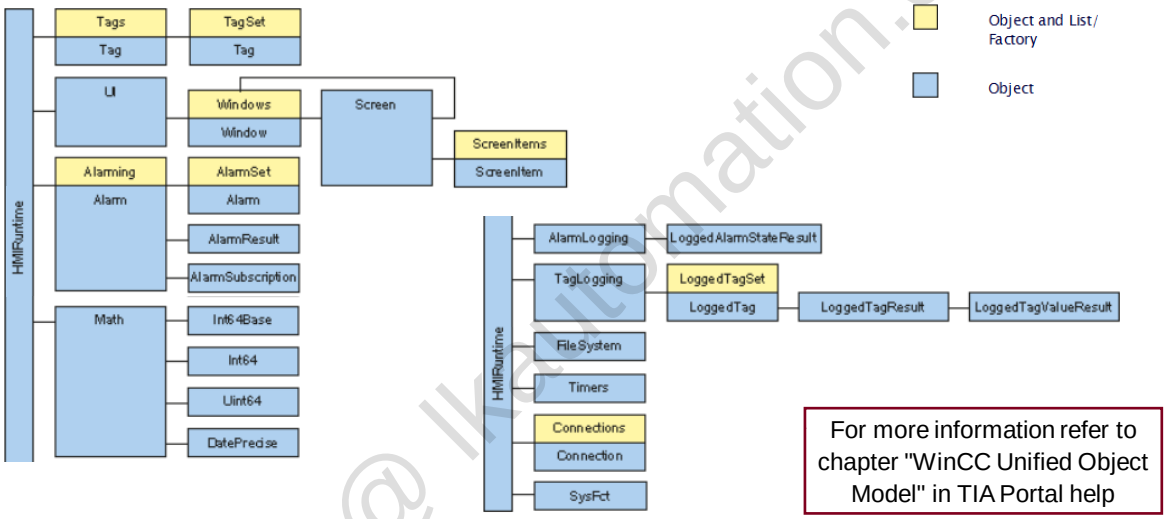


7.4 InfoTip



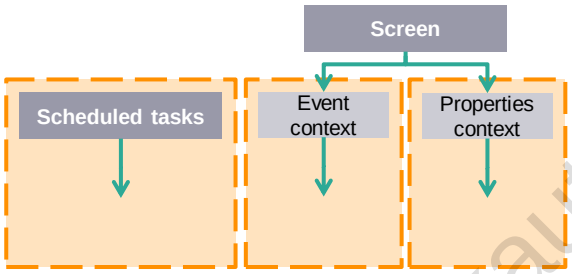
A lot of functions provide InfoTips which explain the function and their parameters.

7.5 Object model



For more information refer to chapter "WinCC Unified Object Model" in TIA Portal help

7.6 Execution contexts



- The different contexts work parallel in Runtime, therefore you can handle several scripts at the same time
- Each screen has its own scripting context
- Each script context has its own namespace

A context is the runtime environment where a specific script is performed.

Scheduled tasks		
Cyclic	**Tag triggered**	**Alarm triggered**
Triggered cyclically fastest cycle is 100ms	Triggered by value change of one or multiple tags	Triggered depending on the defined alarm criteria
Cyclic scripts should be avoided because of the performance	React on value changes	React on changes in alarm system

Hints:
- All jobs share a script context
- Different tasks can access common global tags
- Each required module must be referenced by means of the 'import' statement
- No access to screen items

Screen	
Event context	**Properties context**
Perform an action based on an event, e.g. click on a button or property "change"	Can be cyclic, tag based
	Cyclic scripts should be avoided because of the performance

Hints:
- The script of a property cannot access global tags of an event even in the same screen.
- Each script context references the modules that it has imported using the 'import' statement. However, each script context receives its own copy of the tags defined there

7.7 JavaScript Syntax

Preferred way:

- const variable = value
  (const is for constant variables)
- let variable = value
  (standard way to declare variables)

Outdated way:

- var variable = value (older version of variable declaration, scoping problems)

Some of the snippets or example code have the old version of variable declaration.

Variable declaration.

To work with strings you can use:

- "Text" (to add a specific text)
- + (to connect strings to each other)
- variable (use the value of a variable)

Trace example:

HMIRuntime.trace("Text"+variable+"Text"+Value*Value…)

Working with strings.

- || (logical or)
- && (logical and)
- ! (logical not)

Example:

If ((variable1 || variable2) && variable3) { }

Logical Operators.

```
export function Funtion (parameter1, parameter2){
 let tag1 = Tags('MyTag1');
 tag1.Write(123);

 let tag2 = Tags('MyTag2');
 tag1.Write(123);

 if (tag1 > tag2) {
  UI.Root                    'MyScreen1';
  HMIRutnt                   ess');
  } else {
   HMIRunt                   cess');
  }
}
```

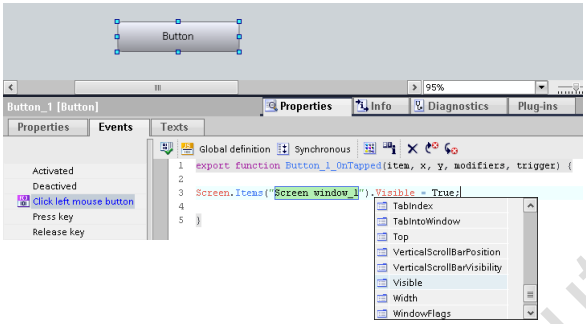
- < / > (smaller or bigger)
- == (equal to , also combinable with smaller or bigger)
- === (equal value and equal type)
- != (not equal)
- !== (not equal value or not equal type)

Example:

if((variable1==variable2) || (variable1===variable3)){}

Comparison Operators.

7.8 Access local screen items

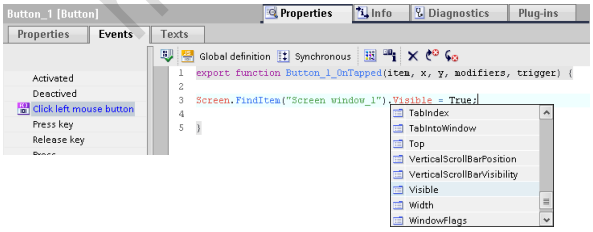


Screen.Items()

- Access to all items in the same hierarchy level
- Properties of items are known and can be selected via IntelliSense
- If item name is changed, Screen.Items()-item is also adjusted automatically
- Item path can be addressed with **relative** or **absolute** prefix

Screen.Items() can only access items of the current screen.

7.9 Access global screen items



Screen.FindItem()

- Access to all items independent of the hierarchy level
- Properties of items are known and can be selected via IntelliSense
- Item name is handover as static string
- Item path can be addressed with **relative** or **absolute** prefix

Screen.FindItem() can access all items from different screens/hierarchy levels.

7.10 Absolute / relative addressing

relative

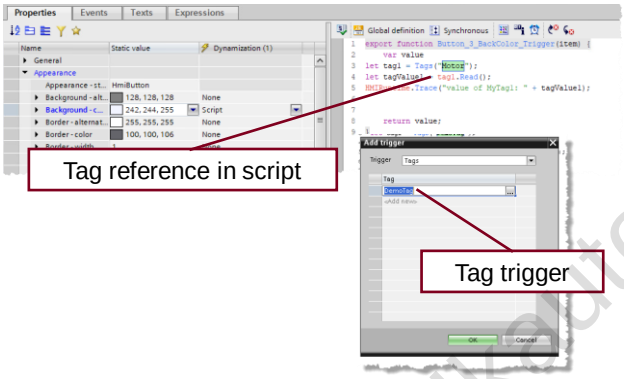
Prefix	Description
".."	References the higher-level screen window (parent) in the context of the current screen window.
"."	References the own screen window (self).
""	A screen item of the current screen window is referenced without prefix.

absolute

Prefix	Description
"/"	References a screen window on the highest level, whose name must follow.
"~"	References the screen window on the highest level in the own screen hierarchy.

- Relative and absolute items paths are distinguished by the prefix of the item path.
- The **relative item path** is specified starting from the screen where the script is called.
- The **absolute item path** is specified starting from the "RootScreenWindow".

7.11 Tag cross reference

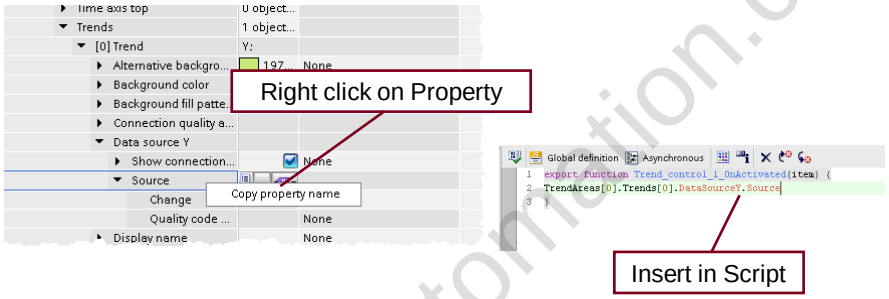


Tags can be used in different variations:

- Event scripts
- System functions
- Property dynamization
- Trigger

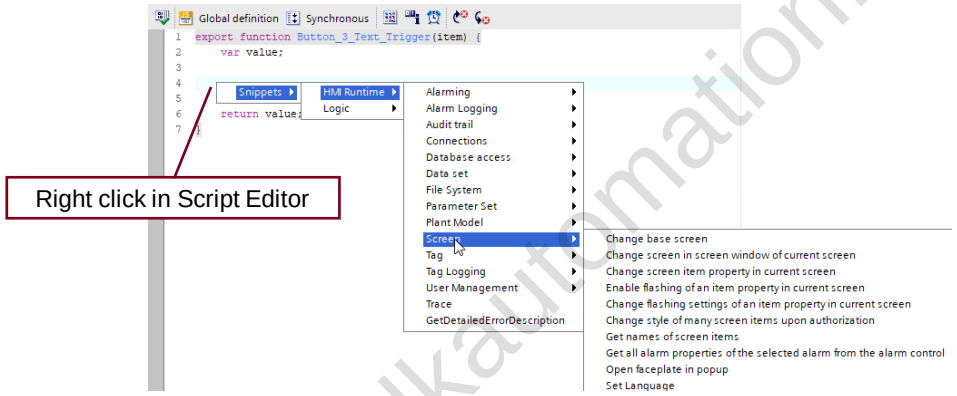
After changing the name of the tag, all references adjust automatically to the new tag name.

7.12 Copy Tag Property



You can right click on the property to copy it and paste it in the script editor.

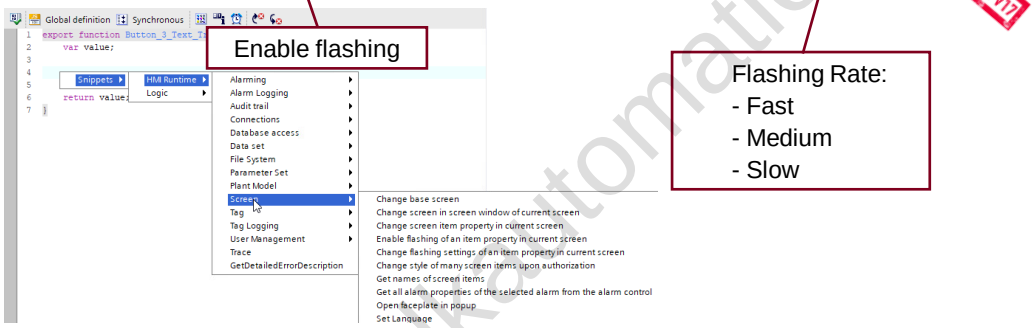
7.13 Snippets



Snippets provide frequently required code templates.

7.14 Flashing snippet

```
Screen.Items("Circle_1").PropertyFlashing("BackColor", true);

Screen.Items("Circle_1").PropertyFlashing("BackColor", true, HMIRuntime.Math.RGB(255, 0, 0), HMIRuntime.Math.RGB(0, 255, 0), UI.Enums.HmiFlashingRate.Fast);
```

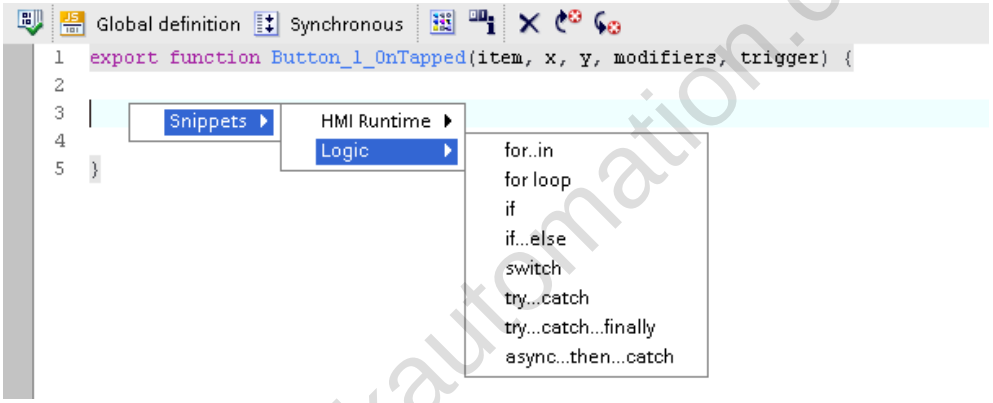


Enable flashing

Flashing Rate:
- Fast
- Medium
- Slow

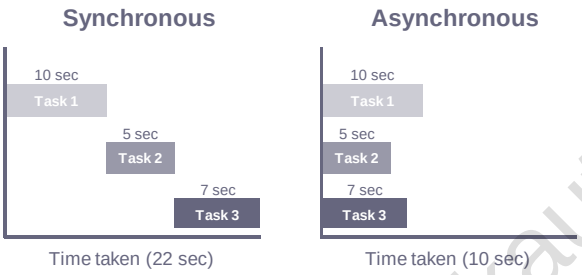
as from V17

7.15 Logic snippets



Snippets for Logic syntax are also available.

7.16 Read & Write methods



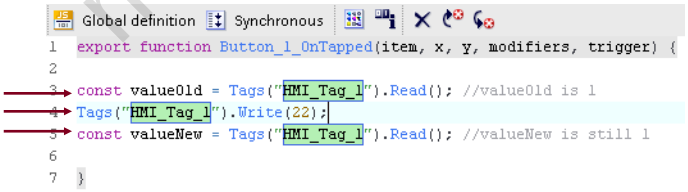
Read

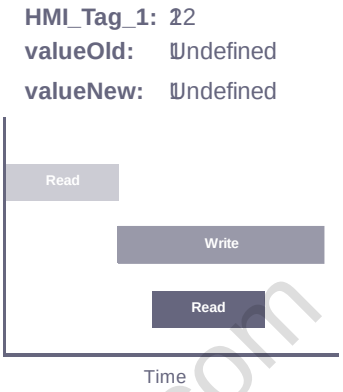
- Read the values of the tag ("Tag" object)
- The **Tag.Read method works synchronously** by default

Write

- Writes the values of the tag ("Tag" object)
- The **Tag.Write method works asynchronously** by default

The Tag.Read method may return unexpected results if the processing of a preceding Write call is not yet complete.





The Tag.Read method may return unexpected results if the processing of a preceding Write call is not yet complete.

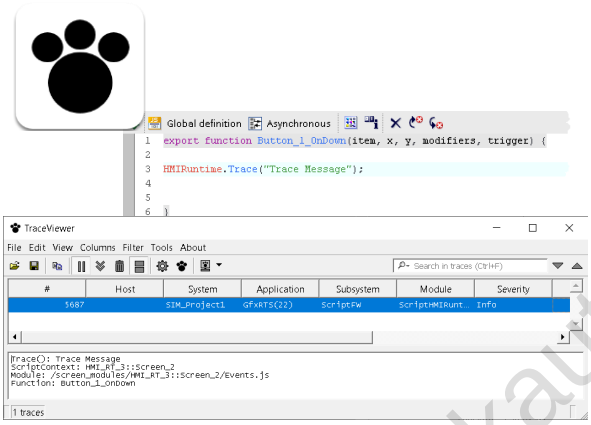
Global definition | Synchronous

```
export function Button_1_OnTapped(item, x, y, modifiers, trigger) {

const valueOld = Tags("HMI_Tag_1").Read();
const valueTNew = 22;
Tags("HMI_Tag_1").Write(valueNew);

}
```

- Store the new value in a local JS variable to use it later in the script

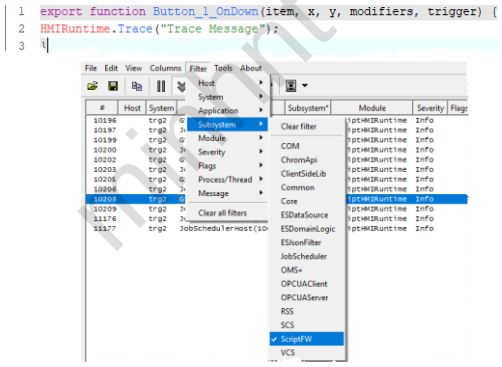
Recommended syntaxis.

7.17 RTIL Trace viewer



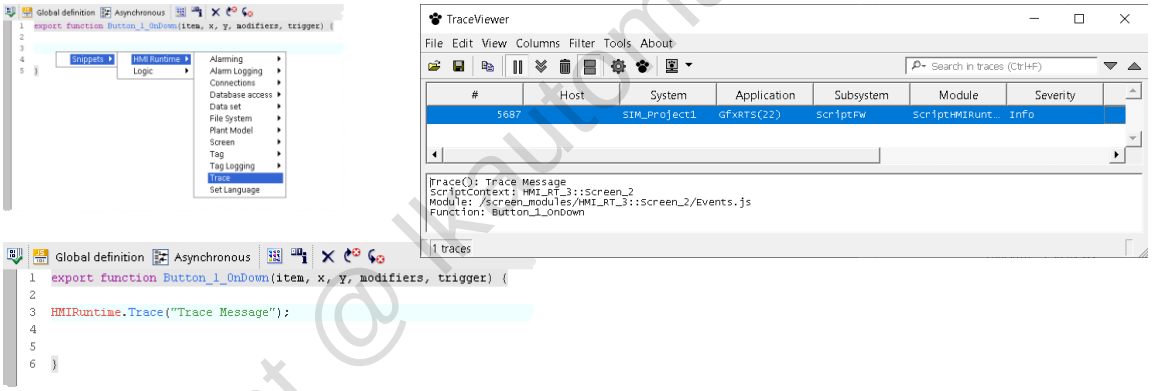
- Trace Viewer displays all runtime alarms which are listed in the configurable TraceCatalog.
- Traces are displayed in tabular form and can be filtered.
- Alarms can be exported in text or CSV files.

RTIL Trace Viewer is a separate application. Location:
C:\Program Files\Siemens\Automation\WinCCUnified\bin\RTILtraceViewer



- Carry out an action in the simulation which starts a JavaScript function.
- Filter by the trace messages "Subsystem > ScriptFW".
- Define additional filter criteria if required.
- Evaluate trace messages if actions in the simulation lead to errors.

If no messages are displayed in the Trace Viewer despite errors in the simulation, reset the filters.



It is possible to output a text or a tag value in the Trace viewer via scripting.

7.18 Exercise 1: Read and write tags

1. Add "Read" button in the Ventilator1 screen. The current value of the setpoint speed is to be read out and displayed in TraceViewer using the HMIRuntime.Trace function.
Tip: Use the corresponding snippet. Select the tag "Ventilator1Data_setpointSpeed" using the object picker (Ctrl+j).

2. Insert "Increase" button in the screen. Each button press is to increase the setpoint speed by 100.
Tip: Additionally use the "WriteTag" snippet.

3. Duplicate the "Increase" button and label it "Increase with limit". Each button press is to increase the setpoint speed by 100. In addition, the setpoint speed is to be monitored for a value greater than 1350 rpm.
Tip: Use snippet for "if".

7.19 Exercise 2: Change screen objects

1. The "Increase with limit" button is to turn red when an increase in the value would exceed 1350 rpm. Otherwise, it is to be black. However, the color change takes place only after clicking.
Tip: The parameter "item" is an object reference to the operated object.

2. Add a circle in the screen as an indicator for the current speed. The solution is to be carried out with JS.

0 rpm	gray background
1 - 1000 rpm	green background
Over 1000 rpm	red background

Tip: Add a script in addition to the "Background - Color" property.

3. Add another "Toggle indicator" button. Its purpose is to toggle the visibility of the circle.

7.20 Detailed exercise description

7.20.1 Exercise 1: Read and write tags

Part 1: Read and write tags

1. Open the "Ventilator1" screen.
2. Insert a new button and label it with "Read" (Properties > Properties > General > Text).
3. Call a script using an event of the "Read" button ("Properties > Events > Press").
4. Now change the editor for system functions to a script editor. To do this, click the following button above the function list:

5. Enter the following code in the function area of the script editor.
   The tag "Ventilator1Data_setpointSpeed" is located in the project path of the F-CPU (Program blocks > Ventilator1Data [DB1] > setpointSpeed).

Figure 7-1

```
export function Funktion1(parameter1, parameter2) {

let tag1 = Tags('Ventilator1Data_setpointSpeed');

let tagValue1 = tag1.Read();

HMIRuntime.Trace('Die aktuelle Soll-Drehzahl von Ventilator 1 beträgt: ' + tagValue1 + ' U/min');

}
```

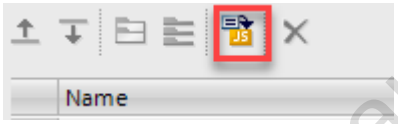
6. Open TraceViewer via "C:/Program Files/Siemens/Automation/WinCCUnified/bin" and start the "RTILtraceViewer.exe" application.
7. Filter TraceViewer by messages
   - Open the "Filter" menu in the TraceViewer menu bar.
   - Open the "Subsystem" category.
   - Select the "ScriptFW" module.

   This completes the settings for the TraceViewer.
8. Start the HMI Runtime. Call the "Ventilator1" screen.
9. Press the "Read" button. The message including the current "set speed" is output via the TraceViewer.

Part 2: Increase/reduce values in increments of 100

1. Open the "Ventilator1" screen.
2. Insert two new buttons and label them with
   - "+ 100"
   - "- 100"

   (Properties > Properties > General > Text).
3. Call a script using an event of the "+ 100" button ("Properties > Events > Press").
4. Now change the editor for system functions to a script editor. To do this, click the following button above the function list:



5. The following program code reads the current value of the tag (line 5, 6), increases the value by 100 and generates a message via the TraceViewer (line 12).
   The value increased by 100 is not automatically written to the tag but instead must be written to the tag with tag1.Write (line 15).

   Tip: The "Read tag" snippet provides you with the structure for reading a tag including TraceViewer message and thus facilitates the programming of the function.

   Figure 7-2

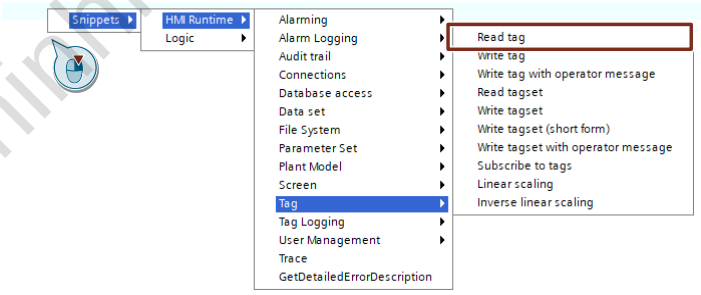
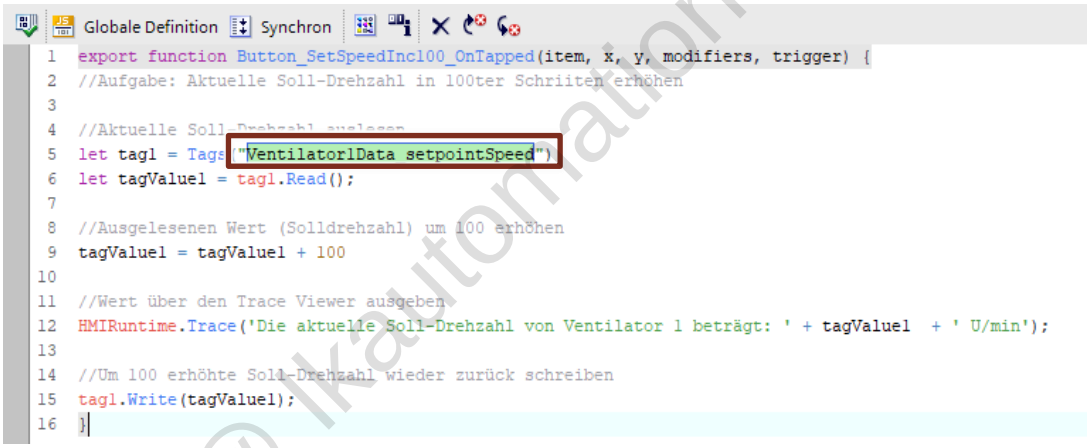


   Figure 7-3



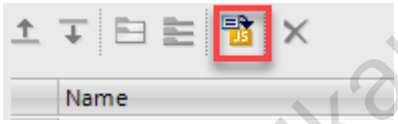
6. Copy the script to an event of the "- 100" button ("Properties > Events > Press"). Change the function list to a script editor again for this.
7. Adapt the code in line 9. Use the "- character" in place of the "+ character".
8. Test the configuration and observe how the setting of the tag values (SetPointSpeed) via the two buttons +100 / -100 behaves.

Note: SetSpeed**Inc**. = Increment; SetSpeed**Dec** = Decrement

Part 3: Limit values to 1350 rpm

1. Open the "Ventilator1" screen.
2. Insert two new buttons and label them with
   a. "+ 100max"
   b. "- 100max"

   (Properties > Properties > General > Text).
3. Call a script using an event of the "+ 100max" button ("Properties > Events > Press").
4. Now change the editor for system functions to a script editor. To do this, click the following button above the function list:



5. Enter the following code.
   (tag used: Program block "Ventilator1Data" > "setpointSpeed"

   Tip:
   Snippets: for an "if-else" instruction, right-click in the Javascript editor and select "Logic > if..else".

   Figure 7-4

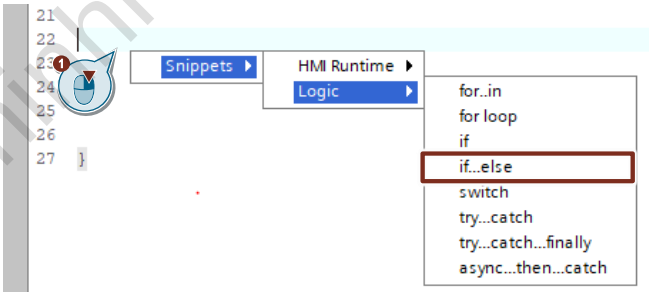
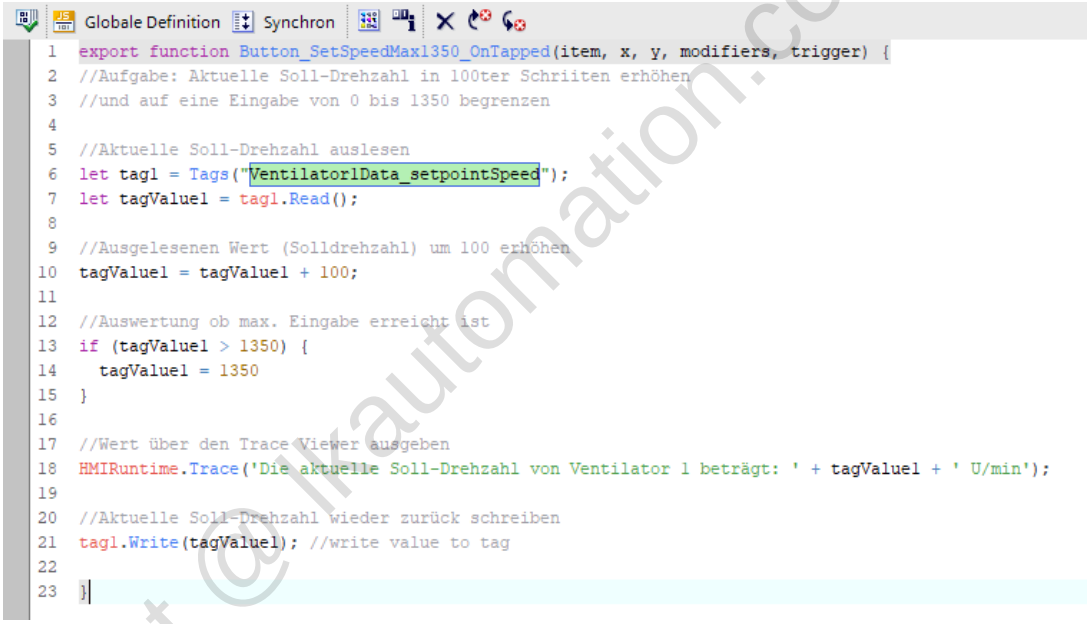


   Figure 7-5



6. Copy the script to an event of the "- 100max" button ("Properties > Events > Press"). Change the function list to a script editor again for this.

7. Adapt the code in line 17.
   Use in place of

```
if (tagValue1 > 1350) {
   tagValue1 = 1350
}
```

   the following code section:

```
if (tagValue1 < 0) {
   tagValue1 = 0
}
```

8. Test the configuration and observe how the setting of the tag values via the two buttons +100max / -100max behaves.

7.20.2 Exercise 2: Animate buttons and objects

The following exercises build on the scripts created in Exercise 1.

Part 4: Change background color of button

1. Add to the script at the press event of the "+ 100max" button from Exercise 1 (Figure 7-5).

   Optional: If your script is to remain, create a new button, copy the function and make the changes in the new script.

2. With the "item.ForeColor" property, you can change the font color of the object for which the script is called.

   Change the button to RGB (255, 0, 0) within the if instruction and insert an else branch with the default color (see Figure 7-6).

   Figure 7-6

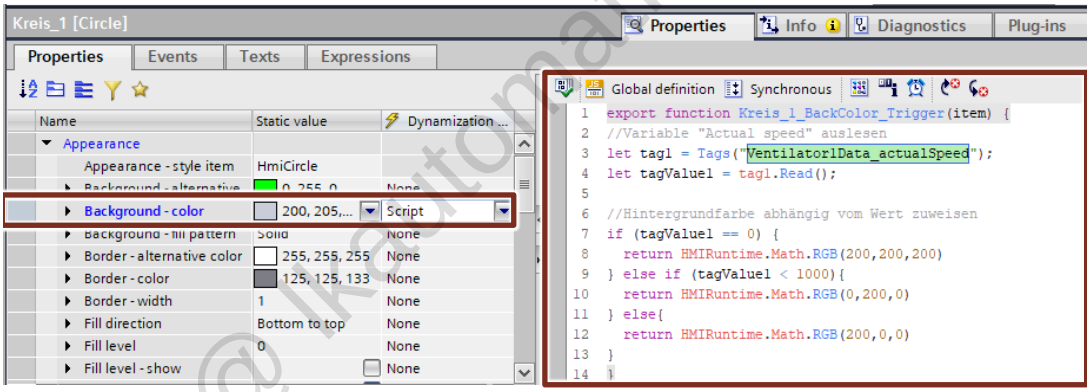
```
8
9  //Ausgelesenen Wert (Solldrehzahl) um 100 erhöhen
10 tagValue1 = tagValue1 + 100;
11
12 //Auswertung ob max. Eingabe erreicht ist
13 if (tagValue1 > 1350) {
14   tagValue1 = 1350;
15   item.ForeColor = HMIRuntime.Math.RGB(255,0,0) //Farbe Rot
16 } else {
17   item.ForeColor = HMIRuntime.Math.RGB(0,0,0) //Farbe schwarz
18 }
19
20 //Wert über den Trace Viewer ausgeben
21 HMIRuntime.Trace('Die aktuelle Soll-Drehzahl von Ventilator 1 beträgt: ' + tagValue1 + ' U/min');
22
```

3. Repeat these steps for the script at the "-100max" button.

Part 5: Change background color of a screen object as a function of the speed

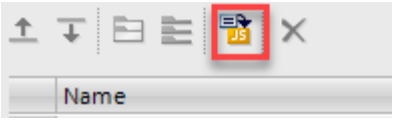
1. Open the "Ventilator1" screen and insert a "Circle" from task card "Toolbox > Basic objects".
2. Select the circle and create a new "Script" dynamization for the "Background - Color" property.
3. Write a function that reads in the tag "Ventilator1Data_actualSpeed" and then returns different colors to the property as a function of the value.

Figure 7-7



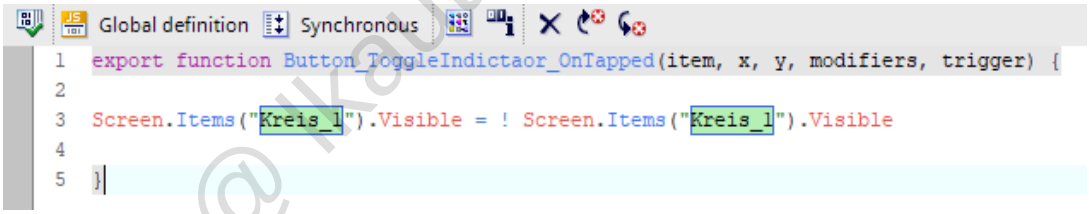
Part 6: Show and hide the circle

1. Insert a new button and label it with "Toggle indicator" (Properties > Properties > General > Text).
2. Call a script using an event of the "Toggle indicator" button ("Properties > Events > Press").
3. Now change the editor for system functions to a script editor. To do this, click the following button above the function list:



Enter the following code in the function area of the script editor.

Figure 7-8

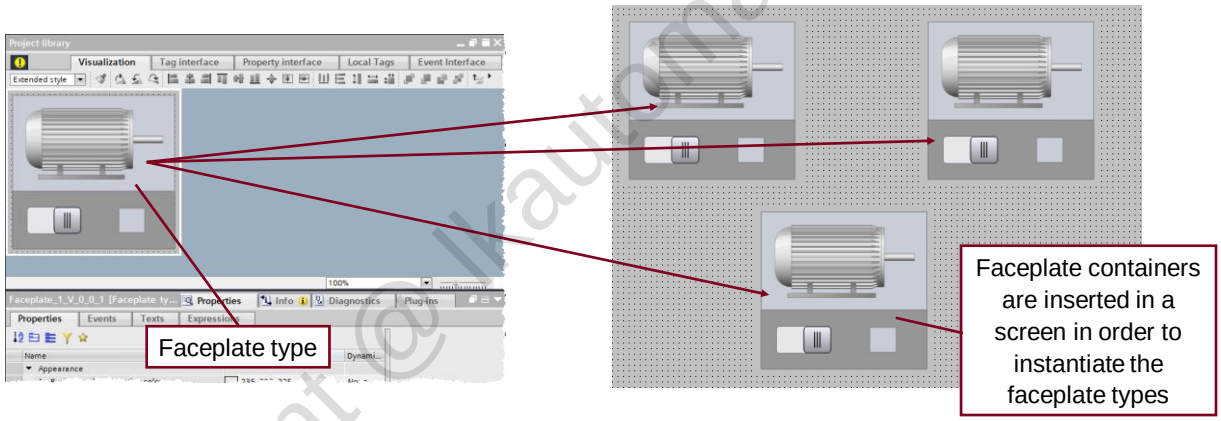


4. Save the screen and test the functionality in runtime.

8 Faceplates

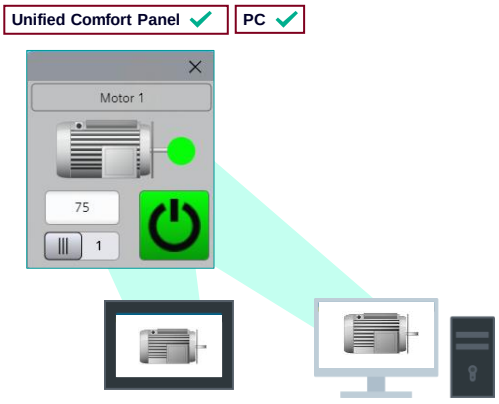
8.1 General information

8.1.1 Faceplate types and Faceplate instances



Configure once … **… use multiple times**

8.1.2 Unified Faceplates as enclosed graphical objects



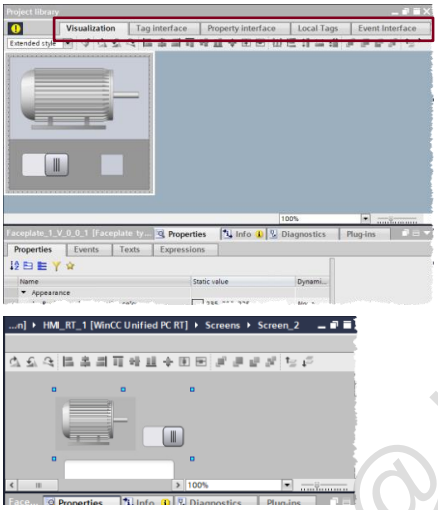
Create individual library templates (faceplate types) with dynamic process connection

Use the same Faceplates
for panel and PC systems using a Faceplate container control

Standardized operation elements for **flexible reuse** in all Unified devices and projects

Efficient operation window (popup)
for assets e.g., motor, pump

8.1.3 Engineering



TYPE-INSTANCE-Concept

- Create once, use many times

Define Faceplates

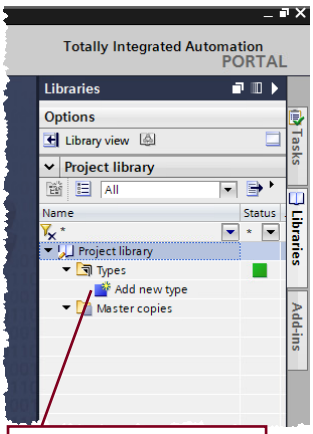
- Interfaces – project independent (simple tag types, PLC UDT types)
- Property interface (colors, text resource lists, Integer, …)
- Visualization for object and operation window, engineering like a screen (without advanced controls)

Use Faceplates in screens

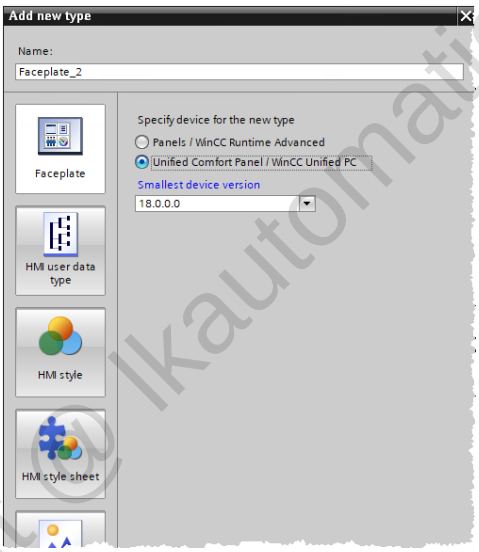
- Drag and Drop
- Assign interface and properties

8.2 Working with Faceplates

8.2.1 Create a new faceplate type



Location: folder "Types" in the project library

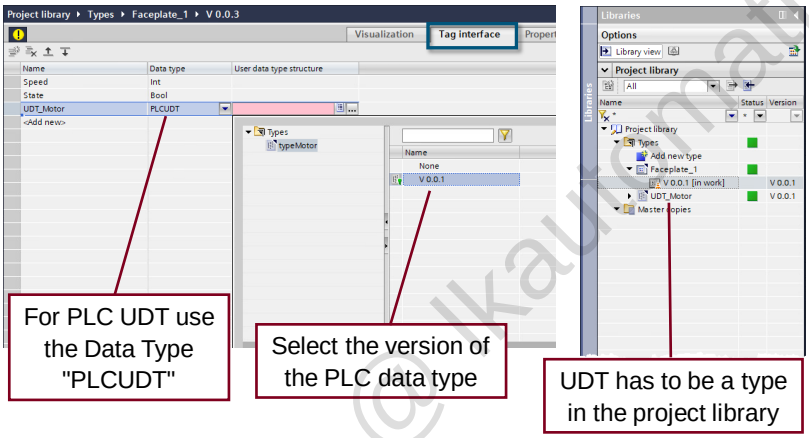


Hints:

- Dynamic SVGs are supported
- Some controls are supported
- System functions can be called in a function list
- Versioning in the library is supported

8.2.2 Faceplate Editor

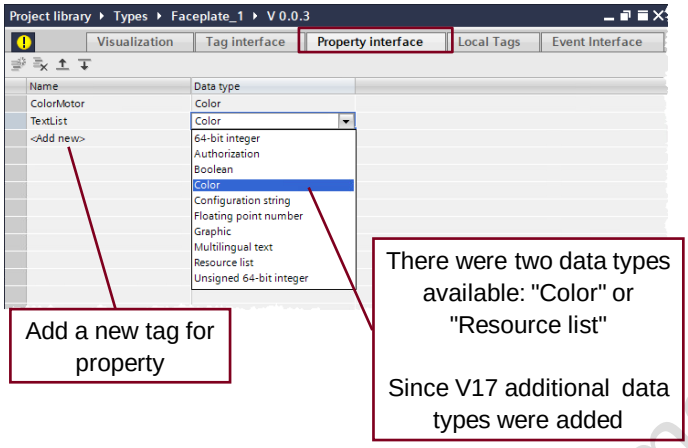
8.2.2.1 Tag interface



Hints:

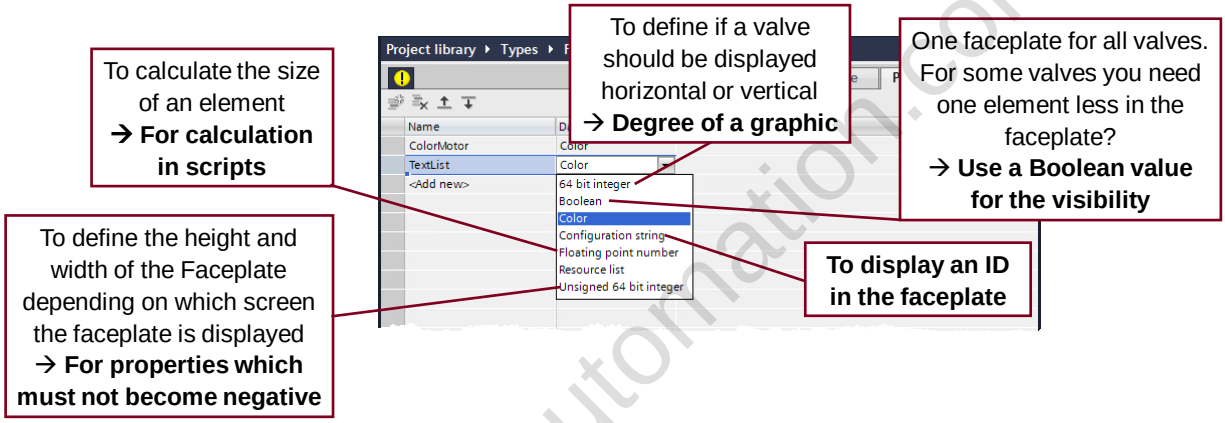
- Only tags from the Interface can be connected in visualization
- UDT in UDT works
- A UDT containing a Struct works
- Arrays **are** supported

8.2.2.2 Property interface

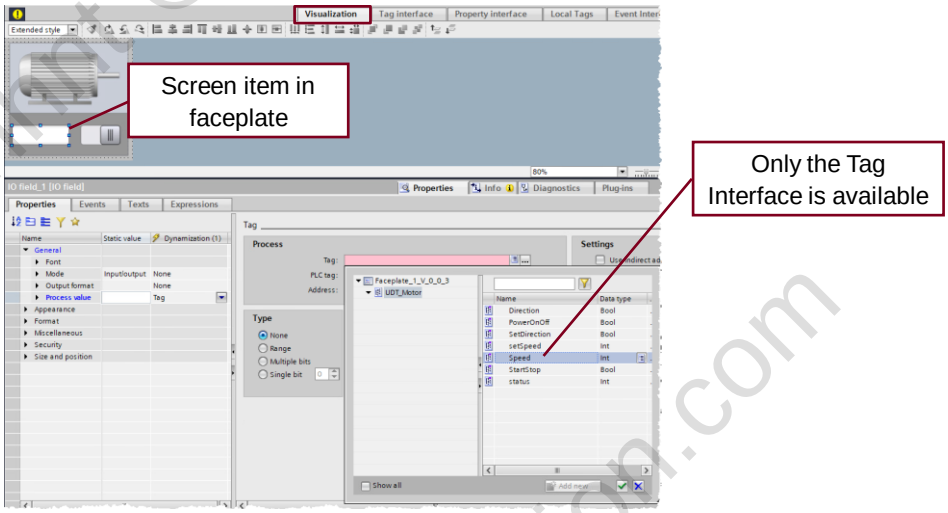


Hints:

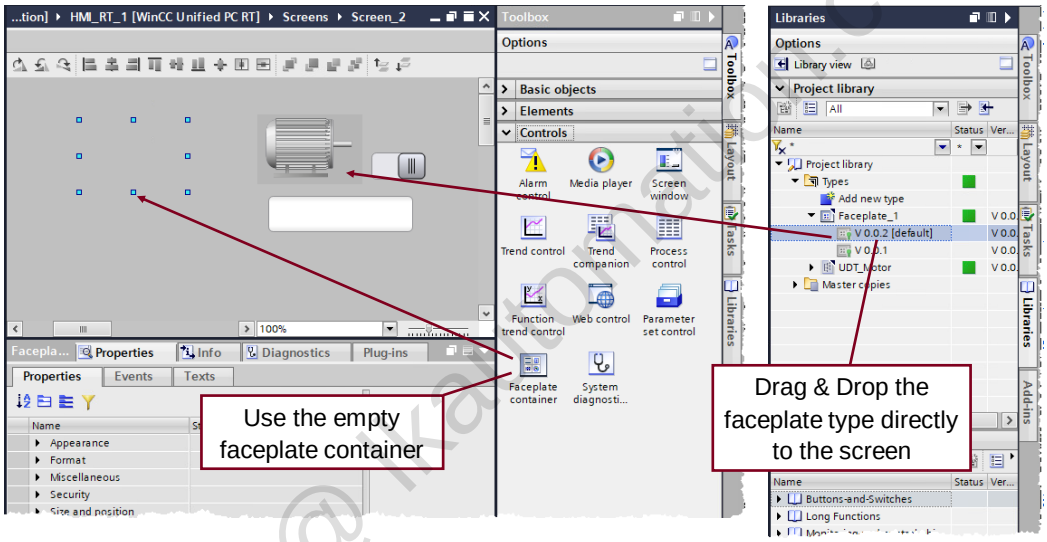
- The Graphic list works in Faceplates
- The Text list works on the symbolic IO field



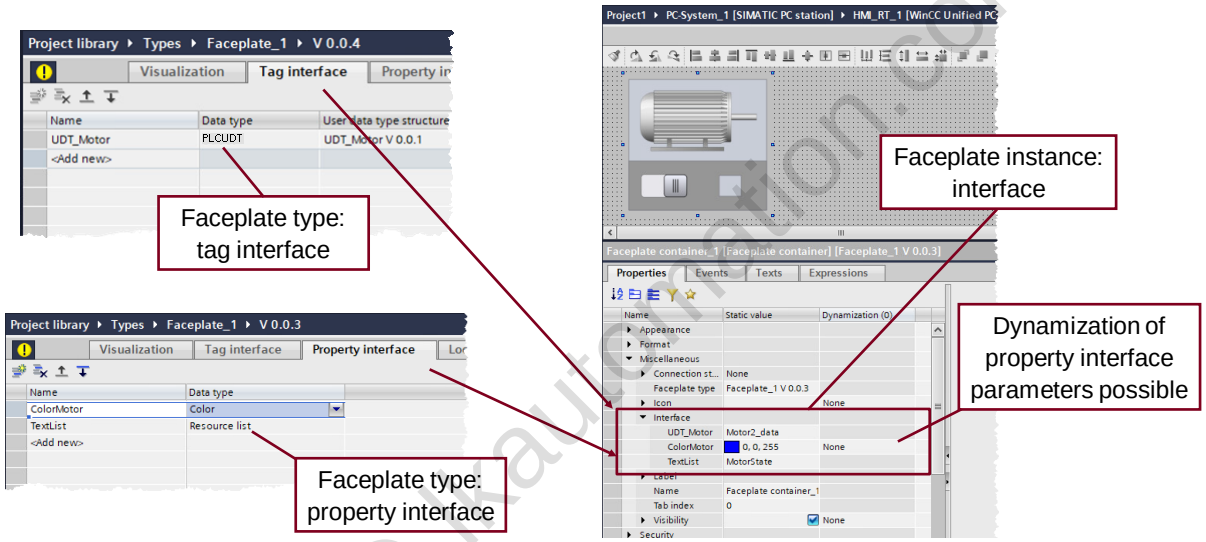
8.2.2.3 Visualization



8.2.3 Working with instances

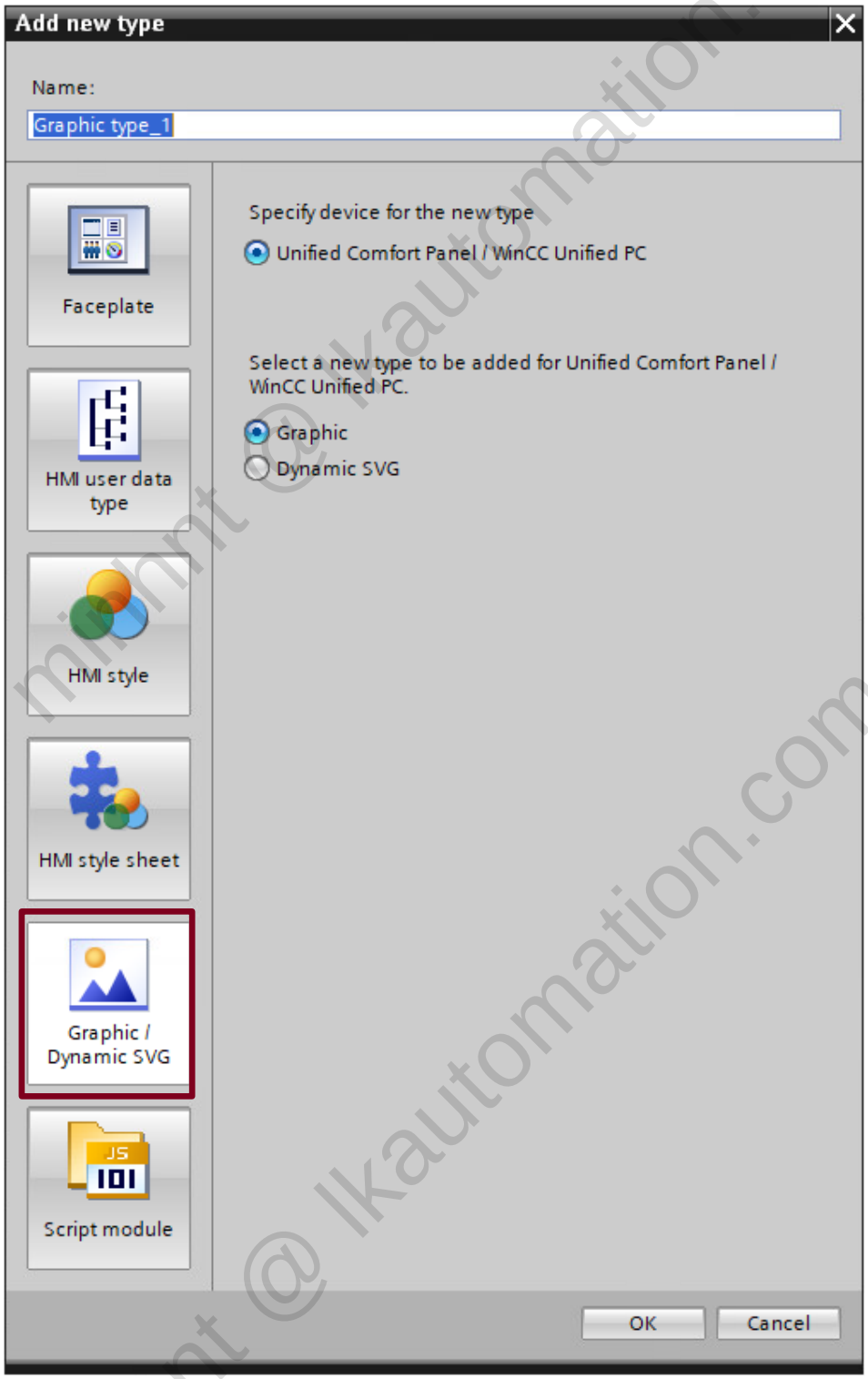


8.2.4 Connect instances



8.3 Using a Graphic type

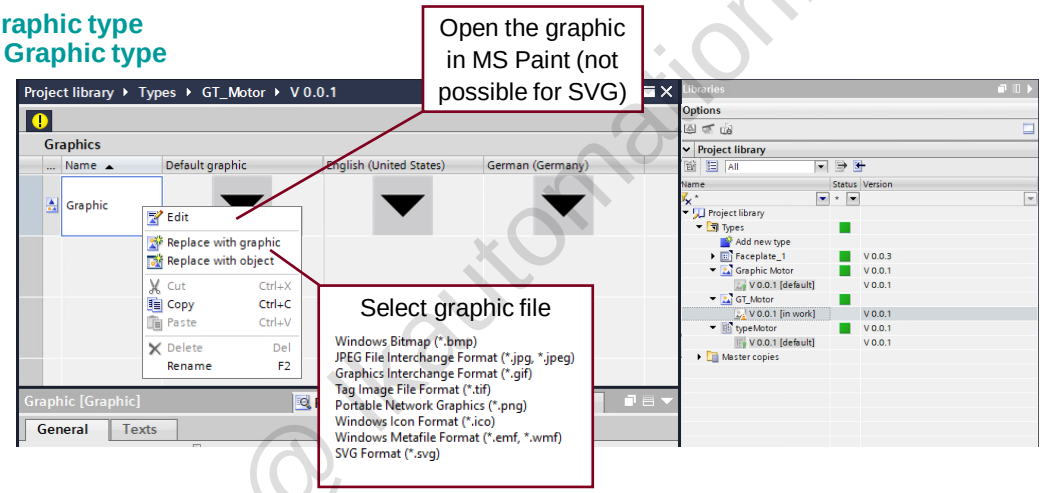
8.3.1 Add new Graphic type



SIMATIC WinCC Unified V18

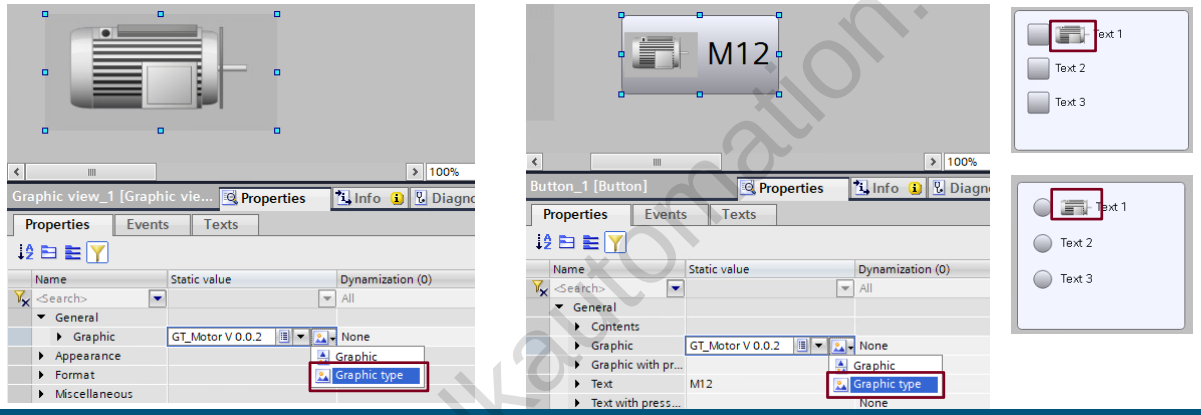
8.3.2 Edit a Graphic type

Edit a Graphic type
Using a Graphic type



If you change the Default graphic, it will be used for all languages.

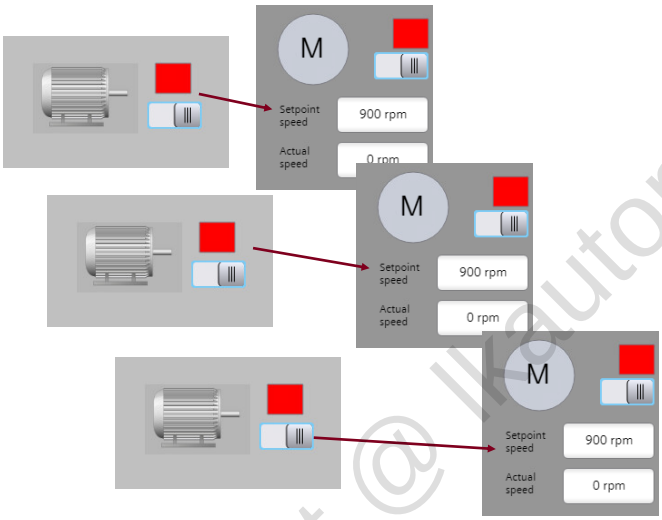
8.3.3 Graphic views and buttons



Graphic types can be used with Graphic views, Checkboxes, Radiobuttons and with buttons. The can not be used in Graphiclists.

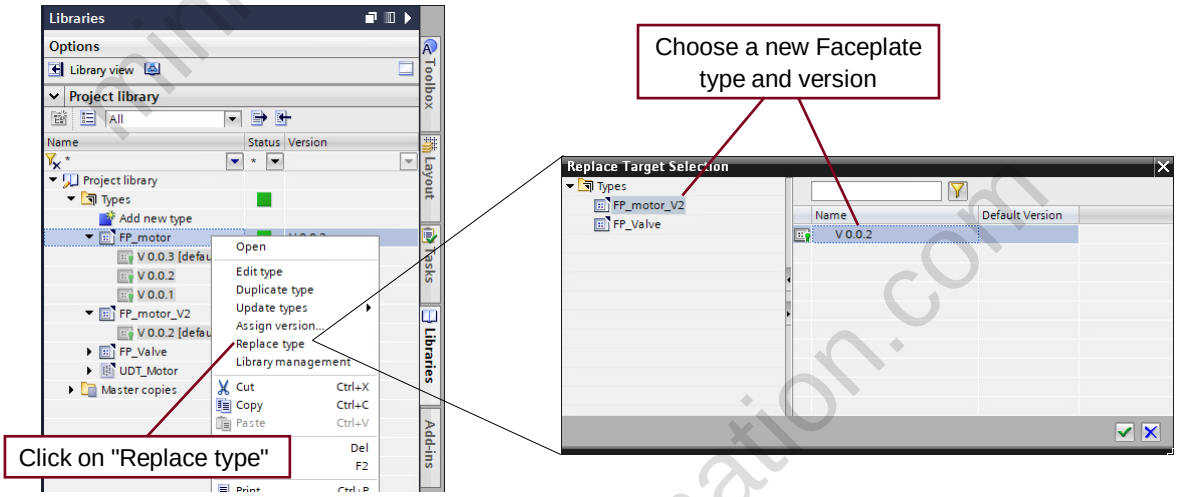
8.4 Replace Faceplates

8.4.1 Replace type

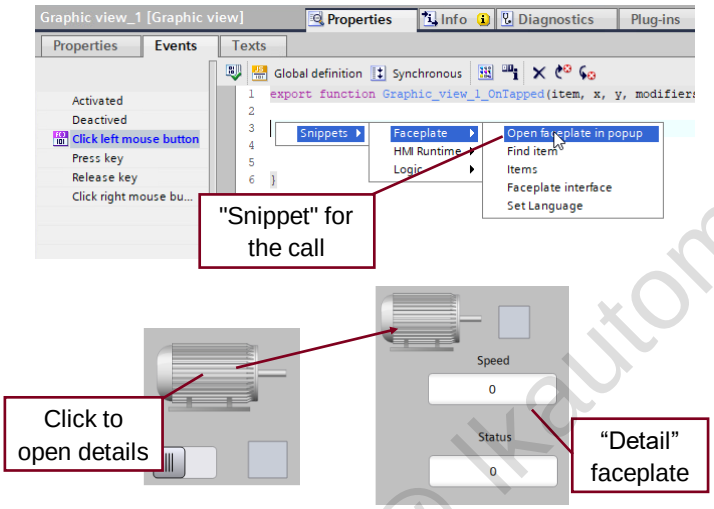


- **Task:** Replace all instances of one Faceplate Type by instances of another Faceplate type.

- **Efficient engineering:** Use central "Replace type" function. All instances will be changed.

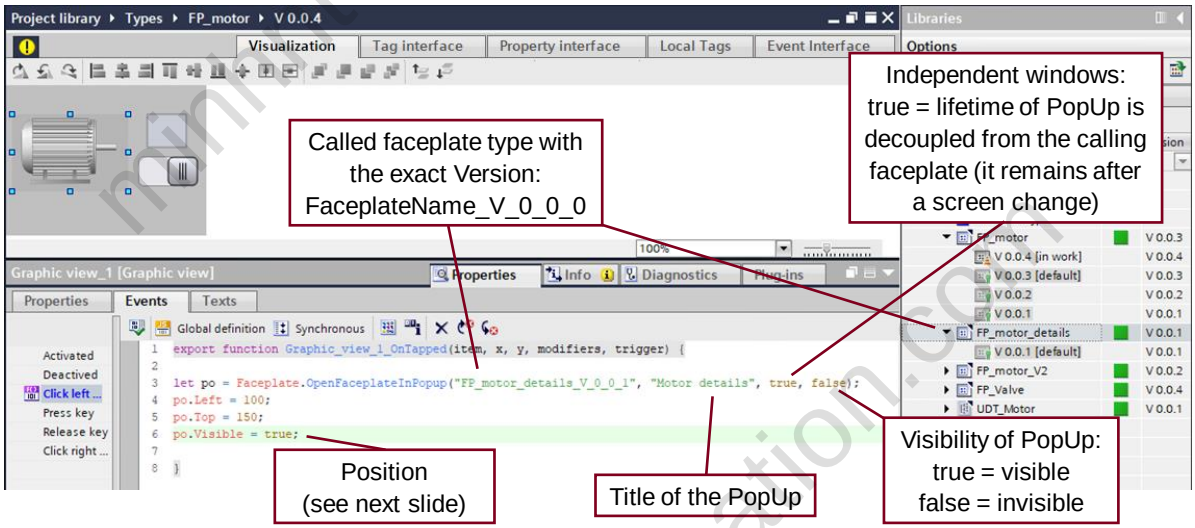


8.5 Faceplate opens another faceplate in popup

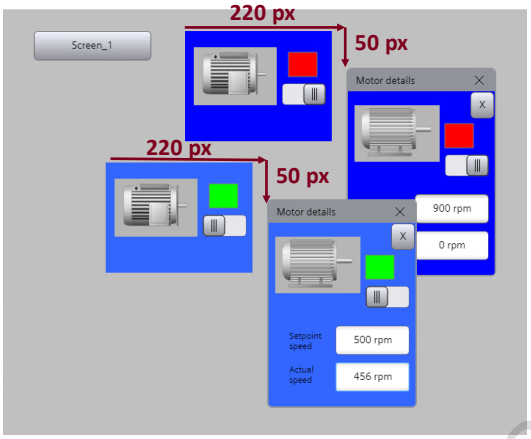


Faceplate Popups

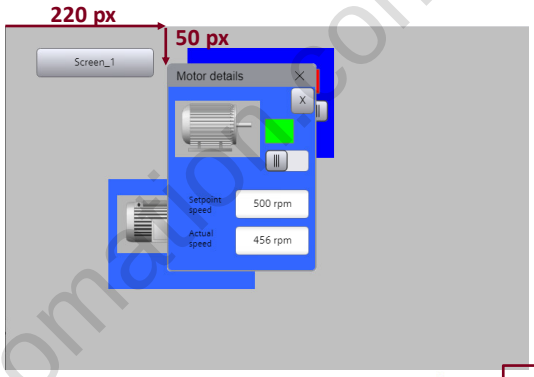
- Identical interfaces of icon and operation window
- Popup is created dynamically at runtime. No need to set placeholder objects.
- Faceplate interface is passed through at runtime



8.5.1 Independent window




```
let po = Faceplate.OpenFaceplateInPopup("FP_motor_d_V_0_0_3", "Motor details", false, false);
po.Left = 220;
po.Top = 50;
po.Visible = true;
```

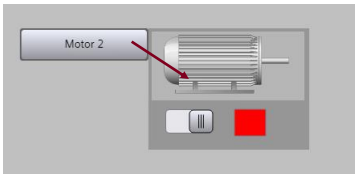



```
let po = Faceplate.OpenFaceplateInPopup("FP_motor_d_V_0_0_3", "Motor details", true, false);
po.Left = 220;
po.Top = 50;
po.Visible = true;
```

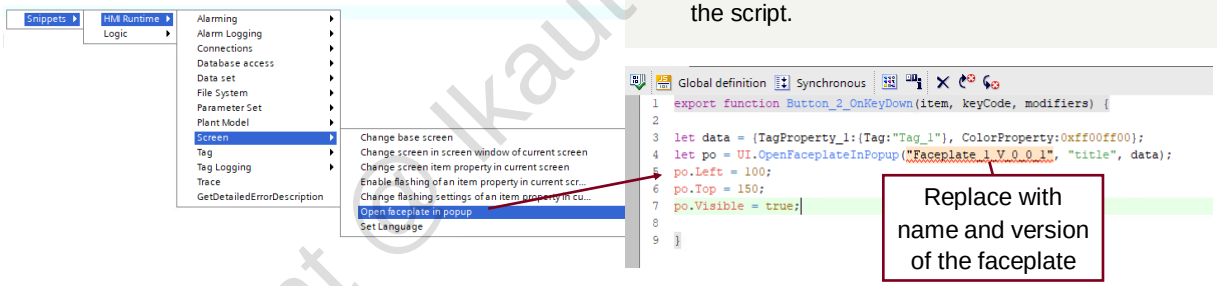
Independent windows

8.5.2 Screen object (Button) opens a faceplate in popup

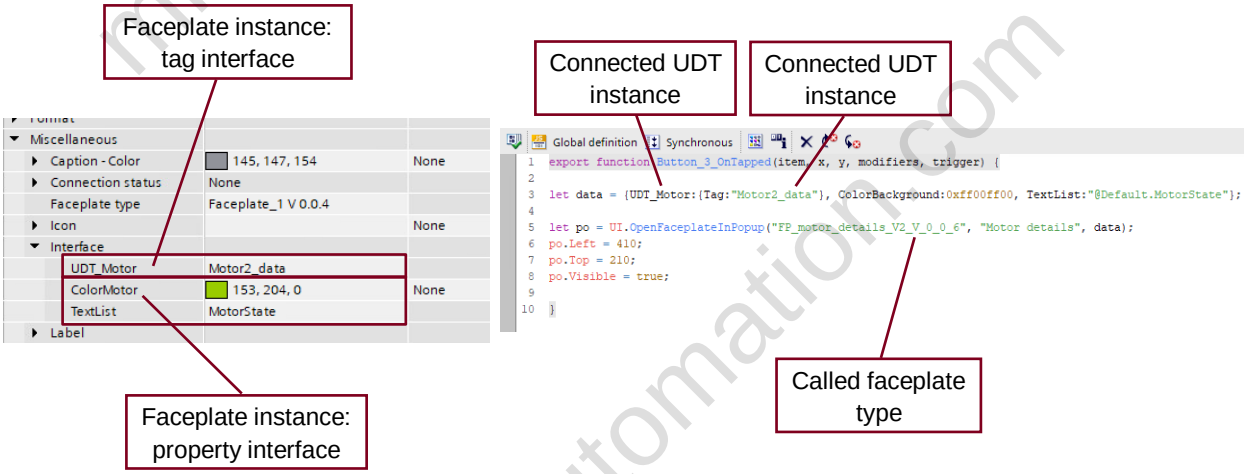
Screen object (Button) opens a faceplate in popup
Faceplate Popups



- It is also possible to call Faceplates as PopUp directly from the HMI screen (via button for example).
- For the call of a Faceplate in a PopUp window from a HMI screen a code snippet is available.
- The interface has to be manually connected in the script.

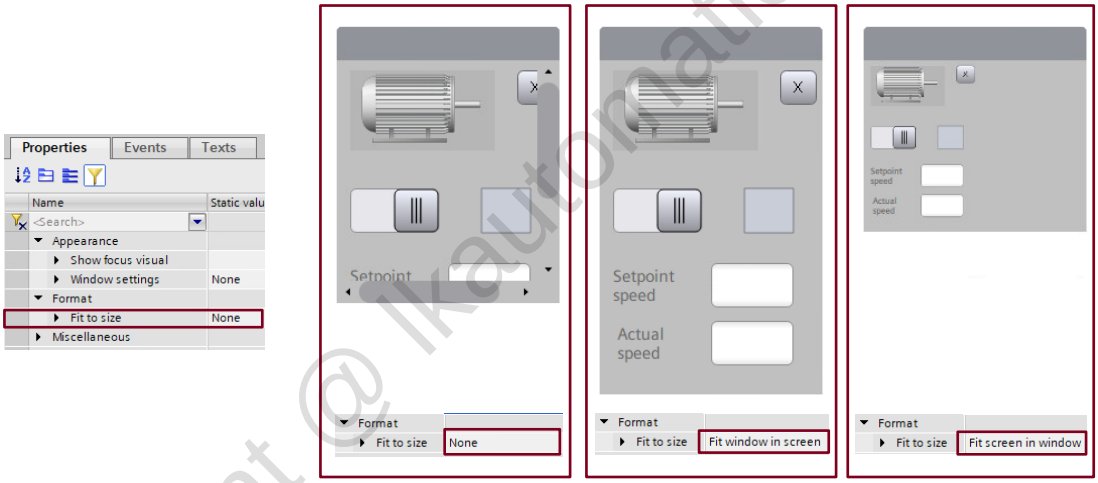


SIEMENS

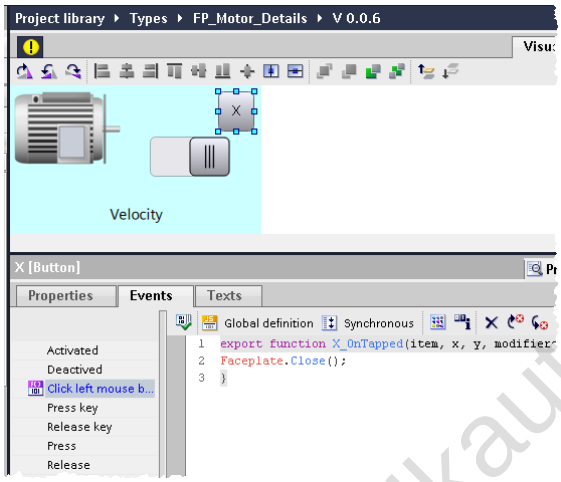


8.6 Faceplates container Properties

8.6.1 Window settings and Fit to size



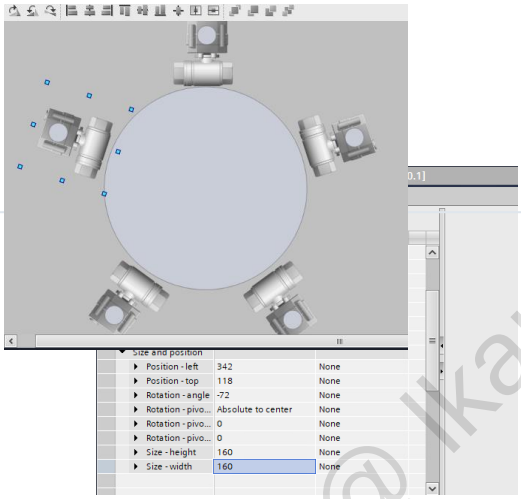
8.6.2 Close Faceplates instances with a separate button



Task:
Create a button for closing the faceplate instance inside the faceplate type.

Advantage:
Faceplate instance is totally deleted.

8.6.3 Rotation of Faceplate Instances



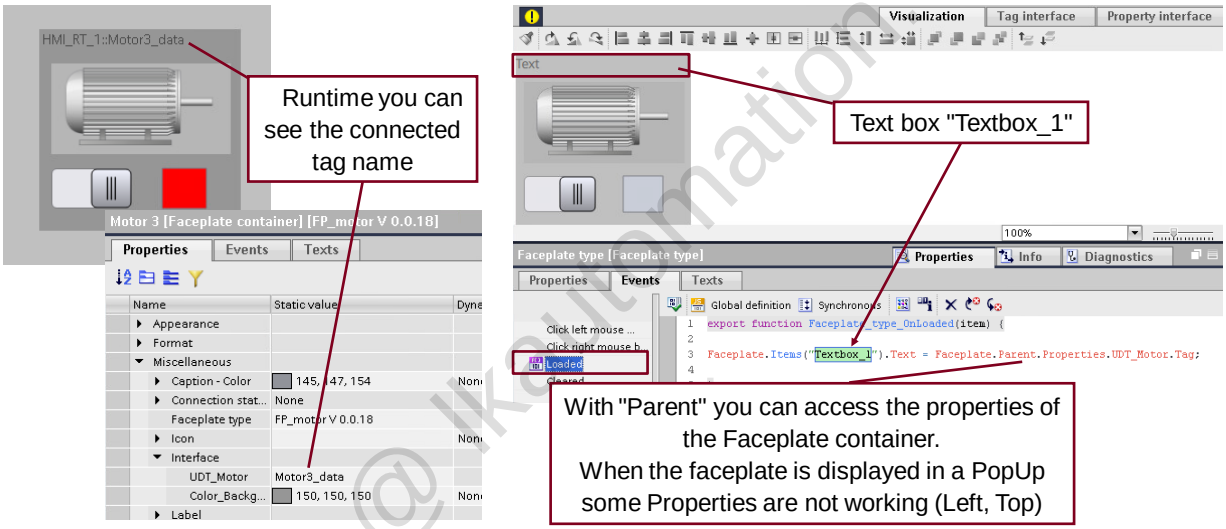
Rotating of Faceplate Instances

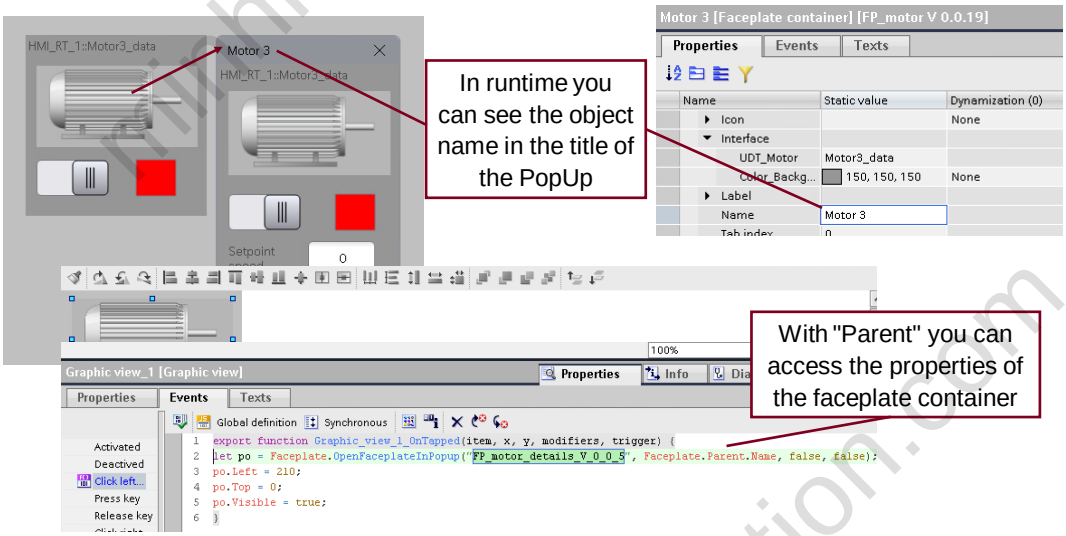
Faceplate Instances can rotate like standard screen item (Rotation)

Dynamize Rotation of Faceplate Instances

Rotation and Rotation Point can be dynamized via Tag or Script

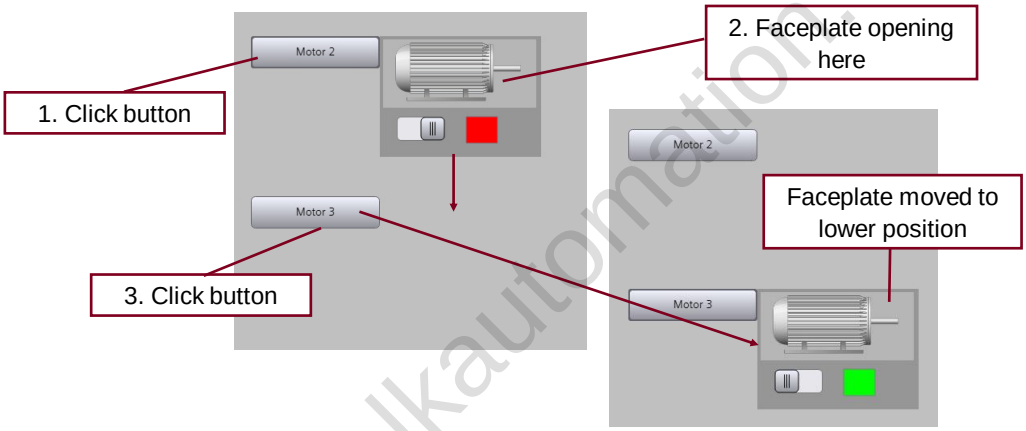
8.6.4 Parent Properties



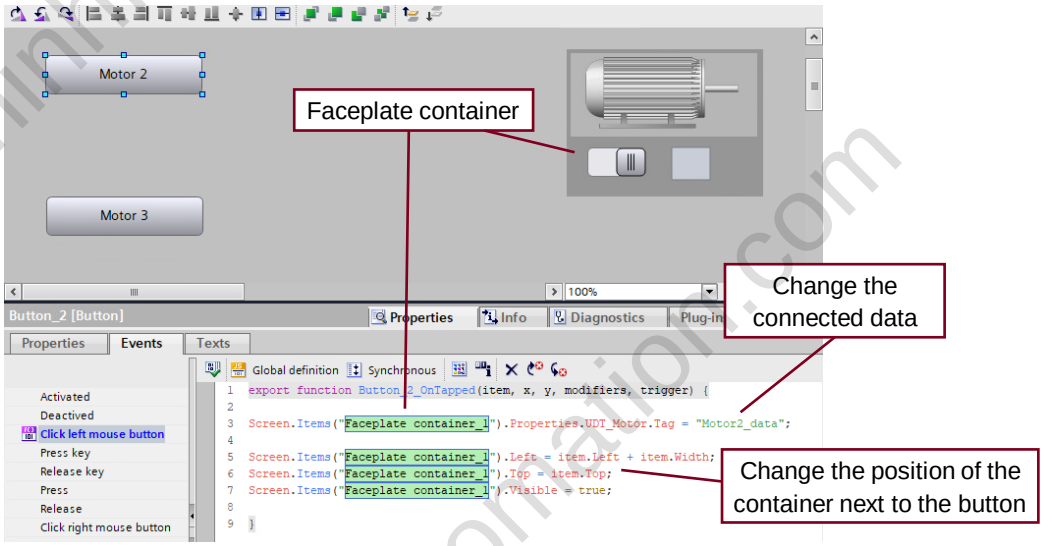


8.7 Tips and Tricks

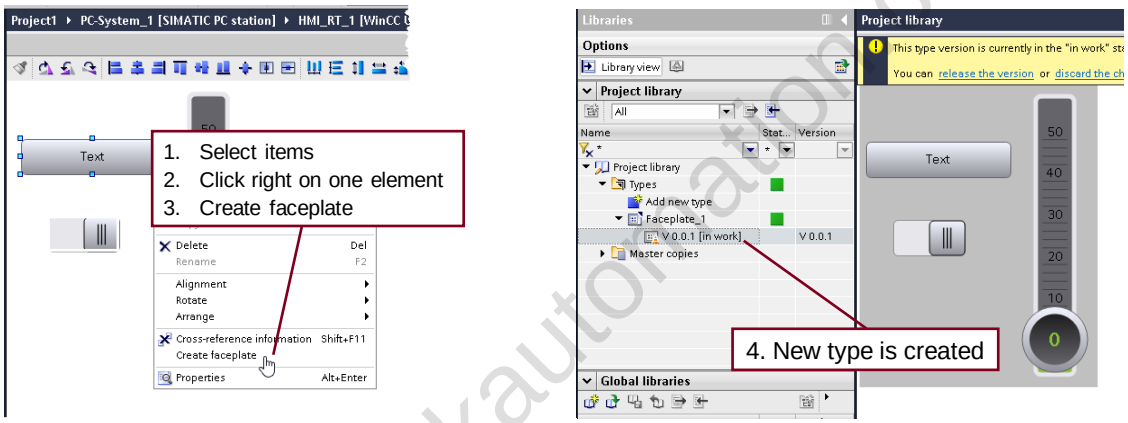
8.7.1 Change the Interface and the Position of a Faceplate Container in Runtime



Use one Faceplate container for more than one faceplate instance, e.g. "Motor 2" and "Motor 3".

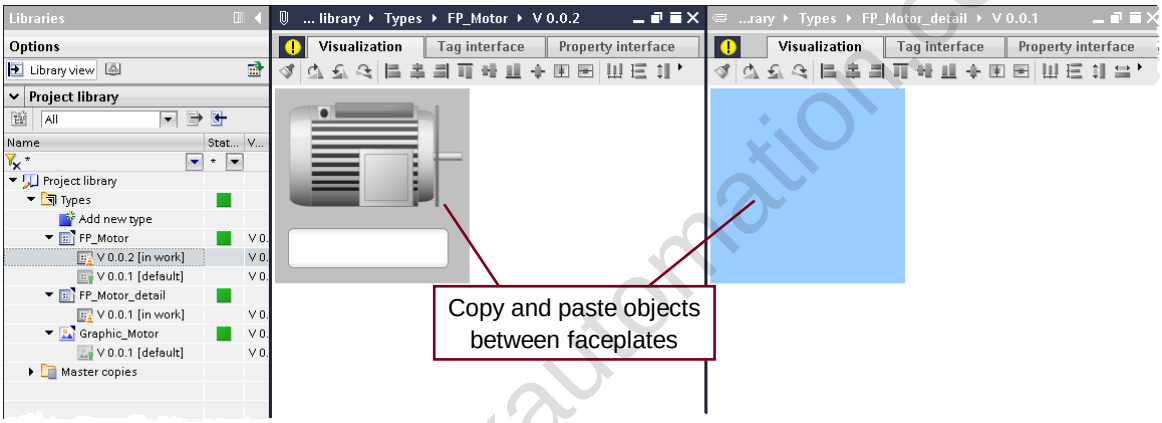


8.7.2 Create Faceplate from Screen



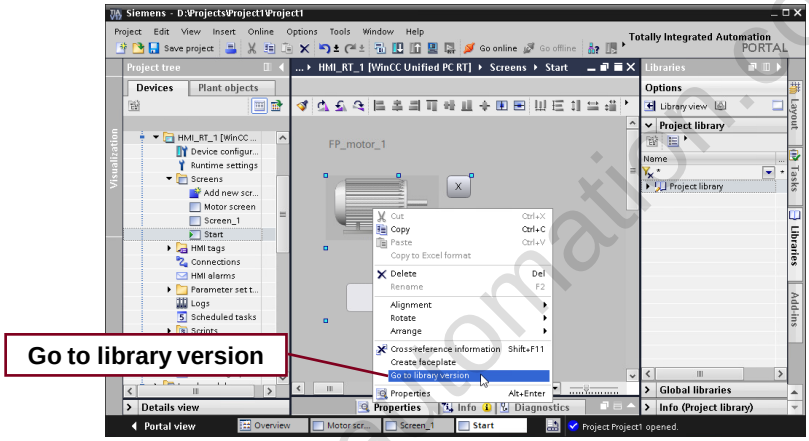
Note: only screen elements which are supported from the Faceplate can be used. Everything else is hidden

8.7.3 Copy Object between Faceplates



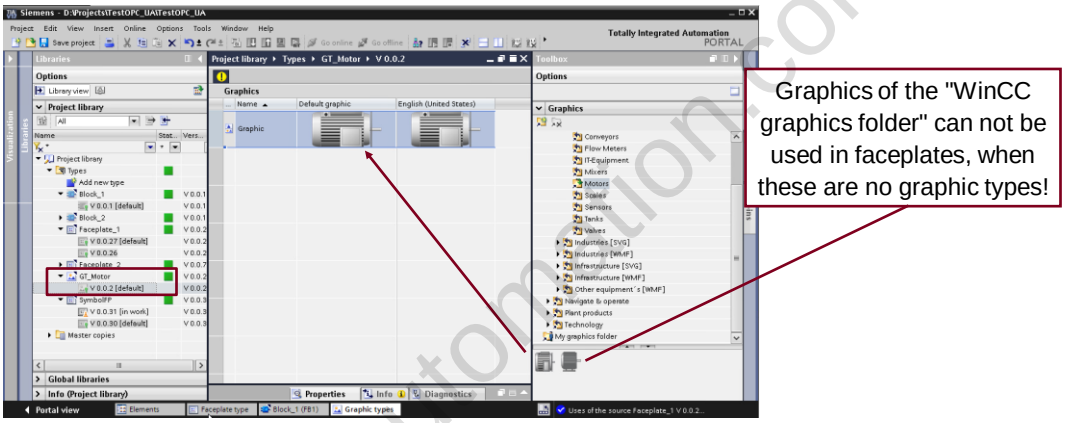
Via drag & drop you can move an object. Press CTRL + drag & drop to copy an object.

8.7.4 Open faceplates from screeneditor



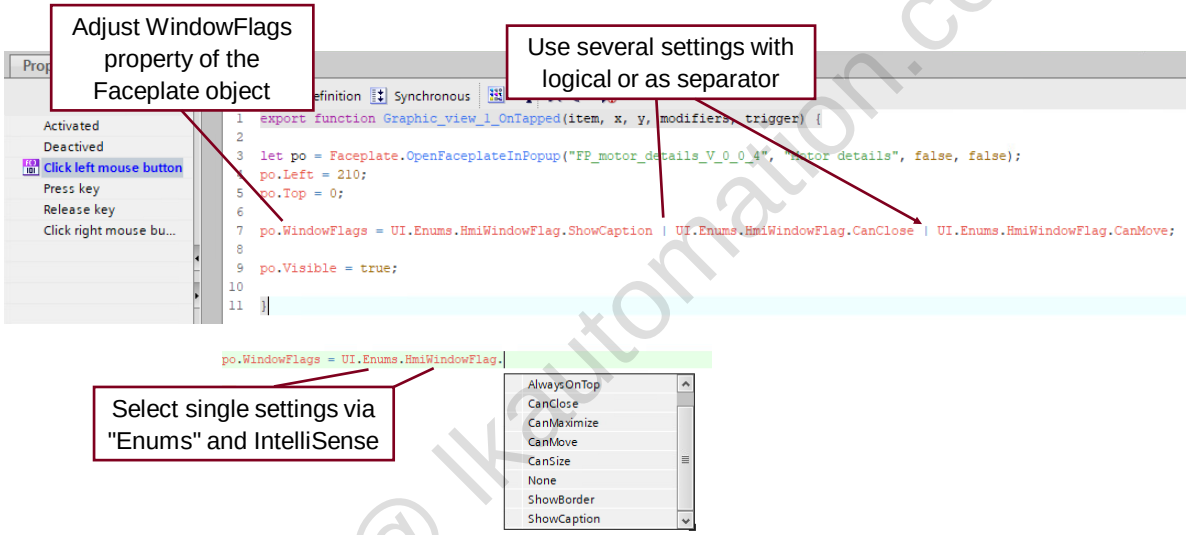
Go to library version highlights the version of the selected faceplate in the project library

8.7.5 Graphic views and buttons



To use graphics of the WinCC graphic folder in faceplates, create a graphic type of the graphic first. → Graphic types can be used in faceplates.

8.7.6 Faceplates instances in Popups - Window settings



8.8 Differences between Faceplate.Close() and Faceplate.Parent.Visible = False

```
Faceplate.Close();
```

Faceplate is **not existing** anymore after execution.

```
Faceplate.Parent.Visible = false;
```

Faceplate is **still existing**
Scripts of Faceplate are still executed!
To many executed scripts can be problematic for performance

Set a faceplate only invisible, when it is used furthermore!

→ Use Faceplate.Close() for deletion of the faceplate. Make invisible for further usage.

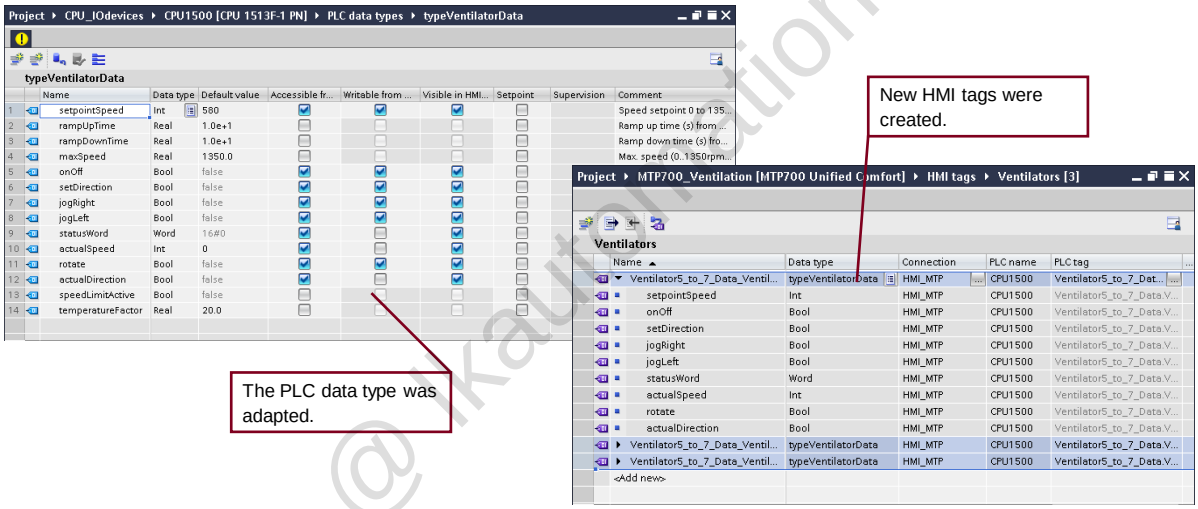
8.9 JavaScript Styleguide

Standardize your JavaScript Code

Use speaking names to make it **easy to read** countValves cVlvs	Use type specific compares to **avoid mistakes** === !== == !=	Structure and modularize your code to make it **reusable** (e.g. **use global modules**)	Do not use obsolete and remove unused functions to achieve a **long-lasting runtime** of the program	And much more …

→ Follow an unique programming style
https://support.industry.siemens.com/cs/ww/de/view/109758536

8.10 Exercise 1: Adapt PLC UDT and create HMI tags



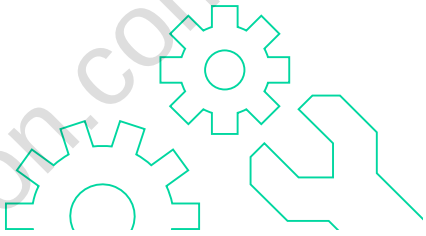
8.10.1 Procedure for exercise 1:

1. Adapt the PLC data type

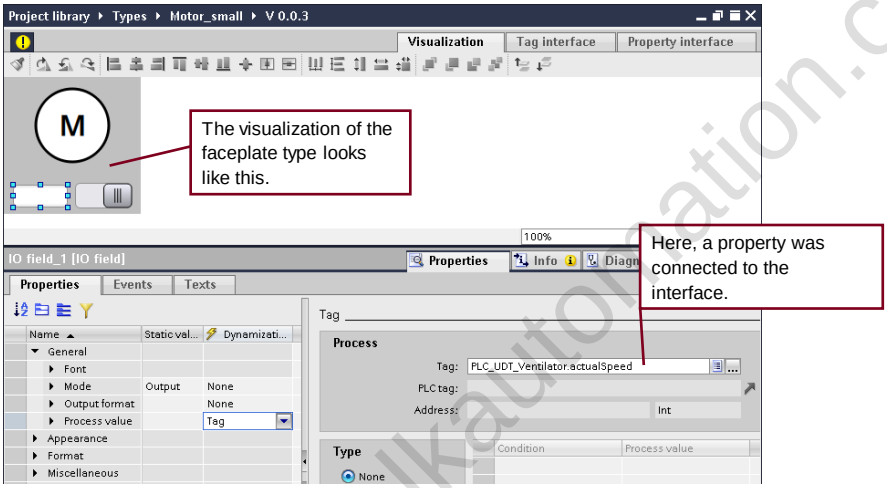
- Open the PLC data type, adapt it and release it as a new version
- Compile the PLC

2. Create tags for Ventilator 5 to Ventilator 7

- Create new tag table for Unified PC runtime
- Select data block with tags for Ventilators 5 to 7 and drag the tags from the Details view to the open tag table
- Save project

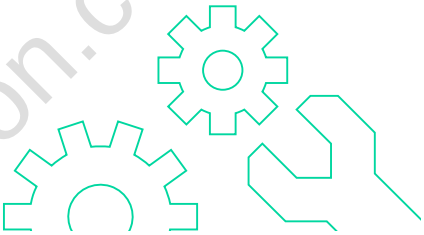


8.11 Exercise 2: Create and instantiate a faceplate type



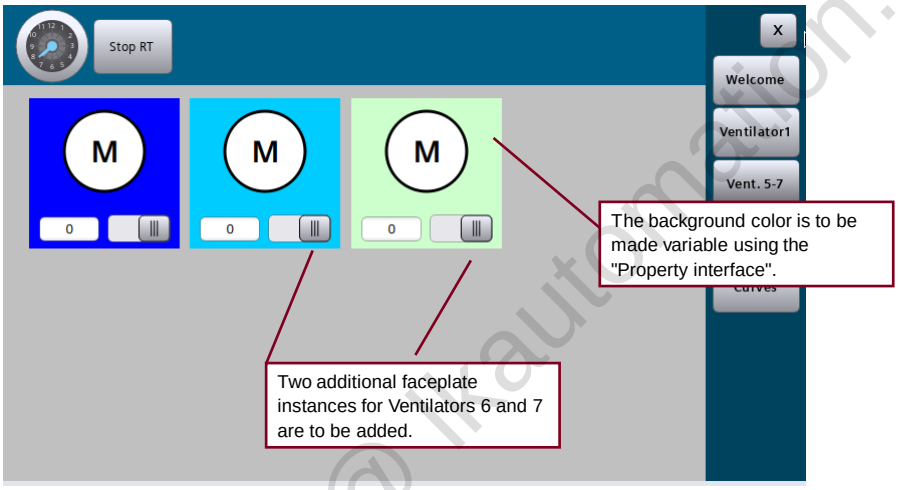
8.11.1 Procedure for exercise 2:

1. **Create new faceplate types**
   - Insert a new faceplate type and rename it to "Motor_small".
2. **Create visualization**
   - Adapt the size: Width: 150, Height 150.
   - Add the following objects: Circle, text field, I/O field and switch.
   - Adapt objects as in the screen (I/O field: Mode = "Output").
3. **Define tag interface**
   - Create a new interface "PLC_UDT_Ventilator".
   - Connect the interface to the user data type structure "typeVentilatorData".
4. **Connect object properties to interface**
   - Connect the I/O field to the structure element "actualSpeed".
   - Connect the switch to the structure element "onOff".



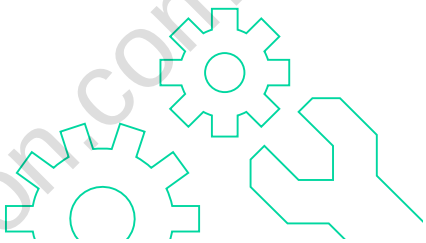
5. **Release new version of the faceplate type**
6. **Insert faceplate container in screen**
   Create a new "Ventilator_5-7" screen
   Add a button in the "Menu" screen for calling the new screen in runtime
   Insert an instance of the faceplate in the "Ventilator_5-7" screen
   Connect the instance of the faceplate to the instance of the user data type of Ventilator 5
   Test the new screen in the PC runtime

8.12 Exercise 3: Faceplate containers for additional ventilators

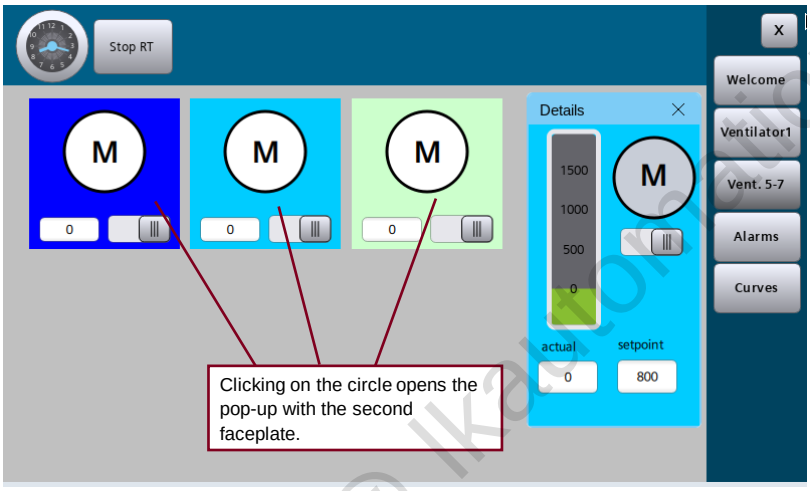


8.12.1 Procedure for exercise 3:

1. **Add background color to property interface**
   - Add "BackgroundColor" to the property interface for the faceplate type
   - Connect the background color of the faceplate type to the "BackgroundColor" interface.
2. **Add faceplate instances for Ventilators 6 and 7**
   - The background color of the faceplate instance can now be changed to blue
   - Insert additional faceplate instances for Ventilators 6 and 7.
   - The background colors should be light blue and light green here.

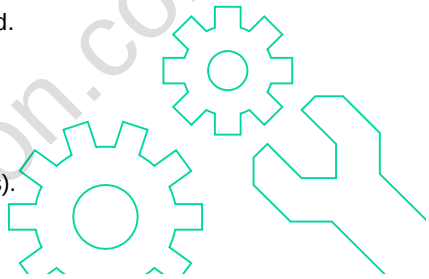


8.13 Exercise 4: Call a faceplate as a pop-up



8.13.1 Procedure for exercise 4:

1. **Create a second faceplate type**
   - Create a second faceplate type by duplicating "Motor_small".
   - Name it "Motor_details".
2. **Expand visualization**
   - Adapt the size: Width: 170, Height 300
   - Add additional objects: e.g. I/O field for setpoint speed and bar for current speed.
   Suggestion: See notes
3. **Interface remains the same**
   - The tag interface and the property interface should remain unchanged.
4. **Connect object properties to interface**
   - Connect a second I/O field to the structure element "setpointSpeed".
   - Connect the bar to the structure element "actualSpeed".
5. **Open a second faceplate as a pop-up**
   - Expand the Java script in the "Motor_small" faceplate type (see notes).
   - Update the instances of "Motor_small" to the new version.
   - Download the project to your Panel and check the functionality



8.14 Detailed exercise description

8.14.1 Exercise 1 (Adapt the PLC UDT)

1. Adapt PLC data type

→ Open the PLC data type, adapt it and release it as a new version.

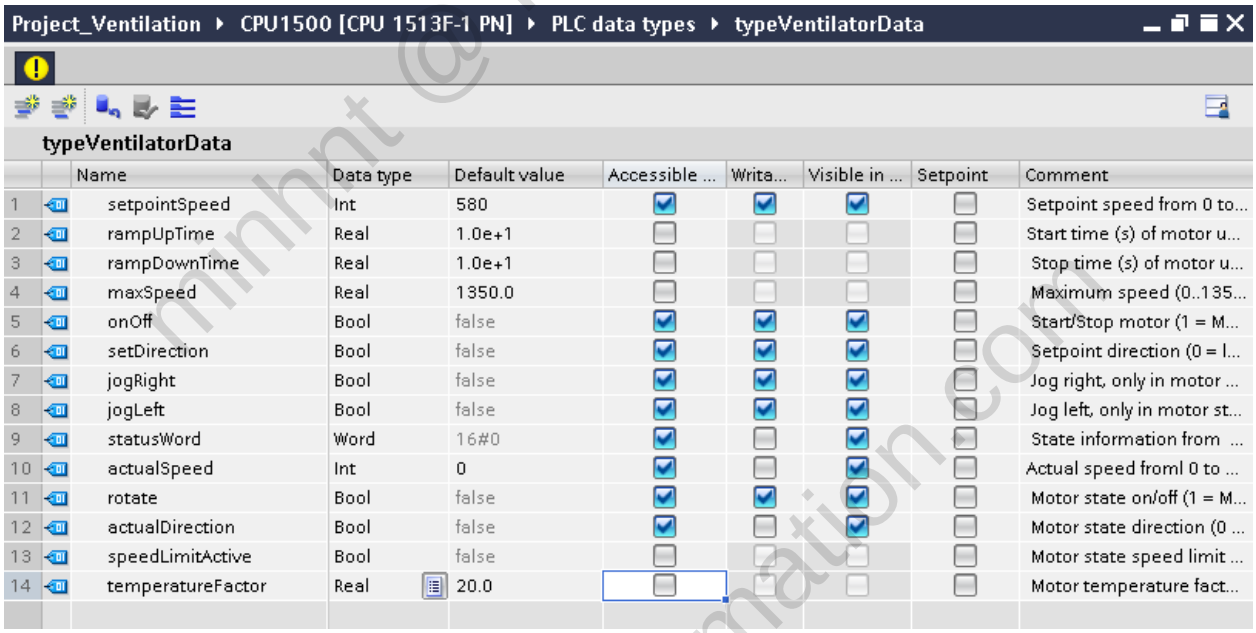
→ Compile the PLC program.

Notes:

Project tree > Project > CPU1500 > PLC data types > typeVentilatorData
Double-click the data type to open it.
A notice is displayed: "The editor is write-protected because it is connected with a type in the library. To make changes, you must edit the type." If you do not receive a notice, click on the yellow exclamation mark.

Click on the underlined text.

Adapt the data type as shown in the following figure:



	Name	Data type	Default value	Accessible ...	Writa...	Visible in ...	Setpoint	Comment
1	setpointSpeed	Int	580	☑	☑	☑	☐	Setpoint speed from 0 to...
2	rampUpTime	Real	1.0e+1	☐	☐	☐	☐	Start time (s) of motor u...
3	rampDownTime	Real	1.0e+1	☐	☐	☐	☐	Stop time (s) of motor u...
4	maxSpeed	Real	1350.0	☐	☐	☐	☐	Maximum speed (0..135...
5	onOff	Bool	false	☑	☑	☑	☐	Start/Stop motor (1 = M...
6	setDirection	Bool	false	☑	☑	☑	☐	Setpoint direction (0 = l...
7	jogRight	Bool	false	☑	☑	☑	☐	Jog right, only in motor ...
8	jogLeft	Bool	false	☑	☑	☑	☐	Jog left, only in motor st...
9	statusWord	Word	16#0	☑	☐	☑	☐	State information from ...
10	actualSpeed	Int	0	☑	☐	☑	☐	Actual speed froml 0 to ...
11	rotate	Bool	false	☑	☑	☑	☐	Motor state on/off (1 = M...
12	actualDirection	Bool	false	☑	☐	☑	☐	Motor state direction (0 ...
13	speedLimitActive	Bool	false	☐	☐	☐	☐	Motor state speed limit ...
14	temperatureFactor	Real	20.0	☐	☐	☐	☐	Motor temperature fact...

After making the adaptations, the data type must be released. To do this, click on the yellow icon with exclamation mark and click "Release the version". The version is incremented automatically.

Since the data type is used in the PLC program, you still have to recompile the PLC programming. To do this, select the CPU1500 and then select the "Compile" button from the toolbar.

2. Create tags for Ventilator 5 to Ventilator 7

→ Create a new tag table for PC runtime.

→ Select the data block with tags for Ventilators 5 to 7 and drag the tags from the Details view to the opened tag table.

Notes:

Project tree > Project > HMI tags > "Add new tag table"

Rename the tag table to "Ventilators" via the shortcut menu.

Double-click to open it.

Project tree > Project > CPU1500 > Program blocks > Select (but do not open) "Ventilator5_to_7_Data". Expand the Details view in the project tree. Select the three rows containing "VentilatorArray[5]" to "VentilatorArray[7]" and use drag-and-drop to move them into the open "Ventilators" tag table.

Save project.

8.14.2 Exercise 2 (Create and instantiate a faceplate type)

1. Create new faceplate types

→ Add a new faceplate type.

→ Rename it to "Motor_small".

Notes:

Libraries > Project library > Types
Double-click on "Add new type".

Select HMI faceplate > "Unified Comfort Panel / WinCC Unified PC".
Change the name of "Faceplate_1" to "Motor_small".
Click the "OK" button.

2. Create visualization

→ Adapt the size: Width: 150, Height 150.

→ Add the following objects: Circle, text field, I/O field and switch.

→ Adapt the objects as shown in the figure (I/O field: Mode = "Output").

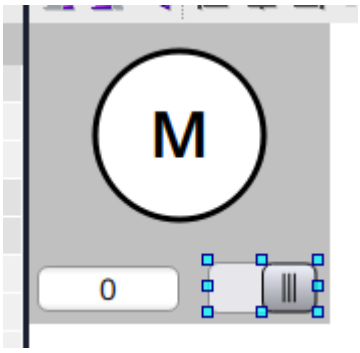
Notes:

Go to the "Visualization" tab in the work area.
Change the following values in the properties:
"Size and position > Width": 150
"Size and position > Height": 150

Insert the following objects:

Circle, text field, I/O field and switch

Appearance and layout should be adapted as follows:



Additionally, change the property "General > Mode" of the I/O field to "Output".

3. Define tag interface

→ Create a new interface "PLC_UDT_Ventilator".

→ Connect it to the user data type structure "typeVentilatorData".

Notes:

Go to the "Tag interface" tab in the work area.

Double-click on "Add" to add a new interface.

Change the name to "PLC_UDT_Ventilator".

Change the data type to "PLCUDT".

Select "typeVentilatorData" as the user data type structure.

4. Connect object properties to interface

→ Connect the I/O field to the structure element "actualSpeed".

→ Connect the switch to the structure element "onOff".

Notes:

Go to the "Visualization" tab in the work area.
Select the I/O field.
Under Property "General > Process value", change the value in the "Dynamization" column from "None" to "Tag". You can then connect an element of the user data type structure "typeVentilatorData" for "Tag" in the right part of the Inspector window. Select the "actualSpeed" element.

Select the switch.
Under Property "General > Switch state", change the value in the "Dynamization" column from "None" to "Tag". You can then connect an element of the user data type structure "typeVentilatorData" for "Tag" in the right part of the Inspector window. Select the "onOff" element.

Additional exercise:
Dynamize the background color of the circle as a function of the actual speed.

5. Release the new version of the faceplate type

Notes:

Open the note window by clicking on the exclamation mark icon: 
Click "Release the version".
Click "OK".

6. Insert faceplate container into screen

→ Create a new screen "Ventilator_5-7".

→ Add a button in the "Menu" screen to be able to call the new screen in runtime.

→ Insert an instance of the faceplate into the "Ventilator_5-7" screen.

→ Connect the instance of the faceplate to the instance of the user data type of Ventilator 5.

→ Test the new screen in the Panel runtime.

Notes:

Create a new screen and rename it to "Ventilator_5-7". The height is 400 pixels, and the width is 800 pixels.

In the "Menu" screen, copy an existing button and adapt the label and event. This should open the new screen "Ventilator_5-7" in runtime.

Libraries > Project library > Types
Using drag-and-drop, move the "Motor_small" faceplate type into the new "Ventilator_5-7" screen.

Select the inserted faceplate instance (faceplate container).
Under Property "Miscellaneous > Interface > PLC_UDT_Ventilator", select "Ventilator5_to_7_Data_VentilatorArray{5}".

Save your project.

Compile the project, load it into your PC station and check the new screen.

8.14.3 Exercise 3 (Faceplate containers for additional ventilators)

1. Add background color to property interface

→ Add "BackgroundColor" to the interface properties for the faceplate type.

→ Connect the background color of the faceplate type to the "BackgroundColor" interface.

Notes:

Libraries > Project library > Types
Double-click "Motor_small" to open the faceplate type.

Optional, if the faceplate is not already in edit mode:
Open the note window by clicking on the exclamation mark icon: 
Click "Edit the type".

Go to the "Property interface" tab in the work area.

Add a new property and name it "BackgroundColor".
The data type is "Color".

Go to the "Visualization" tab in the work area. Do not select any of the objects, click on the background instead.

Change the following values in the properties:
"Appearance > Background color": Set the "Dynamization" column to "Property interface" and then select "BackgroundColor" for the name in the right part of the Inspector window.

Click the note window by clicking the exclamation mark icon: 
Click "Release the version".
Click "OK".

2. Add faceplate instances for Ventilators 6 and 7

→ You can now change the background color of the faceplate instance to blue.

→ Add additional faceplate instances for Ventilators 6 and 7. Their background colors are to be light blue and light green.

Notes:

Open the "Ventilator_5-7" screen.
Select the faceplate instance (faceplate container).
Under Property "Miscellaneous > Interface > Background color", select the color blue (0; 0; 255).
Note:

The change of background color is first displayed in runtime.

Copy the faceplate instance and paste it twice in the screen.

For the first copy, change the "Miscellaneous > Interface > PLC_UDT_Ventilator" property to "Ventilator5_to_7_Data_VentilatorArray{6}" and the "Miscellaneous > Interface > BackgroundColor" property to light-blue (0; 204; 255).

For the second copy, change the "Miscellaneous > Interface > PLC_UDT_Ventilator" property to "Ventilator5_to_7_Data_VentilatorArray{7}" and the "Miscellaneous > Interface > BackgroundColor" property to light green (204; 255; 204).

Save your project.

Compile the project, load it into your PC station and check the new screen.

8.14.4 Exercise 4 (Call a faceplate as a pop-up)

1. Create a second faceplate type

→ Create a second faceplate type by duplicating "Motor_small".

→ Rename it to "Motor_details".

Notes:

Libraries > Project library > Types
Create a copy of "Motor_small": Right-click "Motor_small" > "Duplicate type"

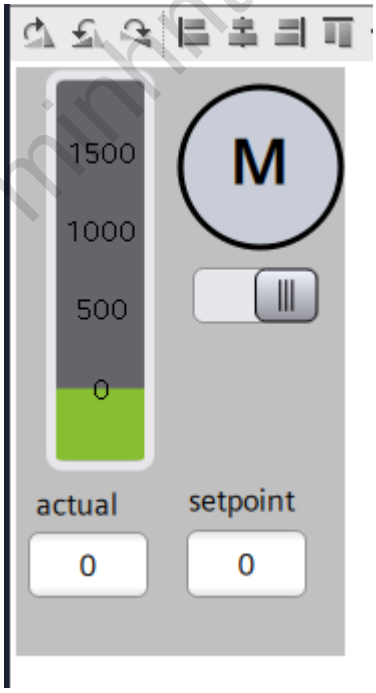
Change the name of the copy to "Motor_details".

Then open "Motor_details" by right-clicking > "Edit type".

2. Expand visualization

→ Adapt the size: Width: 170, Height 300

→ Add additional objects: e.g. I/O field for setpoint speed and bar for current speed.
Suggestion: See Fig.



Properties of bar:
General > Adjust linear scale
Miscellaneous > Process value indicator mode: Bar

3. Interface stays the same

→ The tag interface and the property interface should remain unchanged.

4. Connect object properties to interface

→ Connect a second I/O field to the structure element "setpointSpeed".

→ Connect the bar to the structure element "actualSpeed".

Notes:

Go to the "Visualization" tab in the work area.
Connect the properties to the interface similar to Step 4 in Exercise 2.

5. Call the second faceplate as a pop-up

→ Add the JavaScript in faceplate type "Motor_small" (see notes).

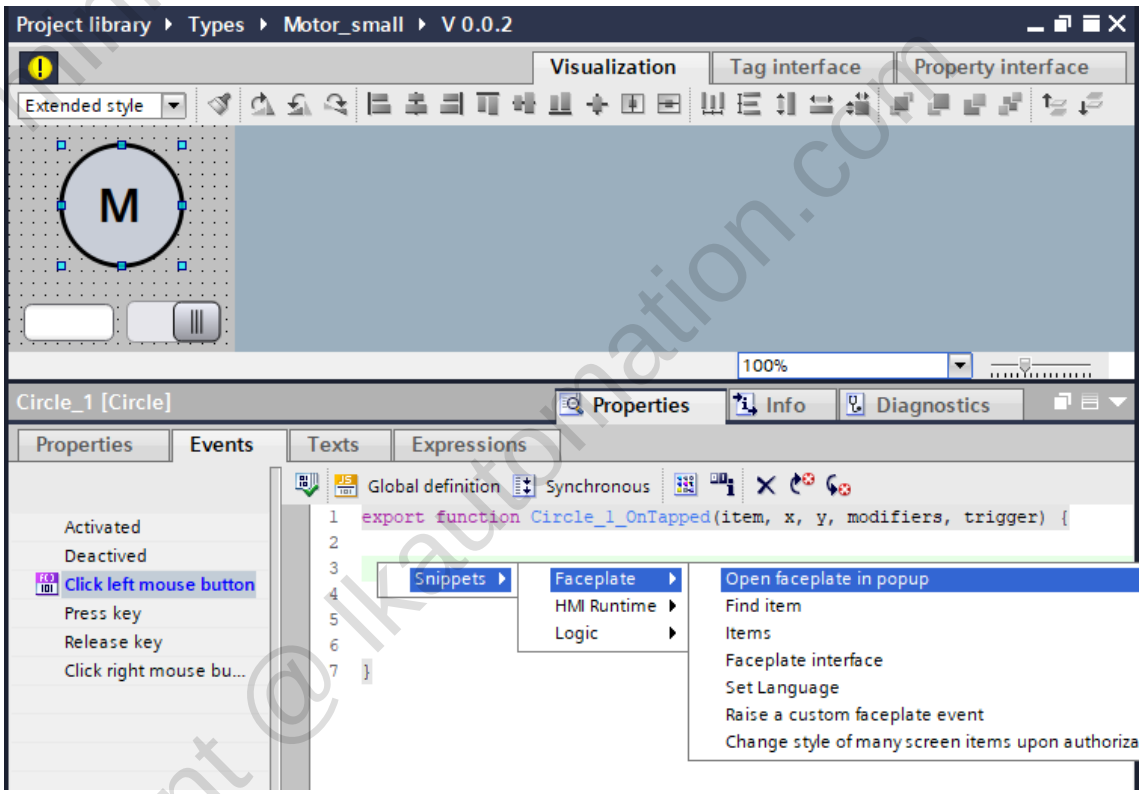
→ Download the project to your Panel and check the functionality.

Notes:

Stay in the editor for Unified faceplate types. The following changes are made to the **"Motor_small"** faceplate type.
Select the circle and open the "Events" tab in the Inspector window.

Select the "Click left mouse button" event and convert the function list to a JavaScript with the "JS" button.

Use a snippet to create the script:



Then adapt the script manually:
Use Ctrl + J to select the faceplate and its version:

```
export function Circle_1_OnTapped(item, x, y, modifiers, trigger) {

let po = Faceplate.OpenFaceplateInPopup("Motor_details_V_0_0_2", "Details", false, false);
po.Left = 10;
po.Top = 10;
po.Visible = true;

}
```

Copy the script and paste it also for the "M" text field.

Open the note window by clicking on the exclamation mark icon: 
Click "Release the version".
Click "OK".

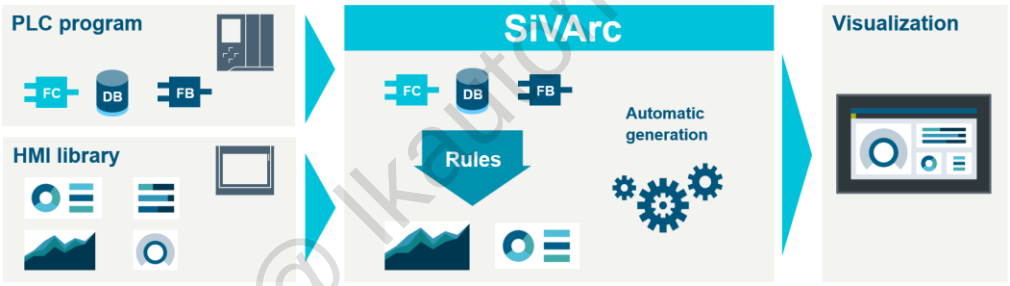
Save your project.

Compile the project, load it into your PC station and check the new screen.

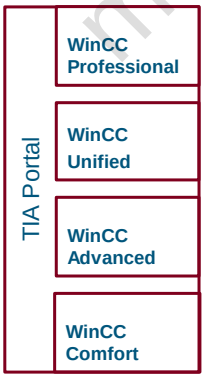
9 SiVArc

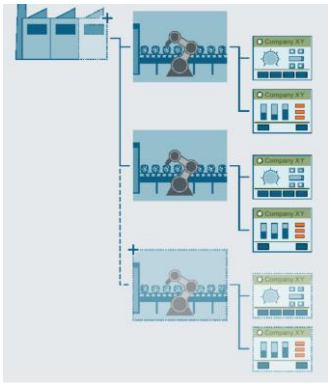
9.1 Overview

SiVArc is an option package in the TIA Portal.
You use generation rules to specify which HMI objects are generated for which blocks and devices.



SiVArc automatically generates your visualization.

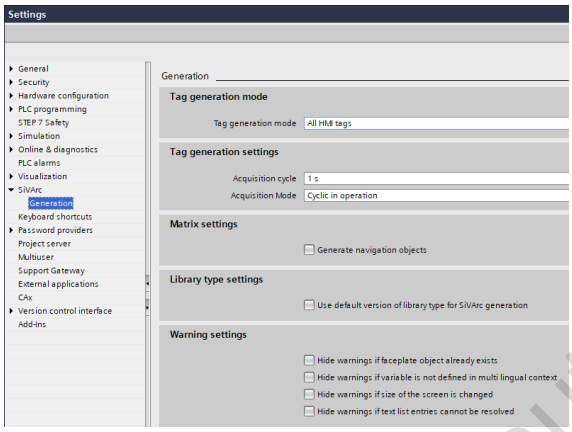




- Automatic generation of the visualization including process connection
- Uniform layout of user interfaces
- Consistent naming of operating elements
- Structured storage of configuration data

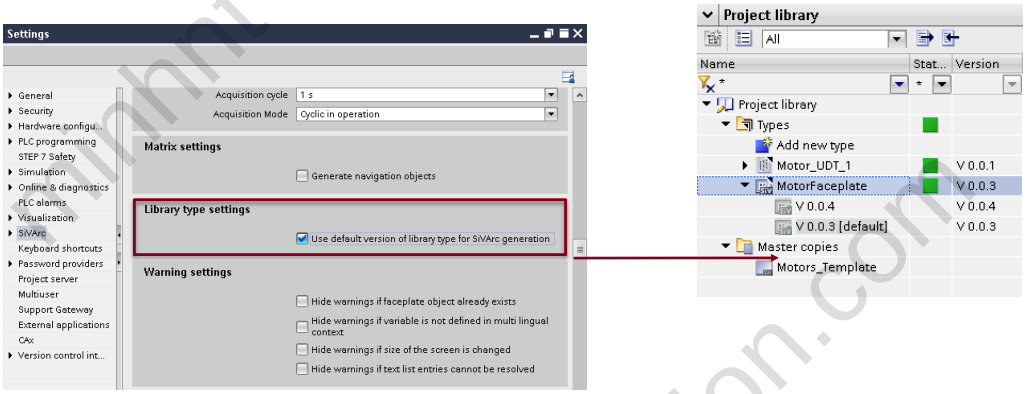
SiVArc promotes standardization in your project.

9.2 Basic settings



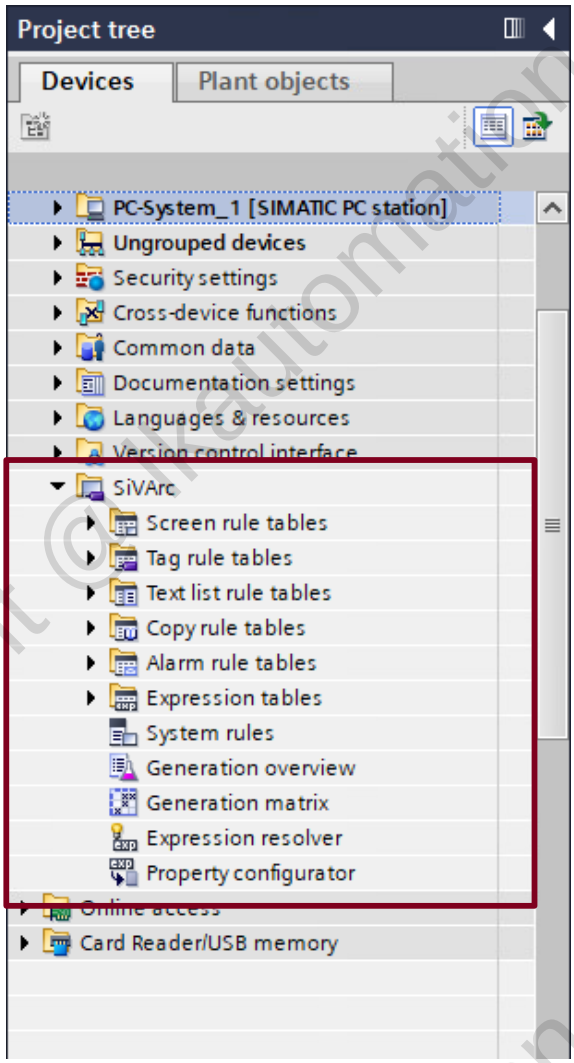
- Generation settings of the visualization including process connection.
- Acquisition cycle of user interfaces.
- Warning settings customize which warnings will be displayed.

SiVArc settings can be edited under TIA Portal Options > Settings.

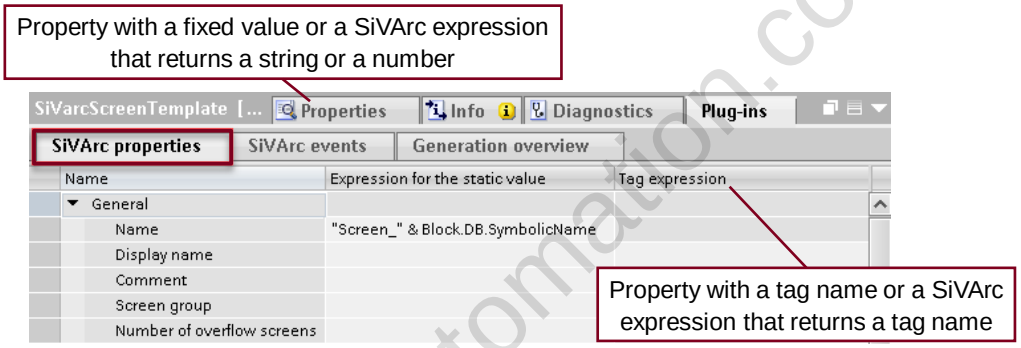


Select for SiVArc generation if you want to use the default version of library type.

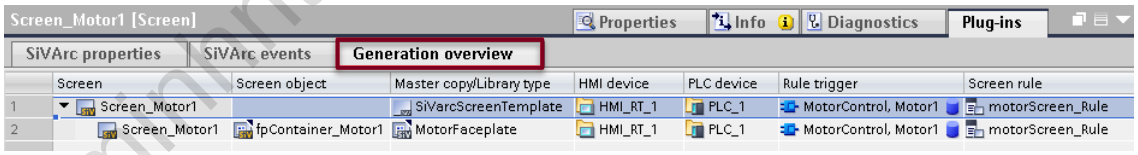
9.3 Editors



SiVArc editors are located in the project tree.

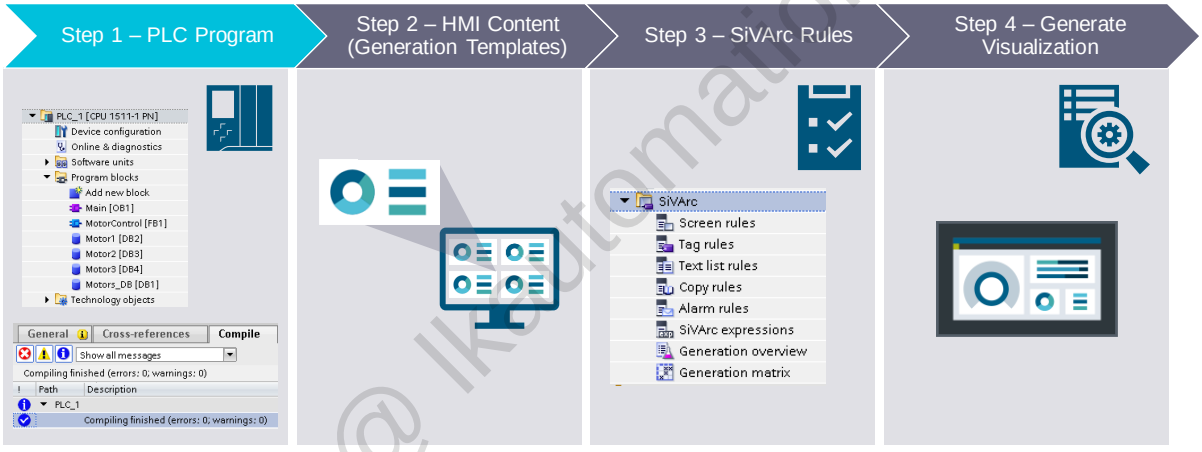


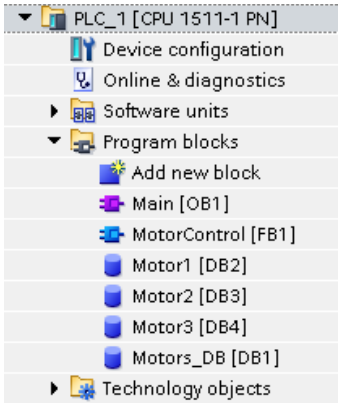
A SiVArc property is an object property that you configure either statically or dynamically with a SiVArc expression.



Displays which objects were actually generated in the active screen.

9.4 PLC Program

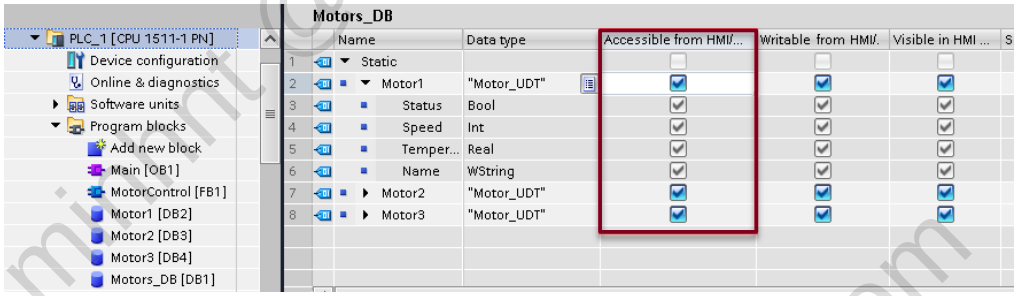




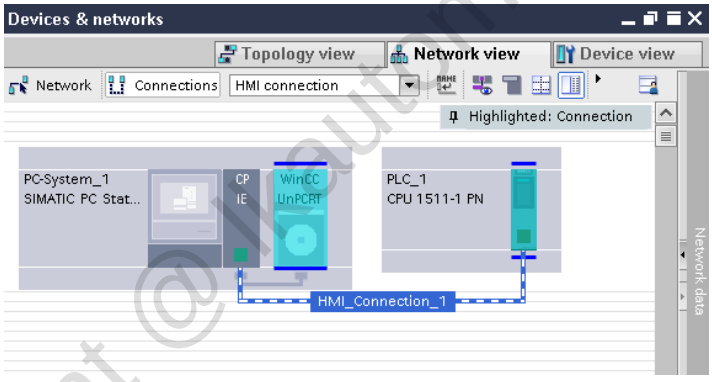
Supported objects:

- FC and (FB) in LAD, FBD, STL and SCL and global and instance DB.
- All elementary data types that the operator panel can display and the Array, Struct and UDT data types.

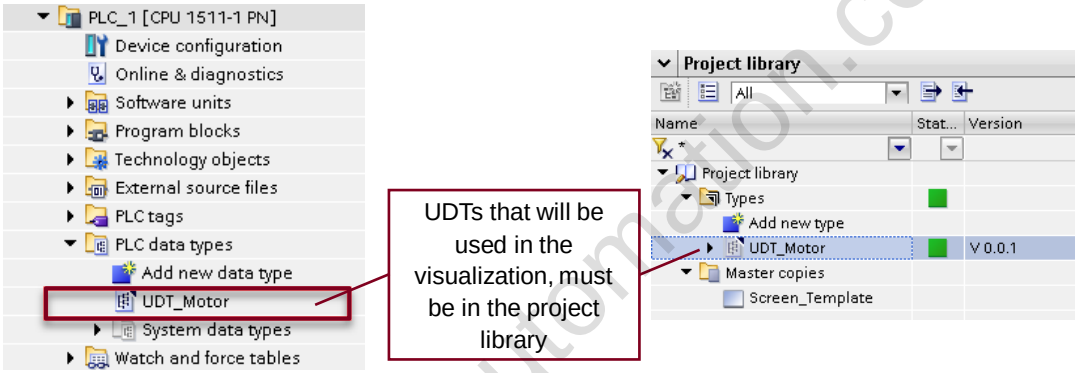
PLC must be compiled without errors before SiVArc generation.



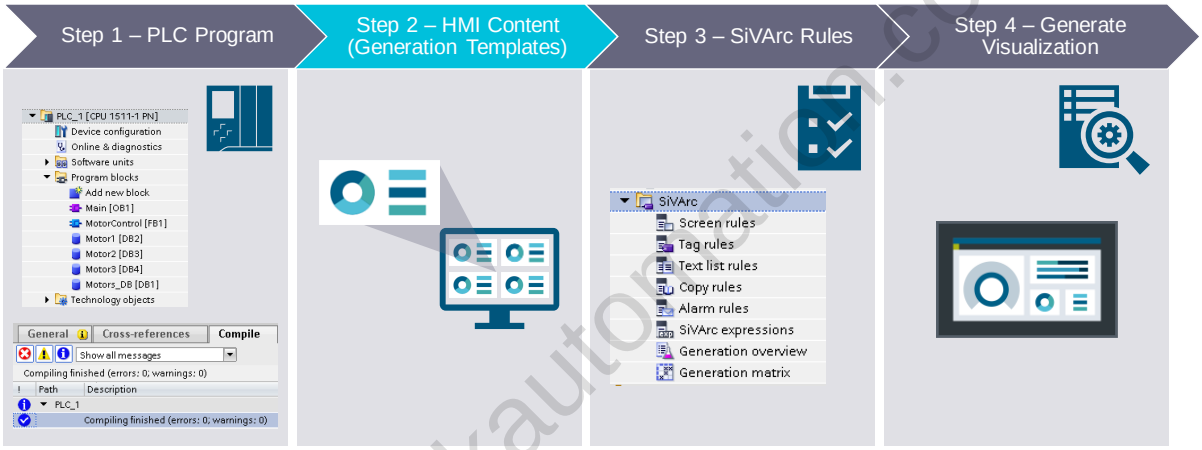
Set PLC tags to "Accessible from HMI".

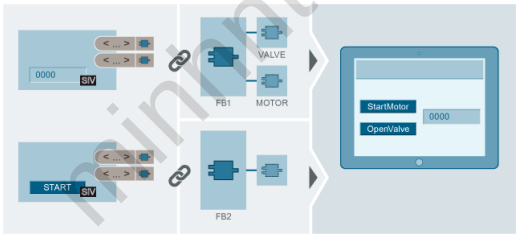


An "HMI connection" must be established.



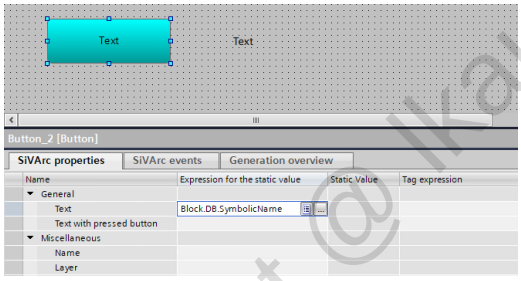
9.4.1 Generation Templates





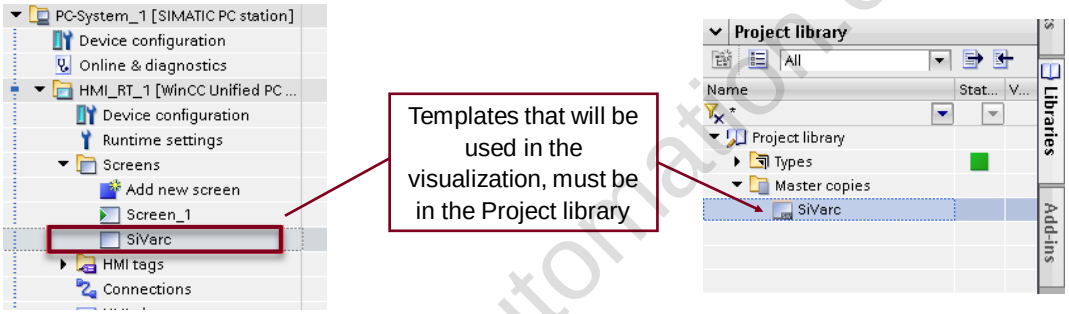
- Generation templates are HMI objects from the library which are not only configured with fixed defined WinCC properties but also with **SiVArc properties**.
- A SiVArc property is an object property, which is first assigned as **tag/expression**.
- SiVArc properties are only filled with texts, such as the **object name, labels or a tag** designation during the generation.

"SiVArc properties" tab is only available for objects supported by SiVArc.

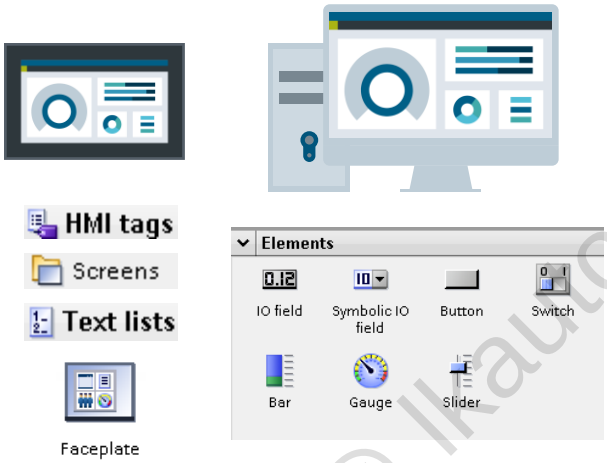


- You generate one generation template per HMI object.
- SiVArc properties are evaluated when the HMI objects are generated. Object properties, such as "Label" or "Name", are generated.

You can configure the templates using the plug-ins editor.



9.4.2 Supported elements



Devices:

- Unified PC Runtime
- Unified Comfort Panel

Generation of:

- Text list
- HMI tags
- Screens & Screen objects
- Faceplates

Screen objects:

- Text box
- IO fields / Symbolic IO field
- Button /Switch
- Bar / Slider
- Gauge

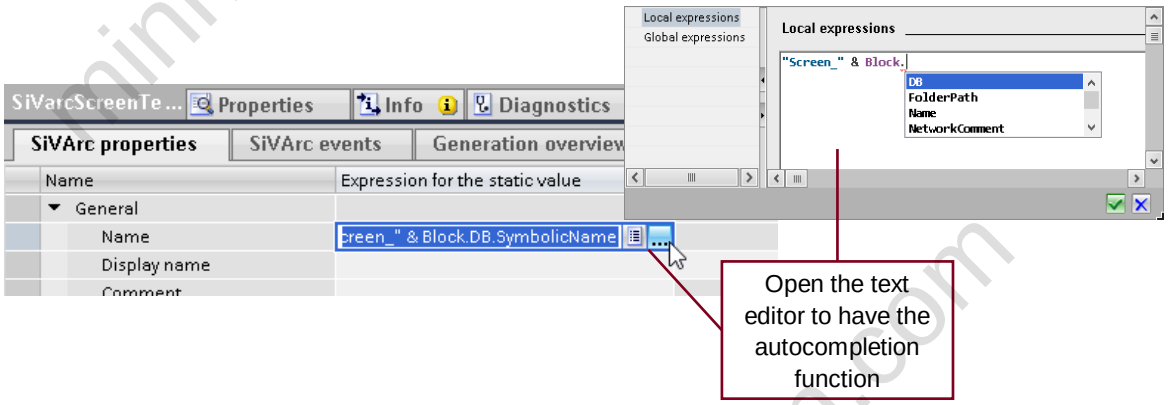
9.4.3 Expressions

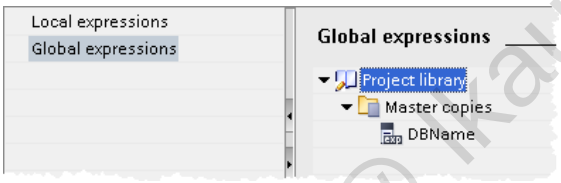
SiVArc expression **accesses text sources** via SiVArc objects. The SiVArc objects address blocks in the program call **in STEP 7**, in the **HMI device** or **library** data.

SiVArc expressions are used for the following configurations:

- **Formulate conditions** for generating HMI objects.
- **Dynamic generation** of properties, events and animations for the visualization.

A SiVArc expression is a function that returns a text.





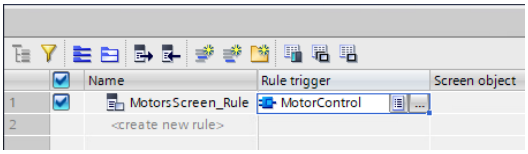
Local expressions:

- Expression that can be used only in the element in which it was configured.

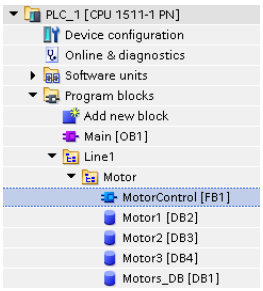
Global expressions:

- Expression which can be used within the project library and the global library.
- Can be used globally by invoking the name of the expression.

There are two types of SiVArc expressions.



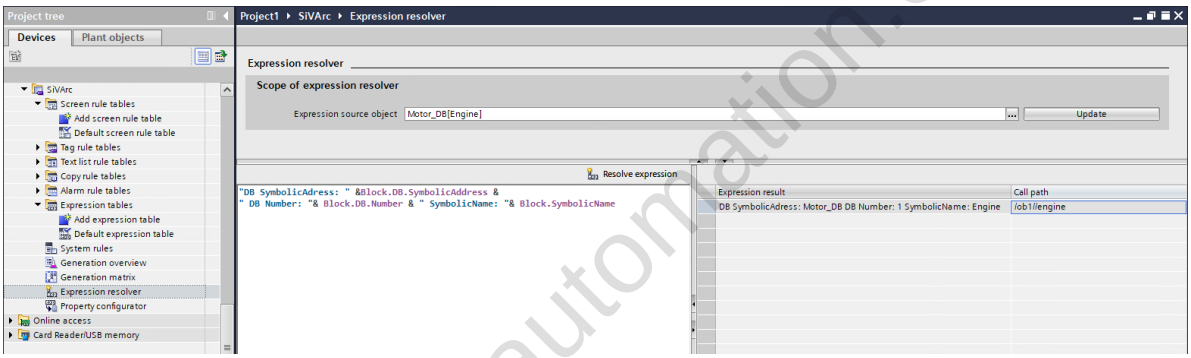
You can find all the available references in TIA Portal Help under SiVArc > References Chapter



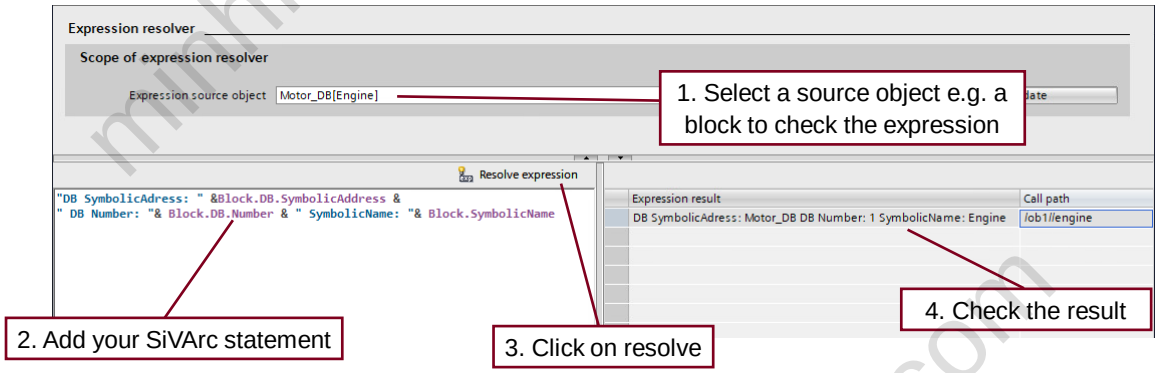
SiVArc Expression	Result
“MyBlock”	MyBlock
“My\”Block"	My"Block
Block.SymbolicName	MotorControl
Block.DB.SymbolicName	Motor1 / Motor2 / Motor3
“MyBlock_” & Block.SymbolicName	MyBlock_MotorControl

You should keep in mind uniqueness and consistency when naming data blocks and network titles.

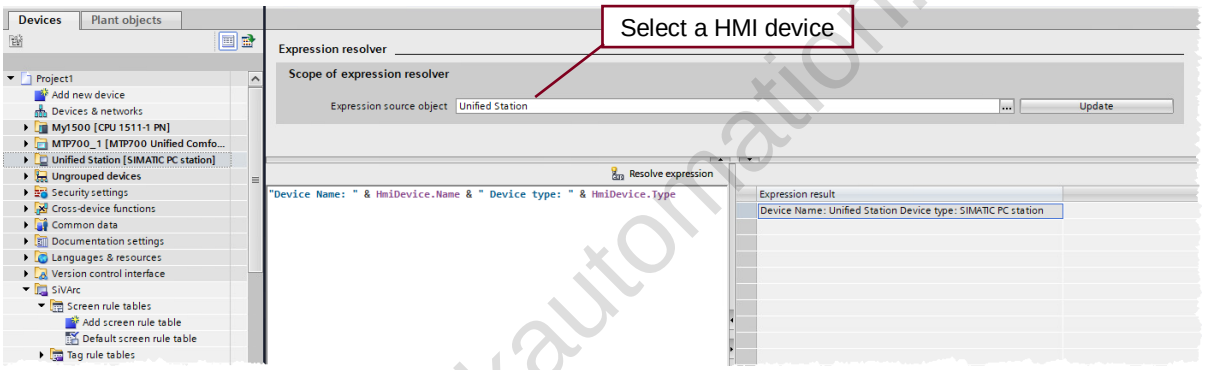
9.4.4 Expression resolver



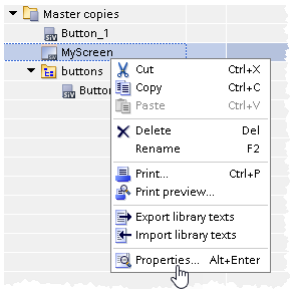
Check you SiVArc expressions without starting a generation, reduce the time for "try and error"-runs.

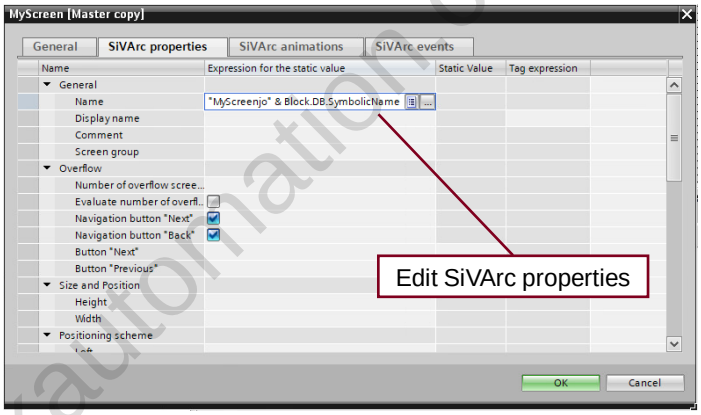


9.4.5 Expression resolver HMI device based



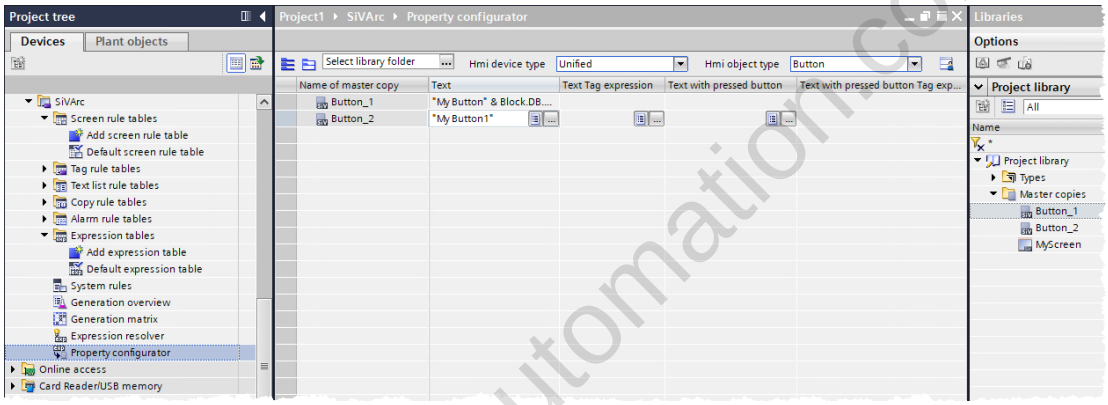
9.4.6 Editing properties





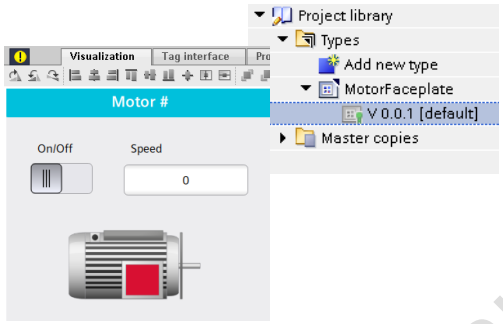
Edit easy master copies without the need of replacing it in the Library.

9.4.7 Property configurator for master copies



Editor for bulk modification of SiVArc master copies.

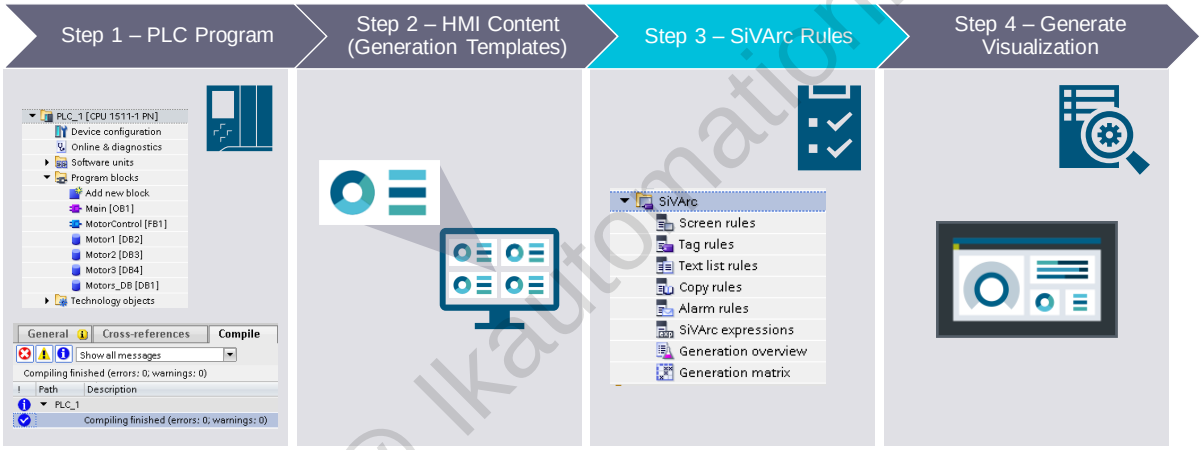
9.4.8 Faceplates



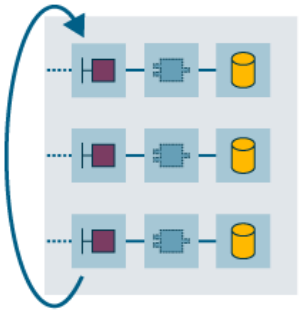
- Generation template of the faceplate type is stored in the library.
- Upon SiVArc generation, unified faceplate is generated as faceplate container.

You achieve the best reusability by using the type-instance concept from the library.

9.5 Rules



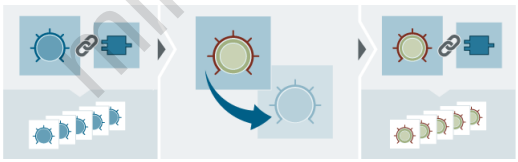
9.5.1 Processing of rules



The following principles apply to screen rules:

- You must define a screen rule for each screen object to be generated.
- If you want to generate different screen objects from a program block, you must define a screen rule with a condition for each screen object.
- The objects are created in the order of the screen rules.

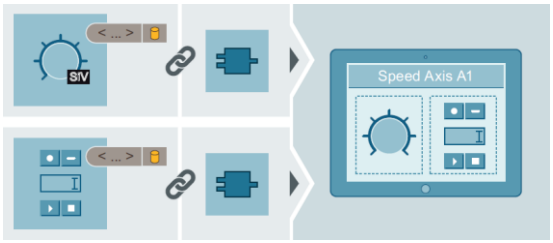
SiVArc executes the user program during generation.
If a rule applies to a function block, the rule is executed.



During next generation:

- When you change a SiVArc rule, generated objects based on this rule are overwritten.
- When you delete a rule, generated objects associated with this rule are automatically removed.

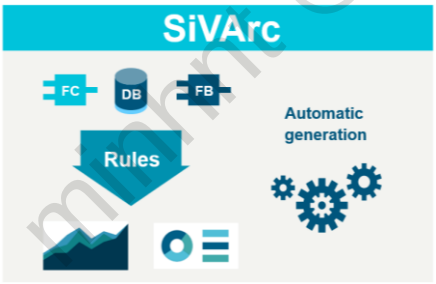
You can add, delete or modify a rule at any time within any rule editor.



Changes to the controller:

- If you delete a PLC with which you have already generated a visualization, all objects generated with this PLC are deleted during the next generation.
- If you delete/add a block call in the user program and generate it once again, the objects generated with SiVArc (HMI objects, screens, tags) are automatically deleted/added.

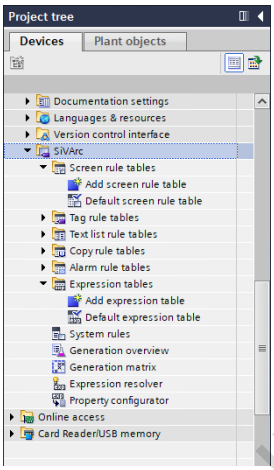
You can modify the PLC program at any time and apply those changes to the visualization during the next generation.



- SiVArc rules are a key functionality of SiVArc and have a direct relationship to the PLC program.
- Changes can therefore be implemented centrally and throughout the project.
- The various SiVArc rules define different generation tasks.

SiVArc rules define how HMI objects are processed during generation.

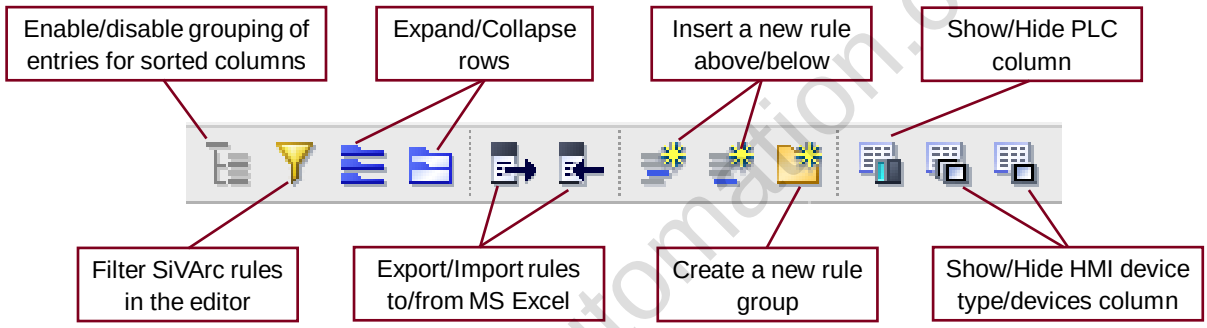
9.5.2 Rules editors



- Screen rules define which control objects are created.
- Tag rules store the external tags generated by SiVArc in a structured manner.
- Text list rules create text lists in the device.
- Copy rules copy HMI objects without a PLC connection from the library.
- Generation overview displays the screens, screen objects, tags and text lists in the project that were generated by SiVArc.
- Generation matrix displays the screens and screen objects of a selected device.

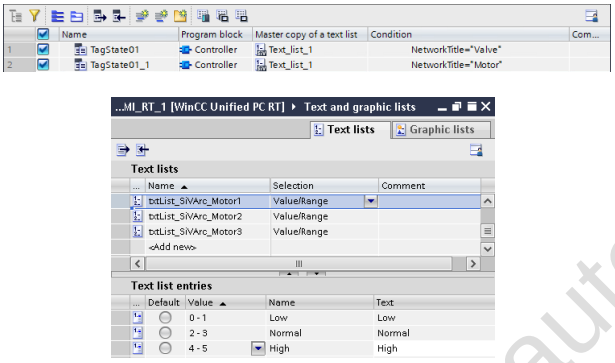
SiVArc editors are located in the project tree.

9.5.3 Rules editor toolbar



Icons in the toolbar depends on the selected rule editor.

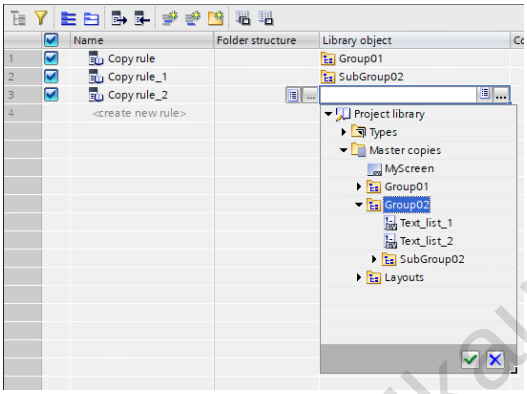
9.5.4 Generating text list



- In the "Text list rules" editor, you define SiVArc rules according to which **text lists are generated** for various devices.
- When you generate the visualization, the text list is created in the **"Text and Graphic Lists"** editor.
- Text lists used for the generation must be in the **Library**.

A text list rule specifies which text list is generated for a program block.

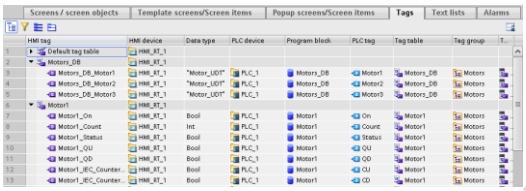
9.5.5 Copy rules



- Use library rules to systematically **copy a large number of objects** according to specific project criteria.
- When you generate, objects that are interconnected to **scripts**, you use the copy rules to ensure that these scripts also exist on the HMI device.
- Create **internal tags and tag tables**, initially for only one HMI device. SiVArc can copy them automatically to all other devices.

A copy rule copies HMI objects based on master copies or types when generating the visualization.

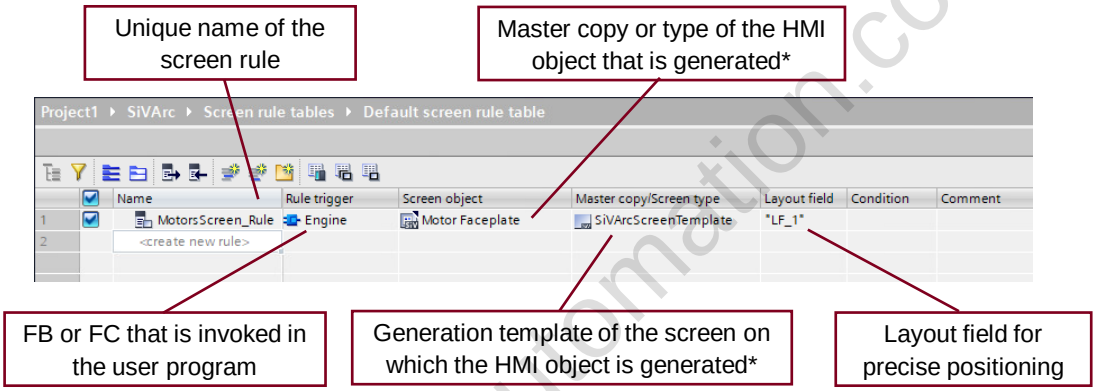
9.5.6 Generating tags



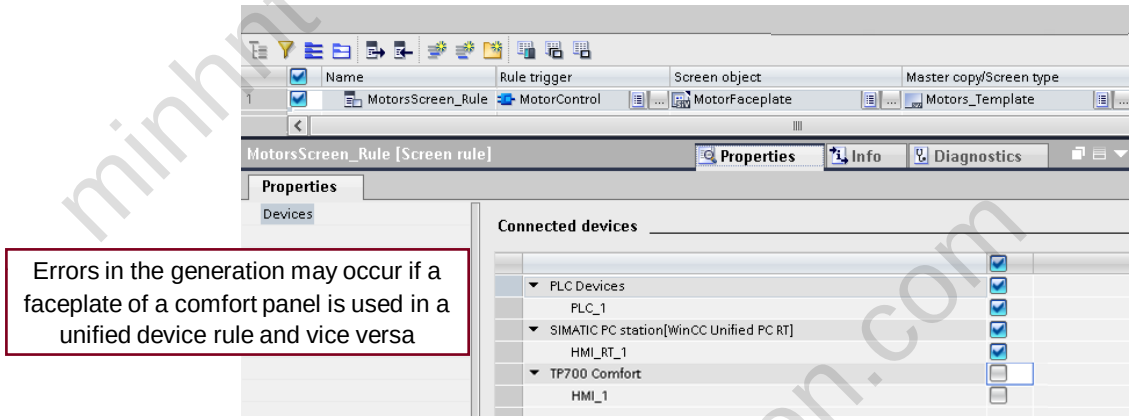
- According to the settings, SiVArc generates either all tags or only those used in the project.
- External tags are stored in tag tables and groups in the project tree according to your structuring specifications.
- SiVArc generates external tags for instance data blocks and global data blocks.

SiVArc generates an external HMI tag for each PLC tag with the option "Accessible from HMI".

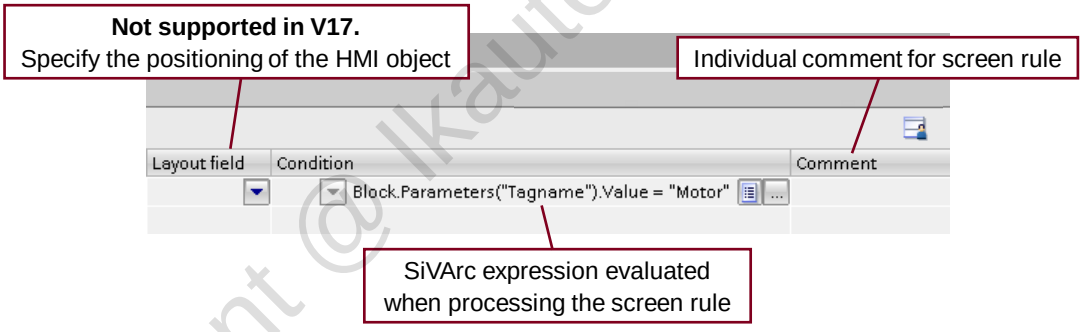
9.5.7 Screen rules editor



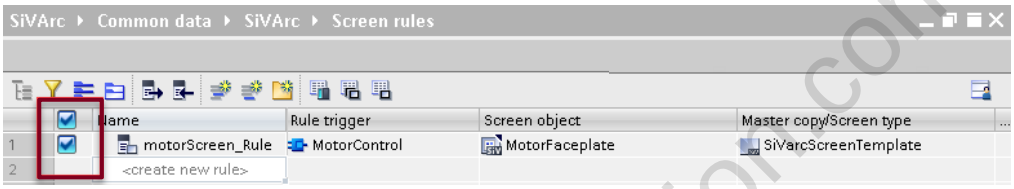
*** must be stored in a library.**



In the properties of each rule, the equipment for which this rule applies must be specified.

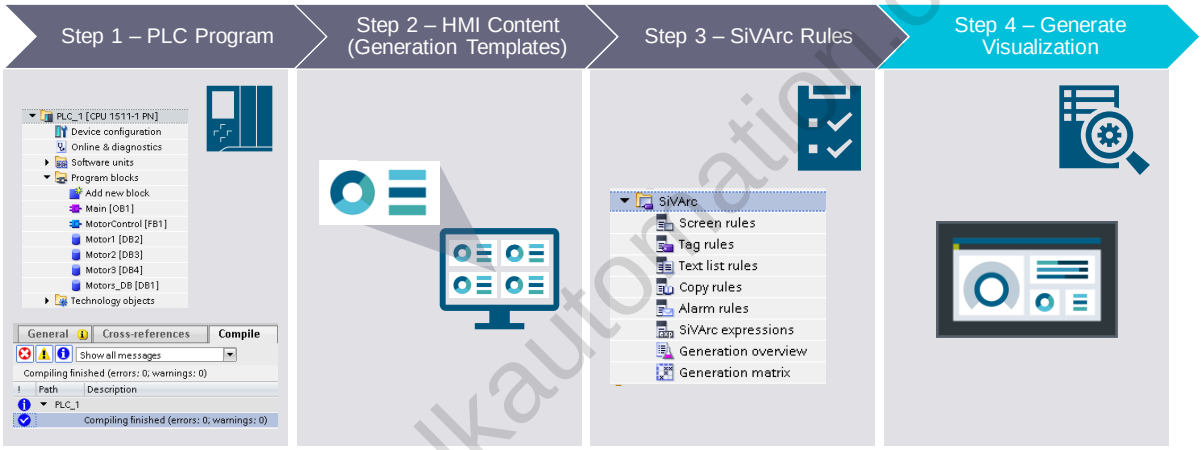


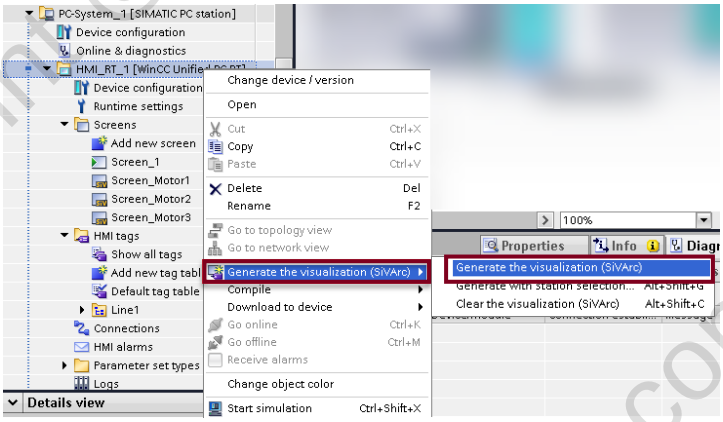
The SiVArc rules editors can be found in TIA Portal in the project tree, "Common data > SiVArc".



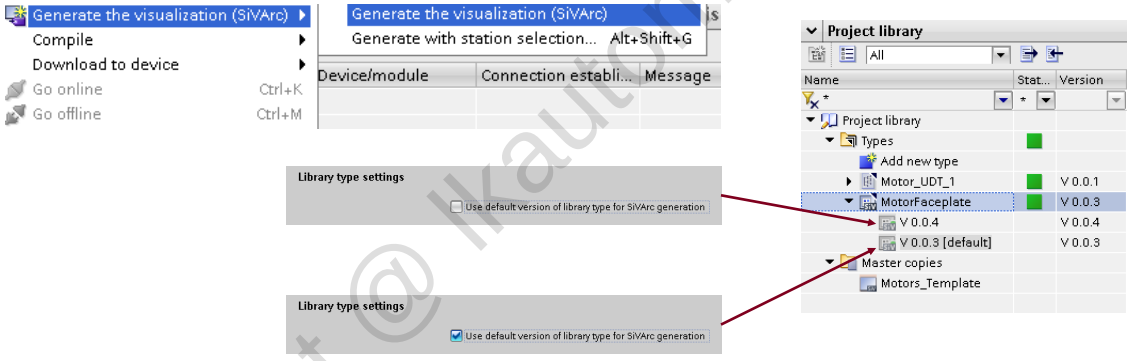
You can select which screen rules will be executed in the next generation.

9.6 Generate visualization

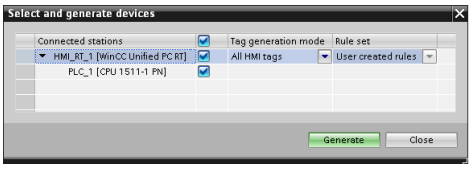




Right-click the HMI Device and select "Generate the visualization (SiVArc)".

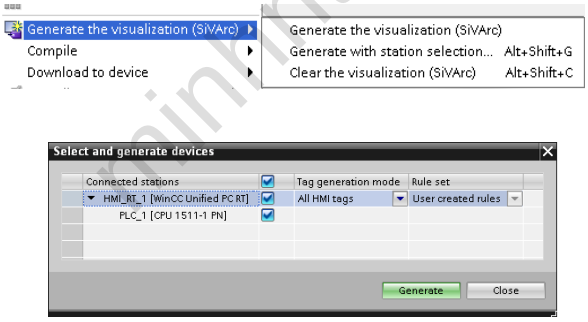


Whether the generated instance is the "default" type or the most current version, depends on the SiVArc settings.



- A dialog for station selection is displayed the **first time generation** is started in a project.
- In the dialog **select the devices** for which SiVArc is to generate the visualization.
- The available selection of stations is stored after the first generation. Every **following generation** is based on this selection.

If your project contains several HMI devices or connected PLCs, SiVArc generates the visualization for the HMI devices and PLCs you have selected.

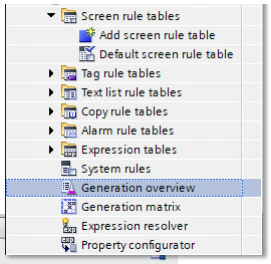


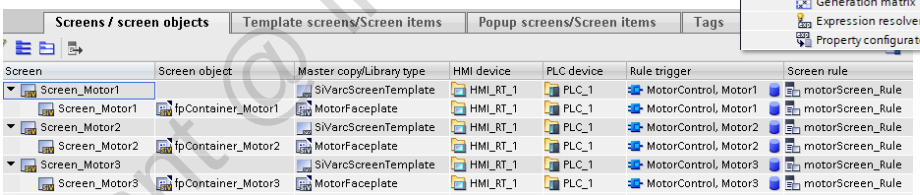
Generation types:

- **Without station selection:** The generation runs with the station selection configured in the first time generation dialog.
- **With station selection:** Displays the dialog to select the station selection.

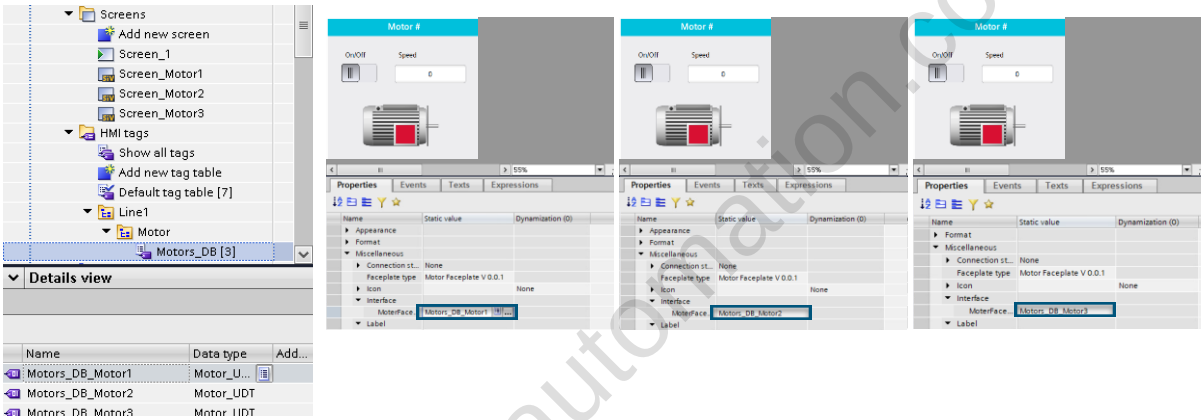
SiVArc generates the HMI objects based on the SiVArc rules and saves them according to the configuration.

You can find the complete overview of all the elements generated with SiVArc in **"Common data > SiVArc"**

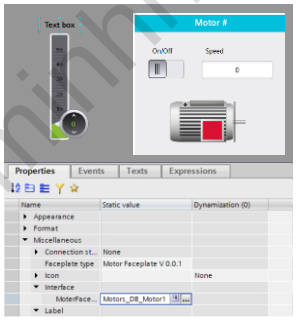




The generation overview is created during the first generation and updated in future generations.



Screens, Tags and faceplates generated via SiVArc.



- After SiVArc generation you can add elements to your screen or change the appearance or position of the elements, these changes are stored and maintain even after further SiVArc generations.

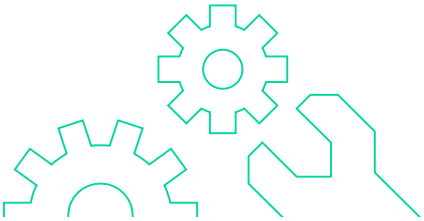
Changes made to the visualization are retained even after another SiVArc generation.

9.7 Exercise: Create SiVArc rules

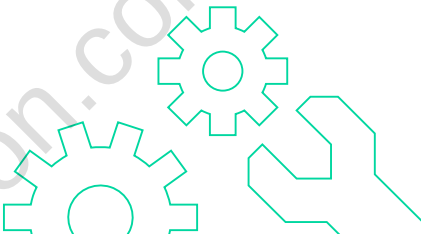
9.7.1 Procedure

1. **Create tag rule**
   - Delete existing HMI tags
   - SiVArc tag rule table -> Create new rule
   - Tag group "HmiTag.DB.FolderPath" and tag table "HmiTag.DB.SymbolicName"
   - Generate HMI tags

2. **Prepare faceplate type "Motor_small"**
   - Duplicate faceplate type
   - Name: "Motor_small_SiVArc"



3. **Set up automatic tag connection**
   - Prepare faceplate (delete dynamization, Change tag interface to "Motor_UDT_FP"
   - Dynamize the tag interface in the SiVArc properties ("Motors_DB_" & Block.DB.SymbolicName")
4. **Prepare screen rule**
   - Create screen (name: SiVArc)
   - Create text field (SiVArc Properties -> Text: Block.DB.SymbolicName) and save as master copy, name: Motor_label
   - Adapt size and position
   - Create layout fields: Insert 2 rectangles (SiVArc Properties -> Use as layout field)
   - Store screen in master copies

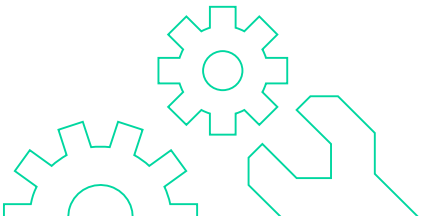


5. **Create screen rule 1**
   - Trigger: MotorControl, screen object: Motor_small, master copy: SiVArc, layout field: LF_1

6. **Create screen rule 2**
   - Trigger: MotorControl, screen object: Motor_label, master copy: SiVArc, layout field: LF_2

7. **Generate screens**

8. **Start SiVArc generation**



9.8 Detailed exercise description

9.8.1 Exercise (SiVArc)

Generate HMI tags

1. Delete all existing HMI tag tables as well as the tags in the default tag table.
2. In the project tree, select > SiVArc > Tag rule table > "Default tag table".
3. Create a new rule:
   Name: Tag rule, tag group: HmiTag.DB.FolderPath, tag table: HmiTag.DB.SymbolicName
4. Select the HMI station and then click the "Start SiVArc generation with station selection dialog" icon

   Note
   Alternatively: Right-click on Station > "Generate the visualization (SiVArc)" > "Generate with station selection…"
5. The "Select and generate devices" window opens. For "Tag generation mode", select the desired CPU and "All HMI tags" and click "Generate".
6. Check whether the HMI tags have been generated.

Prepare faceplate

For the following exercises, we will work with a copy of the "Motor_small" faceplate. To do this, proceed as follows:

1. In Project Library > Types: Right-click on the faceplate type "Motor_small" > Duplicate type.
2. Change the type name to "Motor_small_SiVArc" and confirm with "OK".

Automatic tag connection

Since the MotorControl block has a tag interface that is different from the one set up for the "Motor_small" faceplate, the "Motor_small_SiVArc" faceplate must now be adapted.

1. Right-click on the newly created faceplate "Motor_small_SiVArc" > Edit type.
2. Delete all dynamizations in the properties.
3. In the "Tag interface" tab, delete the interface and create a new one.
   Name: Motor_UDT, data type: PLCUDT, user data type structure: Motor-UDT_FP V0.0.1
4. Back in the "Visualization" tab, navigate to Plug-ins > SiVArc Properties > Tag interface. Enter the following expression as "Expression for the static value" (with leading and trailing characters): "Motors_DB_" & Block.DB.SymbolicName
5. Release the type version.

   Note
   If the faceplate is instantiated using the screen rules, each instance is now associated with the corresponding data block.

Prepare screen rules

1. Project tree > Project > PC Station > Screens:
   Double-clicking on "Add new screen" creates a new screen.
   Select the screen names and rename them to "SiVArc" with F2.
   In the screen -> Plug-ins -> SiVArc Properties -> General -> enter the name "Block.DB.SymbolicName".
2. Place a text field on the screen:
   Properties > "Font": Siemens Sans, 19pt"
   Properties > "Size – Width" 150
   Plug-ins > SiVArc Properties > "Text": Block.DB.SymbolicName
3. Drag-and-drop the text field into the "Master copies" folder of the project library.
   Rename the template (F2) to "Motor_label".
4. Delete the text field from the screen.
5. Place 2 rectangles on the "SiVArc" screen.
   Note
   These are the placeholders that will later be replaced by the "Motor_small_SiVArc" faceplate and the text field you just created during automatic screen creation.

   Rectangle 1 should have the size of the faceplate "Motor_small_SiVArc":
   Properties > "Size – Width" 150
   Properties > "Size – Height": 150

   Rectangle 2 should have the size of the text field:
   Properties > "Size – Width" 150
   Properties > "Size – Height": 40

   Rectangle 2 is to be placed above rectangle 1 (label for motor).

6. Select rectangle 1 > Plug-ins > SiVArc Properties > "Use as layout field": Set a check mark.
   SiVArc Properties > "Layout field name": LF_1
7. Select rectangle 2 > Plug-ins > SiVArc Properties > "Use as layout field": Set a check mark.
   SiVArc Properties > "Layout field name": LF_2
8. Drag the "SiVArc" screen to the "Master copies" folder, where the "Motor_label" text field is also located.
9. Save your project.

Create screen rules

1. In the project tree, select > SiVArc > Screen rule table > "Standard screen rule table".
2. Create a new rule with the following properties:
   - Name: Screen_rule_1
   - Rule trigger: MotorControl [FB3]
     (Click "…" > Project name > CPU1500 > Program blocks > SiVArc)
   - Screen object: Motor_small_SiVArc
     (Click "…" > Project library > Types)
   - Master copy/screen type: SiVArc
     (Click "…" > Project library > Master copies)
   - Layout field: LF_1

Note
The rule is therefore triggered as often as FB3 "Motor control" is present in the project. In this case, 3 times.
Each time a screen is created based on the master copy screen "SiVArc".

3. Create another rule with the following properties:
   - Name: Screen_rule_2
   - Rule trigger: MotorControl [FB3]
     (Click "…" > Project name > CPU1500 > Program blocks > SiVArc)
   - Screen object: **Motor_label**
     (Click "…" > Project library > Types)
   - Master copy/screen type: SiVArc
     (Click "…" > Project library > Master copies)
   - Layout field: **LF_2**

   Note
   With the second rule, the text fields are created and described with the corresponding motor name.

4. Start the SiVArc generation.

5. Check that three screens "Motor1", "Motor2" and "Motor3" have been generated.
   These should each contain the "Motor_small_SiVArc" faceplate and a label.

10 Final exercise

10.1 Final exercise - Overview Topics, requirements and goals

The final exercise contains topics from the following chapters:

- Scripting Basics
- Libraries
- Faceplates
- SiVArc

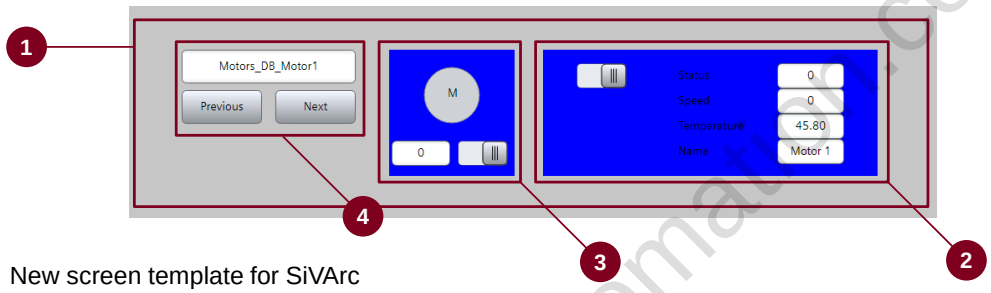
The final exercise assumes the following elements from chapter "SiVArc" and uses them:

- HMI tags table "Motors_DB"
- Faceplate "Motor_small_SiVArc“

The aims of this exercise are:

- Create a new "details" faceplate
- Generate a new screen with SiVArc
- Changing the tag connection of a faceplate instance interface by using Scripting

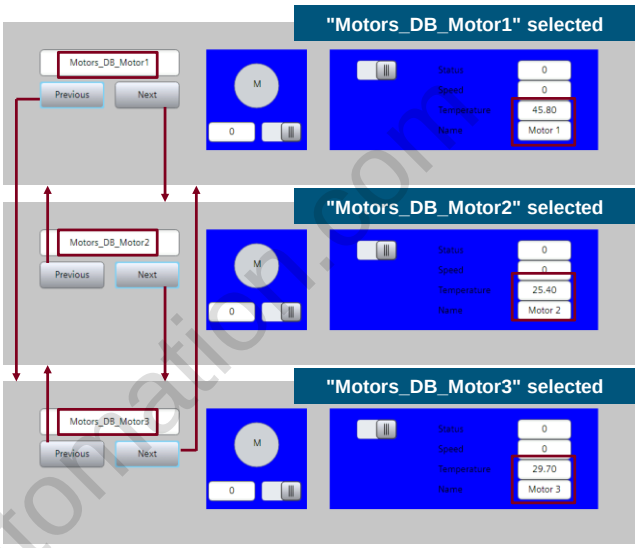
10.1.1 Final exercise - Elements



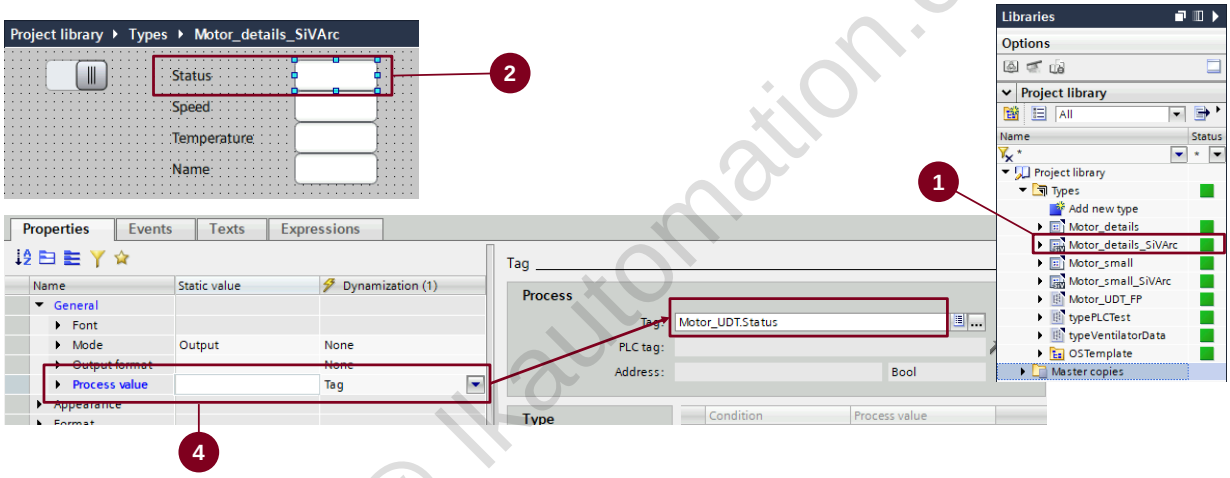
1. New screen template for SiVArc
2. New "details" faceplate
3. Faceplate from the chapter "SiVArc"
4. Faceplate instance interface selection

10.1.2 Final exercise - Elements - Handling

- When the screen is started, the faceplate interfaces use the tag "Motors_DB_Motor1".
- Each click on "Next" switches to the next variable.
- Each click on "Previous" switches to the previous variable.
- If the first or last tag is reached, the program switches to the last or first tag.

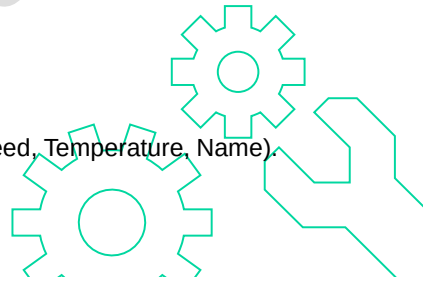


10.2 Exercise 1: Create a new faceplate



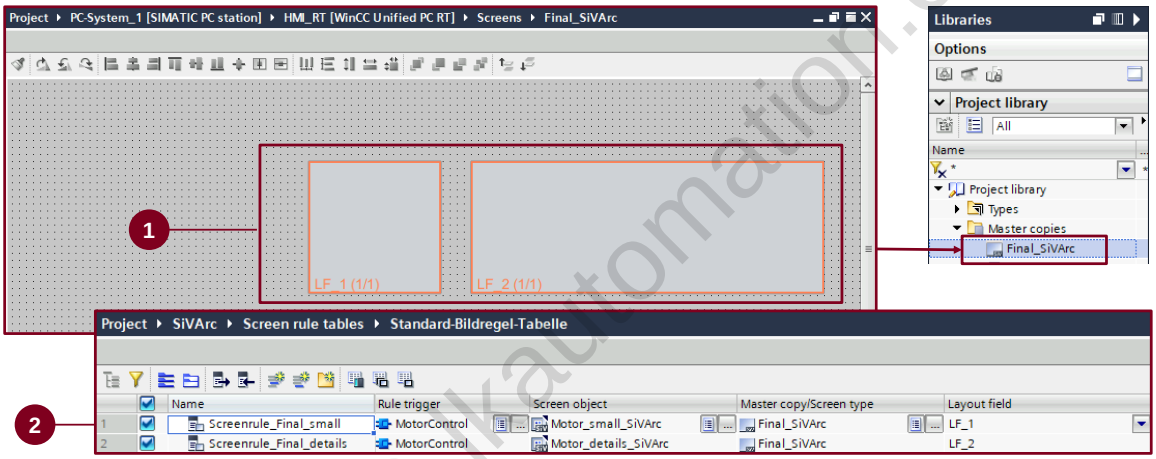
10.2.1 Procedure for exercise 1: Create a new faceplate

1. **Create a second faceplate type**
   - Create by duplicating "Motor_small_SiVArc" a second faceplate type.
   - Rename it to "Motor_details_SiVArc ".
2. **Adapt visualization**
   - Adapt the size: Height: 150, Width: 400.
   - Remove the motor symbol (Circle with the text field) and the IO field.
   - Add IO fields for elements of the tag interface (Status, Speed, Temperature, Name) and label them with the corresponding text fields.
3. **Interface remains the same**
   - The tag and the property interfaces should remain unchanged.
4. **Object properties connection with the interface**
   - Connect the IO fields with the elements of the tag interface (Status, Speed, Temperature, Name).
   - Release new version of the faceplate type.



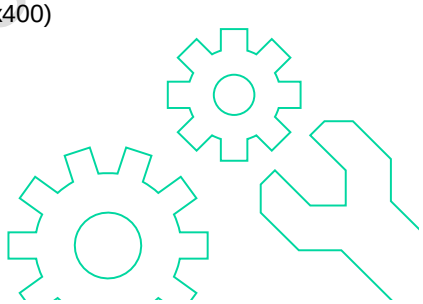
Link to the detailed exercise description

10.3 Exercise 2: Generate a new screen with SiVArc



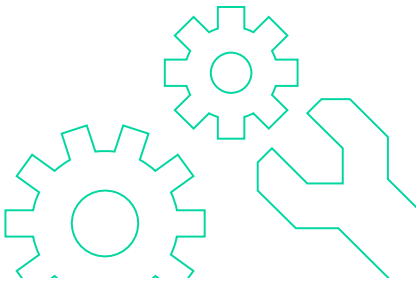
10.3.1 Procedure for exercise 2: Generate a new screen with SiVArc

1. **Create a template**
   - Add new screen (Name: Final_SiVArc)
   - Adapt the screen name and the screen group
     (SiVArc properties > Name: Block.SymbolicName)
     (SiVArc properties > Screen group : Block.SymbolicName)
   - Create layout fields: Add 2 rectangles
     (SiVArc properties > Use as layout field)
     (Height x Width : Rectangle 1: 150x150, Rectangle 2: 150x400)
   - Store screen in master copies



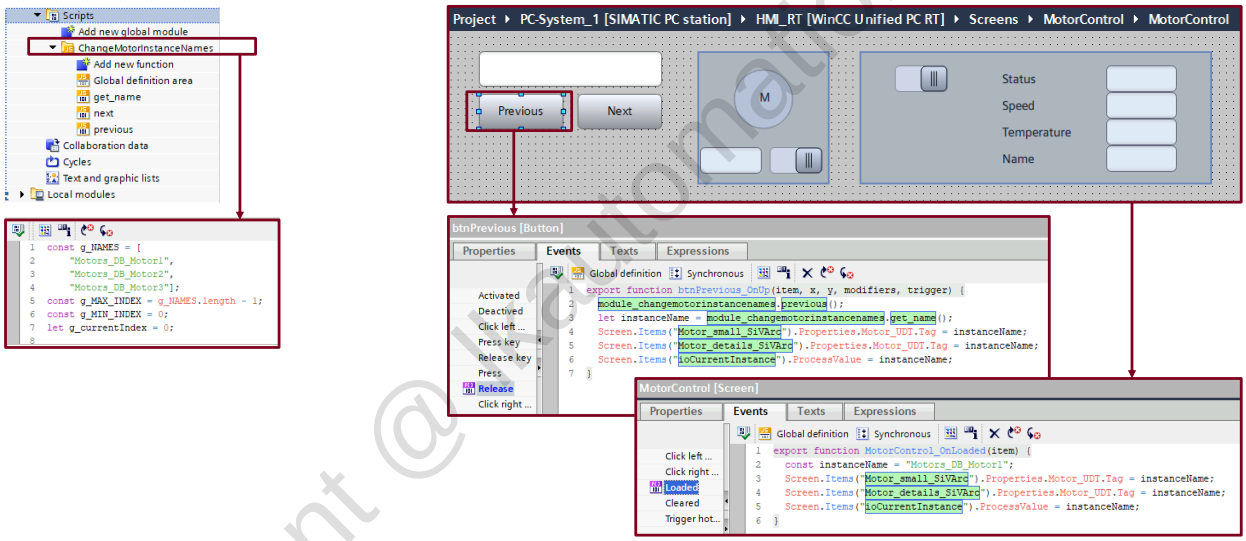
10.3.2 Procedure for exercise 2: Generate a new screen with SiVArc

2. **Create screen rules**
   - Screen rule 1
     (Trigger: MotorControl)
     (Screen object: Motor_small_SiVArc)
     (Master copy: Final_SiVArc)
     (Layout field: LF_1)
   - Screen rule 2
     (Trigger: MotorControl)
     (Screen object: Motor_details_SiVArc)
     (Master copy: Final_SiVArc)
     (Layout field: LF_2)
3. **Start SiVArc generation**
4. **Load the project**



Link to the detailed exercise description

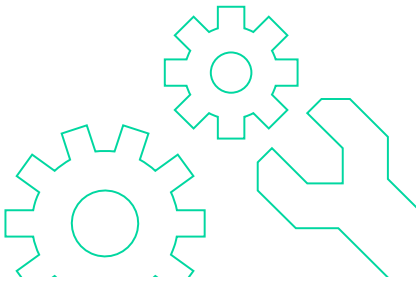
10.4 Exercise 3: Create faceplate instance interface selection



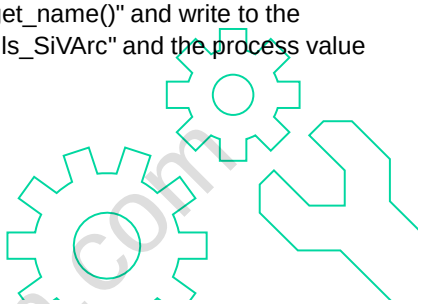
10.4.1 Procedure for exercise 3: Create faceplate instance interface selection

1. **Create global scripting module**
   - Add new global module by using "Skripts", rename it to "ChangeMotorInstanceNames„.
   - Create in „Global definition area":
     - Array "g_NAMES" for names of instance tags: ["Motors_DB_Motor1", "Motors_DB_Motor2", "Motors_DB_Motor3"];
     - Constant " g_MAX_INDEX " for largest array index => 2
     - Constant " g_MIN_INDEX " for smallest array index => 0
     - Tag " g_currentIndex" for current index, the current index should be 0.
   - Create function "get_name" (should return the name at the current index from the array "g_NAMES")
   - Create function "next" (if the current index is not yet the largest index, increase the current index, otherwise start again at the smallest index)
   - Create function "previous" (if the current index is not yet the largest index, decrease the current index, otherwise start again at the largest index)

2. **Create screen objects in the generated SiVArc screen "MotorControl"**
   - Add two buttons („Previous" and „Next") to change the interface tag.
   - Change the names of the buttons to "btnPrevious" respectively "btnNext".
   - Add IO field to display the current interface tag.
   - Change the name of the IO field to "ioCurrentInstance".
   - In the next steps the objects will be extended with functionalities.



3. **Create events scripts of the screen and buttons**
   - In "Loaded" event of the screen "MotorControl" write the start value of the tag "Motors_DB_Motor1" to the faceplate interfaces and IO field "ioCurrentInstance".
   - Import global module "ChangeMotorInstance" into "Global definition".
   - In **"btnPrevious"**: Call the function "previous()", read the instance name using "get_name()" and write to the interfaces of the faceplate containers "Motor_small_SiVArc", "Motor_details_SiVArc" and the process value of the IO field "ioCurrentInstance".
   - In **"btnNext"**: Call the function "next()", read the instance name using "get_name()" and write to the interfaces of the faceplate containers "Motor_small_SiVArc", "Motor_details_SiVArc" and the process value of the IO field "ioCurrentInstance".
4. **Run and test the project**



Link to the detailed exercise description

10.5 View - GSC Supplier Training

Target:

- The participant masters the structure of the GSC software structure and can implement it when programming a standard project.
- The participant is able to create his PLC basic program for the GSC with the help of the "SmartAutomationSuite" software.
- The participant knows the standard-compliant structure of an automation system and can program it in accordance with standards.
- The participant masters the relationship in a standard project between the PLC program, SiVArc and the visualization and can apply it in a standard-compliant manner.

Contents

- Development of the SW structure of the GSC standard (history, components, concept).
  With the help of the virtual training cell, it can be set up as a production cell according to the standard.
- Integration Hardware (Manual)
- Software structure GSC (program structure, operating modes)
- Integration of a panel and presentation of the HMI
- Insight SiVArc
- Reporting system (user login)
- HMI user detail images
- HMI button
- Create User Module with HMI Object
- Safety (Emergency Stop, Gate, Misc)
- Introduction of SAS
- Interaction between EPLAN and SAS
- Interaction between TIA and SAS
- Rejection of TIA components
- SOV+ LineSim
- Type Management
- Actuators
- Phase model
- Fault processing
- Safety programming
- Libraries
- Robotics

10.6 Detailed exercise description

10.6.1 Exercise 1 (Create a new faceplate)

Create a second faceplate type

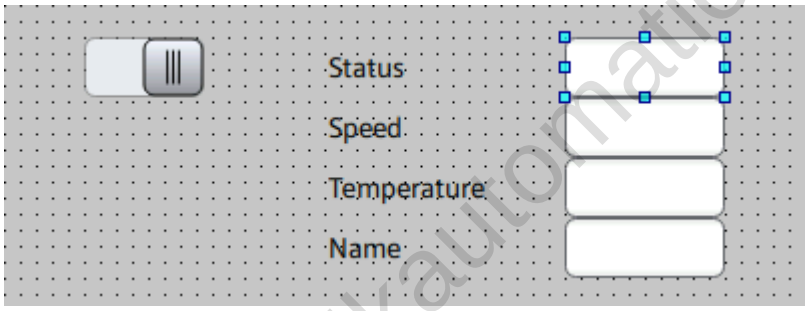
1. Libraries > Project library > Types.
   Create a copy of "Motor_small_SiVArc": Right click on "Motor_small_SiVArc" > "Duplicate type".
2. Rename the copy to "Motor_details_SiVArc".
3. Open "Motor_details_SiVArc" with right click on it > "Edit type".

Adapt visualization

1. Adapt the size: Height: 150, Width: 400.
   Properties > „Size – height“: 150
   Properties > „Size – width“: 400
2. Remove the motor symbol (Circle with the text field) and the IO field.
3. Add IO fields for elements of the tag interface (Status, Speed, Temperature, Name) and label them with the corresponding text fields.

Notes:
Go to the "Visualization" tab in the work area.
Change the following values in the properties:
„Size and position“ > „Size – height“: 150
„Size and position“ > „Size – width“: 400
Add the following objects:
Five text fields and five IO fields.

Appearance and layout should be adapted as follows:



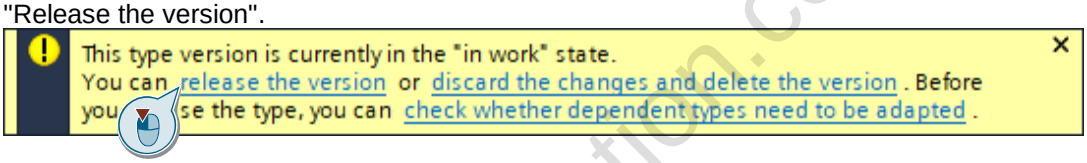
Interface remains the same

→ In the work area the „Tag interface“ and „Property interface“ tabs should remain unchanged.

Object properties connection with the interface

1. Connect the first IO field to the structure element "Status".
2. Connect the second IO field to the structure element "Speed".
3. Connect the third IO field to the structure element "Temperature".

4. Connect the fourth IO field to the structure element "Name".
5. Release the new version of the faceplate type by clicking on the exclamation mark icon at "Release the version".



Notes:

Go to the "Visualization" tab in the work area.

Select the I/O field.

Under Property "General > Process value", change the value in the "Dynamization" column from "None" to "Tag". You can then connect an element of the user data type structure "Motor_UDT" for "Tag" in the right part of the Inspector window. Select the "Status" element.

Other IO fields "Speed", "Temperature" as well as "Name" should be dynamized following the same procedure as for the IO field „Status“.

Open the note window by clicking on the exclamation mark icon: 
Click "Release the version".
Click "OK".

BACK

10.6.2 Exercise 2 (Generate a new screen with SiVArc)

Create a template

1. Project tree > Project > SIMATIC PC station > WinCC Unified PC RT > Screens:
   With right click on "Add new screen" create a new screen.
   Select the new screen in the project tree and with F2 rename it to "Final_SiVArc".

   Navigate in the screen to Plug-ins > SiVArc properties > General > Name. Enter the name "Block.SymbolicName".

2. Add 2 rectangles to the screen „Final_SiVArc“.

   Notes:

   These are the placeholders that will later be replaced by the faceplates „Motor_small_SiVArc“ and „Motor_details_SiVArc“ during automatic screen creation.

   Rectangle 1 should have the size of the faceplate „Motor_small_SiVArc“. Change the following values in the properties:

   „Size and position“ > „Size – height“: 150
   „Size and position“ > „Size – width“: 150
   „Size and position“ > „Size – left“: 344
   „Size and position“ > „Size – top“: 96

   Rectangle 2 should have the size of the faceplate „Motor_details_SiVArc“ besitzen. Change the following values in the properties:

   „Size and position“ > „Size – height“: 150
   „Size and position“ > „Size – width“: 400
   „Size and position“ > „Size – left“: 528
   „Size and position“ > „Size – top“: 96

   Rectangle 2 is now located to the right of rectangle 1 („details“ to the right of „small“).

   Select rectangle 1.

   Navigate to Plug-ins > SiVArc properties > General.

   Change the following values:

   „Use as layout field“: Set a check mark ☑

   „Layoutfield name“: LF_1

3. Select rectangle 2.

   Navigate to Plug-ins > SiVArc properties > General.

   Change the following values:

   „Use as layout field“: Set a check mark ☑

   „Layoutfield name“: LF_2

4. Libraries > Project library > Master copies.
   Drag the „Final_SiVArc“ screen to the "Master copies" folder.
5. Save your project.

Create screen rules

1. In the project tree, select > SiVArc > Screen rule tables > "Standard screen rule table" and open it by double clicking.
2. Create a new rule with the following properties:

„Name“: Screenrule_Final_small
„Rule trigger“: MotorControl [FB3]
(Click on „…“ > Project > CPU1500 > Program blocks > SiVArc)
„Screen object“: Motor_small_SiVArc
(Click on „…“ > Project library > Types)
„Master copy/Screen type“: Final_SiVArc
(Click on „…“ > Project library > Master copies)
„Layout field“: LF_1

Notes:
The rule is therefore triggered as often as the FB3 „MotorControl“ exists in the project. In this case 3 times.

Since the SiVArc property for the name in the master screen „Final_SiVArc“ is the name of the type of the function block FB3 „MotorControl“, the screen will be created only once. The switching of the instances is configured manually in exercise 3.

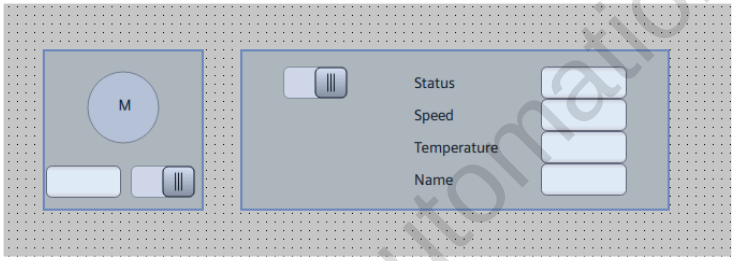
3. Create the next rule with the following properties:

„Name“: Screenrule_Final_details
„Rule trigger“: MotorControl [FB3]
(Click on „…“ > Project > CPU1500 > Program blocks > SiVArc)
„Screen object“: Motor_details_SiVArc
(Click on „…“ > Project library > Types)
„Master copy/Screen type“: Final_SiVArc
(Click on „…“ > Project library > Master copies)
„Layout field“: LF_2

Start SiVArc generation

1. Start the SiVArc generation.

2. Project tree > Project > SIMATIC PC station > WinCC Unified PC RT > Screens. Check that the screen „MotorControl“ in the group „MotorControl“ has been generated. The screen should contain the „Motor_small_SiVArc“ and „Motor_details_SiVArc“ faceplates.



3. Add the button in the start menu, to switch to the previously created „MotorControl“ screen.

Load the project

1. Load the project.
2. Open the screen „MotorControl“.
3. Check if the both faceplates are displayed.

BACK

10.6.3 Exercise 3 (Create faceplate instance interface selection)

Create global scripting module

1. In the project tree, select „HMI Unified PC RT“ > Scripts > „Add new global module“ by double clicking.
2. Rename the module to „ChangeMotorInstanceNames“.
3. By double-click, open the editor „Global definition area“. Enter here the following code:

```
const g_NAMES = ["Motors_DB_Motor1", "Motors_DB_Motor2", "Motors_DB_Motor3"];
const g_MAX_INDEX = g_NAMES.length - 1;
const g_MIN_INDEX = 0;
let g_currentIndex = 0;
```

   Notes:

   - The prefix „`g_`“ is to symbolize here that it is a tag or constant defined in the global definition area.
   - The elements of the array must be entered manually as strings, the "Object picker" (CTRL + j) cannot be used here.
   - The expression `g_NAMES.length - 1` automatically calculates the maximum index based on the number of elements in the array `g_NAMES`. Alternatively, for this exercise you can specify the maximum index with a constant number (`2`).

4. Create the function „get_Name“ in the module „ChangeMotorInstanceNames“ by double-click at „Add new function“.

   Rename the new function to „get_name“ and open it.

   Delete in „Properties > General“ both parameters „parameter1“ and „parameter2“. They are created by default when a function is created and are not required for this exercise.

   Enter here the following code:

```
return g_NAMES[g_currentIndex];
```

5. Create the function „next“ corresponding to the function „get_name“.

   Rename the new function to „next“ and open it.

   Delete in „Properties > General“ both parameters „parameter1“ and „parameter2“. They are created by default when a function is created and are not required for this exercise.

   Enter here the following code:

```
if (g_currentIndex < g_MAX_INDEX )
{
  g_currentIndex = g_currentIndex + 1;
}
else
{
  g_currentIndex = g_MIN_INDEX;
}
```

6. Create the function „previous“ corresponding to the function „get_name“.

   Rename the new function to „previous“ and open it.

   Delete in „Properties > General“ both parameters „parameter1“ and „parameter2“. They are created by default when a function is created and are not required for this exercise.

   Enter here the following code:

```
if (g_currentIndex > g_MIN_INDEX )
{
  g_currentIndex = g_currentIndex - 1;
}
else
{
  g_currentIndex = g_MAX_INDEX;
}
```

7. You have created the global module. Save the project.

Create screen objects in the generated SiVArc screen "MotorControl"

1. Open the generated screen „MotorControl“.
2. Add a new button with the following properties:

   „General“ > „Text“: Previous
   „Miscellaneous“ > „Name“: btnPrevious
   „Size and position“ > „Position – left“: 96
   „Size and position“ > „Position – top“: 144
   „Size and position“ > „Size – height“: 40
   „Size and position“ > „Size – width“: 96

3. Add the next new button with the following properties:

   „General“ > „Text“: Next
   „Miscellaneous“ > „Name“: btnNext
   „Size and position“ > „Position – left“: 208
   „Size and position“ > „Position – top“: 144
   „Size and position“ > „Size – height“: 40
   „Size and position“ > „Size – width“: 96

4. Add a new IO field with the following properties:

   „Miscellaneous“ > „Name“: ioCurrentInstance
   „Size and position“ > „Position – left“: 96
   „Size and position“ > „Position – top“: 96
   „Size and position“ > „Size – height“: 40
   „Size and position“ > „Size – width“: 208

Create events scripts of the screen and buttons

1. Select the screen object in the screen „MotorControl“ by clicking on its background surface.
2. Add in „Events > Loaded“ by clicking at the script for the event „Loaded“.
3. Enter here the following code:

```
const instanceName = "Motors_DB_Motor1";
Screen.Items("Motor_small_SiVArc").Properties.Motor_UDT.Tag = instanceName;
Screen.Items("Motor_details_SiVArc").Properties.Motor_UDT.Tag = instanceName;
Screen.Items("ioCurrentInstance").ProcessValue = instanceName;
```

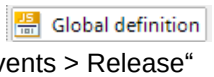
4. Select the button „btnPrevious“.
5. Add in „Events > Release“ by clicking at the script for the event „Release“.
6. Open the global definition area for events in this screen by clicking Global definition.

7. Enter here the following code:

```
import * as module_changemotorinstancenames from "ChangeMotorInstanceNames";
```

Notes:

The code makes the previously created global module „ChangeMotorInstanceNames“ available under the name „module_changemotorinstancenames“ in the events of this image. The events of all screen objects share the same global definition area, so this step needs to be performed only once and is available for all events in the screen.

8. Close the global definition area for events in this screen by clicking 

again. You will automatically switch back to the script in „btnPrevious > Events > Release“
9. In the script in „btnPrevious > Events > Release“, enter the following code in the function body:

```
module_changemotorinstancenames.previous();
let instanceName = module_changemotorinstancenames.get_name();
Screen.Items("Motor_small_SiVArc").Properties.Motor_UDT.Tag = instanceName;
Screen.Items("Motor_details_SiVArc").Properties.Motor_UDT.Tag = instanceName;
Screen.Items("ioCurrentInstance").ProcessValue = instanceName;
```

10. Select the button „btnNext“
11. Add in „Events > Release“ by clicking at  the script for the event „Release“.
12. In the script in „btnNext > Events > Release“, enter the following code in the function body:

```
module_changemotorinstancenames.next();
let instanceName = module_changemotorinstancenames.get_name();
Screen.Items("Motor_small_SiVArc").Properties.Motor_UDT.Tag = instanceName;
Screen.Items("Motor_details_SiVArc").Properties.Motor_UDT.Tag = instanceName;
Screen.Items("ioCurrentInstance").ProcessValue = instanceName;
```

Run and test the project

1. Load the project.
2. Open the screen „MotorControl“.
3. Check whether the tag connections can be switched by clicking on the buttons. To do this, observe the name in the IO field and the values in the faceplates.

BACK

11 Training & Support

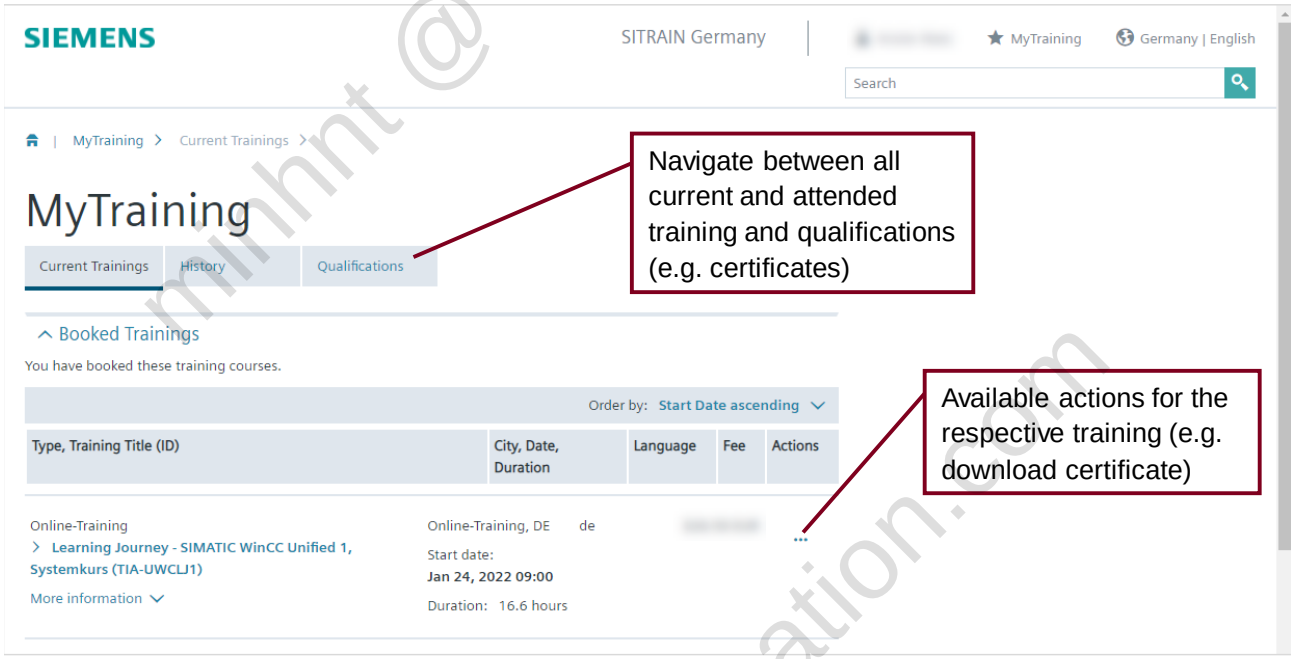
11.1 Your current and attended training

11.1.1 MyTraining – Your personal learning overview

The following actions are available to you in MyTraining:

- View current and attended training
- Download course materials
- Download certificates of participation and other certificates
- Give feedback

You will find MyTraining at www.siemens.com/sitrain-mytraining.

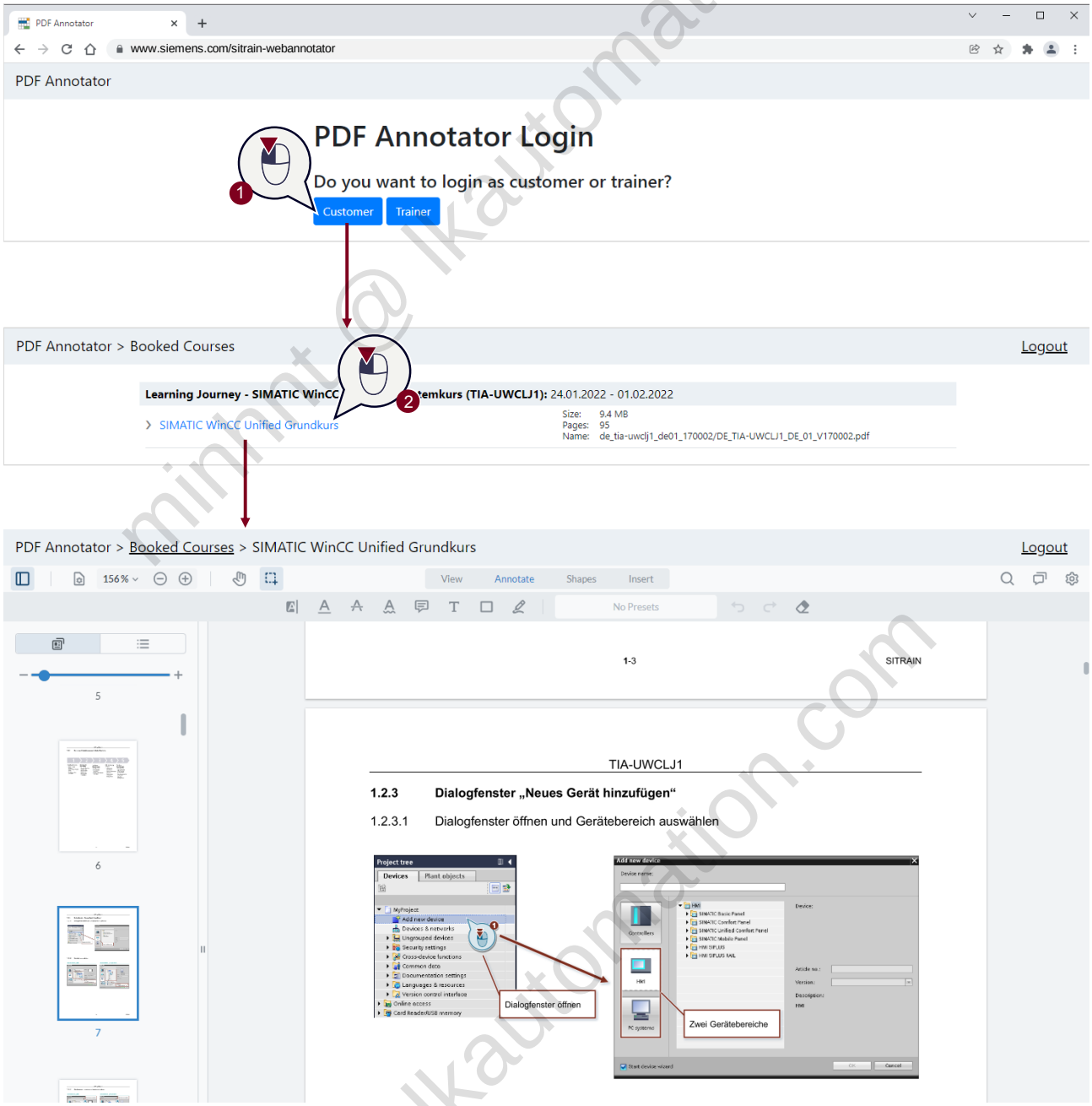


To find out how to use MyTraining and what you can do there, watch this video:
www.siemens.com/sitrain-video-mytraining

11.1.2 Annotating documents

During the training you can add notes to your course materials using a special program.

You will find this program at: www.siemens.com/sitrain-webannotator



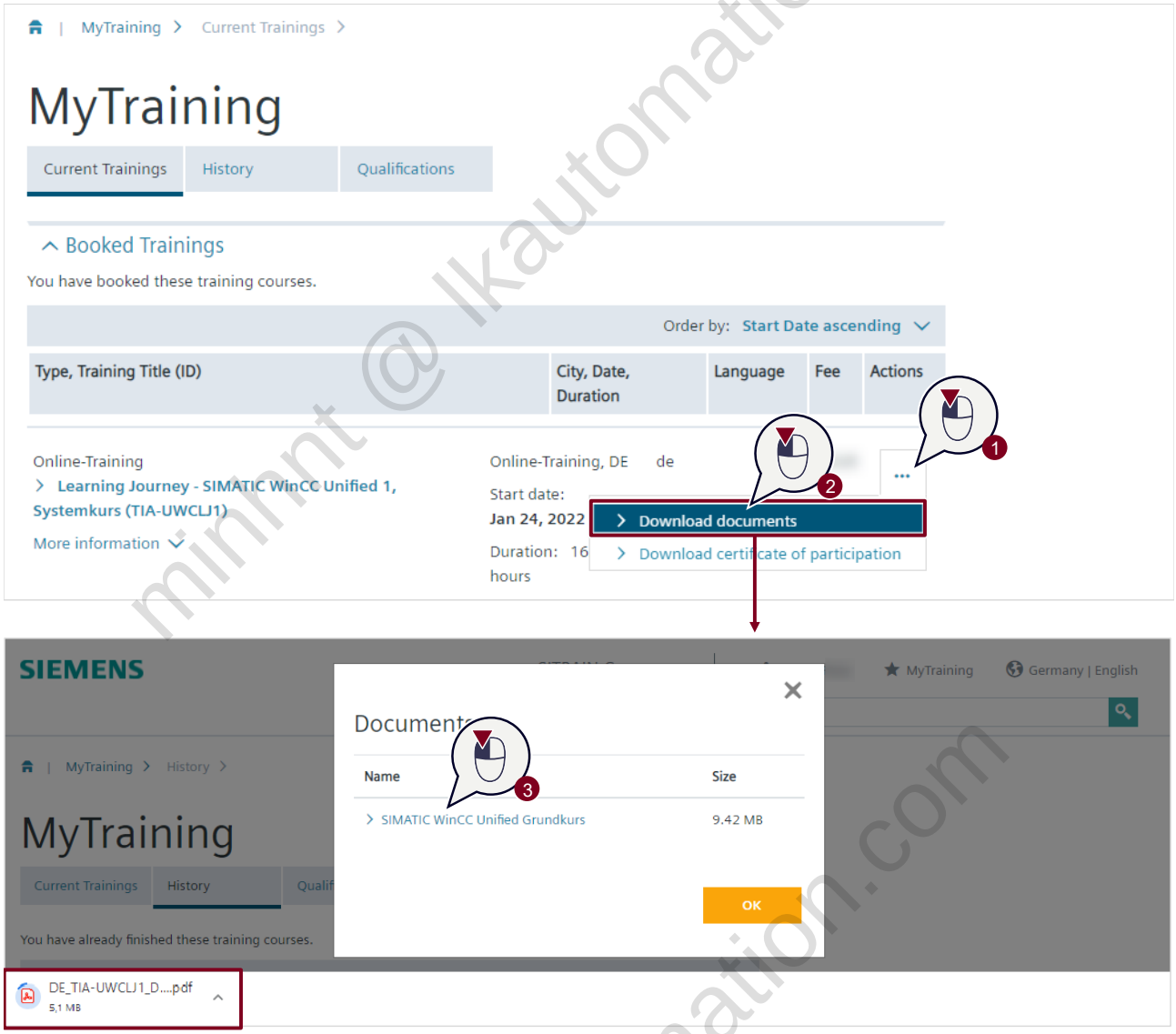
Note

The document can only be annotated up to the end of the training. You must then download your materials in order to hold onto them permanently. Once the document has been downloaded, it can no longer be annotated with the PDF annotator.

For details on how to use the PDF annotator, watch this video: www.siemens.com/sitrain-video-annotationen

11.1.3 Downloading documents and materials

To have your course materials permanently available, you need to download them at the end of the training. You download them in MyTraining:



Note The course documentation with your annotations is available to you for download for 30 days **with** annotations and 365 days **without** annotations after the end of the course.

11.1.4 Downloading certificates of participation and other certificates

You can download your certificates of participation and other certificates after the end of the course in MyTraining, as well:

11.2 Additional learning offers for advancing your knowledge

11.2.1 SITRAIN enables the learning of tomorrow – today!

SITRAIN offers you a comprehensive range of knowledge on Siemens Industry products – directly from the manufacturer, for all industry sectors and applications and for novices and experts alike.

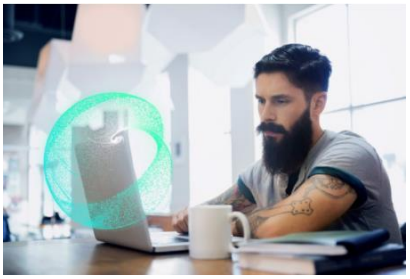
You will find all the important information about our needs-based learning offer at www.siemens.com/sitrain

Three learning formats to meet every need:

Learning Journey
The optimal combination of methods for more sustainable learning success.

www.siemens.com/sitrain-learning-journey

Learning Membership
Secure your knowledge, through independent and continuous learning.



www.siemens.com/sitrain-learning-membership

Learning Event
Build knowledge quickly, compactly and guided.

www.siemens.com/sitrain-learning-event

Which learning format is right for you?

Feature	Journey	Membership	Event
Typical duration	Several weeks	1 year	Several days
Share of independent learning	++	+++	+
Share of guided learning with a learning consultant	++	0	+++
Individual coaching by a learning consultant	+++	0	++
Sustainability of what has been learned	+++	++	++
Exercise portion	+++	+	+++
Individual learning pace	++	+++	+
Integration into your work routine	++	+++	+
Diversity of methods	+++	+	++
On-demand knowledge availability	+++	+++	+
Recommendation	We recommend a Learning Journey for customers who want to take advantage of both independent and guided learning – for **sustained** learning success	We recommend a Learning Membership for customers who want to learn **independently** and **continuously**.	We recommend a Learning Event for customers who especially value **fast, guided** knowledge building.

11.2.2 Optimal preparation and follow-up for maximum learning success

A Learning Membership for SITRAIN allows you to optimally prepare for your training and consolidate and reinforce your knowledge afterward. Depending on the topic, you will find a vast collection of E-learning courses here: www.siemens.com/sitrain-recommendations

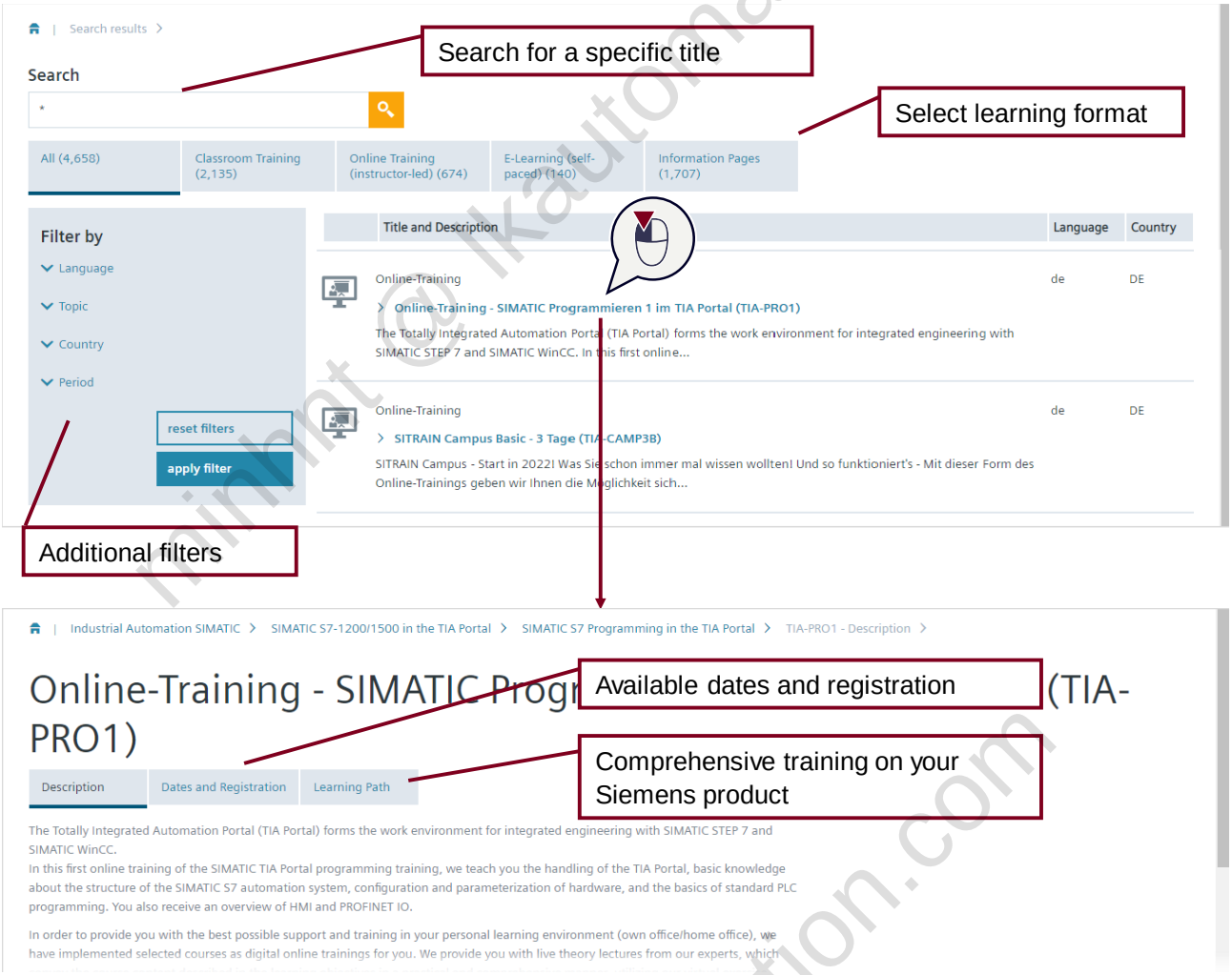
For more information on the advantages of a SITRAIN Learning Membership, go to: www.siemens.com/sitrain-learning-membership

11.2.3 Finding learning offers

Searching for specific learning offers

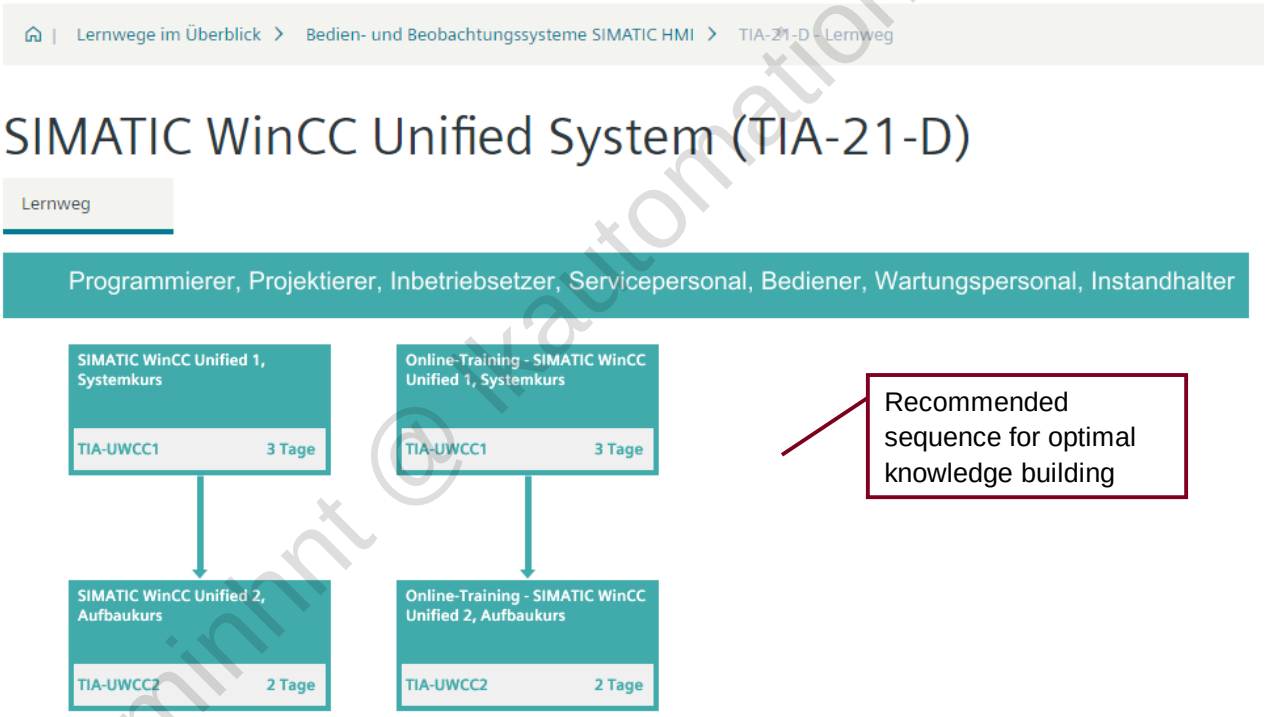
How to quickly find the SITRAIN offer that meets your learning needs:

www.sitrain-learning.siemens.com



Finding learning paths

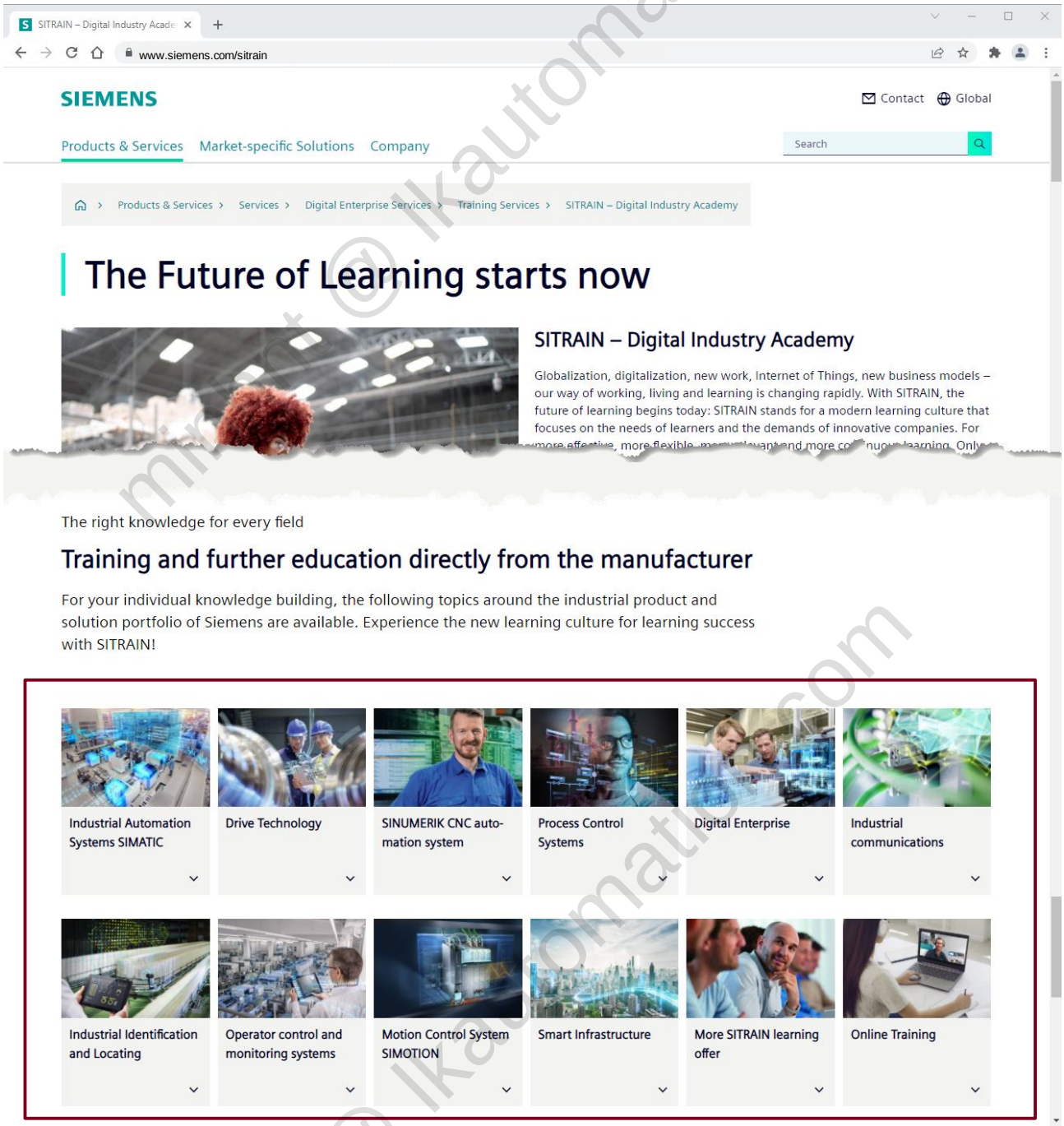
With the help of the interactive learning paths, you can find the optimal comprehensive training for your Digital Industry product.



Browsing topics

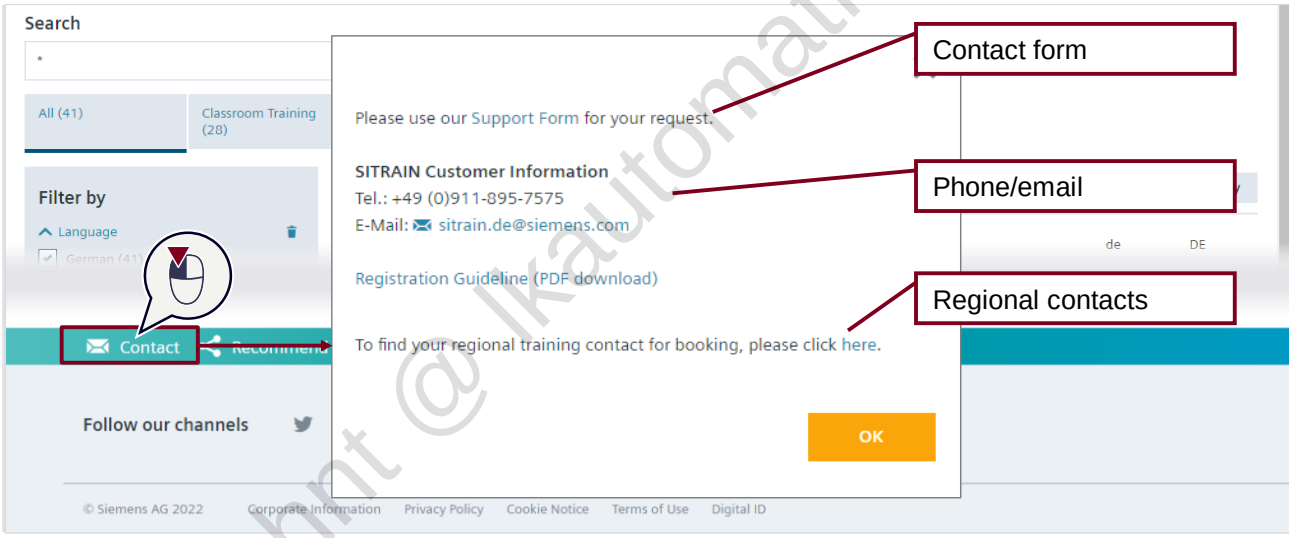
By browsing our learning offers by topic, you can find a wide range of training for your Siemens product.

You can get to the topic areas via our SITRAIN homepage: www.siemens.com/sitrain



11.2.4 Getting advice

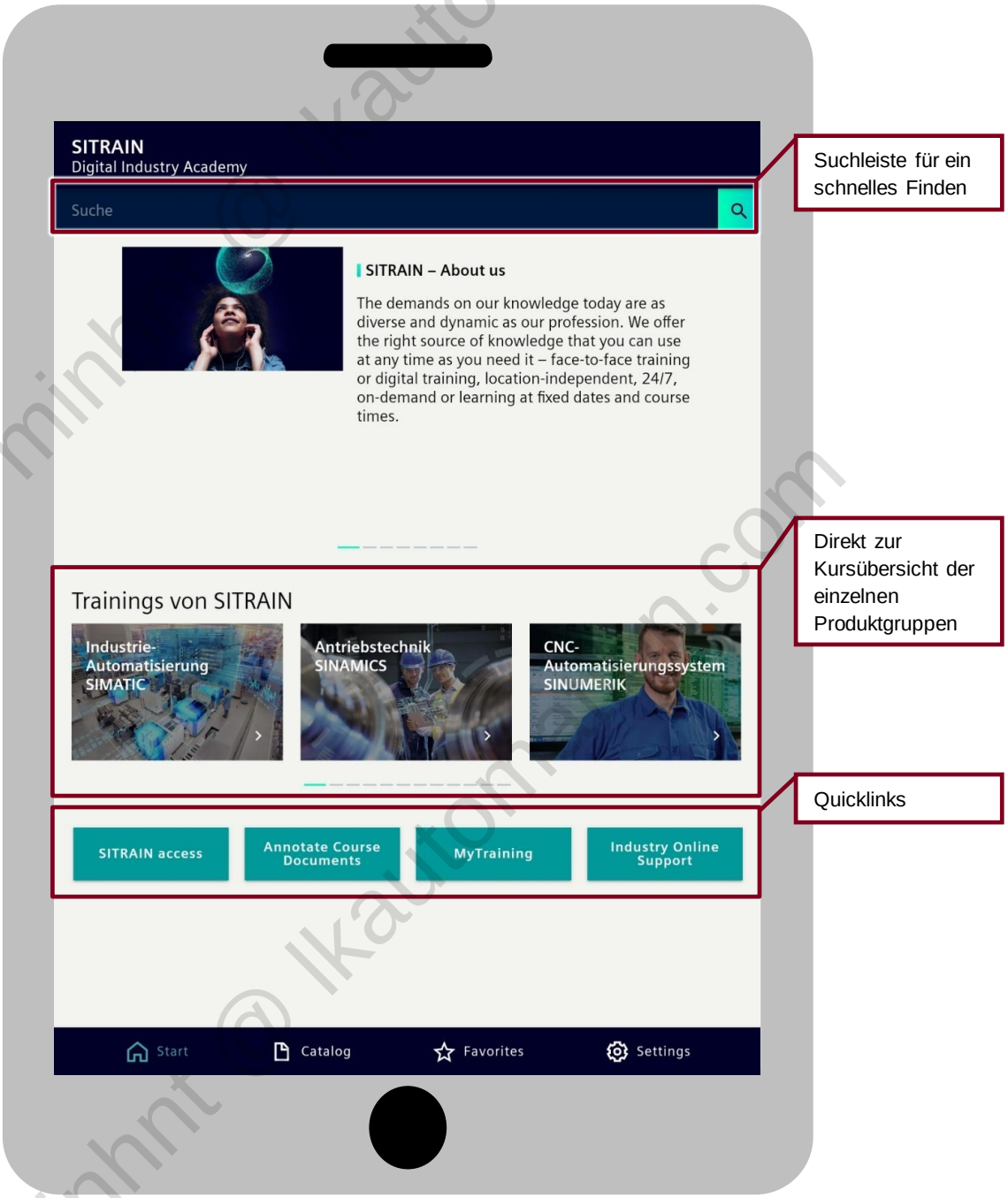
If you have questions about our offer or about registration, feel free to contact our Customer Support:



11.2.5 Everything in the palm of your hand – The SITRAIN app

With the SITRAIN app, you have all the important information and functions for your learning success in a single application.

You can download the SITRAIN app for free in your operating system's app store:



11.3 Information and support for the industrial product portfolio

11.3.1 An answer to every question – The Industry Online Support

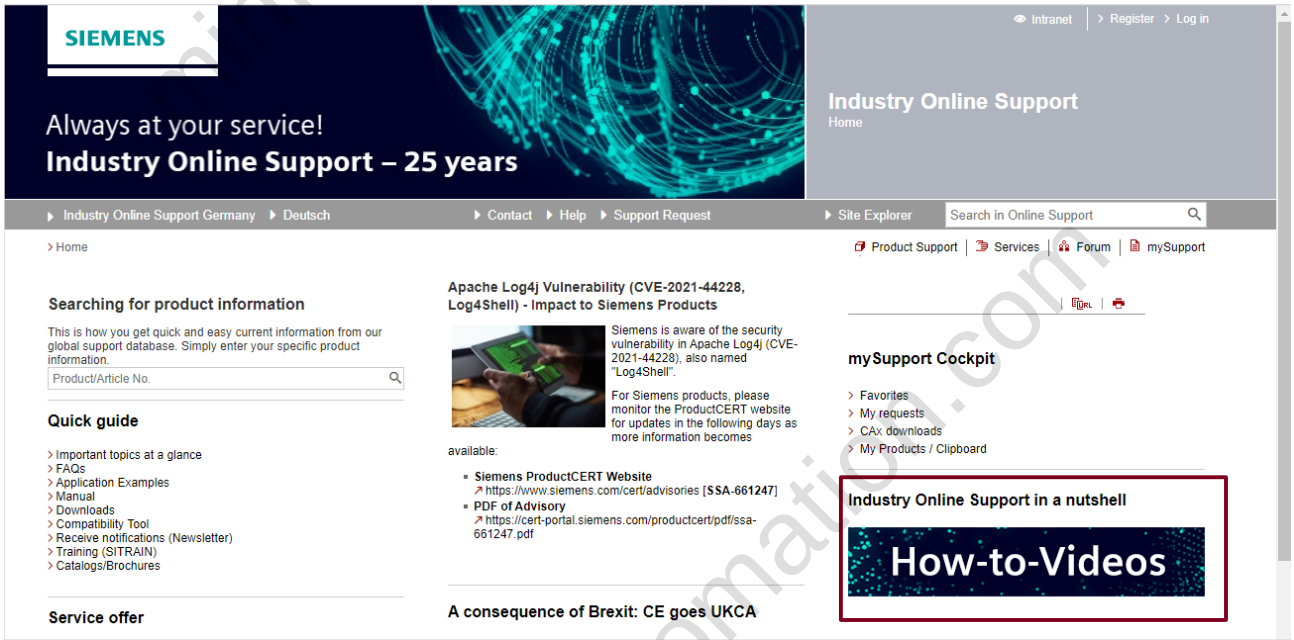
The Industry Online Support provides you with technical information and solutions for all Siemens industrial products.

Here you will find:

- Manuals
- FAQs
- Application examples
- Multimedia content
- Forum for exchanging information with the community
- Support Requests to Technical Support

How to access the Industry Online Support: www.siemens.com/online-support

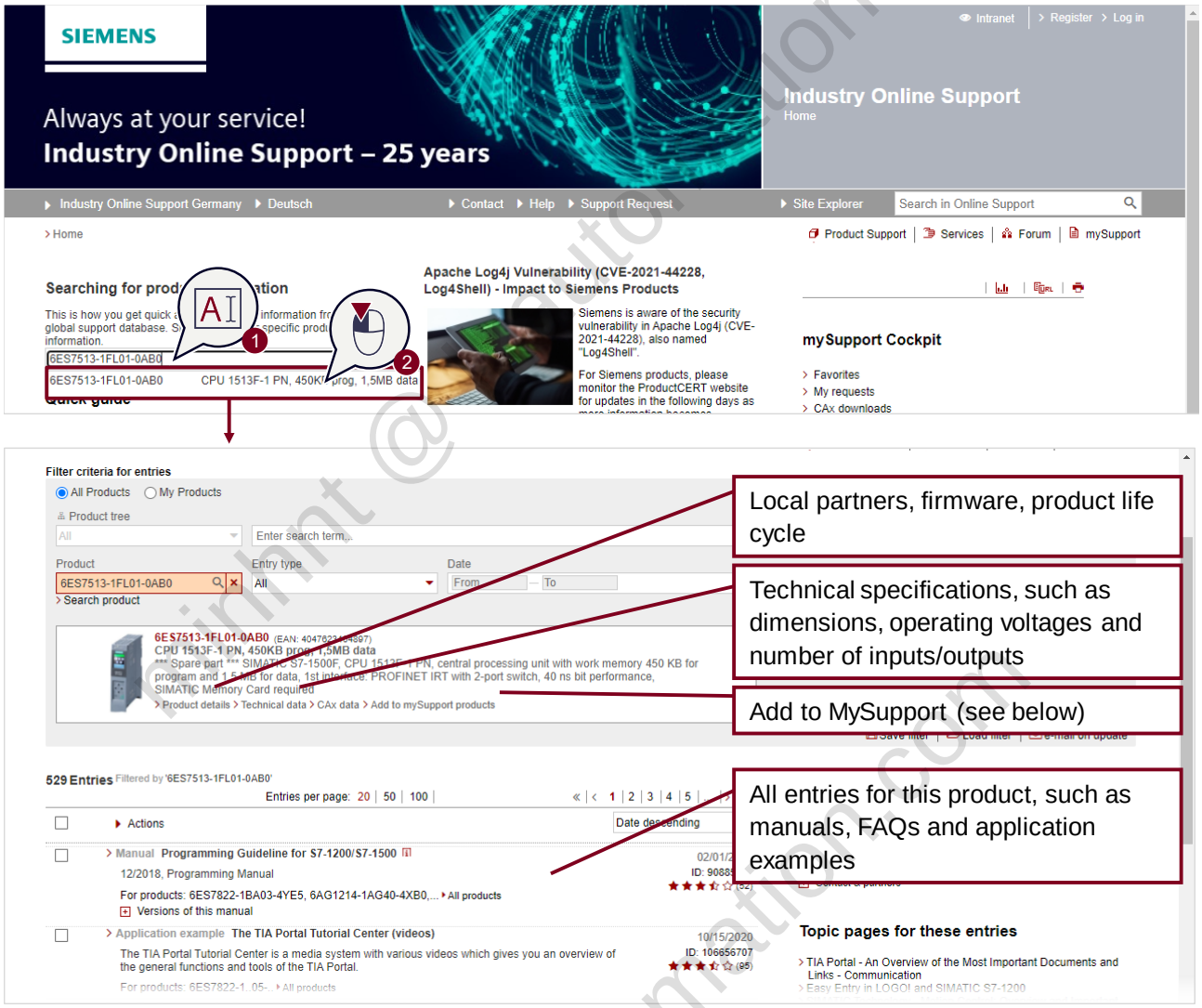
Detailed how-to videos demonstrating the benefits and use of the Industry Online Support can be found here:



11.3.2 Highlights of the Industry Online Support

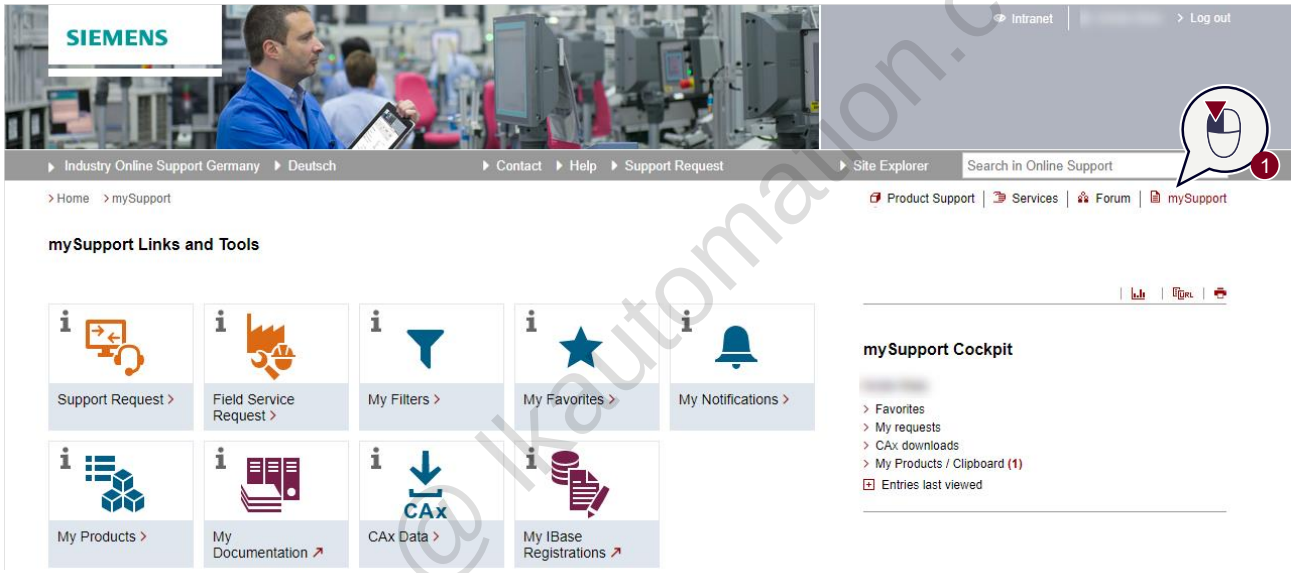
At a glance – Complete product information

Enter the article number to obtain complete product information:



Your personal work area – MySupport

You will find all personal settings and notifications in your MySupport area:



Note

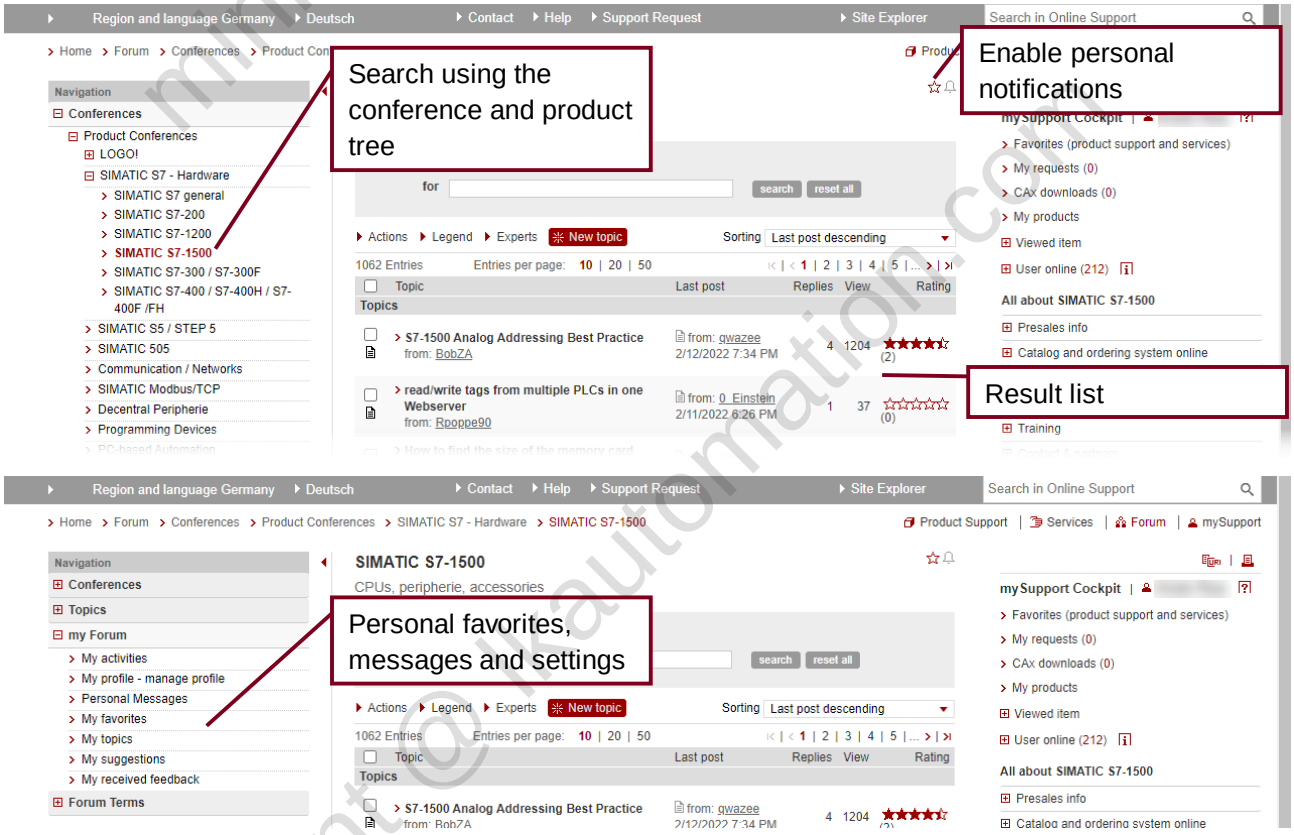
By setting up individual notifications and newsletters for yourself, you will stay up to date regarding your Siemens products. The how-to videos of the Industry Online Support show you how to store personal messages in MySupport.

From user to user – Forum

In the Industry Online Support Forum, you can quickly access help from and engage in exchange of information with other users of the industrial product and solution portfolio.

You can find the forum at: www.siemens.com/industry/forum

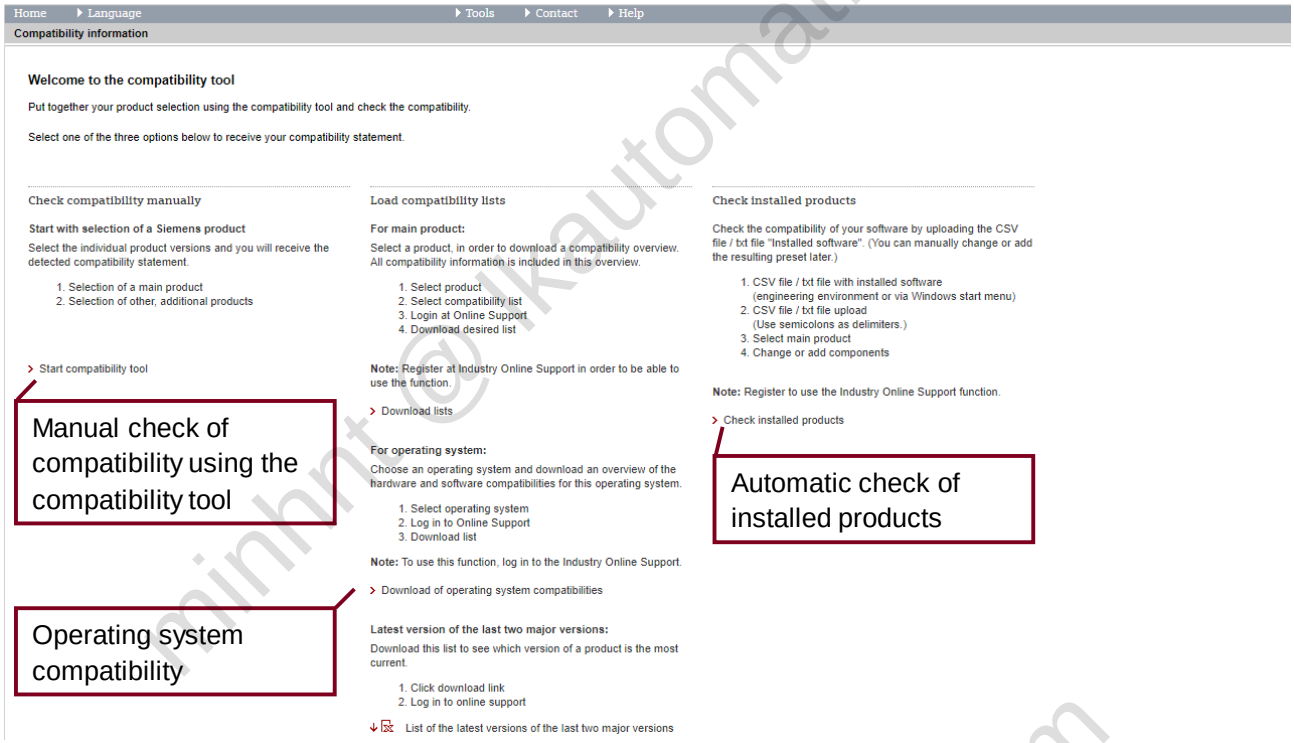
The forum provides you with a clear filter structure and many individual setting options, such as personal notifications.

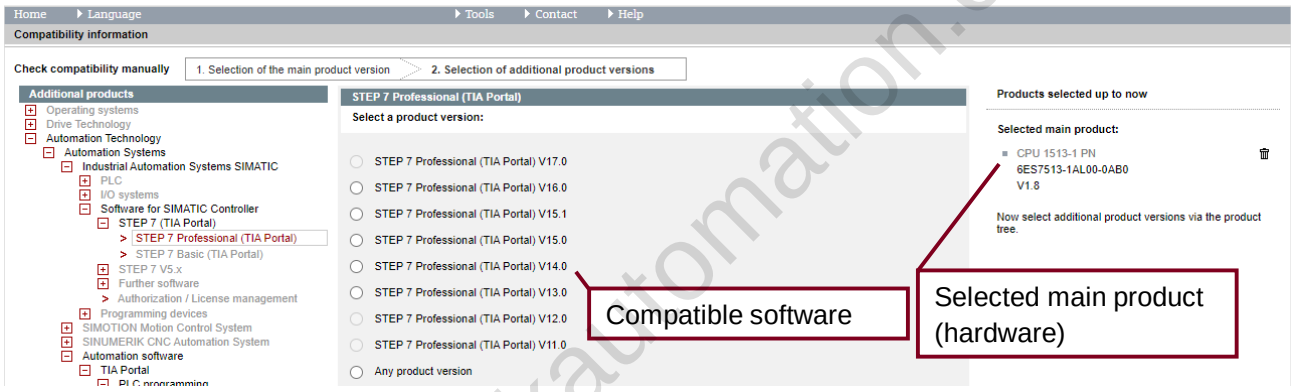


Always the right software – The compatibility tool

With the compatibility tool, you can quickly and easily check the combinability of your Siemens products.

You can find the tool at: http://www.siemens.com/compatool





You can find more information and explanatory videos on the compatibility tool in this entry: https://support.industry.siemens.com/cs/ww/en/view/64847781

11.3.3 Your personal problem solution – Technical Support

Do you have a problem or a question that you are unable to solve using our Industry Online Support knowledge database? Our Technical Support is ready to help you.

Simply send them a Support Request:

Pay attention to the following when creating your request:

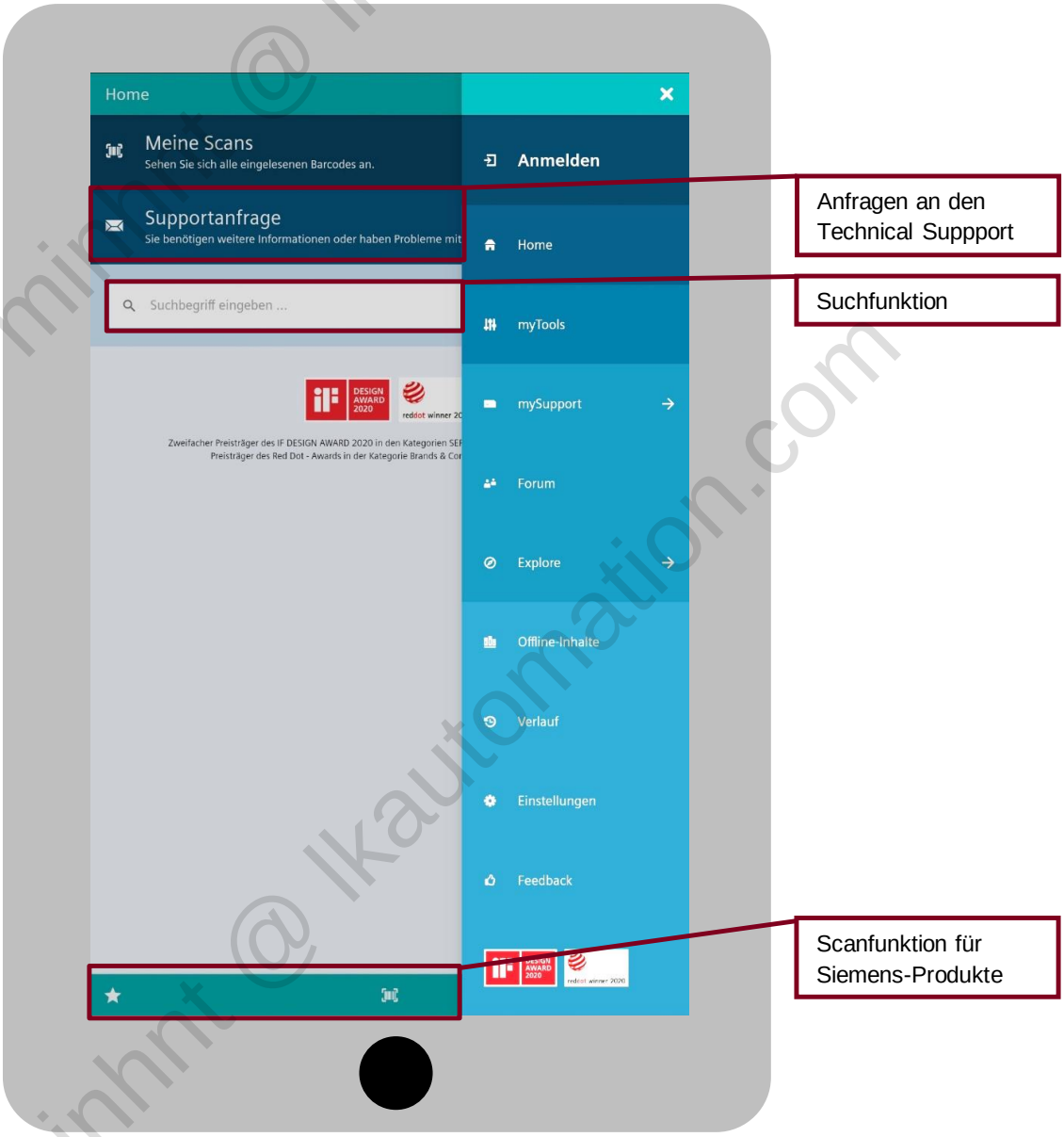
- Is your problem case already known? Consult the Industry Online Support knowledge database and the forum.
- Be precise when selecting your product and avoid choosing "Other".
- Describe your problem in as much detail as possible.
- Pictures and explanatory attachments help our team to better understand your problem.
- Check your contact information before sending.

11.3.4 Support anywhere at any time – The Industry Online Support app

The free Industry Online Support app is available in your operating system's app store:

Examples of what's available in the app:

- A scan function for quickly capturing the product you are searching for
- Access to the global community for knowledge sharing
- A wide variety of entries and products available offline
- A convenient way to send queries to Technical Support (Support Request)



11.4 Links

Here you will find all the relevant links for your learning success summarized in one place:

Table 11-1

What you need	Where you can find it
General overview of the SITRAIN offer	SITRAIN – Digital Industry Academy www.siemens.com/sitrain
Your personal learning overview (including download of materials and certificates)	MyTraining www.siemens.com/sitrain-mytraining
Tool for annotating your course materials	PDF Annotator www.siemens.com/sitrain-webannotator
Learning offers with sustained learning success	SITRAIN Learning Journey www.siemens.com/sitrain-learning-journey
Continuous independent learning	SITRAIN Learning Membership www.siemens.com/sitrain-learning-membership
Fast and concise learning	SITRAIN Learning Events www.siemens.com/sitrain-learning-event
Offers for course preparation and follow-up	SITRAIN recommendations www.siemens.com/sitrain-recommendations
Find specific learning offers	SITRAIN Course Program www.sitrain-learning.siemens.com
Information and support for the Siemens industrial product portfolio	Siemens Industry Online Support www.siemens.com/online-support
Personal support for your Siemens product	Technical Support https://support.industry.siemens.com/